Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mim functions

Synopsis

The functions prefixed with Mim make up the Image Processing module. The basic Image Processing module allows the user to perform point-to-point, statistical, spatial filtering, morphological, wavelet, and geometric transformation operations. These operations allow you to perform image enhancements, distortion corrections, and analyses.

Functions

MimAlloc
MimAllocResult
MimArith
MimArithMultiple
MimAugment
MimBinarize
MimBinarizeAdaptive
MimBoundingBox
MimClip
MimClose
MimConnectMap
MimControl
MimConvert
MimConvolve
MimCountDifference
MimDeadPixelCorrection
MimDeinterlace
MimDetectEvent
MimDifferential
MimDilate
MimDistance
MimDraw
MimEdgeDetect
MimErode
MimFilterAdaptive
MimFilterBinomial
MimFilterMajority
MimFindExtreme
MimFindOrientation
MimFlatField
MimFlip
MimFree
MimGet
MimGetResult
MimGetResult1d
MimGetResult2d
MimGetResultSingle
MimHistogram
MimHistogramEqualize
MimHistogramEqualizeAdaptive
MimInquire
MimLabel
MimLocateDifference

MimLocateEvent
MimLocatePeak1d
MimLutMap
MimMatch
MimMorphic
MimMorphicShape
MimOpen
MimPolarTransform
MimProjection
MimPut
MimRank
MimRearrange
MimRemap
MimResize
MimRestore
MimRotate
MimSave
MimShift
MimStatCalculate
MimStream
MimThick
MimThin
MimTransform
MimTranslate
MimUnwarpAlongPath
MimWarp
MimWarpList
MimWatershed
MimWaveletDenoise
MimWaveletSetFilter
MimWaveletTransform
MimZoneOfInfluence

MimAlloc

Synopsis

Allocate an image processing context.

Syntax

```
AIL_ID MimAlloc(
    AIL_ID SysId,          //in
    AIL_INT64 ContextType, //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ContextImIdPtr //out
)
```

Description

This function allocates an image processing context on the specified system.

Note that image processing contexts allocated with **MimAlloc**() can be saved using either MimStream() or MimSave() and restored using MimRestore().

When the image processing context is no longer required, release it using MimFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the image processing context's lifetime and you must not manually free it.

Parameters

SysId

Specifies the system on which to allocate the context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType INQ

Specifies the type of image processing context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUGMENTATION_CONTEXT	Specifies an image processing context that can be used with MimAugment().	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARIZE_ADAPTIVE_CONTEXT	Specifies an image processing context that can be used with MimBinarizeAdaptive() to perform an adaptive binarization.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT	Specifies an image processing context that can be used with MimBinarizeAdaptive() to perform an adaptive binarization using seeds.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEAD_PIXEL_CONTEXT	Specifies an image processing context that can be used with MimDeadPixelCorrection().	c		e	f	g	a		i		j	k	l	m	n	b
M_DEINTERLACE_CONTEXT	Specifies an image processing context that can be used with MimDeinterlace().	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_MAJORITY_CONTEXT	Specifies an image processing context that can be used with MimFilterMajority().	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND_ORIENTATION_CONTEXT	Specifies an image processing context that can be used with MimFindOrientation().	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_FIELD_CONTEXT	Specifies an image processing context that can be used with MimFlatField().	c		e	f	g	a		i		j	k	l	m	n	b
M_HISTOGRAM_EQUALIZE_ADAPTIVE_CONTEXT	Specifies an image processing context that can be used with MimHistogramEqualizeAdaptive().	c		e	f	g	a		i		j	k	l	m	n	b
M_IM_REMAP_CONTEXT	Specifies an image processing context that can be used with MimRemap().	c		e	f	g	a		i		j	k	l	m	n	b
M_LINEAR_FILTER_IIR_CONTEXT	Specifies an image processing context that can be used with MimConvolve() or MimDifferential().	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_PEAK_1D_CONTEXT	Specifies an image processing context that can be used with MimLocatePeak1d().	c		e	f	g	a		i		j	k	l	m	n	b
M_MATCH_CONTEXT	Specifies an image processing context that can be used with MimMatch().	c		e	f	g	a		i		j	k	l	m	n	b
M_REARRANGE_CONTEXT	Specifies an image processing context that can be used with MimRearrange().	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_CONTEXT	Specifies an image processing context that can be used with MimStatCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_CUMULATIVE_CONTEXT	Specifies an image processing context that can be used with MimStatCalculate() when calculating statistics from multiple images.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNWARP_ALONG_PATH_CONTEXT	Specifies an image processing context that can be used with MimUnwarpAlongPath().	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_TRANSFORM_CONTEXT	Specifies an image processing context that can be used with MimWaveletTransform() and MimWaveletDenoise().	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_TRANSFORM_CUSTOM_CONTEXT	Specifies a custom image processing context that can be used with MimWaveletTransform() and MimWaveletDenoise().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextImIdPtr

Specifies the address of the variable in which to write the image processing context identifier or specifies the data type that the function should use to return the image processing context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated image processing context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated image processing context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_IM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the image processing context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the image processing context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_IM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the context types listed above are not available in Aurora Imaging Library Lite. See the context's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimAllocResult

Synopsis

Allocate an image processing result buffer.

Syntax

```
AIL_ID MimAllocResult(
    AIL_ID SysId,         //in
    AIL_INT NbEntries,    //in
    AIL_INT64 ResultType, //in
    AIL_ID *ResultImIdPtr //out
)
```

Description

This function allocates a result buffer with the specified number of entries, for use with some of the Image Processing module's functions (for example, the statistical functions).

When the image processing result buffer is no longer required, release it using MimFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the image processing result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

NbEntries INQ

Specifies the number of buffer entries to allocate for the specified result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of buffer entries																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that Aurora Imaging Library automatically establishes the size of the result buffer. You can only use this value when the ResultType parameter is set to M_AUGMENTATION_RESULT, M_COUNT_LIST, M_LOCATE_DIFFERENCE_RESULT, M_LOCATE_PEAK_1D_RESULT, M_STATISTICS_RESULT, or M_WAVELET_TRANSFORM_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies the number of buffer entries. This value should take into account the type of result buffer being allocated. A specific number of entries must be specified when the ResultType parameter is set to M_EVENT_LIST, M_EXTREME_LIST, M_FIND_ORIENTATION_LIST, M_HIST_LIST, or M_PROJ_LIST. A specific number of entries can be specified when the ResultType parameter is set to M_LOCATE_DIFFERENCE_RESULT to limit the number of results. A specific number of entries cannot be specified when the ResultType parameter is set to M_COUNT_LIST, M_LOCATE_PEAK_1D_RESULT, M_STATISTICS_RESULT, or M_WAVELET_TRANSFORM_RESULT. Note that for NbEntries other than M_DEFAULT, M_EVENT_LIST and M_LOCATE_DIFFERENCE_RESULT results from a multi-core processing application might differ from run to run.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType INQ

Specifies the type of data that will be stored in this result buffer. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of data																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUGMENTATION_RESULT	Specifies that the buffer can hold MimAugment() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_COUNT_LIST	Specifies that the buffer can hold MimCountDifference() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVENT_LIST +	Specifies that the buffer can hold MimLocateEvent() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTREME_LIST +	Specifies that the buffer can hold MimFindExtreme() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND_ORIENTATION_LIST +	Specifies that the buffer can hold MimFindOrientation() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_LIST	Specifies that the buffer can hold MimHistogram() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_DIFFERENCE_RESULT	Specifies that the buffer can hold MimLocateDifference() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_PEAK_1D_RESULT	Specifies that the buffer can hold MimLocatePeak1d() results. In this case, you must set the NbEntries parameter to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJ_LIST +	Specifies that the buffer can hold MimProjection() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATISTICS_RESULT	Specifies that the buffer can hold MimStatCalculate() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_TRANSFORM_RESULT	Specifies that the buffer can hold results from MimWaveletTransform() and MimWaveletDenoise(). Such results can themselves be processed. That is, you can use the wavelet result as the source with which to perform MimWaveletTransform() and MimWaveletDenoise().	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_EVENT_LIST; M_EXTREME_LIST; M_FIND_ORIENTATION_LIST; M_PROJ_LIST.

You can add the following value to the above-mentioned values to set the type of data stored in the result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_PROJ_LIST, M_EXTREME_LIST, or M_EVENT_LIST to change the type of data to store

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Allocates a result buffer of type *float*. You should allocate a *float* result type buffer when the source buffer specified by MimProjection(), MimFindExtreme(), or MimLocateEvent() is of type *float*. If the specified source buffer is of type *float* and the result buffer is not, the functions can produce inaccurate results. Note that allocating a result buffer of type *float* when the source buffer is not, will slow down the operations performed by the MimProjection(), MimFindExtreme(), or MimLocateEvent() function.	c		e	f	g	a		i		j	k	l	m	n	b

ResultImIdPtr

Specifies the address of the variable in which to write the image processing result buffer identifier or specifies the data type that the function should use to return the image processing result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated image processing result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated image processing result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_IM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the image processing result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated image processing result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the image processing result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_IM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the result buffer types listed above are not available in Aurora Imaging Library Lite. See the result buffer's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.

DLL	Requires ail11.dll; ailim11.dll.

MimArith

Synopsis

Perform a point-to-point arithmetic operation.

Syntax

```
void MimArith(
    AIL_DOUBLE Src1ImageBufIdOrConst, //in
    AIL_DOUBLE Src2ImageBufIdOrConst, //in
    AIL_ID DstImageBufId,             //out
    AIL_INT64 Operation               //in
)
```

Description

This function performs the specified point-to-point operation on two images, an image and a constant, an image, or a constant, storing results in the specified destination image buffer.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

Src1ImageBufIdOrConst

Specifies the data source of the first operand. This parameter can be given an image buffer identifier or a constant. When using a constant, it will be considered to have the same type as the destination buffer unless M_FLOAT_PROC or M_FIXED_POINT has been added to the operation type. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

> Note that when the destination buffer is unsigned and Src1ImageBufIdOrConst refers to a constant, it will be considered unsigned as well. In this case, you should not use a negative value for Src1ImageBufIdOrConst, since it might cause unexpected results.

> Note that logical operations (for example M_AND and M_OR) cannot be performed on a floating-point buffer.

Src2ImageBufIdOrConst

Specifies the data source of the second operand. This parameter can be given an image buffer identifier or a constant. If the selected operation uses only one operand, set this parameter to **M_NULL**. When using a constant, it will be considered to have the same type as the destination buffer unless M_FLOAT_PROC or M_FIXED_POINT has been added to the operation type. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

> Note that when the destination buffer is unsigned and Src2ImageBufIdOrConst refers to a constant, it will be considered unsigned as well. In this case, you should not use a negative value for Src2ImageBufIdOrConst, since it might cause unexpected results.

> Note that logical operations (for example M_AND and M_OR) cannot be performed on a floating-point buffer.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer identifier. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Operation

Specifies the operation to perform. This parameter should be set in accordance to the operands.

Operations using two image buffer operands are the following:

For specifying the type of operation

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ABS_SUB +	Subtracts the absolute values of the second image from the corresponding absolute values of the first image.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADD +	Adds the values of the first image to the corresponding values of the second image.	c		e	f	g	a		i		j	k	l	m	n	b
M_AND +	Performs a bitwise AND operation between the values of the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b
M_ATAN2	Performs the two-argument arctangent function, atan2(*x*, *y*), using the first image for the X-coordinates and the second image for the Y-coordinates. This results in the counter-clockwise angle between the positive X-axis and each X- and Y-coordinate. > Note that M_ATAN2 uses the standard mathematical convention, where the Y-axis is oriented upwards. If you want to use the Aurora Imaging Library angle convention, negate the Y-coordinate buffer using **MimArith**() with M_NEG before you use this operation. Aurora Imaging Library establishes the resulting angle according to the type of the source image buffer, and its minimum and maximum values. For unsigned buffers, Aurora Imaging Library maps 0° to the minimum value of the two images, and 360° to the maximum value plus 1 (for 8-bit buffers, a maximum possible value of 256 maps to 360°). For signed buffers, Aurora Imaging Library maps -180° to the minimum value of the two images, and +180° to the maximum value plus 1. For float buffers, Aurora Imaging Library expects values between 0 and 1, and maps them to angles between 0° and 360°.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVG +	Averages the values of the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIV +	Divides the values of the first image by the values of the second image. Note that dividing a value by 0 will generate a destination pixel with an unpredictable value. This will not generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXP +	Raises the values of the first image to the power of the second image's values.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOG +	Takes the logarithm of the values of the second image, to the base of the values of the first image.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Compares the values of the first image with the values of the second image, and takes the maximum of the two.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_ABS +	Compares the absolute values of the first image with the absolute values of the second image, and takes the maximum of the two.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Compares the values of the first image with the values of the second image, and takes the minimum of the two.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_ABS +	Compares the absolute values of the first image with the absolute values of the second image, and takes the minimum of the two.	c		e	f	g	a		i		j	k	l	m	n	b
M_MULT +	Multiplies the values of the first image with the values of the second image.	c		e	f	g	a		i		j	k	l	m	n	b
M_NAND +	Performs a bitwise NAND operation between the values of the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOR +	Performs a bitwise NOR operation between the values of the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b

M_OR +	Performs a bitwise OR operation between the values of the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB +	Subtracts the values of the second image from the values of the first image (for example, image 1 - image 2).	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_ABS +	Takes the absolute value of the result of subtracting the values of the second image from the values of the first image.	c		e	f	g	a		i		j	k	l	m	n	b
M_XNOR +	Performs a bitwise exclusive NOR operation between the values of the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b
M_XOR +	Performs a bitwise exclusive OR operation between the first and second images.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_MAX_ABS; M_MIN_ABS.

You can add the following value to the above-mentioned values to set the destination buffer to the source value of the selected pixel instead of the pixel's value used in the comparison.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to save the source value																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SOURCE_VALUE	Specifies to write the source value of the selected pixel into the destination buffer. When corresponding pixels from the two source images are compared based on the specified operation, **MimArith**() writes the source value of the pixel from the selected image into the destination buffer instead of the value used during the comparison. For example, if combined with M_MAX_ABS, after finding the maximum absolute value between the corresponding pixels in the two source images, **MimArith**() writes the source value of the bigger absolute value in the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b

For a signed or unsigned destination buffer, the following logical operations will result in either the highest unsigned buffer value (regardless if the buffer is signed) or 0. For a floating-point buffer, the result will either be 0 or 1. Logical operations using two image buffer operands include the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For logical operations using two image buffers																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL	Determines if the values of the first and second images are equal.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Determines if the values of the first image are greater than the second image.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Determines if the values of the first image are greater than or equal to the second image.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Determines if the values of the first image are less than the second image.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Determines if the values of the first image are less than or equal to the second image.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_EQUAL	Determines if the values of the first and second images are not equal.	c		e	f	g	a		i		j	k	l	m	n	b

The following are operations that use an image buffer operand and a constant. If the operation is not commutative, you must pass

the constant as either the first or second operand, as can be determined from the predefined operation name.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For operations using one image buffer and a constant																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ADD_CONST +	Adds a constant to the values of an image.	c		e	f	g	a		i		j	k	l	m	n	b
M_AND_CONST +	Performs a bitwise AND operation on the image and a constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONST_DIV +	Divides a constant by the values of an image (for example, constant / image). Note that dividing the constant by 0 will generate a destination pixel with an unpredictable value. This will not generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONST_EXP +	Raises a constant to the power specified by the values of an image.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONST_LOG +	Takes the logarithm of the values of an image, to the base of a constant (for example LOG $_{constant}$ Image).	c		e	f	g	a		i		j	k	l	m	n	b
M_CONST_PASS	Copies the constant to each destination pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONST_SUB +	Subtracts the image from the constant (for example, constant - image).	c		e	f	g	a		i		j	k	l	m	n	b
M_DIV_CONST +	Divides the values of an image by a constant (for example, image / constant). Note that dividing a value by 0 will generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXP_CONST +	Raises the values of the image to the power specified by the constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOG_CONST +	Takes the logarithm of a constant, to the base of the values of an image (for example LOG $_{image}$ Constant).	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_CONST	Compares the values of one image with a constant, and takes the maximum of the two.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_CONST	Compares the values of one image with a constant, and takes the minimum of the two.	c		e	f	g	a		i		j	k	l	m	n	b
M_MULT_CONST +	Multiplies the values of the image by the constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_NAND_CONST +	Performs a bitwise NAND operation with an image and a constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOR_CONST +	Performs a bitwise NOR operation with an image and a constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_OR_CONST +	Performs a bitwise OR operation with an image and a constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_CONST +	Subtracts a constant from the values of an image (for example, image - constant).	c		e	f	g	a		i		j	k	l	m	n	b
M_SUB_CONST_ABS +	Takes the absolute value of the subtraction of a constant value from the values of the first image.	c		e	f	g	a		i		j	k	l	m	n	b

M_XNOR_CONST +	Performs a bitwise exclusive NOR operation with an image and a constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_XOR_CONST +	Performs a bitwise exclusive OR operation with an image and a constant.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DIV; M_ADD_CONST; M_CONST_DIV; M_CONST_EXP; M_CONST_LOG; M_CONST_SUB; M_DIV_CONST; M_EXP_CONST; M_LOG_CONST; M_MULT_CONST; M_SUB_CONST.

You can add one of the following values to the above-mentioned values to specify that the operation is performed in fixed-point or floating-point format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to perform operations in fixed-point or floating-point format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIXED_POINT	Forces the operation to be performed in a fixed point format. Operations using this attribute result in an 0.8 fixed-point format for an 8-bit destination. That is, the 8 bits comprise only the fractional portion of the value, and no integer portion. When the destination image buffer is 16-bit, operations that use the M_FIXED_POINT attribute result in an 8.8 fixed-point format (where the eight most-significant bits are the integer portion and the eight least-significant bits are the fractional portion). For a 32-bit destination, the result is in a 16.16 fixed-point format. Note that this attribute cannot be used when using a floating-point destination buffer and it can only be used with M_CONST_DIV, M_DIV and M_DIV_CONST. In addition, this attribute cannot be used in combination with M_FLOAT_PROC.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT_PROC	Forces the operation to be performed in floating-point format. Note that, if the destination buffer is not a floating-point buffer, the resulting data of the operation will be truncated accordingly. This attribute cannot be used with M_DIV. In addition, this attribute cannot be used in combination with M_FIXED_POINT.	c		e	f	g	a		i		j	k	l	m	n	b

For a signed or unsigned destination buffer, the following logical operations will result in either the highest unsigned buffer value (regardless if the buffer is signed) or 0. For a floating-point buffer, the result will either be 0 or 1. Logical operations using one image buffer and a constant include the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For logical operations using one image buffer and a constant																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL_CONST	Determines if the values of an image are equal to the specified constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_CONST	Determines if the values of an image are greater than the specified constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL_CONST	Determines if the values of an image are greater than or equal to the specified constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_CONST	Determines if the values of an image are less than the specified constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL_CONST	Determines if the values of an image are less than or equal to the specified constant.	c		e	f	g	a		i		j	k	l	m	n	b

M_NOT_EQUAL_CONST	Determines if the values of an image are not equal to the specified constant.	c		e	f	g	a		i		j	k	l	m	n	b

Operations using only one image buffer operand are the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For operations using one image buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ABS	Takes the absolute value of the image values.	c		e	f	g	a		i		j	k	l	m	n	b
M_CEIL +	Rounds the image values up to the next integer. > This operation is only supported with float source buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_CUBE +	Raises the image values to the power of 3 (x^3).	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOOR +	Rounds the image values down to the previous integer. > This operation is only supported with float source buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTEGRAL +	Takes the integral of the image. This computes the cumulative sum of the source image at the corresponding pixels in the destination. Each pixel in the destination buffer contains the total sum of the pixels in the rectangular sections delimited by the pixel at (0, 0) and itself, inclusively.	c		e	f	g	a		i		j	k	l	m	n	b
M_LN +	Performs a natural logarithm on the image values.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOG2 +	Performs a base-2 logarithm on the image values.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOG10 +	Performs a base-10 logarithm on the image values.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEG +	Performs a unary negation operation (2's complement) on an image. For example, if a pixel has a binary value of 0x01000000 (corresponding to a decimal value of 64 in a signed buffer) and you perform an M_NEG operation on it, the resulting value will be 0x11000000 (corresponding to a decimal value of -64). > This operation is buffer dependent. That is, a signed buffer with a value of 64 will be converted to -64, whereas an unsigned buffer with a value of 64 will be converted to 192.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT +	Performs a bitwise NOT operation (1's complement) on an image. For example, if a pixel has a binary value of 0x01000000 (corresponding to a decimal value of 64 in an unsigned buffer) and you perform an M_NOT operation on it, the resulting value will be 0x10111111 (corresponding to a decimal value of 191). > This operation is buffer dependent. That is, a signed buffer with a value of 64 will be converted to -63, whereas an unsigned buffer with a value of 64 will be converted to 191.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Copies the source buffer to the destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_ROUND +	Rounds image values to the nearest integer value, and values with decimal portion 0.5 are rounded to the nearest integer value, away from 0. For example, 1.5 is rounded to 2, while -1.5 is rounded to -2. > This operation is only supported with float source buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROUND_HALF_UP +	Rounds image values to the nearest integer value, and values with decimal portion 0.5 are rounded up to the next highest integer value. For example, 1.5 is rounded to 2, while -1.5 is rounded to -1. > This operation is only supported with float source buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE +	Raises the image values to the power of 2 (x^2).	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE_ROOT +	Performs a square root operation on the image values.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUNC +	Removes the fractional portion of the image values, such that only the integer portion remains. For example, -1.3 truncates to -1, and 2.9 truncates to 2. > This operation is only supported with float source buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_AND; M_NAND; M_NOR; M_OR; M_XNOR; M_XOR; M_AND_CONST; M_NAND_CONST; M_NOR_CONST; M_OR_CONST; M_XNOR_CONST; M_XOR_CONST; M_NOT.

You can add the following value to the above-mentioned values to specify to use the logical operation rather than the bitwise operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to perform logical operations																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOGICAL	Specifies to use the logical operation rather than the bitwise operation. The bitwise operation performs the specified operation on each binary digit. The logical operation determines the boolean state of the image pixel or constant based on whether it is equal to 0, and then performs the specified operation on that boolean state. In the destination buffer, a result of TRUE or FALSE is represented as 1 or 0, respectively. **M_AND Bitwise Logical** 7 0111 7≠0 TRUE 3 0011 3≠0 TRUE 0011 TRUE	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ABS_SUB; M_ADD; M_AVG; M_EXP; M_LOG; M_MAX_ABS; M_MIN_ABS; M_MULT; M_SUB; M_SUB_ABS; M_ADD_CONST; M_CONST_EXP; M_CONST_LOG; M_CONST_SUB; M_EXP_CONST; M_LOG_CONST; M_MULT_CONST; M_SUB_CONST; M_SUB_CONST_ABS; M_CEIL; M_CUBE; M_FLOOR; M_INTEGRAL; M_LN; M_LOG2; M_LOG10; M_NEG; M_ROUND; M_ROUND_HALF_UP; M_SQUARE; M_SQUARE_ROOT; M_TRUNC.

You can add the following value to the above-mentioned values to specify that the operation result should be saturated if necessary.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to saturate results that overflow or underflow																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_SATURATION	Forces the operation to saturate any resulting pixel values that overflow or underflow the possible range of the destination buffer. The pixel values are clipped to fit within the buffer's range, rather than wrapped around the range. If you do not specify M_SATURATION and there is an overflow or underflow, the resulting pixel values are either clipped (saturated) or wrapped around the buffer's range (not saturated), depending on which is faster for your hardware.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- Optimized in-place processing is supported (source equals destination), but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimArithMultiple

Synopsis

Perform a point-to-point arithmetic operation using multiple source images.

Syntax

```
void MimArithMultiple(
    AIL_DOUBLE Src1ImageBufId,        //in
    AIL_DOUBLE Src2ImageBufIdOrConst, //in
    AIL_DOUBLE Src3ImageBufIdOrConst, //in
    AIL_DOUBLE Src4Const,             //in
    AIL_DOUBLE Src5Const,             //in
    AIL_ID DstImageBufId,             //out
    AIL_INT64 Operation,              //in
    AIL_INT64 OperationFlag           //in
)
```

Description

This function performs the specified point-to-point operation using multiple images, images and constants, or constants, storing results in the specified destination image buffer. Note that this function does not take 1-bit buffers in any of its parameters.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

Src1ImageBufId

Specifies the data source of the first operand; this parameter must be given an image buffer identifier. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

See the Parameter associations section for possible values that can be specified.

Src2ImageBufIdOrConst

Specifies the data source of the second operand; this parameter can be given an image buffer identifier or a constant. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

See the Parameter associations section for possible values that can be specified.

Src3ImageBufIdOrConst

Specifies the data source of the third operand; this parameter can be given an image buffer identifier or a constant. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

See the Parameter associations section for possible values that can be specified.

Src4Const

Specifies the data source of the fourth operand; this parameter must be given a constant.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Src5Const

Specifies the data source of the fifth operand; this parameter must be given a constant.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

DstImageBufId

Specifies the identifier of the destination of the results; this parameter must be given an image buffer identifier. If you specify an

image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

OperationFlag

Reserved for future expansion. Must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **Src1ImageBufId**, **Src2ImageBufIdOrConst**, **Src3ImageBufIdOrConst**, **Src4Const**, **Src5Const**, and **Operation** parameters.

- For the Operation, Src1ImageBufId, Src2ImageBufId, Src3ImageBufId, Src4ImageBufId, and Src5ImageBufId parameters

> Note that any unused parameters should be set to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the Operation, Src1ImageBufId, Src2ImageBufId, Src3ImageBufId, Src4ImageBufId, and Src5ImageBufId parameters																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Src1ImageBufId																
Src2ImageBufIdOrConst																
Src3ImageBufIdOrConst																
Src4Const																
Src5Const																
M_MULTIPLY_ACCUMULATE_1 +	Performs a point-to-point multiply and accumulate 1 operation using the equation that follows. $\text{DestImage} = \frac{(\text{Src1Image} * \text{Src2Const}) + \text{Src3Const}}{\text{Src4Const}}$															
Src1ImageBufId	Specifies the image buffer identifier required for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src2ImageBufIdOrConst	Specifies the constant required for the second operand. Note that the constant is cast to the same type as the buffer specified for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src3ImageBufIdOrConst	Specifies the constant required for the third operand. Note that the constant is cast to the same type as the buffer specified for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src4Const	Specifies the constant required for the fourth operand; this constant must be a power of 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_MULTIPLY_ACCUMULATE_2 +	Performs a point-to-point multiply and accumulate 2 operation using the equation that follows. $\text{DestImage} = \frac{(\text{Src1Image} * \text{Src2Const}) + (\text{Src3Image} * \text{Src4Const})}{\text{Src5Const}}$	c		e	f	g	a		i		j	k	l	m	n	b
Src1ImageBufId	Specifies the image buffer identifier required for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b

Src2ImageBufIdOrConst	Specifies the constant required for the second operand. Note that the constant is cast to the same type as the buffer specified for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src3ImageBufIdOrConst	Specifies the image buffer identifier required for the third operand. Note that the image buffer's data type must either match that of the first operand, or it must be of type M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b
Src4Const	Specifies the constant required for the fourth operand. Note that the constant is cast to the same type as the buffer specified for the third operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src5Const	Specifies the constant required for the fifth operand; this constant must be a power of 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_GAIN +	Performs a per-pixel gain and offset correction operation using the equation that follows. $\text{DestImage} = \frac{(\text{Src1Image} - \text{Src2Image}) * \text{Src3Image}}{\text{Src4Const}}$	c		e	f	g	a		i		j	k	l	m	n	b
Src1ImageBufId	Specifies the image buffer identifier required for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src2ImageBufIdOrConst	Specifies the image buffer identifier required for the second operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src3ImageBufIdOrConst	Specifies the image buffer identifier required for the third operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src4Const	Specifies the constant required for the fourth operand; this constant must be a power of 2.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHTED_AVERAGE +	Performs a weighted average operation using the equation that follows. $\text{DestImage} = \left(\frac{1}{\text{Src2Const}}\right) * \text{Src1Image} + \left(1 - \frac{1}{\text{Src2Const}}\right) * \text{Src3Image}$	c		e	f	g	a		i		j	k	l	m	n	b
Src1ImageBufId	Specifies the image buffer identifier required for the first operand.	c		e	f	g	a		i		j	k	l	m	n	b
Src2ImageBufIdOrConst	Specifies the constant required for the second operand; this constant must be a power of 2.	c		e	f	g	a		i		j	k	l	m	n	b
Src3ImageBufIdOrConst	Specifies the image buffer identifier required for the third operand. Note that the image buffer's data type must either match that of the first operand, or it must be of type M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For the Operation, Src1ImageBufId, Src2ImageBufId, Src3ImageBufId, Src4ImageBufId, and Src5ImageBufId parameters.

You can add the following value to the above-mentioned values to specify that the operation result should be saturated if necessary.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the Operation parameter																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_SATURATION	Forces the operation to saturate any resulting pixel values that overflow or underflow the possible range of the destination buffer. The pixel values are clipped to fit within the buffer's range, rather than wrapped around the range. If you do not specify M_SATURATION and there is an overflow or underflow, the resulting pixel values are either clipped (saturated) or wrapped around the buffer's range (not saturated), depending on which is faster for your hardware.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- Optimized in-place processing is supported (source equals destination), but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimAugment

Synopsis

Perform an augmentation operation on an image.

Syntax

```
void MimAugment(
    AIL_ID AugmentationContextImId,             //in
    AIL_ID SrcImageBufId,                       //in
    AIL_ID DstImageBufOrAugmentationResultImId, //out
    AIL_INT64 SeedValue,                        //in
    AIL_INT64 ControlFlag                       //in
)
```

Description

This function performs an augmentation operation that allows you to create a plausible variation of an image. This is done by applying a specified ordered set of image processing operations with randomized settings within a specified range. To establish the image processing operations that **MimAugment**() can perform, call MimControl() with M_AUG_..._OP.

MimAugment() places its results in the specified destination buffer, which can be an image or an augmentation result.

The image processing operations that you specify **MimAugment**() to perform (M_AUG_..._OP) are prioritized as follows:

1. Affine operations, such as: • M_AUG_ROTATION_OP • M_AUG_SCALE_OP • M_AUG_TRANSLATION_X_OP • M_AUG_TRANSLATION_Y_OP • M_AUG_ASPECT_RATIO_OP • M_AUG_SHEAR_X_OP • M_AUG_SHEAR_Y_OP	2. Structure operations, such as: • M_AUG_DILATION_OP • M_AUG_EROSION_OP • M_AUG_DILATION_ASYM_OP • M_AUG_EROSION_ASYM_OP	3. Geometric operations, such as: • M_AUG_FLIP_OP • M_AUG_CROP_OP • M_AUG_REDUCE_OP
4. Intensity operations, such as: • M_AUG_INTENSITY_MULTIPLY_OP • M_AUG_INTENSITY_ADD_OP • M_AUG_GAMMA_OP • M_AUG_LIGHTING_DIRECTIONAL_OP • M_AUG_SATURATION_GAIN_OP • M_AUG_HUE_OFFSET_OP • M_AUG_HSV_VALUE_GAIN_OP	5. Linear filter operations, such as: • M_AUG_SMOOTH_DERICHE_OP • M_AUG_SMOOTH_GAUSSIAN_OP • M_AUG_BLUR_MOTION_OP • M_AUG_SHARPEN_DERICHE_OP	6. Noise operations, such as: • M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP • M_AUG_NOISE_MULTIPLICATIVE_OP • M_AUG_NOISE_SALT_PEPPER_OP

To modify the order in which **MimAugment**() prioritizes the specified image processing operations, call MimControl() with M_PRIORITY.

Parameters

AugmentationContextImId

Specifies the identifier of the augmentation context. The context must have been allocated using MimAlloc() with

M_AUGMENTATION_CONTEXT.

SrcImageBufId

Specifies the identifier of the source image buffer. If you are preprocessing the augmentation context (M_PREPROCESS), set this parameter to **M_NULL**.

The image buffer can be a 1-, 8-, or 16-bit unsigned buffer, or a 32-bit floating-point buffer (with values that are normalized between 0.0 and 1.0). You must allocate this buffer using MbufAlloc...() with M_IMAGE + M_PROC.

Note that you cannot perform certain image processing operations (M_AUG_..._OP) on 1-bit unsigned images. Also note that some image processing operations are supported only for 3-band image buffers.

1-bit unsigned images do not support the following: • M_AUG_BLUR_MOTION_OP • M_AUG_GAMMA_OP • M_AUG_HSV_VALUE_GAIN_OP • M_AUG_HUE_OFFSET_OP • M_AUG_INTENSITY_ADD_OP • M_AUG_INTENSITY_MULTIPLY_OP • M_AUG_LIGHTING_DIRECTIONAL_OP • M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP • M_AUG_NOISE_MULTIPLICATIVE_OP • M_AUG_NOISE_SALT_PEPPER_OP • M_AUG_SATURATION_GAIN_OP • M_AUG_SHARPEN_DERICHE_OP • M_AUG_SMOOTH_DERICHE_OP • M_AUG_SMOOTH_GAUSSIAN_OP	Only 3-band images support the following: • M_AUG_HUE_OFFSET_OP • M_AUG_SATURATION_GAIN_OP • M_AUG_HSV_VALUE_GAIN_OP

When specifying a color (3-band) image, **MimAugment**() assumes an RGB color space by default. Operations that require a different color space (for example, HSV) internally convert the source image's color space.

DstImageBufOrAugmentationResultImId

Specifies the identifier of the destination in which to write the result of the augmentation transformation. The identifier must be for an image buffer or an augmentation image processing result buffer. If you are preprocessing the augmentation context (M_PREPROCESS), set this parameter to **M_NULL**.

The image buffer can be a 1-, 8-, or 16-bit unsigned buffer, or a 32-bit floating-point buffer. You must allocate this buffer using MbufAlloc...() with M_IMAGE + M_PROC.

The augmentation result buffer must have been allocated on the required system using MimAllocResult() with M_AUGMENTATION_RESULT.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

SeedValue

Specifies the initial value that will generate the randomized settings for the image processing operations used to perform the augmentation. This parameter must be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the seed that will generate the randomized settings																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies the default value. This depends on the seed mode, which is set using MimControl() with M_AUG_SEED_MODE. To use M_DEFAULT, the seed mode must be set to M_RNG_AUTO (default) or M_RNG_INIT_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that you are preprocessing the augmentation context (M_PREPROCESS).	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < *AIL_INT32_MAX*	Specifies a user-defined seed value. You can only specify a user-defined seed if MimControl() with M_AUG_SEED_MODE is set to M_USER_DEFINED_SEED.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies whether **MimAugment**() should perform the augmentation, or should explicitly preprocess the augmentation context. If you do not explicitly preprocess the context (M_PREPROCESS), **MimAugment**() preprocesses it the first time you call the function.

This parameter must be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Performs the augmentation.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Preprocesses the specified augmentation context explicitly. In this case, you must set the SrcImageBufId, DstImageBufOrAugmentationResultImId, and SeedValue parameters to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimBinarize

Synopsis

Perform a point-to-point binary threshold operation.

Syntax

```
AIL_INT MimBinarize(
    AIL_ID SrcImageBufId,             //in
    AIL_ID DstImageBufId,             //out
    AIL_INT64 ConditionAndThreshMode, //in
    AIL_DOUBLE LowParam,              //in
    AIL_DOUBLE HighParam              //in
)
```

Description

This function performs a binary threshold operation on the specified image. Each pixel that meets the specified condition is set to the highest unsigned destination buffer value, while other pixels are set to 0. For example, the highest buffer value for an 8-bit buffer is 0xff (regardless if the source buffer is signed). If a floating-point destination buffer is specified, pixels that meet the condition are set to 1.

There are 6 ways to establish the threshold value(s): two that require you to explicitly specify the threshold value(s) (M_FIXED and M_PERCENTILE_VALUE) and four that automatically base the threshold value(s) on the histogram (M_BIMODAL, M_DOMINANT, M_TRIANGLE_BISECTION_DARK, and M_TRIANGLE_BISECTION_BRIGHT).

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

DstImageBufId

Specifies the identifier of the destination image buffer in which to write the results. In general, this parameter must be given an image buffer identifier.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

To return only the automatically calculated threshold value, if applicable, set the DstImageBufId parameter to **M_NULL**. Note that if the DstImageBufId parameter is set to **M_NULL**, the **MimBinarize**() function will be forced to run synchronously.

ConditionAndThreshMode

Specifies the condition, as well as the threshold mode to use to establish the threshold value(s) of the condition.

See the Parameter associations section for possible values that can be specified.

LowParam

Specifies either the lower limit of the selected condition or the minimum value to use when establishing the threshold.

See the Parameter associations section for possible values that can be specified.

HighParam

Specifies either the upper limit of the selected condition or the maximum value to use when establishing the threshold.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ConditionAndThreshMode**, **LowParam**, and **HighParam** parameters.

- For controlling the threshold determination mode

The following conditions allow you to control the threshold determination mode used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the threshold determination mode																		
ConditionAndThreshMode		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	LowParam																	
	HighParam																	
M_BIMODAL +		Specifies to automatically establish the threshold value for the condition from the source image's histogram, using the bimodal threshold mode. This threshold mode can only be used if the condition uses one limit. In this mode, if the source image's histogram contains only two peaks, the threshold value is set to the intensity value at the lowest point between these two peaks of the histogram, on the assumption that these peaks represent the object and the background. If the histogram contains more than two peaks, the threshold value will typically be set between the intensity values at the two principal peaks, although exceptions exist for the situations where the principal peaks occur are closest to the minimum intensity value (pure black) or the maximum intensity value (full saturation). Note that the bimodal threshold mode works best when the histogram contains two visible peaks. You can limit the intensity value range of the histogram from which the threshold value is established. To do so, use the LowParam and HighParam parameters. You must specify a combination value from the following table: • To specify a threshold condition															
	LowParam	Sets the minimum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when HighParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum pixel value. The minimum value must be less than or equal to the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
	HighParam	Sets the maximum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when LowParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum pixel value. The maximum value must be greater than or equal to the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b

M_DOMINANT +	Specifies to automatically establish the threshold value for the condition from the source image's histogram, using the dominant threshold mode. This threshold mode can only be used if the condition uses one limit. In this mode, the threshold value is set to isolate the dominant peak of the source image's histogram. If the histogram contains two peaks, the threshold is set to the intensity value at the lowest point between these two peaks. If the histogram contains more than two peaks, the threshold value is set at the lowest point between the dominant peak and its nearest peak on the right or left, depending on the pixel distribution in the histogram. If there are more pixels represented on the left side of the dominant peak, then the threshold will be set to the left of the peak. If there are more pixels represented on the right side of the dominant peak, then the threshold will be set to the right of the peak. Note that the dominant threshold mode works best when the histogram contains clearly identifiable peaks. You can limit the intensity value range of the histogram from which the threshold value is established. To do so, use the LowParam and HighParam parameters. You must specify a combination value from the following table: • To specify a threshold condition	c		e	f	g	a		i		j	k	l	m	n	b
LowParam	Sets the minimum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when HighParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum pixel value. The minimum value must be less than or equal to the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
HighParam	Sets the maximum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when LowParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum pixel value. The maximum value must be greater than or equal to the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIXED +	Specifies that the threshold value(s) for the condition are specified explicitly. You must specify a combination value from the following table: • To specify a threshold condition	c		e	f	g	a		i		j	k	l	m	n	b
LowParam	Sets the lower limit of the selected condition.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use any value that meets the selected condition. You must use this setting for M_NON_FINITE.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Sets the threshold value to use as the lower limit of the selected condition. The lower limit must be less than or equal to the upper limit.	c		e	f	g	a		i		j	k	l	m	n	b

HighParam	Sets the upper limit of the selected condition.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies not to use an upper limit. Use this setting when the condition uses only one limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the threshold value to use as the upper limit of the selected condition. The upper limit must be greater than or equal to the lower limit.	c		e	f	g	a		i		j	k	l	m	n	b
M_PERCENTILE_VALUE +	Specifies that the threshold value(s) for the condition are specified as a percentage of the source image's histogram data. In this mode, you specify a threshold value by specifying the percentage of pixels that must be equal to or less than the required threshold value. You must specify a combination value from the following table: • To specify a threshold condition	c		e	f	g	a		i		j	k	l	m	n	b
LowParam	Sets the lower limit of the selected condition.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the lower limit as a percentage of the source image's histogram data. The lower limit must be less than or equal to the upper limit.	c		e	f	g	a		i		j	k	l	m	n	b
HighParam	Sets the upper limit of the selected condition.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies not to use an upper limit. Use this setting when the condition uses only one limit.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 100	Specifies the upper limit as a percentage of the source image's histogram data. The upper limit must be greater than or equal to the lower limit.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRIANGLE_BISECTION_BRIGHT +	Specifies to automatically establish the threshold value for the condition from the source image's histogram, using the bright triangle bisection threshold mode. This threshold mode can only be used if the condition uses one limit. In bright triangular bisection threshold mode, Aurora Imaging Library establishes the tallest peak in the source image's histogram (the most commonly-occurring value) and the lowest point (the least commonly-occurring value) when heading from the peak toward its maximum intensity value. It then connects these two points by a line. The threshold value is the intensity value at the point furthest below that line (the inflection point). Note that all points above the line are ignored. You can limit the intensity value range of the histogram from which the threshold value is established. To do so, use the LowParam and HighParam parameters. You must specify a combination value from the following table: • To specify a threshold condition	c		e	f	g	a		i		j	k	l	m	n	b
LowParam	Sets the minimum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b

		M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when HighParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the minimum pixel value. The minimum value must be less than or equal to the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
	HighParam		Sets the maximum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when LowParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the maximum pixel value. The maximum value must be greater than or equal to the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRIANGLE_BISECTION_DARK +			Specifies to automatically establish the threshold value for the condition from the source image's histogram, using the dark triangle bisection threshold mode. This threshold mode can only be used if the condition uses one limit. In dark triangular bisection threshold mode, Aurora Imaging Library establishes the tallest peak in the source image's histogram (the most commonly-occurring value) and the lowest point (the least commonly-occurring value) when heading from the peak toward its minimum intensity value. It then connects these two points by a line. The threshold value is the intensify value at the point furthest below that line (the inflection point). Note that all points above the line are ignored. You can limit the intensity value range of the histogram from which the threshold value is established. To do so, use the LowParam and HighParam parameters. You must specify a combination value from the following table: • To specify a threshold condition	c		e	f	g	a		i		j	k	l	m	n	b
	LowParam		Sets the minimum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when HighParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the minimum pixel value. The minimum value must be less than or equal to the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
	HighParam		Sets the maximum pixel value in the source image's histogram to use when establishing the threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
		M_NULL	Specifies to use the full range of pixel values in the histogram. This setting is only valid when LowParam is also set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the maximum pixel value. The maximum value must be greater than or equal to the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For controlling the threshold determination mode.

You must add one of the following values to the above-mentioned values to specify a threshold condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a threshold condition																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL	Sets pixel values equal to the threshold value to the highest buffer value possible, while other pixels are set to zero. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Sets pixel values greater than the threshold value to the highest buffer value possible, while other pixels are set to zero. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Sets pixel values greater than or equal to the threshold value to the highest buffer value possible, while other pixels are set to zero. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_IN_RANGE	Sets pixels with values between the lower threshold value and the higher threshold value, inclusive, to the highest buffer value. Other pixels are set to 0. > Note that this is only applicable to constants that can use two limits (M_FIXED and M_PERCENTILE_VALUE). Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Sets pixel values less than the threshold value to the highest buffer value possible, while other pixels are set to zero. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Sets pixel values less than or equal to the threshold value to the highest buffer value possible, while other pixels are set to zero. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_NON_FINITE	Sets floating-point pixel values that are non-finite to the highest buffer value possible, while other pixels are set to zero. This condition can only be used with M_FIXED. HighParam must be set to M_NULL. The source must be a 32-bit floating-point buffer. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_EQUAL	Sets pixel values not equal to the threshold value to the highest buffer value possible, while other pixels are set to zero. Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_RANGE	Sets pixels with values less than the lower threshold value or greater than the higher threshold value, to the highest buffer value. Other pixels are set to 0. > Note that this is only applicable to constants that can use two limits (M_FIXED and M_PERCENTILE_VALUE). Note that if using a floating-point destination buffer, pixels that meet the condition are set to 1.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

When DstImageBufId is given an image buffer identifier, this function returns the specified value of the LowParam parameter. When DstImageBufId is set to **M_NULL** and the ConditionAndThreshMode parameter is set to one of the automatic threshold modes (M_BIMODAL, M_TRIANGLE_BISECTION_DARK, or M_TRIANGLE_BISECTION_BRIGHT), this function returns the automatically calculated threshold value. If the ConditionAndThreshMode parameter is set to M_PERCENTILE_VALUE, this function returns the threshold value established from the specified lower limit. When dealing with floating-point source buffers, the returned threshold is truncated to an integer value. Therefore, the returned threshold and the actual threshold used to binarize can differ.

Remark

- Optimized in-place processing is supported (source equals destination), but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimBinarizeAdaptive

Synopsis

Perform a point-to-point adaptive binarization on an image.

Syntax

```
void MimBinarizeAdaptive(
    AIL_ID AdaptiveBinarizeContextImId, //in
    AIL_ID SrcImageBufId,               //in
    AIL_ID SeedImage1BufId,             //in
    AIL_ID SeedImage2BufId,             //in
    AIL_ID BinarizedImageBufId,         //out
    AIL_ID ThresholdImageBufId,         //out
    AIL_INT64 ControlFlag               //in
)
```

Description

This function performs an adaptive binary threshold operation on the specified image and places the resulting binarized image in the specified buffer. This function also provides the threshold values with which it binarized the image.

To perform the adaptive binarization, this function establishes a threshold value for each pixel in the source, according to the context's threshold mode. Each source pixel that is greater than its corresponding threshold value is set to the highest unsigned destination buffer value. For example, for 1-, 8-, and 16-bit destination buffers (signed or unsigned), the highest values are 1, 0xFF, and 0xFFFF, respectively. For 32-bit floating-point destination buffers, the highest value is 1. Source pixels that are less than or equal to their corresponding threshold value are set to 0, for destination buffers of any type.

To specify a particular mode with which to calculate threshold values, call MimControl() with M_THRESHOLD_MODE. Available threshold modes generally refer to adaptive algorithms, which perform multiple calculations on individual sections of an image. When compared to a conventional binarization (MimBinarize()), which calculates threshold values using an image as a whole, an adaptive binarization is more robust at processing images with objects that are difficult to identify. Such difficulties typically come from changes in illumination and poor or varying contrast. This type of robustness can cause an adaptive binarization to take longer to execute than a conventional one.

You can modify controls other than threshold mode to customize calculations, such as indicating a maximum (M_GLOBAL_MAX) or minimum (M_GLOBAL_MIN) threshold value. Available controls depend on the type of the adaptive binarization context.

1-bit, 8-bit, and 16-bit buffers are supported; their type can be signed or unsigned. 32-bit buffers are also supported; their type must be float. For best results, all image buffers should have the same number of bands and be the same type.

Parameters

AdaptiveBinarizeContextImId

Specifies the identifier of the adaptive binarization context. The context must have been allocated using MimAlloc() with either M_BINARIZE_ADAPTIVE_CONTEXT or M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT.

SrcImageBufId

Specifies the identifier of the source image buffer with which to perform the adaptive binarization. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

SeedImage1BufId

Specifies the identifier of the image buffer to use as the first seed image. If the type of the specified context is M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT, this function uses seeds as positional information in the source image to establish threshold values, given the threshold mode control (M_THRESHOLD_MODE). For contexts of type M_BINARIZE_ADAPTIVE_CONTEXT, seed information is not required; set this parameter to M_NULL.

For contexts of type M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT, set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first seed image buffer

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Establishes seeds automatically.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to ignore this parameter. Use M_NULL if the context is of type M_BINARIZE_ADAPTIVE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the first seed image. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

SeedImage2BufId

Specifies the identifier of the image buffer to use as the second seed image. If the type of the specified context is M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT, this function uses seeds as positional information in the source image to establish threshold values, given the threshold mode control (M_THRESHOLD_MODE). For contexts of type M_BINARIZE_ADAPTIVE_CONTEXT, seed information is not required; set this parameter to M_NULL.

For contexts of type M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT, set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the second seed image buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Establishes seeds automatically. This is only valid if the threshold mode control is set to M_TOGGLE, and the SeedImage1BufId parameter is set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to ignore this parameter. Use M_NULL if the threshold mode control is set to M_RECONSTRUCT or M_LEVEL (or the context is of type M_BINARIZE_ADAPTIVE_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the second seed image. This is only valid if the SeedImage1BufId parameter specifies an image buffer identifier, and the threshold mode control is set to M_TOGGLE. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

BinarizedImageBufId

Specifies the identifier of the image buffer in which to put the result of the adaptive binarization operation. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

To not generate the binarized image result, set this parameter to **M_NULL**. This is only valid if the ThresholdImageBufId parameter is not set to **M_NULL**.

ThresholdImageBufId

Specifies the identifier of the image buffer in which to put the resulting threshold values with which to binarize the source image. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

To not generate the threshold image result, set this parameter to **M_NULL**. This is only valid if the BinarizedImageBufId parameter is not also set to **M_NULL**.

This parameter must be set to **M_NULL** if the threshold type (MimControl() with M_THRESHOLD_TYPE) is set to M_IN_RANGE or M_OUT_RANGE.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimBoundingBox

Synopsis

Compute the bounding box that contains valid pixels according to a given condition.

Syntax

```
void MimBoundingBox(
    AIL_ID SrcImageBufId,             //in
    AIL_INT64 Condition,              //in
    AIL_DOUBLE CondLow,               //in
    AIL_DOUBLE CondHigh,              //in
    AIL_INT64 BoxDefinitionType,      //in
    AIL_INT *TopLeftXPtr,             //out
    AIL_INT *TopLeftYPtr,             //out
    AIL_INT *BottomRightXOrLengthPtr, //out
    AIL_INT *BottomRightYOrLengthPtr, //out
    AIL_INT64 ControlFlag             //in
)
```

Description

This function computes the bounding box that contains valid pixels according to a given condition, based on a lower limit and an upper limit. For example, you can define a condition such that pixels are valid only within a specified value range.

You can specify to return the bounding box in one of two ways: using coordinates of 2 opposite corners, or using coordinates of 1 corner and the width and height (in pixel units).

You can limit this function's results to a region of the source image buffer using a region of interest (ROI) set using MbufSetRegion().

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be set to the Aurora Imaging Library identifier of a 1-band image buffer.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Condition

Specifies the condition under which pixels are considered valid.

For specifying the condition	
Value	Description
M_EQUAL	Specifies that only pixels with values equal to CondLow are considered valid. CondHigh must be set to 0.
M_GREATER	Specifies that only pixels with values greater than CondLow are considered valid. CondHigh must be set to 0.
M_GREATER_OR_EQUAL	Specifies that only pixels with values greater than or equal to CondLow are considered valid. CondHigh must be set to 0.
M_IN_RANGE	Specifies that pixel values between CondLow and CondHigh, inclusive, are considered valid.
M_LESS	Specifies that only pixels with values less than CondHigh are considered valid. CondLow must be set to 0.
M_LESS_OR_EQUAL	Specifies that only pixels with values less than or equal to CondHigh are considered valid. CondLow must be set to 0.

M_NOT_EQUAL	Specifies that only pixels with values not equal to CondLow are considered valid. CondHigh must be set to 0.
M_OUT_RANGE	Specifies that pixels with values less than CondLow, or greater than CondHigh, are considered valid.

CondLow

Specifies the lower limit of the selected condition.

CondHigh

Specifies the upper limit of the selected condition.

BoxDefinitionType

Specifies the way in which the bounding box is returned.

For specifying the way in which the bounding box is returned	
Value	Description
M_BOTH_CORNERS	Specifies to return the bounding box using the coordinates of its top-left and bottom-right corners.
M_CORNER_AND_DIMENSION	Specifies to return the bounding box using the coordinates of its top-left corner, and the box's width and height.

TopLeftXPtr

Specifies the address of the variable in which to write the X-coordinate of the top-left corner of the bounding box.

TopLeftYPtr

Specifies the address of the variable in which to write the Y-coordinate of the top-left corner of the bounding box.

BottomRightXOrLengthPtr

Specifies the address of the variable in which to write the X-coordinate of the bottom-right corner or the width of the bounding box.

BottomRightYOrLengthPtr

Specifies the address of the variable in which to write the Y-coordinate of the bottom-right corner or the height of the bounding box.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimClip

Synopsis

Perform a point-to-point clipping operation.

Syntax

```
void MimClip(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT64 Condition,  //in
    AIL_DOUBLE CondLow,   //in
    AIL_DOUBLE CondHigh,  //in
    AIL_DOUBLE WriteLow,  //in
    AIL_DOUBLE WriteHigh  //in
)
```

Description

This function clips each image pixel that meets the specified condition. If the condition has one clipping point, each pixel that satisfies this condition is replaced with the specified WriteLow value. If it has two clipping points, the pixels are either replaced with the WriteLow or WriteHigh value depending on the condition. Pixels that do not satisfy the condition are not replaced. Instead, they are copied as is to the destination buffer.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcImageBufId

Specifies the identifier of the image data source. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DstImageBufId

Specifies the identifier of the destination image buffer. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Condition

Specifies the clipping condition. This parameter must be set to one of the values below.

The following are conditions that use two clipping points:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For clipping																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IN_RANGE	Replaces pixel values in the range of CondLow and CondHigh, inclusive, with WriteLow. CondLow = 100 CondHigh = 150 WriteLow = 0	c		e	f	g	a		i		j	k	l	m	n	b

M_OUT_RANGE	Replaces pixel values less than CondLow with WriteLow and those greater than CondHigh with WriteHigh. CondLow = 100 CondHigh = 150 WriteLow = 0 WriteHigh = 0	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Saturates source values according to the minimum and maximum values of the destination image buffer's data type.	c		e	f	g	a		i		j	k	l	m	n	b

The following are conditions that use one clipping point:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying conditions that use one clipping point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL	Replaces pixel values equal to CondLow with WriteLow. CondLow = 25 WriteLow = 255	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Replaces pixel values greater than CondLow with WriteLow. CondLow = 100 WriteLow = 255	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Replaces pixel values greater than or equal to CondLow with WriteLow.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Replaces pixel values less than CondLow with WriteLow. CondLow = 100 WriteLow = 255	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Replaces pixel values less than or equal to CondLow with WriteLow.	c		e	f	g	a		i		j	k	l	m	n	b
M_NON_FINITE	Replace floating-point pixel values that are non-finite with WriteLow. CondLow must be set to M_DEFAULT. The source must be a 32-bit floating-point buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_NOT_EQUAL	Replaces pixel values not equal to CondLow with WriteLow. CondLow = 25 WriteLow = 255	c		e	f	g	a		i		j	k	l	m	n	b

CondLow

Specifies the lower clipping point of the selected condition. If the condition uses only one limit, use this parameter to specify the required limit.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the lower clipping point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to use all non-finite values.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the lower limit is based on the minimum value of the destination image buffer's data type. You must set CondLow to this value when the condition is set to M_SATURATION.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower limit. This value is cast to the source buffer's data type and interpreted accordingly. Note, if the source buffer is binary, CondLow must be equal to 0 or 1.	c		e	f	g	a		i		j	k	l	m	n	b

CondHigh

Specifies the upper clipping point of the selected condition. If the condition uses only one limit, this parameter is ignored and should be set to M_NULL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the upper clipping point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that the upper limit is based on the maximum value of the destination image buffer's data type, or that the condition only uses one limit. You must set CondHigh to this value when the condition is set to M_SATURATION.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit. This value is cast to the source buffer's data type and interpreted accordingly. Note, if the source buffer is binary, CondHigh must be equal to 0 or 1.	c		e	f	g	a		i		j	k	l	m	n	b

WriteLow

Specifies the value to write to the destination buffer when a pixel satisfies the specified low clipping condition. If the condition uses only one replacement value, use this parameter to specify the required value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the value to write to the destination buffer when a pixel satisfies the specified low clipping point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that the value to write is the minimum value of the destination image buffer's data type. You must set WriteLow to this value when the condition is set to M_SATURATION.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the value to write when the pixel satisfies the low clipping condition. This value is cast to the destination buffer's data type. Note, if the destination buffer is binary, WriteLow must be equal to 0 or 1.	c		e	f	g	a		i		j	k	l	m	n	b

WriteHigh

Specifies the value to write to the destination buffer when a pixel satisfies the specified high clipping condition. If the condition uses only one replacement value, this parameter is not used and must be set to M_NULL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the value to write to the destination buffer when a pixel satisfies the specified high clipping point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that the value to write is the maximum value of the destination image buffer's data type, or that the condition only uses the low replacement value. You must set WriteHigh to this value when the condition is set to M_SATURATION.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the value to write when the pixel satisfies the high clipping condition. This value is cast to the destination buffer's data type. Note, if the destination buffer is binary, WriteHigh must be equal to 0 or 1.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- Optimized in-place processing is supported (source equals destination), but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimClose

Synopsis

Perform a binary or grayscale closing-type morphological operation.

Syntax

```
void MimClose(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbIteration,  //in
    AIL_INT64 ProcMode    //in
)
```

Description

This function performs a binary or grayscale closing operation on the given source image for the specified number of iterations. A closing is a dilation followed by an erosion.

In binary mode, this function uses a 3x3 full rectangular structuring element; in grayscale mode, a 3x3 empty one.

The overscan pixels are automatically set to the lowest possible buffer value for the dilation and the highest possible buffer value for the erosion; this will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

NbIteration

Specifies the number of times to iterate the operation.

When this parameter is set to 0 and ProcMode is set to M_BINARY, the source image is binarized and the result is copied into the destination image buffer.

When this parameter is set to 0 and ProcMode is set to M_GRAYSCALE, the source image is copied into the destination image buffer.

ProcMode

Specifies the processing mode to use. This parameter can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Treats non-zero pixels as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Uses the source image's gray values for processing. The resulting buffer will also contain gray values.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimConnectMap

Synopsis

Perform a 3x3 binary connectivity mapping.

Syntax

```
void MimConnectMap(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_ID LutBufId       //in
)
```

Description

This function performs a 3x3 binary connectivity mapping. It calculates a connectivity code for each pixel in the source image, treating the source image as if it were binary (that is, all non-zero pixels are treated as 1); it then maps the codes through the specified LUT buffer. Specifically, for each source pixel, this function concatenates the binary values of the pixels in a pixel's 3x3 neighborhood, and then uses this 9-bit number to address the specified LUT. The value at this LUT address is then written in the specified destination buffer at the pixel's corresponding position.

Pixel connectivity codes are determined in the following order:

$$\begin{bmatrix} n_3 & n_2 & n_1 \\ n_4 & n_8 & n_0 \\ n_5 & n_6 & n_7 \end{bmatrix} \text{ where } n_i \text{ is either } 0 \text{ or } 1$$

$$\text{Connectivity code} = \sum_{i=0}^{8} 2^i n_i$$

Result = LUTMAP[*Connectivity code*]

The overscan pixels are automatically set to 0 (zero), which will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

LutBufId

Specifies the identifier of the LUT buffer. Since each connectivity code has 9 bits, you should supply a LUT buffer with at least 512 (2^9) entries; otherwise, unpredictable results can occur.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimControl

Synopsis

Control an image processing context or result buffer setting.

Syntax

```
void MimControl(
    AIL_ID ContextOrResultImId, //out
    AIL_INT64 ControlType,      //in
    AIL_DOUBLE ControlValue     //in
)
```

Description

This function allows you to control an image processing context or result buffer setting. All the control type settings can be inquired using MimInquire().

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MimControl()** internally calls the *AIL_INT64* version of this function (MimControlInt64()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MimControlDouble()).

Parameters

ContextOrResultImId

Specifies the identifier of the image processing context or result buffer. The image processing context or result buffer must have been previously allocated on the system using MimAlloc() or MimAllocResult(), respectively.

See the Parameter associations section for possible values that can be returned.

ControlType

Specifies the processing feature to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value needed for the control. When the ControlValue is an image buffer, the internal representation of the image stored in the image processing context might not be the same as the original image.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **ControlValue** parameters and possible values returned to the **ContextOrResultImId** parameter.

- For specifying the control type and control value for an image processing context or result buffer

The following ContextOrResultImId, ControlType, and ControlValue parameter settings can be specified for different types of image processing contexts or result buffers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the control type and control value for an image processing context or result buffer																	
ContextOrResultImId - Possible values returned			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlType																	
		ControlValue																

Adaptive binarize context ID	Specifies an adaptive binarize context, allocated using MimAlloc() with M_BINARIZE_ADAPTIVE_CONTEXT, and used in MimBinarizeAdaptive() operations. The main setting with which to control an adaptive binarize context is M_THRESHOLD_MODE. In general, all other control settings are used by the specified thresholding process to establish the threshold values with which to binarize.														
M_AVERAGE_MODE	Sets how Aurora Imaging Library establishes average pixel values that can be required to determine threshold values. This is typically used when M_THRESHOLD_MODE is set to M_NIBLACK or M_LOCAL_MEAN. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAUSSIAN	Specifies a Gaussian type average.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies a uniform type average.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE	Sets whether the objects to binarize are lighter or darker than the background. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FOREGROUND_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the objects to binarize are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the objects to binarize are lighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_MAX	Sets the maximum threshold value. The threshold destination image (MimBinarizeAdaptive()) cannot hold values higher than M_GLOBAL_MAX. Higher threshold values are clipped. If the source image (MimBinarizeAdaptive()) also has a maximum value restriction (MbufControl() with M_MAX), Aurora Imaging Library uses the lower maximum value as the actual maximum. Before calling MimBinarizeAdaptive(), if both M_GLOBAL_MAX and M_GLOBAL_MIN are set to values, then M_GLOBAL_MAX must be greater than or equal to M_GLOBAL_MIN. By default, Aurora Imaging Library binarizes pixels with an intensity higher than the maximum as part of the foreground (object). To change this behavior, use the M_FOREGROUND_VALUE control. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that there is no maximum threshold value restriction imposed by M_GLOBAL_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum threshold value.	c		e	f	g	a		i		j	k	l	m	n	b

M_GLOBAL_MIN	Sets the minimum threshold value. The threshold destination image (MimBinarizeAdaptive()) cannot hold values lower than M_GLOBAL_MIN. Lower threshold values are clipped. If the source image (MimBinarizeAdaptive()) also has a minimum value restriction (MbufControl() with M_MIN), Aurora Imaging Library uses the greater minimum value as the actual maximum. Before calling MimBinarizeAdaptive(), if both M_GLOBAL_MIN and M_GLOBAL_MAX are set to values, then M_GLOBAL_MIN must be less than or equal to M_GLOBAL_MAX. By default, Aurora Imaging Library binarizes pixels with an intensity lower than the minimum as part of the background. To change this behavior, use the M_FOREGROUND_VALUE control. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that there is no minimum threshold value restriction imposed by M_GLOBAL_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the minimum threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_OFFSET	Sets the offset to add to each threshold value. MimBinarizeAdaptive() uses the adjusted threshold values. The specified offset is reflected in the threshold destination image (MimBinarizeAdaptive()). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_OFFSET_SECOND_PASS	Sets the offset to apply to the threshold values for the second pass of a hysteresis adaptive binarization. Aurora Imaging Library applies the offset for the second pass in the same way that it applies the offset for the first pass (M_GLOBAL_OFFSET). For M_GLOBAL_OFFSET_SECOND_PASS to have an effect, you must specify a threshold type other than M_SINGLE. For an M_NIBLACK threshold mode, either M_GLOBAL_OFFSET_SECOND_PASS must have a value different from M_GLOBAL_OFFSET, or M_NIBLACK_BIAS_SECOND_PASS must have a value different from M_NIBLACK_BIAS; otherwise, Aurora Imaging Library generates an error. For other threshold modes, Aurora Imaging Library generates an error if M_GLOBAL_OFFSET_SECOND_PASS and M_GLOBAL_OFFSET have the same value. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b

M_LOCAL_DIMENSION	Specifies the size of the neighborhood that the threshold mode uses to establish threshold values. For an M_NIBLACK or M_LOCAL_MEAN threshold mode, the size should be the largest square that represents a uniform background. The size should also be greater than the object's expected thickness. For an M_BERNSEN threshold mode, the size should be slightly larger than half the object's expected thickness. For an M_PSEUDOMEDIAN threshold mode, the size should be half the object's expected thickness. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 15 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the size of the neighborhood, in pixels. Only *integer* values accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST	Sets the minimum contrast between background and foreground (object) pixels. Aurora Imaging Library binarizes (classifies) pixels in a neighborhood as background if they do not meet the minimum contrast. An M_LOCAL_MEAN threshold mode ignores the minimum contrast. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 5.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b
M_NIBLACK_BIAS	Sets the bias for Niblack's binarization mode. This value only has an effect if M_THRESHOLD_MODE is set to M_NIBLACK. The bias gives you some general control over thresholding. A higher bias binarizes fainter values as part of the object. A lower bias binarizes fainter values as part of the background. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.2.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the bias. Typical values range from 0.1 to 0.3.	c		e	f	g	a		i		j	k	l	m	n	b
M_NIBLACK_BIAS_SECOND_PASS	Sets the bias for the second pass of a Niblack adaptive binarization that uses a hysteresis process. For this control type to have an effect, you must specify an M_NIBLACK threshold mode and you must specify a threshold type other than M_SINGLE. Either M_NIBLACK_BIAS_SECOND_PASS must have a value different from M_NIBLACK_BIAS, or M_GLOBAL_OFFSET_SECOND_PASS must have a value different from M_GLOBAL_OFFSET; otherwise, Aurora Imaging Library generates an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.2.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the bias.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE		Sets how Aurora Imaging Library establishes the threshold values with which to binarize the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NIBLACK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BERNSEN	Specifies that threshold values are established using Bernsen's adaptive threshold algorithm. This represents a type of morphological erosion and dilation. This threshold results in the fastest process.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOCAL_MEAN	Specifies that threshold values are established using adaptive local mean calculations. This is a simplified version of M_NIBLACK. M_LOCAL_MEAN usually results in a faster, though less precise, binarization than M_NIBLACK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NIBLACK	Specifies that threshold values are established using Niblack's adaptive threshold algorithm. This setting offers the highest precision. The processing time is usually quite quick.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PSEUDOMEDIAN	Specifies that threshold values are established using adaptive pseudomedian calculations. This is similar to an M_BERNSEN threshold, except it represents a type of morphological open or close process instead of erosion or dilation.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_TYPE		Sets the type of threshold to use for the adaptive binarization. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_SINGLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HYSTERESIS	Specifies that the adaptive binarization uses a hysteresis process. In this case, Aurora Imaging Library performs a geodesic reconstruction (a type of morphological operation) after a second pass of the specified threshold. Regardless of the threshold mode, Aurora Imaging Library uses M_GLOBAL_OFFSET_SECOND_PASS instead of M_GLOBAL_OFFSET during the second pass of MimBinarizeAdaptive(). For an M_NIBLACK threshold mode, the second pass uses M_NIBLACK_BIAS_SECOND_PASS instead of M_NIBLACK_BIAS. Aurora Imaging Library generates an error if every M_..._SECOND_PASS value it uses is the same as its first pass counterpart.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IN_RANGE	Specifies to use the values inside the range defined by the two passes as the foreground.	c		e	f	g	a		i		j	k	l	m	n	b

		M_OUT_RANGE	Specifies to use the values outside the range defined by the two passes as the foreground.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SINGLE	Specifies to use a single pass of the specified threshold. Hysteresis is not used.	c		e	f	g	a		i		j	k	l	m	n	b
Adaptive binarize from seed context ID			Specifies an adaptive binarize context that uses seeds, allocated using MimAlloc() with M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT, and used in MimBinarizeAdaptive() operations. The main setting with which to control an adaptive binarize context that uses seeds is M_THRESHOLD_MODE. In general, all other control settings are used by the specified thresholding process to establish the threshold values with which to binarize.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_VALUE		Sets whether the objects to binarize are lighter or darker than the background. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_FOREGROUND_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FOREGROUND_BLACK	Specifies that the objects to binarize are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FOREGROUND_WHITE	Specifies that the objects to binarize are lighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL_OFFSET		Sets the offset to add to each established threshold value. Binarization uses the adjusted threshold values, however offsets do not change the threshold values themselves. The content of the threshold destination image (MimBinarizeAdaptive() with ThresholdImageBufId) remains unaltered. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
		Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NB_ITERATIONS		Sets the number of times to perform the adaptive threshold process specified with M_THRESHOLD_MODE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_TO_IDEMPOTENCE.	c		e	f	g	a		i		j	k	l	m	n	b
		M_TO_IDEMPOTENCE	Specifies that the threshold process iterates until idempotence is reached. This is the number of iterations at which subsequent iterations do not alter results.	c		e	f	g	a		i		j	k	l	m	n	b
		Value > 0	Specifies the number of iterations. Only *integer* values accepted. The threshold process for an M_TOGGLE threshold mode is always performed once.	c		e	f	g	a		i		j	k	l	m	n	b

M_NB_SEED_ITERATIONS	Sets the number of iterations with which to internally establish the seeds that the threshold mode requires. This value only has an effect if you do not specify your own seed images with MimBinarizeAdaptive(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of iterations. Only *integer* values accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE	Sets how Aurora Imaging Library uses seeds to establish the threshold values with which to binarize the source image. You can provide the required seed images when you call MimBinarizeAdaptive(). If you do not, Aurora Imaging Library internally establishes the seed data. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RECONSTRUCT.	c		e	f	g	a		i		j	k	l	m	n	b
M_LEVEL	Specifies that threshold values are established using an adaptive leveling. This essentially performs two geodesic reconstructions. One that processes the foreground as white, and the other that processes the foreground as black. This results in M_LEVEL generally taking twice as long as M_RECONSTRUCT. M_LEVEL uses one seed image.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECONSTRUCT	Specifies that threshold values are established using an adaptive geodesic reconstruction. This represents a type of morphological erosion or dilation. M_RECONSTRUCT uses one seed image. M_RECONSTRUCT is typically faster than M_LEVEL and slower than M_TOGGLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOGGLE	Specifies that threshold values are established as one of two possibilities, defined by the seeds. Aurora Imaging Library compares the source pixel (including the offset) to each seed. The value of the closest seed is the threshold value for that pixel. M_TOGGLE uses two seed images (typically min and max values). M_TOGGLE is the fastest threshold mode.	c		e	f	g	a		i		j	k	l	m	n	b
Augmentation context ID	Specifies an augmentation context, allocated using MimAlloc() with M_AUGMENTATION_CONTEXT, and used in MimAugment() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP +	Sets whether to enable the aspect ratio operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_ASPECT_RATIO_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_ASPECT_RATIO_OP.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_ASPECT_RATIO_OP_MAX	Sets the maximum aspect ratio, to apply for M_AUG_ASPECT_RATIO_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 16.0	Specifies the maximum aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP_MIN	Sets the minimum aspect ratio, to apply for M_AUG_ASPECT_RATIO_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 16.0	Specifies the minimum aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP_MODE	Sets the aspect ratio mode, to apply for M_AUG_ASPECT_RATIO_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_BOTH.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTH	Specifies that Aurora Imaging Library sets the aspect ratio mode. That is, either M_INVERT or M_NORMAL is randomly set.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVERT	Specifies to apply the aspect ratio as height/width. In this case, the width stays the same while the height changes.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMAL	Specifies to apply the aspect ratio as width/height. In this case, the height stays the same while the width changes.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP +	Sets whether to enable the operation that applies a motion blur kernel with random direction and kernel size. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_BLUR_MOTION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_BLUR_MOTION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_ANGLE_MAX	Sets the maximum value for the motion blur angle's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the maximum value for the motion blur angle's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_BLUR_MOTION_OP_ANGLE_MIN	Sets the minimum value for the motion blur angle's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the minimum value for the motion blur angle's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_SIZE_MAX	Sets the maximum value for the kernel size's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 15.	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 15	Specifies the maximum value for the kernel size's random distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_SIZE_MIN	Sets the minimum value for the kernel size's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 5.	c		e	f	g	a		i		j	k	l	m	n	b
5 <= *Value* <= 15	Specifies the min value for the kernel size's random distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP +	Sets whether to enable the crop operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP_FACTOR_X	Sets the crop width factor, to apply for M_AUG_CROP_OP. The actual width with which to crop depends on the specified factor and the source image (MimAugment()); that is: *CropWidth* = M_AUG_CROP_OP_FACTOR_X x *SourceImageWidth* . INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the width factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP_FACTOR_Y	Sets the crop height factor, to apply for M_AUG_CROP_OP. The actual height with which to crop depends on the specified factor and the source image (MimAugment()); that is: *CropHeight* = M_AUG_CROP_OP_FACTOR_Y x *SourceImageHeight* . INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the height factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP_RESIZE	Sets the crop resize option, to apply for M_AUG_CROP_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies to not resize.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies to resize the crop dimensions according to the destination buffer size (MimAugment()).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_ASYM_OP +	Sets whether to enable the asymmetrical dilation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_DILATION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_DILATION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_ASYM_OP_NB_ITERATIONS_MAX	Sets the maximum number of iterations for dilation, to apply for M_AUG_DILATION_ASYM_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_OP +	Sets whether to enable the dilation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_DILATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_DILATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_OP_NB_ITERATIONS_MAX	Sets the maximum number of iterations, to apply for M_AUG_DILATION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_EROSION_ASYM_OP +	Sets whether to enable the asymmetrical erosion operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_EROSION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_EROSION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_ASYM_OP_NB_ITERATIONS_MAX	Sets the maximum number of iterations for dilation, to apply for M_AUG_EROSION_ASYM_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_OP +	Sets whether to enable the erosion operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_EROSION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_EROSION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_OP_NB_ITERATIONS_MAX	Sets the maximum number of iterations, to apply for M_AUG_EROSION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_FLIP_OP +	Sets whether to enable the flip operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_FLIP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_FLIP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_FLIP_OP_DIRECTION	Sets the direction with which to flip the image, to apply for M_AUG_FLIP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_BOTH.	c		e	f	g	a		i		j	k	l	m	n	b

M_BOTH	Specifies to flip in either the horizontal or vertical direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP_HORIZONTAL	Specifies to flip exclusively in the horizontal direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP_VERTICAL	Specifies to flip exclusively in the vertical direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP +	Sets whether to enable the gamma correction operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP_DELTA	Sets the range for the random distribution of gamma values, to apply for M_AUG_GAMMA_OP. The range for the random distribution of gamma values can be expressed as: [gamma - delta, gamma + delta]. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 10.0	Specifies the range for the random distribution of gamma values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP_MODE	Sets the gamma correction mode, to apply for M_AUG_GAMMA_OP. If you call MimAugment() with a 1-band buffer, Aurora Imaging Library applies the gamma correction to band 0, regardless of the specified mode (for example, to retrieve the gamma correction value applied by MimAugment(), you must call MimGetResult() with M_AUG_GAMMA_VALUE_BAND_0). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ALL_BANDS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies to apply the same gamma correction to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_PER_BAND	Specifies to apply a different random gamma correction to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP_VALUE	Sets the gamma value, to apply for M_AUG_GAMMA_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 10.0	Specifies the gamma value.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HSV_VALUE_GAIN_OP +		Sets whether to enable the operation that multiplies the image's value band (in the HSV color space) by a random factor. Multiplying an image's value band can be seen as a type of gain operation. When specifying a color (3-band) image, MimAugment() assumes an RGB color space by default. Operations that require a different color space (for example, HSV) internally convert the source image color. If your source image is not RGB, the conversion produces erroneous results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_HSV_VALUE_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_HSV_VALUE_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HSV_VALUE_GAIN_OP_MAX		Sets the maximum value for the random value factor's distribution [min, max], to use for M_AUG_HSV_VALUE_GAIN_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum value for the random value factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HSV_VALUE_GAIN_OP_MIN		Sets the minimum value for the random value factor's distribution [min, max], to use for M_AUG_HSV_VALUE_GAIN_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the minimum value for the random value factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET_OP +		Sets whether to enable the operation that adds a random hue angle to hue values. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_HUE_OFFSET_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_HUE_OFFSET_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET_OP_MAX		Sets the maximum value for the random hue angles' distribution [min, max], to use for M_AUG_HUE_OFFSET_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Specifies the default value; the default value is 360.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the maximum value for the random hue angles' distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET_OP_MIN	Sets the minimum value for the random hue angles' distribution [min, max], to use for M_AUG_HUE_OFFSET_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the minimum value for the random hue angles' distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP +	Sets whether to enable the intensity addition operation. Adding to an image's intensity can be seen as a type of offset operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_INTENSITY_ADD_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_INTENSITY_ADD_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP_DELTA	Sets the range for the random distribution of intensity addition (offset) values, to apply for M_AUG_INTENSITY_ADD_OP. The random distribution of intensity addition (offset) values can be expressed as: [offset - delta, offset + delta]. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the data type and data depth of the source image buffer with which to perform MimAugment(). Source image buffer: M_AUTO_VALUE 8-bit unsigned: 127.0 16-bit unsigned: 32767.0 32-bit floating-point: 0.5	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of intensity addition (offset) values.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_INTENSITY_ADD_OP_MODE	Sets the intensity addition value (offset) mode, to apply for M_AUG_INTENSITY_ADD_OP. When specifying a color (3-band) image, MimAugment() assumes an RGB color space by default. Operations that require a different color space (for example, HSV) internally convert the source image color. If your source image is not RGB, the conversion produces erroneous results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies to apply the intensity addition value (offset) to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the number of bands of the source image buffer with which to perform MimAugment(). This is the same as M_LUMINANCE for color (3-band) image buffers, and the same as M_ALL_BANDS for all other image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies to apply the intensity addition value (offset) to the luminance band only. This value is only supported for color (3-band) image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP_VALUE	Sets the intensity addition value (offset), to apply for M_AUG_INTENSITY_ADD_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the intensity addition value (offset).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP +	Sets whether to enable the intensity multiplication operation. Multiplying an image's intensity can be seen as a type of gain operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_INTENSITY_MULTIPLY_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_INTENSITY_MULTIPLY_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP_DELTA	Sets the range for the random distribution of the intensity multiplication (gain) values, to apply for M_AUG_INTENSITY_MULTIPLY_OP. The random distribution of the intensity multiplication (gain) values can be expressed as: [gain - delta, gain + delta]. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the range for the random distribution of the intensity multiplication (gain) values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP_MODE		Sets the multiplication value (gain) mode, to apply for M_AUG_INTENSITY_MULTIPLY_OP. When specifying a color (3-band) image, MimAugment() assumes an RGB color space by default. Operations that require a different color space (for example, HSV) internally convert the source image color. If your source image is not RGB, the conversion produces erroneous results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL_BANDS	Specifies to apply the multiplication value (gain) to each band.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_VALUE	Specifies a value that is automatically established according to the number of bands of the source image buffer with which to perform MimAugment(). This is the same as M_LUMINANCE for color (3-band) image buffers, and the same as M_ALL_BANDS for all other image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LUMINANCE	Specifies to apply the multiplication value (gain) to the luminance band only. This value is only supported for color (3-band) image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP_VALUE		Sets the intensity multiplication value (gain), to apply for M_AUG_INTENSITY_MULTIPLY_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the intensity multiplication value (gain).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP +		Sets whether to enable the directional lighting operation. Modifying an image's directional lighting can be seen as a type of add-ramp operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_LIGHTING_DIRECTIONAL_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_LIGHTING_DIRECTIONAL_OP.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_LIGHTING_DIRECTIONAL_OP_ANGLE_MAX	Sets an illumination to add to the image at a random angle (ramp), to apply for M_AUG_LIGHTING_DIRECTIONAL_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the maximum angle (ramp) with which to add the illumination, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP_INTENSITY_MAX	Sets the maximum light intensity, to use for M_AUG_LIGHTING_DIRECTIONAL_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum light intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP_INTENSITY_MIN	Sets the minimum light intensity, to use for M_AUG_LIGHTING_DIRECTIONAL_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum light intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP_MODE	Sets the directional lighting operation mode, to apply for M_AUG_LIGHTING_DIRECTIONAL_OP. When specifying a color (3-band) image, MimAugment() assumes an RGB color space by default. Operations that require a different color space (for example, HSV) internally convert the source image color. If your source image is not RGB, the conversion produces erroneous results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies to apply the directional lighting to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the number of bands of the source image buffer with which to perform MimAugment(). This is the same as M_LUMINANCE for color (3-band) image buffers, and the same as M_ALL_BANDS for all other image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies to apply the directional lighting to the luminance band only. This value is only supported for color (3-band) image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP +	Sets whether to enable the operation that applies a Gaussian additive noise with the option of a random noise standard deviation. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP_STDDEV	Sets standard deviation for noise, to apply for M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the data type and data depth of the source image buffer with which to perform MimAugment(). Source image buffer: M_AUTO_VALUE 8-bit unsigned: 25.0 16-bit unsigned: 3553.5 32-bit floating-point: 0.5	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the standard deviation for noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP_STDDEV_DELTA	Sets the range for the random distribution of noise standard deviation values, to apply for M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP. The range for the random distribution of noise standard deviation values can be expressed as: [stddev - delta, stddev + delta]. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the data type and data depth of the source image buffer with which to perform MimAugment(). Source image buffer: M_AUTO_VALUE 8-bit unsigned: 25.0 16-bit unsigned: 3553.5 32-bit floating-point: 0.5	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of noise standard deviation values.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_NOISE_MULTIPLICATIVE_OP +	Sets whether to enable the operation that applies a multiplicative noise with option of a random noise standard deviation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_NOISE_MULTIPLICATIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_NOISE_MULTIPLICATIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_DISTRIBUTION	Sets the uniform distribution for noise generation, to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAUSSIAN	Specifies a Gaussian distribution for noise generation.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies a uniform distribution for noise generation.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_INTENSITY_MIN	Sets an optional minimum noise intensity value (white noise threshold), to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies an optional minimum noise intensity value (white noise threshold).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_STDDEV	Sets the standard deviation for noise, to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the standard deviation for noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_STDDEV_DELTA	Sets the range for the random distribution of noise standard deviation values, to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. The range for the random distribution of noise standard deviation values can be expressed as: [stddev - delta, stddev + delta]. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of noise standard deviation values.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_NOISE_SALT_PEPPER_OP +	Sets whether to enable salt and pepper noise with the option of random density. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_NOISE_SALT_PEPPER_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_NOISE_SALT_PEPPER_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_SALT_PEPPER_OP_DENSITY	Sets the density value for salt and pepper noise, to use for M_AUG_NOISE_SALT_PEPPER_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.01.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the density value for salt and pepper noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_SALT_PEPPER_OP_DENSITY_DELTA	Sets the range for the random distribution of noise density values, to use for M_AUG_NOISE_SALT_PEPPER_OP. The range for the random distribution of noise density values can be expressed as: [density - delta, density + delta]. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.005.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the range for the random distribution of noise density values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP +	Sets whether to enable the reduce operation. M_AUG_REDUCE_OP scales down an image by a random factor between M_AUG_REDUCE_OP_FACTOR_MIN and M_AUG_REDUCE_OP_FACTOR_MAX. The scaled down image is positioned with a random X and Y offset, and the operation will add a background of color M_AUG_REDUCE_OP_BACKGROUND_COLOR, maintaining the original image size. The offset can be retrieved by using MimGetResult() with M_AUG_REDUCE_OFFSET_X and M_AUG_REDUCE_OFFSET_Y. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP_BACKGROUND_COLOR	Sets the background color to use for M_AUG_REDUCE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Same as M_COLOR_BLACK.	c		e	f	g	a		i		j	k	l	m	n	b
		M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies the RGB value to use as the background color.	c		e	f	g	a		i		j	k	l	m	n	b
			Parameters															
			Red Specifies the red band, as a value between 0 and 255.															
			Green Specifies the green band, as a value between 0 and 255.															
			Blue Specifies the blue band, as a value between 0 and 255.															
		M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_DONT_CARE_CLASS	Specifies the color (255, 255, 255).	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies a grayscale value. The buffer can be a 1-band or multi-band buffer. The specified value is cast to the buffer type and depth. Note that a grayscale value can be any integer or floating-point number. If the given value exceeds the range of the possible values that can be stored in each band of the destination buffer, the least significant bits of the value are used.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP_FACTOR_MAX		Sets the maximum scale factor by which to reduce the image for M_AUG_REDUCE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.9.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 1.0	Specifies the maximum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP_FACTOR_MIN		Sets the minimum scale factor by which to reduce the image for M_AUG_REDUCE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 1.0	Specifies the minimum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_RNG_INIT_VALUE		Sets the explicit initialization value for the internal random number generator, when M_AUG_SEED_MODE is set to M_RNG_INIT_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < *AIL_INT32_MAX*	Specifies the explicit initialization value for the internal random number generator.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_ROTATION_OP +	Sets whether to enable the rotation operation. To rotate the image, a random rotation angle is selected from the following range: AngleMin <= (AngleRef + i * AngleStep) ± (AngleDelta / 2) < AngleMax, where *AngleMin* represents the minimum rotation angle (M_AUG_ROTATION_OP_ANGLE_MIN), *AngleRef* represents the reference angle (M_AUG_ROTATION_OP_ANGLE_REF), *i* is a randomly generated integer, *AngleStep* represents the step size (M_AUG_ROTATION_OP_ANGLE_STEP), *AngleDelta* represents the delta angle (M_AUG_ROTATION_OP_ANGLE_DELTA), and *AngleMax* represents the maximum rotation angle (M_AUG_ROTATION_OP_ANGLE_MAX). If the randomly selected rotation angle is outside the range of *AngleMin* and *AngleMax*, it will be discarded and another will be generated (with another randomly selected *i*). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_ROTATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_ROTATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_DELTA	Sets the delta angle, to apply for M_AUG_ROTATION_OP. The delta angle is the angular range, relative to (AngleRef + i * AngleStep). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the delta angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_MAX	Sets the maximum rotation angle, to apply for M_AUG_ROTATION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 360°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the maximum rotation angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_MIN	Sets the minimum rotation angle, to apply for M_AUG_ROTATION_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0°.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the minimum rotation angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_REF	Sets the reference angle. All other angles for M_AUG_ROTATION_OP will be calculated relative to this angle. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the reference angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_STEP		Sets the step angle, to apply for M_AUG_ROTATION_OP. The step angle is the distance, or gap, relative to the reference angle. The random integer i determines the number of rotation steps from the reference angle to reach the angle range from which to randomly select a rotation angle. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0°.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the reference angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN_OP +		Sets whether to enable the operation that multiplies the image's saturation band (in the HSV color space) by a random factor. Multiplying an image's saturation band can be seen as a type of gain operation. When specifying a color (3-band) image, MimAugment() assumes an RGB color space by default. Operations that require a different color space (for example, HSV) internally convert the source image color. If your source image is not RGB, the conversion produces erroneous results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SATURATION_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SATURATION_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN_OP_MAX		Sets the maximum value for the random saturation factor's distribution [min, max], to use for M_AUG_SATURATION_GAIN_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum value for the random saturation factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN_OP_MIN		Sets the minimum value for the random saturation factor's distribution [min, max], to use for M_AUG_SATURATION_GAIN_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0.0	Specifies the minimum value for the random saturation factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_OP +	Sets whether to enable the scale operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_SCALE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_SCALE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_OP_FACTOR_MAX	Sets the maximum scale factor, to apply for M_AUG_SCALE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 16.0	Specifies the maximum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_OP_FACTOR_MIN	Sets the minimum scale factor, to apply for M_AUG_SCALE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 16.0	Specifies the minimum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SEED_MODE	Sets the mode with which to initialize seeds. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_RNG_AUTO.	c		e	f	g	a		i		j	k	l	m	n	b
M_RNG_AUTO	Specifies that the initialization value of the internal random number generator is chosen at random.	c		e	f	g	a		i		j	k	l	m	n	b
M_RNG_INIT_VALUE	Specifies an explicit initialization value for the internal random number generator, using M_AUG_RNG_INIT_VALUE. Specifying the same initialization value lets you rerun the augmentation with the same randomized settings. This initialization value is saved with the augmentation context.	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED_SEED	Specifies a user-defined value for the seeds, using MimAugment() with the SeedValue parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHARPEN_DERICHE_OP +	Sets whether to enable the operation that applies a Deriche sharpening filter with a random filter smoothness value. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies to disable M_AUG_SHARPEN_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SHARPEN_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHARPEN_DERICHE_OP_FACTOR_MAX		Sets the maximum value for the sharpen filter smoothness value's random distribution [min, max], to apply for M_AUG_SHARPEN_DERICHE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the maximum value for the sharpen filter smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHARPEN_DERICHE_OP_FACTOR_MIN		Sets the minimum value for the sharpen filter smoothness value's random distribution [min, max], to apply for M_AUG_SHARPEN_DERICHE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the minimum value for the sharpen filter smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X_OP +		Sets whether to enable the X-direction shear operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SHEAR_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SHEAR_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X_OP_MAX		Sets the maximum value for the X-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_X_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the maximum value for the X-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X_OP_MIN		Sets the minimum value for the X-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_X_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 1.0	Specifies the minimum value for the X-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y_OP +		Sets whether to enable the Y-direction shear operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SHEAR_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SHEAR_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y_OP_MAX		Sets the maximum value for the Y-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_Y_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the maximum value for the Y-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y_OP_MIN		Sets the minimum value for the Y-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_Y_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.5.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the minimum value for the Y-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_DERICHE_OP +		Sets whether to enable the operation that performs a Deriche filter smoothing with random filter smoothness value. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SMOOTH_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SMOOTH_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_DERICHE_OP_FACTOR_MAX		Sets the maximum value for the Deriche smoothness value's random distribution [min, max], to use for M_AUG_SMOOTH_DERICHE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 100.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the maximum value for the Deriche smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_SMOOTH_DERICHE_OP_FACTOR_MIN		Sets the minimum value for the Deriche smoothness value's random distribution [min, max], to use for M_AUG_SMOOTH_DERICHE_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the minimum value for the Deriche smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_OP +		Sets whether to enable the operation that performs a blurring of the image with a Gaussian blurring kernel calculated using a Gaussian function with a random standard deviation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SMOOTH_GAUSSIAN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SMOOTH_GAUSSIAN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_OP_STDDEV_MAX		Sets the maximum value for the Gaussian standard deviation's random distribution [min, max], to apply for M_AUG_SMOOTH_GAUSSIAN_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 3.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum value for the Gaussian standard deviation's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_OP_STDDEV_MIN		Sets the minimum value for the Gaussian standard deviation's random distribution [min, max], to apply for M_AUG_SMOOTH_GAUSSIAN_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the minimum value for the Gaussian standard deviation's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_X_OP +		Sets whether to enable the X-direction translation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_TRANSLATION_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to enable M_AUG_TRANSLATION_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_X_OP_MAX	Sets the maximum translation in the X-direction, to apply for M_AUG_TRANSLATION_X_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum translation in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_Y_OP +	Sets whether to enable the Y-direction translation operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_TRANSLATION_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_TRANSLATION_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_Y_OP_MAX	Sets the maximum translation in the Y-direction, to apply for M_AUG_TRANSLATION_Y_OP. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum translation in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
Cumulative statistics image processing context ID	Specifies a cumulative statistics image processing context, allocated using MimAlloc() with M_STATISTICS_CUMULATIVE_CONTEXT, and used in MimStatCalculate() operations. To process multiple images, call MimStatCalculate() multiple times, each time with a different source image. Note that the source image must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. Note, use multiple calls to this function to enable multiple statistical operations. Enabling fewer statistics will help increase the speed of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_X	Sets the width of a typical source image. This value is only used when not providing a source image to MimStatCalculate() with M_PREPROCESS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the width of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_SOURCE_SIZE_Y	Sets the height of a typical source image. This value is only used when not providing a source image to MimStatCalculate() with M_PREPROCESS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the height of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX	Sets whether to calculate the maximum pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS	Sets whether to calculate the maximum absolute pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN	Sets whether to calculate the mean pixel value for each pixel location across the different source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN	Sets whether to calculate the minimum pixel value for each pixel location across the different source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS	Sets whether to calculate the minimum absolute pixel value for each pixel location across the different source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_NUMBER	Sets whether to keep track of the number of source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION	Sets whether to calculate the standard deviation. Note that Aurora Imaging Library calculates the standard deviation using the following formula: INQ $$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2}$$	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM	Sets whether to calculate the sum of the pixel value for each pixel location across the different source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS	Sets whether to calculate the sum of the absolute pixel values for each pixel location across the different source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES	Sets whether to calculate the sum of the squared pixel values for each pixel location across the different source images. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
Dead pixel correction image processing context ID	Specifies a dead pixel correction image processing context, allocated using MimAlloc() with M_DEAD_PIXEL_CONTEXT, and used in MimDeadPixelCorrection() operations.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEAD_PIXELS	Sets the dead pixels image buffer used to identify dead pixels in the source image, where all non-zero pixels are considered dead pixels. Note that you should only use this control type if you have not specified a series of values that identify the dead pixels, using MimPut() with M_XY_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
Dead pixel mask image buffer ID	Specifies identifier of the image buffer containing the dead pixel mask. The buffer must be a single-band image buffer, allocated using MbufAlloc...(). This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_MODE	Sets the interpolation mode. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AVERAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to overwrite a dead pixel with an interpolation performed using a weighted average of all its neighboring pixels in the source image.	c		e	f	g	a		i		j	k	l	m	n	b
Deinterlacing image processing context ID	Specifies a deinterlacing image processing context, allocated using MimAlloc() with M_DEINTERLACE_CONTEXT, and used in MimDeinterlace() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEINTERLACE_TYPE	Sets the deinterlacing algorithm to use. The chosen algorithm can either be applied to all the pixels in the source image or to the pixels that are part of an object in motion (adaptive version of the algorithm). To determine if a pixel is part of a moving object, the adaptive algorithm compares it with the pixel at the same location in neighboring frames. If the difference between the maximum and minimum pixel intensity exceeds a set threshold (M_MOTION_DETECT_THRESHOLD), then the pixel is considered to be part of a moving object. Otherwise, the pixel is considered to be part of the background. The deinterlacing algorithm is not applied to the background pixels. Instead, the background pixels in the output image will be formed by the corresponding pixels in the even or odd field. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISCARD.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_AVERAGE	Specifies that the average algorithm is applied to the pixels that are considered to be part of a moving object and leaves the background pixels unchanged.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_BOB	Specifies that the bob algorithm is applied to the pixels that are considered to be part of a moving object and leaves the background pixels unchanged.	c		e	f	g	a		i		j	k	l	m	n	b

M_ADAPTIVE_DISCARD	Specifies that the discard algorithm is applied to the pixels that are considered to be part of a moving object and leaves the background pixels unchanged.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Performs the averaging algorithm. This algorithm is equivalent to performing the discard algorithm twice, once using the first field in the frame and once using the second. The resulting two frames will then be averaged to form one deinterlaced output frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOB	Performs the bob algorithm. This algorithm performs the discard algorithm twice, once using the first field in the frame and once using the second. The result is two output frames. Therefore, the output frame rate is twice as high as the input frame rate.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISCARD	Performs the discard algorithm. This algorithm takes one field from the source image and discards the other. The second field is then calculated from this field. Each row of the second field is obtained by averaging the two corresponding neighboring rows in the first field. For example, the first row of the second field is calculated from the average of the first and second rows of the first field.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISCARD_FIELD	Sets the field to discard when using the M_DISCARD or M_ADAPTIVE_DISCARD algorithm. Note, in the averaging and bob algorithms, the discard algorithm is called twice; the first field is discarded on the first call and the second field is discarded on the second call. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_EVEN_FIELD.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVEN_FIELD	Specifies that the even field is discarded.	c		e	f	g	a		i		j	k	l	m	n	b
M_ODD_FIELD	Specifies that the odd field is discarded.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_FIELD	Sets the first field to be processed for each input frame and consequently sets the order of the output frames when using the M_BOB or M_ADAPTIVE_BOB algorithm. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_EVEN_FIELD.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVEN_FIELD	Specifies that the even field will be processed first.	c		e	f	g	a		i		j	k	l	m	n	b
M_ODD_FIELD	Specifies that the odd field will be processed first.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_NUM_FRAMES	Sets the number of frames to use for comparison purposes to determine if a pixel is part of an object in motion. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 2.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 1	Specifies the number of frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_OUTPUT		Sets whether the output images of MimDeinterlace() are deinterlaced images or images indicating which pixels are considered to be part of an object in motion (the internal motion detection mask). In the latter case, the pixel values are either 0, if they are part of a background object, or the maximum unsigned value (0xFF for an 8 bit image), if they are part of an object in motion. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the output images are the deinterlaced images.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the output images indicate the background pixels and the pixels that are considered to be part of the object in motion.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_REFERENCE_FRAME		Sets the index of the frame to process within the group of frames that are used for motion detection. This frame is used as the reference frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_CENTER_FRAME.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CENTER_FRAME	Specifies that the center frame in the group is used as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIRST_FRAME	Specifies that the first frame in the group is used as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LAST_FRAME	Specifies that the last frame in the group is used as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_MOTION_DETECT_NUM_FRAMES	Specifies the index of the frame relative to the first frame of the group. The first frame of the group has index 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_THRESHOLD		Sets the threshold value used to differentiate between pixels that are part of objects in motion and background pixels. Each pixel is compared with the pixel at the same location in neighboring frames. If the difference between the maximum and minimum pixel intensity exceeds the specified threshold, the pixel is considered part of a moving object. Otherwise, it is considered part of the background. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_FIRST_IMAGE		Sets the index of the input image used to generate the first deinterlaced image. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the index of the image in the source image array.	c		e	f	g	a		i		j	k	l	m	n	b
Event list image processing result ID		Specifies an event list image processing result buffer, allocated using MimAllocResult() with M_EVENT_LIST, and used in MimLocateEvent() operations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MimGetResult() or MimGetResult1d() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
Filter majority image processing context ID		Specifies a filter majority image processing context identifier, allocated using MimAlloc() with M_FILTER_MAJORITY_CONTEXT, and used in MimFilterMajority() operations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IGNORED_VALUE	Sets a single pixel value to ignore. If a pixel value in the neighborhood matches the ignored value, it will not be counted; the pixel is essentially excluded from the neighborhood. For example, if you want to ignore a background color present in your source image, you can set this control type to the value of the pixels corresponding to the background color. If you want to use this control type, you must also set M_USE_IGNORED_VALUE to M_ENABLE. The default value of this control type is 0. If you set M_USE_IGNORED_VALUE to M_ENABLE but do not specify a value to ignore, pixels whose values are 0 will be ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the pixel value to ignore. The default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

M_TIE_BREAKER_MODE	Sets the method used to break a tie in the neighborhood. In the case of a tie (two or more pixel values occur with the greatest frequency in the neighborhood), this control type specifies which value should be selected to replace the value of the central pixel. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to replace the central pixel value with the greatest value present among the tied values. If you have specified weights for your neighborhood and have set M_USE_FREQUENCY_TIE_BREAKER to M_ENABLE, this control value specifies to replace the central pixel value with the most common (greatest non-weighted frequency) value present among the tied values.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIAN	Specifies to replace the central pixel value with the median value present among the tied values. If you have specified weights for your neighborhood and have set M_USE_FREQUENCY_TIE_BREAKER to M_ENABLE, this control value specifies to replace the central pixel value with the value that has the median non-weighted frequency present among the tied values.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to replace the central pixel value with the smallest value present that appears among the tied values. If you have specified weights for your neighborhood and have set M_USE_FREQUENCY_TIE_BREAKER to M_ENABLE, this control value specifies to replace the central pixel value with the least common (lowest non-weighted frequency) value present among the tied values.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_FREQUENCY_TIE_BREAKER	Sets whether the control type M_TIE_BREAKER_MODE should break a tie in the neighborhood using the values present or, if weights are specified, using the non-weighted frequency at which the tied values appear. If you have specified weights for the pixels in your neighborhood, it is possible that two pixel values are tied despite not occurring with equal frequency in the neighborhood. For example, if your neighborhood contains four pixels with value 5 and weight 1, and two pixels with value 6 and weight 2, they are considered to be tied. You can use this control type to control whether M_TIE_BREAKER_MODE breaks the tie by comparing the actual pixel values (by comparing 5 to 6) or by comparing their non-weighted frequencies (by comparing four pixels present to two pixels present). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to choose the value with which to replace the central pixel by comparing the values present in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to choose the value with which to replace the central pixel by comparing the non-weighted frequency at which the values are found in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_IGNORED_VALUE	Sets whether Aurora Imaging Library should ignore the value in the source image set by M_IGNORED_VALUE when calling MimFilterMajority(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to ignore any value when calling MimFilterMajority().	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to ignore the pixels in the source image whose values match the ignored value when calling MimFilterMajority().	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_IMAGE	Sets the identifier of an image buffer, allocated using MbufAlloc2d(), that defines the neighborhood. When you pass a custom majority filter image processing context to MimFilterMajority(), this is how you define the neighborhood and, optionally, specify different weights, instead of passing a structuring element to the function. This allows you to save the weights with the context when calling MimSave() or MimStream(). The shape and size of the neighborhood will match that of the weight image. For example, if you want a rectangular 3x3 neighborhood, you should set the width and height of the buffer to 3 pixels when allocating it. If you load non-uniform pixel values into this buffer, certain neighborhood pixels will have more influence during the majority filter operation. For example, if you set the central pixel's value to 2, and all others to 1, then the central pixel's value will be counted twice during the majority filter operation. You can use MbufPut2d() to populate your weight image buffer with pixel weights.	c		e	f	g	a		i		j	k	l	m	n	b
Weight image buffer identifier	Specifies the identifier of the weight image buffer that you have previously allocated, using MbufAlloc2d(). Note that floating-point image buffers are not permitted. Additionally, if your weight image buffer is associated with a region of interest, it will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Find orientation image processing context ID	Specifies a find orientation image processing context identifier, allocated using MimAlloc() with M_FIND_ORIENTATION_CONTEXT, and used in MimFindOrientation() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BORDER_ATTENUATION	Sets whether MimFindOrientation() must process the image's borders, or if the operation can ignore spatial patterns occurring close to the ends of the image buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies that MimFindOrientation() cannot ignore the image's borders. The find orientation operation will use the borders.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that MimFindOrientation() can ignore the image's borders. The find orientation operation will not necessarily use the borders.	c		e	f	g	a		i		j	k	l	m	n	b
M_FREQUENCY_CUTOFF_RATIO_HIGH	Specifies the upper limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency; the maximum frequency is dictated by the size of the image. Larger images will have higher maximum frequencies. Frequencies higher than this percentage will be ignored during computation. This can be useful for discarding noise in the image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 95%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the upper limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency. For example, by specifying 95% (the default), frequencies above *MaxFrequency* x 95% will be ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_FREQUENCY_CUTOFF_RATIO_LOW	Specifies the lower limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency; the maximum frequency is dictated by the size of the image. Larger images will have higher maximum frequencies. Frequencies lower than this percentage will be ignored during computation. This can be useful for discarding noise in the image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 5%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the lower limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency. For example, by specifying 5% (the default), frequencies below *MaxFrequency* x 5% will be ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_MODE	Sets the interpolation mode used to internally resize the source image if it is of an inappropriate size. This will only be used if the source image has dimensions that are not a power of 2 (X-size and Y-size). Note that you cannot set an interpolation mode if you are using M_CLIP_CENTER. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AVERAGE.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies averaging interpolation. The M_MODE control type must be set to M_RESIZE_DOWN.	c		e	f	g	a		i		j	k	l	m	n	b

	M_BICUBIC	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERPOLATE	Specifies interpolated resizing. For resizing up, this is equivalent to bilinear; for resizing down, this is equivalent to averaging. This gives the best speed/result compromise for interpolated resizing.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX	Specifies an interpolation based on the maximum pixel value in the source image area. The M_MODE control type must be set to M_RESIZE_DOWN. Note that this can alter the shapes of objects and reduce the robustness of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN	Specifies an interpolation based on the minimum pixel value in the source image area. The M_MODE control type must be set to M_RESIZE_DOWN. Note that this can alter the shapes of objects and reduce the robustness of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE		Sets the resizing mode used if the source image is of an inappropriate size. Resizing will only occur if the source image has dimensions that are not a power of 2 (X-size and Y-size). The find orientation operation is then performed on the resized image, which is stored in a temporary image buffer. The original image is not altered. For more information on resizing, see the Basic geometric transforms section of Chapter 3: Fundamental image processing. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_CLIP_CENTER.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLIP_CENTER	Specifies to perform the find orientation operation using the largest centered portion of the image with dimensions that are a power of 2 (X-size and Y-size).	c		e	f	g	a		i		j	k	l	m	n	b

M_RESIZE_DOWN	Specifies to perform the find orientation operation on a subsampled version of the image with the closest possible dimensions that are a power of 2 (X-size and Y-size).	c		e	f	g	a		i		j	k	l	m	n	b
M_RESIZE_UP	Specifies to perform the find orientation operation on a zoomed version of the image with the closest possible dimensions that are a power of 2 (X-size and Y-size).	c		e	f	g	a		i		j	k	l	m	n	b
Flat-field image processing context ID	Specifies a flat-field image processing context, allocated using MimAlloc() with M_FLAT_FIELD_CONTEXT, and used in MimFlatField() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_CONST	Sets the dark constant value. This is used to remove thermal agitation recorded in the grabbed image (from the CCD) or to remove the darkest possible shade of black when removing uneven lighting from grabbed images. Note that M_DARK_CONST and M_DARK_IMAGE both cannot be set in the same flat-field image processing context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_IMAGE	Sets the identifier of the dark image. This is used to remove thermal agitation recorded in the grabbed image (from the CCD) or to remove the dark shadows from the source image when removing uneven lighting from grabbed images. Note that, this image should be a uniformly dark area (such as, grabbing with the lens cap on your camera). Note that M_DARK_CONST and M_DARK_IMAGE both cannot be set in the same flat-field image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
DarkImageId	Specifies the identifier of the image buffer. This image buffer must be an 8- or 16-bit unsigned processing image buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_CONST	Sets the flat constant value. This is used to remove the variations of CCD sensitivity recorded in the grabbed image (from the CCD) or to reduce the gray in the grabbed image when removing uneven lighting. Note that M_FLAT_CONST and M_FLAT_IMAGE both cannot be set in the same flat-field image processing context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the constant.	c		e	f	g	a		i		j	k	l	m	n	b

M_FLAT_IMAGE	Sets the identifier of the flat image. This is used to remove the variations of CCD sensitivity recorded in the grabbed image (from the CCD) or to reduce the gray in the grabbed image when removing uneven lighting. Note that, this image should be a uniform light gray area. When dealing with CCD sensitivity, the exposure time should be relatively short. Alternatively, when dealing with uneven lighting, the exposure time should be set so that no pixel is saturated. Note that M_FLAT_CONST and M_FLAT_IMAGE both cannot be set in the same flat-field image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
FlatImageId	Specifies the identifier of the image buffer. This image buffer must be an 8- or 16-bit unsigned processing buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAIN_CONST	Sets the gain factor used to normalize (or scale) the result of the flat field calculation back to the full dynamic range of the destination image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTOMATIC	Specifies an automatic gain factor. The automatically generated gain factor is determined by subtracting the flat image (M_FLAT_IMAGE) from the offset image (M_OFFSET_IMAGE) and then taking the average of the resulting image's pixels. If a constant value is specified instead of images (by using M_FLAT_CONST and M_OFFSET_CONST), Aurora Imaging Library returns the result of the subtraction instead.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the gain factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_CONST	Sets the offset constant value. This is used to remove the electrical bias recorded in the grabbed image (from the CCD) or to reduce the black in the flat image when removing uneven lighting. Note that M_OFFSET_CONST and M_OFFSET_IMAGE both cannot be set in the same flat-field image processing context. INQ	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65535	Specifies the constant.	c		e	f	g	a		i		j	k	l	m	n	b

M_OFFSET_IMAGE	Sets the identifier of the offset image. This is used to remove the electrical bias recorded in the image (from the CCD) or to reduce the black in the flat image when removing uneven lighting. Note that, this image should be a uniform dark area. When dealing with electrical bias, the exposure time should be relatively short. Alternatively, when dealing with uneven lighting, the exposure time should be the same as for the flat image (that is, so that no pixel is saturated). Note that M_OFFSET_CONST and M_OFFSET_IMAGE both cannot be set in the same flat-field image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
OffsetImageId	Specifies the identifier of the image buffer. This image buffer must be an 8- or 16-bit unsigned processing buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Histogram equalization adaptive context ID	Specifies a histogram equalization adaptive context, allocated using MimAlloc() with M_HISTOGRAM_EQUALIZE_ADAPTIVE_CONTEXT, and used in MimHistogramEqualizeAdaptive() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALPHA_VALUE	Sets the adjustment factor for M_EXPONENTIAL and M_RAYLEIGH operations. For other operations, M_ALPHA_VALUE is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.4.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the adjustment factor. For an M_EXPONENTIAL operation, greater adjustment values result in less occurrences of the most frequent pixels of the histogram in the resulting image buffer. For an M_RAYLEIGH operation, greater adjustment values result in greater occurrences of the most frequent pixels of the histogram in the resulting image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIP_LIMIT	Sets the maximum percentage of values that a tile's histogram bin can represent. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1%.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 100.0	Specifies the maximum percentage. For example, if a tile has 100 pixels and you specify a maximum limit of 10%, there can be no bin in that tile's histogram with more than 10 values. This essentially limits the contrast. Exceeding values are distributed evenly among the tile's other histogram bins.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_SIZE	Sets the number of bins for each tile's histogram. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_ACCORDING_TO_SOURCE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_SOURCE	Specifies that MimHistogramEqualizeAdaptive() automatically determines the number of bins for each tile's histogram, according to the number of intensities that are possible in the specified image buffer. For example, if you specify an 8-bit unsigned buffer, each tile's histogram will have 256 bins.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 2	Specifies the number of bins for each tile's histogram. Only *integer* values accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TILES_X		Sets the number of tiles along the X-direction of the source image specified with MimHistogramEqualizeAdaptive(). Given the number of tiles in the X- and Y-direction, the size of the source image, and the requirement that tiles be congruent rectangles, MimHistogramEqualizeAdaptive() is able to establish the size of the tiles with which to process the image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 8.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 2	Specifies the number of tiles. Only *integer* values accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TILES_Y		Sets the number of tiles along the Y-direction of the source image specified with MimHistogramEqualizeAdaptive(). Given the number of tiles in the Y- and X-direction, the size of the source image, and the requirement that tiles be congruent rectangles, MimHistogramEqualizeAdaptive() is able to establish the size of the tiles with which to process the image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 8.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 2	Specifies the number of tiles. Only *integer* values accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPERATION		Sets the equalization operation that MimHistogramEqualizeAdaptive() uses. The cumulative probability distribution, $P_f(f)$, of the input image is approximated by its cumulative histogram: $$P_f(f) = \sum_{m=0}^{j} H_F(m) \in [0, 1]$$ For more information, refer to "Pratt, William K. *Digital Image Processing* . United States: John Wiley & Sons, 1978. 318.". INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b

	M_EXPONENTIAL	Specifies an equalization density function which generates an Exponential distribution. Output probability density model: $P_g(g) = \alpha e^{\{-\alpha(g-g_{min})\}} \quad g \geq g_{min}, \alpha > 0$ Transfer function: $g = \begin{cases} min(\ (g_{min} - \frac{1}{\alpha}\ln\{1 - P_f(f)\})\ ,\ g_{max}) & \text{if } P_f(f) < 1.0 \\ g_{max} & \text{if } P_f(f) = 1.0 \end{cases}$	c	e	f	g	a	i	j	k	l	m	n	b
	M_HYPER_CUBE_ROOT	Specifies an equalization density function which generates a Hyperbolic Cube Root distribution. Output probability density model: $P_g(g) = \frac{1}{3}\left[\frac{g^{-2/3}}{\sqrt[3]{g_{max}} - \sqrt[3]{g_{min}}}\right]$ Transfer function: $g = \left[\left(\sqrt[3]{g_{max}} - \sqrt[3]{g_{min}}\right)\left(P_f(f)\right) + \sqrt[3]{g_{min}}\right]^3$	c	e	f	g	a	i	j	k	l	m	n	b
	M_HYPER_LOG	Specifies an equalization density function which generates a Hyperbolic Logarithmic distribution. Output probability density model: $P_g(g) = \frac{1}{g\left[\ln(g_{max}) - \ln(g_{min})\right]}$ Transfer function: $g = g_{min}\left[\frac{g_{max}}{g_{min}}\right]^{P_f(f)}$	c	e	f	g	a	i	j	k	l	m	n	b
	M_RAYLEIGH	Specifies an equalization density function which generates a Rayleigh distribution. Output probability density model: $P_g(g) = \frac{g - g_{min}}{\alpha^2}\left[e^{\left[-\frac{(g-g_{min})^2}{2\alpha^2}\right]}\right] \quad g \geq g_{min}, \alpha > 0$ Transfer function: $g = \begin{cases} min(\ (g_{min} + \left[2\alpha^2 \ln\left\{\frac{1}{1-P_f(f)}\right\}\right]^{\frac{1}{2}})\ ,\ g_{max}) & \text{if } P_f(f) < 1.0 \\ g_{max} & \text{if } P_f(f) = 1.0 \end{cases}$	c	e	f	g	a	i	j	k	l	m	n	b
	M_UNIFORM	Specifies an equalization density function which generates a Uniform distribution. Output probability density model: $P_g(g) = \frac{1}{g_{max} - g_{min}}$ $g_{min} \leq g \leq g_{max}$ Transfer function: $g = \left[g_{max} - g_{min}\right]P_f(f) + g_{min}$	c	e	f	g	a	i	j	k	l	m	n	b
Intensity histogram image processing result ID		Specifies an intensity histogram image processing result buffer, allocated using MimAllocResult() with M_HIST_LIST, and used in MimHistogram() operations.	c	e	f	g	a	i	j	k	l	m	n	b
	M_HIST_BIN_SIZE_MODE	Sets the number of values each histogram bin can represent. To specify the number of histogram bins in the result, use MimAllocResult() and the NbEntries parameter. For example, if you are using an 8-bit unsigned source image, and you want to have one bin for every possible intensity value, you should set the NbEntries parameter to 256. INQ	c	e	f	g	a	i	j	k	l	m	n	b

	M_DEFAULT	Same as M_REGULAR.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIT_SRC_DATA	Specifies that MimHistogram() determines the resulting bin size according to the source image's minimum and maximum intensity values, and the total number of bins. For example, if you call MimHistogram() with a 16-bit signed buffer that holds a source image with a minimum pixel intensity value of 10 and a maximum of 1009, the histogram's bins must account for values between 10 and 1009, which is a total of 1000 possible values. In this case, if the histogram has 100 bins, each bin can represent 10 values (1000/100).	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIT_SRC_RANGE	Specifies that MimHistogram() determines the resulting bin size according to the full range of possible values in the source buffer, and the total number of bins. For example, if you call MimHistogram() with a 16-bit signed buffer, the histogram's bins must account for intensity values ranging from -32768 to 32767, which is a total of 65536 possible values. In this case, if the histogram has 512 bins, each bin can represent 128 values (65536/512). Note that the minimum and maximum values possible for a buffer can be modified using MbufControl() with M_MIN and M_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FREEDMAN	Specifies that MimHistogram() determines the resulting bin size according to the Freedman–Diaconis rule. This is a statistical estimate based on an equation of the general form: $BinSize=2IQR(x)n^{-1/3}$. For more information, see "Freedman, David; Diaconis, Persi. *On the histogram as a density estimator* . Heidelberg, Berlin: Springer, 1981. ".	c		e	f	g	a		i		j	k	l	m	n	b
	M_REGULAR	Specifies that each histogram bin can hold 1 value.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_SMOOTHING_ITERATIONS		Sets the number of smoothing iterations to perform on the histogram after it has been generated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of smoothing iterations. Only *integer* values accepted. The smoothing applied is an integer-based averaging of the histogram; the resulting number of values might therefore be different than the number of values in the source image.	c		e	f	g	a		i		j	k	l	m	n	b
Linear IIR filter image processing context ID		Specifies a linear IIR filter image processing context, allocated using MimAlloc() with M_LINEAR_FILTER_IIR_CONTEXT, and used in MimConvolve() and MimDifferential() operations.	c		e	f	g	a		i		j	k	l	m	n	b

M_FILTER_OPERATION	Sets the linear filter operation to perform. For all derivatives, the result is signed. The destination buffer should be a signed or floating-point buffer (even if an error is not returned). The result of a derivative in an unsigned buffer will wrap around negative numbers. In all cases, if a result does not fit in the buffer, the least-significant n-bits of the result will be written to the destination buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SMOOTH.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_DERIVATIVE_X	Specifies to compute the first derivative of the image, with respect to X, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_DERIVATIVE_Y	Specifies to compute the first derivative of the image, with respect to Y, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SECOND_DERIVATIVE_X	Specifies to compute the second derivative of the image, with respect to X, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SECOND_DERIVATIVE_XY	Specifies to compute the second derivative of the image, with respect to X and Y, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SECOND_DERIVATIVE_Y	Specifies to compute the second derivative of the image, with respect to Y, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SMOOTH	Performs a smoothing operation on the image using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_RESPONSE_TYPE	Sets the filter response type. In all cases, if a result does not fit in the buffer, the least-significant n-bits of the result will be written to the destination buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SLOPE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SLOPE	Specifies a response that is normalized according to the input slope. With this setting, the filter response is consistent with the local slope of the input signal; that is, the rate of change of local intensity values in the input image is in line with the filter response. Any derivative operations performed maintain this consistency. For example, if using a first derivative operation, the response to a ramp input will be its slope. For derivative operations, a floating-point buffer should be used since the result is not remapped and theoretically the result could be an infinite slope. Note that saturation can occur with this setting.	c		e	f	g	a		i		j	k	l	m	n	b

	M_STEP	Specifies a response that is normalized according to a step input. When M_FILTER_STEP_RESPONSE_MAPPING is set to M_ACCORDING_TO_DEST, the response is adjusted so that saturation will not occur; that is, the maximum filter response is set according to the depth of the destination image buffer so that the destination image buffer can hold value extremes without saturation. For derivative operations, the result will be mapped to fit in a signed buffer with the same size as the source. The result is mapped between min +1 and max. For example, for an 8-bit buffer, the result will be mapped between -127 and 127.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS		Specifies the degree of smoothness (strength of the denoising) to apply, according to the filter smoothness type. Larger values result in greater smoothing. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies a default smoothness value that is mid-range. For an M_NORMALIZED filter smoothness type, the value is 50. For an M_SIZE or M_NATIVE filter smoothness type, the default value results in smoothing that is approximately equivalent to setting M_NORMALIZED to 50.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the smoothness value. The range available depends on the smoothness type. For an M_NORMALIZED type, the range is 0 <= Value <= 100. For an M_NATIVE type, the range is 0 < Value < 1. For an M_SIZE type, the value entered reflects the extent to which neighboring pixels affect the central pixel of the region receiving the filter. A size of 10, for example, is roughly equivalent to a 21 x 21 convolution kernel, where the 10 pixels on each side of the central pixel will have an impact on the filter operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS_TYPE		Specifies the filter smoothness type. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NORMALIZED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NATIVE	Specifies to control the alpha (α), beta (β), or sigma (σ) value in the Deriche, Shen-Castan, or Vliet weighting function, depending on the filter type specified with M_FILTER_TYPE. Whereas the M_NORMALIZED and M_SIZE filter smoothness type settings reorganize the data to provide a more intuitive approach (such as the 0-100 range of M_NORMALIZED), this setting makes no such transformation and allows you to control the raw (native) smoothness directly. This is an advanced setting, and requires knowledge of the Deriche, Shen-Castan, or Vliet filter formulas. Set the required alpha (α), beta (β), or sigma (σ) value with M_FILTER_SMOOTHNESS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NORMALIZED	Specifies to use a normalized smoothness value that is between 0 and 100.	c		e	f	g	a		i		j	k	l	m	n	b

M_SIZE	Specifies to use a smoothness value driven by a spatial size. This size specifies the extent (similar to a kernel width) to which neighboring pixels affect the central pixel of the region receiving the filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_STEP_RESPONSE_MAPPING	Sets how to remap a derivative result. Note that smoothing operations (M_FILTER_OPERATION with M_SMOOTH) are not remapped.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_SOURCE.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_DEST	Specifies to remap to the range of the destination buffer. When using this setting, the source and destination buffers cannot be floating-point buffers. Additionally, the destination must be a signed buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_SOURCE	Specifies to remap to a signed (floating-point) result that has the same bit-depth as the source buffer. For example, for an 8-bit unsigned buffer, the result will be mapped between -127 and 127.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_TYPE	Specifies the type of IIR filter to use for the convolution operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DERICHE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DERICHE	Specifies a Deriche infinite support filter. M_DERICHE implements an approximation of the Gaussian filter using an IIR weighted exponential function, of which the general form is: $k\left(a\ \lvert n\rvert+1\right)e^{-a\lvert n\rvert}$ For the Deriche filter, the neighborhoods' influence decreases much slower as the distance from the central pixel increases, compared to the Shen-Castan filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_SHEN	Specifies a Shen-Castan infinite support exponential filter. This is an exponential weighting function, of the general form: $Ke^{-\beta\lvert n\rvert}$ For the Shen-Castan filter, the neighborhoods' influence decreases much faster as the distance from the central pixel increases, compared to the Deriche filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_VLIET	Specifies a Vliet infinite support filter. This is an approximation of the Gaussian function, which has the general form: $Ke^{\frac{-n^2}{\sigma^2}}$ The Vliet filter is an accurate approximation of the Gaussian function, especially when sigma (σ) is large.	c		e	f	g	a		i		j	k	l	m	n	b

Locate difference image processing result ID	Specifies a locate difference image processing result buffer, allocated using MimAllocResult() with M_LOCATE_DIFFERENCE_RESULT, and used in MimLocateDifference() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MimGetResult() or MimGetResult1d() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
Locate peak 1D image processing context ID	Specifies a 1D locate peak image processing context, allocated using MimAlloc() with M_LOCATE_PEAK_1D_CONTEXT, and used in MimLocatePeak1d() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRAME_SIZE	Sets the Y-size of the frames. The Y-size must be set before preprocessing the context. M_FRAME_SIZE will always be the Y-size, regardless of the direction set with M_SCAN_LANE_DIRECTION. If a multi-frame image buffer is used, M_FRAME_SIZE sets the Y-size of each individual frame. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FULL_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FULL_SIZE	Specifies to use the buffer as a single frame.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the Y-size of the frames.	c		e	f	g	a		i		j	k	l	m	n	b

M_MINIMUM_CONTRAST	Sets the minimum contrast (difference) between the intensity of the local 1D background and the minimum acceptable intensity of a pixel in the peak neighborhood. This control type is intended to adjust how M3dmapAddScan() internally processes laser line images. When using MimLocatePeak1d(), you typically set the value using the function's parameters; this control type only sets the default value used by the function. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 100.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FRAMES	Sets the total number of frames for which peaks can be extracted. This is the maximum number of frames for which results can be accumulated. The number of frames must be set before preprocessing the context. Note that when passing a multi-frame image buffer to MimLocatePeak1d(), the number of frames processed by each call to MimLocatePeak1d() can be different from M_NUMBER_OF_FRAMES. If M_NUMBER_OF_FRAMES is set to 10, and a 5-frame source image buffer is passed, you can call MimLocatePeak1d() 2 times to process the total number of frames. Note that, in this example case, the 2 calls need not be successive. You can retrieve results using MimGetResultSingle() at any time after a MimLocatePeak1d() operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_PEAKS	Sets the maximum number of peaks to find along a given lane. If more than the specified number of peaks are found, the highest peaks are kept. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of peaks. Only *integer* values accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_INTENSITY_RANGE	Sets the number of pixels used to calculate the average peak intensity. The pixels are chosen around the peak intensity pixel. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_PEAK_WIDTH_DELTA	Sets the number of pixels that can be added to or subtracted from the nominal width, when determining the range of allowable widths of the peak neighborhood. This control type is intended to adjust how M3dmapAddScan() internally processes laser line images. When using MimLocatePeak1d(), you typically set the value using the function's parameters; this control type only sets the default value used by the function. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_WIDTH_NOMINAL	Sets the nominal (expected average) width of the peak neighborhood. In laser line images, this is the average width of the laser line. This control type is intended to adjust how M3dmapAddScan() internally processes laser line images. When using MimLocatePeak1d(), you typically set the value using the function's parameters; this control type only sets the default value used by the function. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_LANE_DIRECTION	Sets the direction in which to detect peaks. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_VERTICAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL	Detects peaks along the image's X-axis. Typically for laser line images with a vertical laser line.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL	Detects peaks along the image's Y-axis. Typically for laser line images with a horizontal laser line.	c		e	f	g	a		i		j	k	l	m	n	b
Locate peak 1D image processing result ID	Specifies a locate peak 1D image processing result buffer, allocated using MimAllocResult() with M_LOCATE_PEAK_1D_RESULT, and used in MimLocatePeak1d() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_INTENSITY_INVALID_VALUE	Sets the intensity value to use for missing peaks. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies the value -1. This is adequate for most applications.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies any integer value, for applications with specific needs. 0 is a common choice in this case.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_CRITERION	Sets the characteristic with which to sort peaks. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default; the default is M_PEAK_INTENSITY + M_SORT_DOWN. Note that M_SORT_DOWN is not the default sort order when explicitly specifying the sort criterion.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_INTENSITY +	Orders peaks according to their intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_POSITION +	Orders peaks according to their position.	c		e	f	g	a		i		j	k	l	m	n	b
Match image processing context ID	Specifies a match image processing context, allocated using MimAlloc() with M_MATCH_CONTEXT, and used in MimMatch() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE	Sets the image buffer containing the mask image. All non-zero values are considered masked pixels that will be ignored during the match.	c		e	f	g	a		i		j	k	l	m	n	b
MaskImageId	Specifies the identifier of the image buffer containing the mask image. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_SCORE	Sets the maximum score when performing a match using the normalized grayscale correlation mode. When using other matching modes, this control type is ignored. This control type causes Aurora Imaging Library to linearly remap the range of the internal match results to the range established, from the lowest possible value to the maximum score specified. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MAX_DEPTH.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DEPTH	Specifies to establish the maximum score based on the pixel depth of the destination buffer (for example, when dealing with a 16-bit signed buffer, the maximum value would be 32, 767). Note that, when using floating-point destination buffers, the range is [-1, 1]; unless you use a clipping score type (using M_SCORE_TYPE set to either M_NORM_CLIP or M_NORM_CLIP_SQR), in which case the range is [0, 1].	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum score. This score can be any positive value less than the maximum predetermined by the destination buffer's pixel depth.	c		e	f	g	a		i		j	k	l	m	n	b

M_MODE	Sets the type of computation to perform when matching the source image to the model image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_CORRELATE_NORMALIZED.	c		e	f	g	a		i		j	k	l	m	n	b
M_CORRELATE	Computes a grayscale correlation.	c		e	f	g	a		i		j	k	l	m	n	b
M_CORRELATE_NORMALIZED	Computes a normalized grayscale correlation.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN_OF_ABS_DIFFERENCES	Computes the sum of absolute differences divided by the number of pixels used to compute it, resulting in a 8-bit range result.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_OF_ABS_DIFFERENCES	Computes sum of the absolute differences. Note that the result might overflow in 16-bit destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_IMAGE	Sets the image buffer containing the model image.	c		e	f	g	a		i		j	k	l	m	n	b
ModelImageId	Specifies the identifier of the image buffer containing the model image. Note that this buffer must be a 1-band, 8-bit unsigned image buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_STEP	Sets whether to use every pixel or every other pixel in the model image when matching the source image to the model image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as 1.	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies to take every pixel (on both axes) to compute the match.	c		e	f	g	a		i		j	k	l	m	n	b
2	Specifies to take every other pixel (on both axes) to compute the match.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_TYPE	Sets how to compute the final correlation score. Note that this control type is only used when performing a match using normalized grayscale correlation mode (using M_MODE set to M_CORRELATE_NORMALIZED). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NORM_CLIP_SQR.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM	Specifies to use normalized grayscale correlation to compute the final match score.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM_CLIP	Specifies to clip the results from a normalized grayscale correlation when computing the final correlation score. The calculation used is the same as M_NORM, but any value less than 0 is clipped (that is, recorded as 0).	c		e	f	g	a		i		j	k	l	m	n	b

M_NORM_CLIP_SQR	Specifies to clip the results from the square of the normalized grayscale correlation when computing the final correlation score. The calculation used is the same as M_SQR_NORM, but any value less than 0 is clipped (that is, recorded as 0).	c		e	f	g	a		i		j	k	l	m	n	b
M_SQR_NORM	Specifies to use the square of the normalized grayscale correlation to compute the final correlation score.	c		e	f	g	a		i		j	k	l	m	n	b
Rearrangement image processing context ID	Specifies a rearrangement image processing context, allocated using MimAlloc() with M_REARRANGE_CONTEXT, and used in MimRearrange() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE	Sets the processing mode. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_LINES	Specifies that each area to be rearranged is a single horizontal line.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTS	Specifies that each area to be rearranged is a single rectangle.	c		e	f	g	a		i		j	k	l	m	n	b
Remap image processing context ID	Specifies a remap image processing context, allocated using MimAlloc() with M_IM_REMAP_CONTEXT, and used in MimRemap() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DST_END_VALUE	Sets the end pixel value used to compute the destination range. Note, this is only used when M_DST_RANGE is set to M_USE_CONTEXT. When calling MimRemap(), if the value falls outside the range of the destination buffer, it will be saturated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_DST_RANGE	Sets whether to use the destination image buffer or the remap image processing context to determine the destination range. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_BUFFER	Specifies to use the minimum and maximum pixel values of the destination image buffer. By default, this is the full value range according to the buffer type, unless MbufControl() with M_MIN and M_MAX have been set to other values.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_CONTEXT	Specifies to use the context's destination start and end values specified with M_DST_START_VALUE and M_DST_END_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b

M_DST_START_VALUE	Sets the start pixel value used to compute the destination range. Note, this is only used when M_DST_RANGE is set to M_USE_CONTEXT. When calling MimRemap(), if the value falls outside the range of the destination buffer, it will be saturated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_FUNCTION	Sets the function used to map the source image buffer to the destination image buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies to perform a uniform linear mapping.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC_END_VALUE	Sets the end pixel value used to compute the source range. Note, this is only used when M_SRC_RANGE is set to M_USE_CONTEXT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC_RANGE	Sets whether to use the source image buffer or the remap image processing context to determine the source range. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_USE_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_BUFFER	Specifies to use the minimum and maximum pixel values of the source image buffer. By default, this is the full value range according to the buffer type, unless MbufControl() with M_MIN and M_MAX have been set to other values.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_CONTEXT	Specifies to use the context's source start and end values specified with M_SRC_START_VALUE and M_SRC_END_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC_START_VALUE	Sets the start pixel value used to compute the source range. Note, this is only used when M_SRC_RANGE is set to M_USE_CONTEXT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the start pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics image processing context ID	Specifies a statistics image processing context, allocated using MimAlloc() with M_STATISTICS_CONTEXT, and used by MimStatCalculate() as the operation to perform. Note, use multiple calls to this function to enable multiple statistical operations. Enabling fewer statistics will help increase the speed of the operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_COND_HIGH	Sets the upper limit of the selected condition (set using M_CONDITION) that establishes which pixels to include in the statistical operation(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no upper limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit.	c		e	f	g	a		i		j	k	l	m	n	b
M_COND_LOW	Sets the lower limit of the selected condition (set using M_CONDITION) that establishes which pixels to include in the statistical operation(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no lower limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower limit.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONDITION	Sets the condition for the statistical computation that establishes which pixels to include in the statistical operation(s). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no condition is set. Note that, all pixels will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_EQUAL	Specifies that only pixels with values equal to M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Specifies that only pixels with values greater than M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Specifies that only pixels with values greater than or equal to M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b

	M_IN_RANGE	Specifies that pixels with values between M_COND_LOW and M_COND_HIGH, inclusive, will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LESS	Specifies that only pixels with values less than M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LESS_OR_EQUAL	Specifies that only pixels with values less than or equal to M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_EQUAL	Specifies that pixels with values equal to M_COND_LOW will be not used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_OUT_RANGE	Specifies that pixels with values less than M_COND_LOW, or greater than M_COND_HIGH, will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLCM_PAIR_OFFSET_X		Sets the displacement in X between pixel pairs used to generate the co-occurrence matrix. Note that this control type is only valid when calculating co-occurrence statistics (M_STAT_GLCM_...). When used with any other statistic, this control type is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the displacement in X.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLCM_PAIR_OFFSET_Y		Sets the displacement in Y between pixel pairs used to generate the co-occurrence matrix. Note that this control type is only valid when calculating co-occurrence statistics (M_STAT_GLCM_...). When used with any other statistic, this control type is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the displacement in Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLCM_QUANTIFICATION		Sets the depth of the gray level of the co-occurrence matrix. Note that this control type is only valid when calculating co-occurrence statistics (M_STAT_GLCM_...). When used with any other statistic, this control type is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 8 bit.	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 10	Specifies the depth. If the depth is set to a value greater than 10, the co-occurrence matrix is rescaled to 10 bits.	c		e	f	g	a		i		j	k	l	m	n	b

M_SOURCE_SIZE_X	Sets the width of a typical source image. This value is only used when not providing a source image to MimStatCalculate() with M_PREPROCESS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the width of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_Y	Sets the height of a typical source image. This value is only used when not providing a source image to MimStatCalculate() with M_PREPROCESS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the height of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANGULAR_DATA_COHERENCE	Calculates the coherence of the angular data. The coherence represents the directional trend of all the angular data in the image and is a measure of how coherent or parallel the unit vectors are with respect to each other. The coherence is calculated as the ratio between the length (norm) of the vector sum and its maximum possible length (which corresponds to the number of pixels being considered, since each pixel is presumed to have a length of one). Numerically, this quantity lies between 0 and 1, where 0 represents absolutely no coherence (unit vectors pointing in random directions) and 1 is total coherence (unit vectors pointing in same direction). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANGULAR_DATA_MEAN	Sets whether to calculate the dominant direction of the unit vectors. This angle is a representation of the mean (or average) angle of all the angular data. Note that to obtain more representative results, the mean angle should be analyzed in context with the coherence. For example, if your image has a coherence of 0.1 and a mean angle of 65°, the mean angle is not very meaningful because the unit vectors are pointing in random, incoherent directions. However, if your image has a coherence of 0.9 and a mean angle of 65°, this indicates that the unit vectors are more or less pointing in the same direction and their average value is around 65°. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_GLCM_CONTRAST	Sets whether to calculate the contrast of all the grayscale pixel values within a specified neighborhood in relation to the normalized co-occurrence probability of each pixel. Define the neighborhood using M_GLCM_PAIR_OFFSET_X, M_GLCM_PAIR_OFFSET_Y, M_GLCM_QUANTIFICATION, M_TILE_SIZE_X, and M_TILE_SIZE_Y. Note that the image on which the calculation is performed must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CORRELATION	Sets whether to measure the local variations in the grayscale co-occurrence matrix. Define the neighborhood using M_GLCM_PAIR_OFFSET_X, M_GLCM_PAIR_OFFSET_Y, M_GLCM_QUANTIFICATION, M_TILE_SIZE_X, and M_TILE_SIZE_Y. Note that the image on which the calculation is performed must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_DISSIMILARITY	Sets whether to measures the joint probability co-occurrence. The result determines how close your data comes to forming a straight line. Define the neighborhood using M_GLCM_PAIR_OFFSET_X, M_GLCM_PAIR_OFFSET_Y, M_GLCM_QUANTIFICATION, M_TILE_SIZE_X, and M_TILE_SIZE_Y. Note that the image on which the calculation is performed must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_GLCM_ENERGY	Sets whether to calculate the energy between two points in the gray level co-occurrence matrix of a given neighborhood. The energy is the pixel value of the cell divided by the sum of the value of the cells in the neighborhood. Define the neighborhood using M_GLCM_PAIR_OFFSET_X, M_GLCM_PAIR_OFFSET_Y, M_GLCM_QUANTIFICATION, M_TILE_SIZE_X, and M_TILE_SIZE_Y. Note that the image on which the calculation is performed must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_ENTROPY	Sets whether to calculate the inverse of the energy between two points in the normalized co-occurrence probability of a specified neighborhood. Define the neighborhood using M_GLCM_PAIR_OFFSET_X, M_GLCM_PAIR_OFFSET_Y, M_GLCM_QUANTIFICATION, M_TILE_SIZE_X, and M_TILE_SIZE_Y. Note that the image on which the calculation is performed must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_HOMOGENEITY	Sets whether to calculate the similarity between the normalized co-occurrence probability and the gray level co-occurrence matrix of a given neighborhood. Homogeneity weighs values by the inverse of the contrast weight, with weights decreasing exponentially away from the diagonal. Define the neighborhood using M_GLCM_PAIR_OFFSET_X, M_GLCM_PAIR_OFFSET_Y, M_GLCM_QUANTIFICATION, M_TILE_SIZE_X, and M_TILE_SIZE_Y. Note that the image on which the calculation is performed must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_MAX	Sets whether to calculate the maximum pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS	Sets whether to calculate the maximum absolute pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN	Sets whether to calculate the mean value of the pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN	Sets whether to calculate the minimum pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS	Sets whether to calculate the minimum absolute pixel value from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_NUMBER	Sets whether to keep track of the number of pixels, from the source image, that satisfied the condition specified when calling MimStatCalculate(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to keep track.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_ORIENTATION_DATA_COHERENCE	Sets whether to calculate the coherence of the orientation data. The coherence represents the directional trend of all the orientation data in the image and is a measure of how coherent or parallel the orientation of the unit vectors are with respect to each other. The coherence is calculated as the ratio between the length (norm) of the vector sum and its maximum possible length (which corresponds to the number of pixels being considered, since each pixel is presumed to have a length of one). Numerically, this quantity lies between 0 and 1, where 0 represents absolutely no coherence (unit vectors pointing in random directions) and 1 is total coherence (unit vectors pointing in same direction). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ORIENTATION_DATA_MEAN	Sets whether to calculate the dominant orientation of the unit vectors. The orientation of a unit vector is its inclination, without its direction; if two unit vectors have opposite directions (for example, an angle of 0° and 180°), they have the same orientation (0°). Calculating the dominant orientation using unit vectors with opposite directions (0° and 180°) would result in the magnitude of their vector sum being double the original value (2) and a dominant orientation of 0°. Whereas, calculating the dominant orientation using two unit vectors that are separated by 90° (for example, an angle of 0° and 90°) would result in the magnitude of their vector sum being 0, effectively canceling each other out (no dominant orientation), and an invalid orientation angle. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION	Sets whether to calculate the standard deviation from the value of each pixel location across the source image. Note that Aurora Imaging Library calculates the standard deviation using the following formula: INQ $$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2}$$	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM	Sets whether to calculate the sum of the pixel values from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS	Sets whether to calculate the sum of the absolute pixel values from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES	Sets whether to calculate the sum of the squared pixel values from each pixel location across the source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_X	Sets the distance between the neighborhood along the X-axis in which to evaluate the specified statistic. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Width	Specifies the width of the neighborhood, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_Y	Sets the distance between the neighborhood along the Y-axis in which to evaluate the specified statistic. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Height	Specifies the height of the neighborhood, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TILE_SIZE_X	Sets the X-size (width) of the neighborhood in which to evaluate the specified statistic. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Width	Specifies the width of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_TILE_SIZE_Y	Sets the Y-size (height) of the neighborhood in which to evaluate the specified statistic. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Height	Specifies the height of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b

Unwarp along path image processing context ID	Specifies an unwarp along path image processing context, allocated using MimAlloc() with M_UNWARP_ALONG_PATH_CONTEXT, and used in MimUnwarpAlongPath() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_MODE	Sets the interpolation mode used when unwarping along the path. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NEAREST_NEIGHBOR.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE	Sets the type of path along which to unwarp the image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_POLYLINE.	c		e	f	g	a		i		j	k	l	m	n	b
M_POLYLINE	Specifies that the path is defined by a polyline. This path must be set using MimPut() with M_XY_PATH and the width of the path set using **MimControl**()M_SINGLE_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN	Sets how to handle pixels that map outside of the source image buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_OVERSCAN_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_CLEAR	Specifies to set the destination pixel to 0 if the corresponding point falls outside of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Specifies to leave the destination pixels as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Specifies to use pixels from the source buffer's ancestor buffer. If the source buffer is not a child buffer, or if the point falls outside of the ancestor buffer, the destination pixel is left as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_SINGLE_WIDTH	Sets a single width for all points along the path. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the width of the path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNWARP_SAMPLING_MODE		Sets which algorithm to use for the unwarp along path operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ADAPTIVE_AT_JUNCTION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ADAPTIVE_ALONG_PATH	Specifies to spread the deformation over the entire path. Quadrilaterals are defined from the intersection of the top and bottom lines of the path (one half-width from the center line). The deformation will occur over the entire quadrilateral.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ADAPTIVE_AT_JUNCTION	Specifies to deform only at the junction of each path segment. Rectangles are defined from the shortest side (top or bottom) of the quadrilateral. Triangles are generated to fill the rest of the quadrilateral. The deformation will only occur within the triangular region at the junction. This mode preserves the width of the path.	c		e	f	g	a		i		j	k	l	m	n	b
Wavelet image processing context ID		Specifies a wavelet image processing context, allocated using MimAlloc() with M_WAVELET_TRANSFORM_CONTEXT or M_WAVELET_TRANSFORM_CUSTOM_CONTEXT, and used in MimWaveletTransform() or MimWaveletDenoise() operations. Unless otherwise specified, settings apply to both M_WAVELET_TRANSFORM_CONTEXT and M_WAVELET_TRANSFORM_CUSTOM_CONTEXT context types.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFORMATION_MODE		Sets the wavelet transformation mode. Modifying the transformation mode can affect how Aurora Imaging Library samples the data and produces the wavelet results (for example, the diagonal, horizontal, and vertical coefficients). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DYADIC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DYADIC	Specifies a dyadic wavelet transformation. For each transformation level, Aurora Imaging Library samples the wavelet coefficients by a factor of 2. For example, when drawing dyadic results, they are at different sizes, at different levels. Dyadic transformations generally apply to signal coding and data compression. Performing MimWaveletDenoise() with M_DYADIC is typically faster than M_UNDECIMATED, though the quality of denoising is usually lower. In general, M_DYADIC uses less resources (processing time and memory) than M_UNDECIMATED.	c		e	f	g	a		i		j	k	l	m	n	b

	M_UNDECIMATED +	Specifies an undecimated wavelet transformation. Such transformations are designed to overcome the lack of invariance between transformation levels in dyadic wavelet transformations. For example, when drawing undecimated results, they are at the same size regardless of the level. Undecimated transformations generally apply to signal denoising and pattern recognition. Performing MimWaveletDenoise() with M_UNDECIMATED is typically slower than M_DYADIC, though the quality of denoising is usually higher. In general, M_UNDECIMATED uses more resources (processing time and memory) than M_DYADIC.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_TYPE		Sets a predefined type of wavelet filter. Only available if the context type is M_WAVELET_TRANSFORM_CONTEXT. The mathematical domain of complex filter types (M_..._COMPLEX) consists of numbers that have a real part (real numbers) and an imaginary part (imaginary numbers). The mathematical domain of filter types that are not complex consist of real numbers only. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_HAAR.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_1	Specifies a Daubechies wavelet filter that uses 1 vanishing moment and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_2	Specifies a Daubechies wavelet filter that uses 2 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_3	Specifies a Daubechies wavelet filter that uses 3 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_3_COMPLEX	Specifies a Daubechies wavelet filter that uses 3 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_4	Specifies a Daubechies wavelet filter that uses 4 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_5	Specifies a Daubechies wavelet filter that uses 5 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_5_COMPLEX	Specifies a Daubechies wavelet filter that uses 5 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_6	Specifies a Daubechies wavelet filter that uses 6 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_7	Specifies a Daubechies wavelet filter that uses 7 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_7_COMPLEX	Specifies a Daubechies wavelet filter that uses 7 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DAUBECHIES_8	Specifies a Daubechies wavelet filter that uses 8 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HAAR	Specifies a Haar wavelet filter. Haar uses real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_1	Specifies a Symlet wavelet filter that uses 1 vanishing moment and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_2	Specifies a Symlet wavelet filter that uses 2 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_3	Specifies a Symlet wavelet filter that uses 3 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_4	Specifies a Symlet wavelet filter that uses 4 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_5	Specifies a Symlet wavelet filter that uses 5 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_6	Specifies a Symlet wavelet filter that uses 6 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_7	Specifies a Symlet wavelet filter that uses 7 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_8	Specifies a Symlet wavelet filter that uses 8 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ContextOrResultImId=Augmentation context ID: M_AUG_ASPECT_RATIO_OP; M_AUG_BLUR_MOTION_OP; M_AUG_CROP_OP; M_AUG_DILATION_ASYM_OP; M_AUG_DILATION_OP; M_AUG_EROSION_ASYM_OP; M_AUG_EROSION_OP; M_AUG_FLIP_OP; M_AUG_GAMMA_OP; M_AUG_HSV_VALUE_GAIN_OP; M_AUG_HUE_OFFSET_OP; M_AUG_INTENSITY_ADD_OP; M_AUG_INTENSITY_MULTIPLY_OP; M_AUG_LIGHTING_DIRECTIONAL_OP; M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP; M_AUG_NOISE_MULTIPLICATIVE_OP; M_AUG_NOISE_SALT_PEPPER_OP; M_AUG_REDUCE_OP; M_AUG_ROTATION_OP; M_AUG_SATURATION_GAIN_OP; M_AUG_SCALE_OP; M_AUG_SHARPEN_DERICHE_OP; M_AUG_SHEAR_X_OP; M_AUG_SHEAR_Y_OP; M_AUG_SMOOTH_DERICHE_OP; M_AUG_SMOOTH_GAUSSIAN_OP; M_AUG_TRANSLATION_X_OP; M_AUG_TRANSLATION_Y_OP].

You can add the following value to the above-mentioned values to specify the priority (order) with which MimAugment() performs the corresponding operation. Alternatively, you can add a value from For specifying the probability that MimAugment() performs the corresponding operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the priority (order) with which MimAugment() performs the corresponding operation																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_PRIORITY		Sets the priority (order) with which MimAugment() performs the corresponding operation. This overrides the default priority that MimAugment() uses.															
	0 <= *Value* <= 100	Specifies the priority (order) value. Operations with lower priority (order) values are performed before operations with higher priority (order) values. If multiple operations have the same priority (order) value, MimAugment() internally chooses the priority (order).	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ContextOrResultImId=Augmentation context ID: M_AUG_ASPECT_RATIO_OP; M_AUG_BLUR_MOTION_OP; M_AUG_CROP_OP; M_AUG_DILATION_ASYM_OP; M_AUG_DILATION_OP; M_AUG_EROSION_ASYM_OP; M_AUG_EROSION_OP; M_AUG_FLIP_OP; M_AUG_GAMMA_OP; M_AUG_HSV_VALUE_GAIN_OP; M_AUG_HUE_OFFSET_OP; M_AUG_INTENSITY_ADD_OP; M_AUG_INTENSITY_MULTIPLY_OP; M_AUG_LIGHTING_DIRECTIONAL_OP; M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP; M_AUG_NOISE_MULTIPLICATIVE_OP; M_AUG_NOISE_SALT_PEPPER_OP; M_AUG_REDUCE_OP; M_AUG_ROTATION_OP; M_AUG_SATURATION_GAIN_OP; M_AUG_SCALE_OP; M_AUG_SHARPEN_DERICHE_OP; M_AUG_SHEAR_X_OP; M_AUG_SHEAR_Y_OP; M_AUG_SMOOTH_DERICHE_OP; M_AUG_SMOOTH_GAUSSIAN_OP; M_AUG_TRANSLATION_X_OP; M_AUG_TRANSLATION_Y_OP].

You can add the following value to the above-mentioned values to specify the probability that MimAugment() performs the corresponding operation. Alternatively, you can add a value from For specifying the priority (order) with which MimAugment() performs the corresponding operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the probability that MimAugment() performs the corresponding operation																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_PROBABILITY	Sets the probability that MimAugment() performs the corresponding operation.															
0.0 <= *Value* <= 100.0	Specifies the probability, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ContextOrResultImId=Wavelet image processing context ID, and ControlType=M_TRANSFORMATION_MODE, : M_UNDECIMATED].

You can add the following value to the above-mentioned value to specify undecimated wavelet transformations that are centered.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether undecimated wavelet transformations are centered																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CENTER	Specifies undecimated wavelet transformations that are centered. Undecimated wavelet transformations that are not centered can appear misaligned (for example, when drawing results).	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when ContextOrResultImId=Locate peak 1D image processing result ID, and ControlType=M_SORT_CRITERION, : M_PEAK_INTENSITY; M_PEAK_POSITION].

You can add one of the following values to the above-mentioned values to specify the order to sort peaks.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the order to sort peaks																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SORT_DOWN	Sorts peaks in descending order.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_UP	Sorts peaks in ascending order. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the values listed above are not available in Aurora Imaging Library Lite. See the value's corresponding operation function for Aurora Imaging Library Lite availability.

Type-specific versions of the function when using a C compiler under 64-bit

```
void MimControlDouble (AIL_ID ContextOrResultImId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultImId

See ContextOrResultImId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MimControlInt64 (AIL_ID ContextOrResultImId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextOrResultImId

See ContextOrResultImId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimConvert

Synopsis

Perform a color conversion.

Syntax

```
void MimConvert(
    AIL_ID SrcImageBufId,             //in
    AIL_ID DstImageBufId,             //out
    AIL_ID ArrayBufIdOrConversionType //in
)
```

Description

This function performs a color conversion on the source image and places the result in the destination buffer.

If necessary, you can perform a mathematically optimal color-to-grayscale conversion using McolProject() with M_PRINCIPAL_COMPONENT_PROJECTION.

When converting from sRGB (M_SRGB_...), your color data must adhere to standard RGB specifications (sRGB), as defined by the International Electrotechnical Commission (IEC) Project Team 61966-2-1. Conversions to sRGB (M_...SRGB) will also adhere to these standards. If converting from linear sRGB (M_SRGB_LINEAR_...), your color data must not be corrected (gamma corrected). Conversions to linear sRGB (M_...SRGB_LINEAR) will also not be corrected.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Source buffer values must typically be positive, including the values in a signed source buffer. When using an integer source buffer for a LAB or LCH conversion, the full range is taken into account, including the negative values of signed buffers.

If specifying a floating-point buffer, the values of the buffer must be normalized between 0 and 1 before calling this function.

DstImageBufId

Specifies the identifier of the destination image buffer.

This image buffer must be a 1- or 3-band buffer and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The values generated will typically be normalized to the possible range of values in the destination buffer. For example, the hue values from 0 to 360° will generate the values from 0 to 255 in an 8-bit destination buffer.

For a floating-point destination buffer, the values generated will typically be normalized between 0 and 1. This normalization does not apply to the Array buffer identifier conversion type.

For LAB conversions with a floating-point destination buffer, the L band values fall between 0 and 100 (inclusive), while the A and B band values are unbounded. However for most applications, A and B values fall between -150 and 150 (exclusive). When converting to LAB with an integer destination buffer, A and B values are saturated to fall within -128 and 127 (inclusive). Aurora Imaging Library then scales the L, A, and B band values to fall within the range of the destination buffer. The L band is therefore subject to a larger scaling than the two other bands.

For LCH conversions with a floating-point destination buffer, the H band value is an angle, in degrees; otherwise, Aurora Imaging Library rescales the angle to fall within the range of the destination buffer. When converting to LCH with an integer destination buffer, Aurora Imaging Library clips the C value to the lower value between the maximum allowed in the destination buffer and 255. Aurora Imaging Library then rescales the value to fit within the full range of the destination buffer.

ArrayBufIdOrConversionType

Specifies the type of conversion to perform. The conversion can be specified with either a predefined or custom transformation matrix. This parameter can be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of conversion to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HSL_TO_RGB	Specifies an HSL to RGB conversion. Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_HSV_TO_RGB	Specifies an HSV to RGB conversion. Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAB_TO_SRGB	Specifies a LAB (CIELAB) to sRGB conversion. The LAB color space is based on the color's luminance (L), its position between red and green (A), and its position between yellow and blue (B). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAB_TO_SRGB_LINEAR	Specifies a LAB (CIELAB) to linear sRGB conversion. The LAB color space is based on the color's luminance (L), its position between red and green (A), and its position between yellow and blue (B). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_L_TO_RGB	Specifies an L (luminance) to RGB conversion. The luminance (intensity) is repeated in each color band of the destination buffer, creating a monochromatic (gray) RGB buffer. The source buffer must be a 1-band image buffer and the destination buffer must be a 3-band image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LCH_TO_SRGB	Specifies an LCH to sRGB conversion. The LCH color space is similar to the LAB color space, with an emphasis on polar coordinates (absolute black and absolute white). LCH is based on luminance (L), chroma (C), and hue (H). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LCH_TO_SRGB_LINEAR	Specifies an LCH to linear sRGB conversion. The LCH color space is similar to the LAB color space, with an emphasis on polar coordinates (absolute black and absolute white). LCH is based on luminance (L), chroma (C), and hue (H). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB_NORMALIZE	Specifies an RGB-to-normalized-RGB conversion. Each band of every pixel is scaled to a value between 0 and 1(inclusive), for example, the normalized R band is calculated according to $R_Normalized = R / (R + G + B)$. To do so, the sum of the RGB bands of each pixel is computed, and then, each band for that pixel is divided by that sum. If the destination buffer is an integer buffer, this value is then rescaled between 0 and the depth of the buffer. Both the source and destination image buffers must be 3-band image buffers. This normalizing of the RGB color space can help remove unwanted variations in shades from an image.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB_TO_H	Specifies an RGB to H conversion, where H represents the hue of the HSL color space. The source image must be a 3-band image buffer and the destination buffer must be a 1-band image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB_TO_HSL	Specifies an RGB to HSL (hue, luminance (intensity), and saturation) conversion. Both the source and destination image buffers must be 3-band image buffers. The hue (H) is stored in band 0, the luminance (L) is stored in band 2, and the saturation (S) is stored in band 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB_TO_HSV	Specifies an RGB to HSV (hue, saturation, and value) conversion. Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b

M_RGB_TO_L	Specifies an RGB to L conversion, where L represents the luminance (intensity) of the HSL color space. The source image must be a 3-band image buffer and the destination buffer must be a 1-band image buffer. Note that M_RGB_TO_L and M_RGB_TO_Y produce similar results but not identical results because they are calculated to different color spaces.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB_TO_Y	Specifies an RGB to Y conversion, where Y represents the luminance of the YUV color space. The source buffer must have 3 bands. The destination buffer can have 1 or 3 bands. If it has 3 bands, only the first band is overwritten. Note that M_RGB_TO_L and M_RGB_TO_Y produce similar results but not identical results because they are calculated to different color spaces.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRGB_LINEAR_TO_LAB	Specifies a linear sRGB to LAB (CIELAB) conversion. The LAB color space is based on the color's luminance (L), its position between red and green (A), and its position between yellow and blue (B). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRGB_LINEAR_TO_LCH	Specifies a linear sRGB to LCH conversion. The LCH color space is similar to the LAB color space, with an emphasis on polar coordinates (absolute black and absolute white). LCH is based on luminance (L), chroma (C), and hue (H). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRGB_TO_LAB	Specifies an sRGB to LAB (CIELAB) conversion. The LAB color space is based on the color's luminance (L), its position between red and green (A), and its position between yellow and blue (B). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRGB_TO_LCH	Specifies an sRGB to LCH conversion. The LCH color space is similar to the LAB color space, with an emphasis on polar coordinates (absolute black and absolute white). LCH is based on luminance (L), chroma (C), and hue (H). Both the source and destination image buffers must be 3-band image buffers.	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies a general matrix multiplication transform will be used to modify the source image's color information. The matrix multiplication is performed using the provided 1-band, 32-bit floating-point buffer allocated using MbufAlloc2d() with M_ARRAY. You should use MbufPut() to define the values of the matrix. The source image buffer must be a 3-band buffer. The computation is shown below. **M_ARRAY buffer Source pixel Destination pixel** $\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \cdot \begin{bmatrix} Src_{band0} \\ Src_{band1} \\ Src_{band2} \end{bmatrix} = \begin{bmatrix} Dest_{band0} \\ Dest_{band1} \\ Dest_{band2} \end{bmatrix}$ $\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix} \cdot \begin{bmatrix} Src_{band0} \\ Src_{band1} \\ Src_{band2} \\ 1 \end{bmatrix} = \begin{bmatrix} Dest_{band0} \\ Dest_{band1} \\ Dest_{band2} \end{bmatrix}$ $\begin{bmatrix} a & b & c \end{bmatrix} \cdot \begin{bmatrix} Src_{band0} \\ Src_{band1} \\ Src_{band2} \end{bmatrix} = \begin{bmatrix} Dest_{band0} \end{bmatrix}$ $\begin{bmatrix} a & b & c & d \end{bmatrix} \cdot \begin{bmatrix} Src_{band0} \\ Src_{band1} \\ Src_{band2} \\ 1 \end{bmatrix} = \begin{bmatrix} Dest_{band0} \end{bmatrix}$ When the provided array buffer is a 3x3 matrix or a 4x3 matrix (with optional offset values), the destination should be a 3-band buffer. When the provided array buffer is a 3x1 matrix or a 4x1 matrix (with an optional offset value), the destination should be a 1-band buffer. The final values are clipped (saturated) according to the destination buffer's bit depth.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimConvolve

Synopsis

Perform a general convolution operation.

Syntax

```
void MimConvolve(
    AIL_ID SrcImageBufId,               //in
    AIL_ID DstImageBufId,               //out
    AIL_ID FilterContextImOrKernelBufId //in
)
```

Description

This function performs a general convolution operation on the source buffer using the specified filter, storing results in the specified destination buffer. This function supports two types of filters: Finite Impulse Response (FIR) filters and Infinite Impulse Response (IIR) filters. FIR filters operate on a finite neighborhood, while IIR filters take into account all values in an image. You can specify either a predefined FIR filter or a custom FIR or IIR filter.

Note that predefined FIR filters are the fastest type, and are recommended as the first to try if you are not sure which filter will best suit your needs. When using custom filters, relatively small FIR filters are faster and allow control over the kernel contents. If the FIR kernel that you require becomes too large, try an IIR filter. If speed is not an issue, try the adaptive filters available using MimFilterAdaptive(), which remove noise in an image while preserving edges as much as possible.

This function provides a number of predefined FIR filters. For predefined FIR filters, the overscan is set to transparent overscan pixel values, saturation is enabled for results, and the kernel's default center pixel is set as the top-left pixel of the central elements in a neighborhood. When using predefined FIR filters, you cannot change operation control settings except for the overscan.

This function also accepts a custom FIR or IIR filter with which you can customize operation control settings. To define a custom FIR filter, you must allocate a kernel buffer using MbufAlloc1d() or MbufAlloc2d(). Note that the kernel buffer size can be constrained by the available resources. To define a custom IIR filter, you must allocate a linear IIR filter image processing context, using MimAlloc() with M_LINEAR_FILTER_IIR_CONTEXT.

To define a custom FIR filter, you must load the kernel buffer with values, using MbufPut(). When using a custom FIR filter, you can change operation control settings using MbufControl(). To define a custom IIR filter, use MimControl() with a linear IIR filter image processing context and specify appropriate M_FILTER_TYPE, M_FILTER_OPERATION, M_FILTER_RESPONSE_TYPE, M_FILTER_SMOOTHNESS_TYPE, and M_FILTER_SMOOTHNESS operation control type settings.

For predefined and custom FIR filters, when using 32-bit integer buffers and saturation is disabled, the accumulator buffer is a 32-bit internal buffer (AIL_INT); whereas, when saturation is enabled, the accumulator buffer is a 64-bit floating-point internal buffer (double). For custom IIR filters, operations are always performed using floating-point precision.

For custom FIR filters, this function will internally separate a large kernel into two 1-dimensional kernels, if possible, to increase the speed of the convolution operation. For more information on separating kernels, see the Finite Impulse Response (FIR) filters subsection of the Custom spatial filters section of Chapter 4: Advanced image processing.

Custom FIR filters containing a single kernel value (such as uniform kernels or average kernels) are dealt with by a specifically targeted algorithmic optimization.

If the source and destination are multi-band buffers, the same filter is applied to every band of the source buffer.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

If performing an IIR derivative filter operation and M_FILTER_STEP_RESPONSE_MAPPING is set to M_ACCORDING_TO_DEST, the source buffer cannot be a floating-point buffer; otherwise, an error will be generated.

Note that for an FIR filter, the size of the image buffer must be greater or equal to the size of the kernel.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

If performing an IIR derivative filter operation, the destination buffer must be a signed or floating-point buffer. If M_FILTER_STEP_RESPONSE_MAPPING is set to M_ACCORDING_TO_DEST, the destination buffer is further restricted and cannot be a floating-point buffer; otherwise, an error will be generated.

Note that for an FIR filter, the size of the image buffer must be greater or equal to the size of the kernel.

FilterContextImOrKernelBufId

Specifies the filter to use. Set this parameter to a predefined filter, the identifier of the kernel buffer which defines a custom FIR filter, or a linear IIR filter image processing context, which defines a custom IIR filter.

To specify a custom filter, set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a custom filter																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Kernel Buffer ID	Specifies the identifier of the kernel buffer for a custom FIR filter, allocated using MbufAlloc...() with M_KERNEL. For more details, refer to the Custom spatial filters section of Chapter 4: Advanced image processing.	c		e	f	g	a		i		j	k	l	m	n	b
Linear IIR filter image processing context ID	Specifies a linear IIR filter image processing context for a custom IIR filter, allocated using MimAlloc() with M_LINEAR_FILTER_IIR_CONTEXT. Set up the context using MimControl().	c		e	f	g	a		i		j	k	l	m	n	b

To specify a predefined FIR filter, set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a predefined FIR filter																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EDGE_DETECT_PREWITT_FAST +	Computes the gradient of the image using the following operation, which is based on a Prewitt filter. $\left(\left\lvert\begin{bmatrix}-1 & -1 & -1\\ 0 & 0 & 0\\ 1 & 1 & 1\end{bmatrix}\right\rvert + \left\lvert\begin{bmatrix}-1 & 0 & 1\\ -1 & 0 & 1\\ -1 & 0 & 1\end{bmatrix}\right\rvert\right)/2$	c		e	f	g	a		i		j	k	l	m	n	b
M_EDGE_DETECT_SOBEL_FAST +	Computes the gradient of the image using the following operation, which is based on a Sobel filter. $\left(\left\lvert\begin{bmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{bmatrix}\right\rvert + \left\lvert\begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}\right\rvert\right)/2$	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL_EDGE_PREWITT +	Computes the absolute values of the vertical derivative of the image using a Prewitt filter, to detect the horizontal edges. $\left\lvert\begin{bmatrix}-1 & -1 & -1\\ 0 & 0 & 0\\ 1 & 1 & 1\end{bmatrix}\right\rvert$	c		e	f	g	a		i		j	k	l	m	n	b

M_HORIZONTAL_EDGE_SOBEL +	Computes the absolute values of the vertical derivative of the image using a Sobel filter, to detect the horizontal edges. $\left\lvert\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}\right\rvert$	c		e	f	g	a		i		j	k	l	m	n	b
M_LAPLACIAN_4 +	Computes the Laplacian values of the image using the following 4-connected Laplacian edge detection filter. $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_LAPLACIAN_8 +	Computes the Laplacian values of the image using the following 8-connected Laplacian edge detection filter. $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_LAPLACIAN_ISO_8 +	Computes the Laplacian values of the image using the following 8-connected isotropic Laplacian edge detection filter. $\begin{bmatrix} -1 & -2 & -1 \\ -2 & 12 & -2 \\ -1 & -2 & -1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_PREWITT_X +	Computes the values of the horizontal derivative of the image using a Prewitt filter. $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_PREWITT_Y +	Computes the values of the vertical derivative of the image using a Prewitt filter. $\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_SHARPEN_4 +	Performs a sharpening operation on the image using the following 4-connected Laplacian filter combined with the original image. Note that this operation results in a sharpening effect that is not as strong as the one achieved with M_SHARPEN_8. $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_SHARPEN_8 +	Performs a sharpening operation on the image using the following 8-connected Laplacian filter combined with the original image. Note that this operation results in a stronger sharpening effect than M_SHARPEN_4. $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_SMOOTH +	Performs a smoothing operation on the image using the following smoothing filter. $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} /16$	c		e	f	g	a		i		j	k	l	m	n	b

M_SOBEL_X +	Computes the values of the horizontal derivative of the image using a Sobel filter. $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_SOBEL_Y +	Computes the values of the vertical derivative of the image using a Sobel filter. $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL_EDGE_PREWITT +	Computes the absolute values of the horizontal derivative of the image using a Prewitt filter, to detect the vertical edges. $\left\lvert \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \right\rvert$	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL_EDGE_SOBEL +	Computes the absolute values of the horizontal derivative of the image using a Sobel filter, to detect the vertical edges. $\left\lvert \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \right\rvert$	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying a predefined FIR filter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to determine the value of a destination pixel when its associated point falls outside the source buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_DISABLE	Specifies to disable overscanning for this operation. You should disable overscanning to accelerate the operation for applications in which the overscan data is not important in the resulting buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Specifies to enable overscanning for this operation. The overscan uses pixels from the ancestor of the source buffer. If the source buffer is not a child buffer, or if the point falls outside the ancestor buffer, a mirror type overscan is used. A mirror overscan specifies that the overscan pixels will be a mirror copy of the source buffer's border pixels. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimCountDifference

Synopsis

Count the number of pixels that differ in each image.

Syntax

```
AIL_INT MimCountDifference(
    AIL_ID Src1ImageBufId, //in
    AIL_ID Src2ImageBufId, //in
    AIL_ID CountResultImId //out
)
```

Description

This function finds the number of differences between the two specified source buffers and stores the resulting number in the specified result buffer.

You can read the number of differences from the result buffer, using MimGetResult1d() or MimGetResult(), specifying M_VALUE as the result type.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

Src1ImageBufId

Specifies the identifier of the first image data source. This parameter can be given an image buffer identifier. The image buffer must be 1-band. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Src2ImageBufId

Specifies the identifier of the second image data source. This parameter can only be given an image buffer identifier. The image buffer must be 1-band. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

CountResultImId

Specifies the identifier of the buffer in which to store the differences. This parameter must be given the identifier of an image processing result buffer that was allocated with MimAllocResult() and has an M_COUNT_LIST type. The buffer needs only one entry.

You can set this parameter to **M_NULL**, since the function also returns the number of differences between the two specified source buffers.

Return value

If CountResultImId is **M_NULL**, the returned value is the number of differences between the two specified source buffers. If CountResultImId is not **M_NULL**, the returned value is **M_INVALID**. In that case, use either MimGetResult1d() or MimGetResult(), with M_VALUE as the result type, to retrieve the number of differences.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDeadPixelCorrection

Synopsis

Correct dead pixels in an image.

Syntax

```
void MimDeadPixelCorrection(
    AIL_ID DeadPixelContextImId, //in
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageBufId,        //out
    AIL_INT64 ControlFlag        //in
)
```

Description

This function corrects dead pixels in the specified source image and places the resulting image in the destination image buffer. Dead pixels are replaced with an average of their non-dead pixel neighbors. Other pixels are copied from the source image buffer into the destination image buffer.

Before using this function, you must add the list of the coordinates of the dead pixels to the context, using MimPut() and two arrays (one for the X-coordinates and one for the Y-coordinates). Alternatively, you can add an image identifying the locations of the dead pixels to the context, using MimControl() with M_DEAD_PIXELS; this image acts as a dead pixel mask. In the dead pixel mask image, all non-zero pixels are considered dead pixels.

Specify the interpolation mode using MimControl() with M_INTERPOLATION_MODE. By default, the interpolation mode is M_AVERAGE.

The source and destination image buffers must be as large as (or larger than) the dead pixel mask, or equal to or larger than the largest specified coordinates of the dead pixels.

Once the dead pixel correction image processing context is configured, preprocess the context by calling **MimDeadPixelCorrection**() with M_PREPROCESS. If the preprocess operation was not done explicitly (using M_PREPROCESS), it will be done when **MimDeadPixelCorrection**() is first called. When preprocessed explicitly, the source and destination image buffers can be set to **M_NULL**. If, however, the source or destination image buffer is provided, it will be used in the preprocess operation, to better optimize future calls.

Parameters

DeadPixelContextImId

Specifies the identifier of the dead pixels image processing context. The context must have been allocated using MimAlloc() with M_DEAD_PIXEL_CONTEXT. In addition, the dead pixel image processing context must be already configured with the location of the dead pixels.

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer must be a single-band image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

If performing a preprocess operation on the image processing context, set this parameter to the identifier of a typical source image buffer. Otherwise, to exclude this buffer from preprocessing, set this parameter to **M_NULL**.

DstImageBufId

Specifies the identifier of the destination image buffer. The buffer must be a single-band image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

If performing a preprocess operation on the image processing context, set this parameter to the identifier of a typical destination image buffer. Otherwise, to exclude this buffer from preprocessing, set this parameter to **M_NULL**.

ControlFlag

Specifies the control flag. This parameter must be set to one of the following:

l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For preprocessing the dead pixel image processing context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Corrects the dead pixels present in the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Preprocesses the specified image processing context. Note that, if not called explicitly, this operation will be performed automatically upon the first call to **MimDeadPixelCorrection**(). If the identifier of the source and/or destination buffer is not set to **M_NULL**, information about the source and/or destination image buffer will be used in the preprocessing operation.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDeinterlace

Synopsis

Produce a sequence of deinterlaced images from a sequence of images acquired from an interlaced camera.

Syntax

```
void MimDeinterlace(
    AIL_ID DeinterlaceContextImId,  //in
    const AIL_ID *SrcImageArrayPtr, //in
    const AIL_ID *DstImageArrayPtr, //in
    AIL_INT SrcImageCount,          //in
    AIL_INT DstImageCount,          //in
    AIL_INT64 ControlFlag           //in
)
```

Description

This function produces a sequence of deinterlaced images from a sequence of images grabbed from an interlaced camera. When an image is acquired using an interlaced camera, the even and odd fields are not taken at the exact same moment in time. If the object in the image was in motion, there is an offset between the position of the object in one field and its position in the other. Simply combining these two fields to produce a deinterlaced image results in noticeable deformities in moving objects, called interlacing artifacts. **MimDeinterlace**() uses averaging techniques to reduce or remove these interlacing artifacts and produce a higher quality deinterlaced image. To perform the deinterlacing operation, you can select one of several deinterlacing algorithms using MimControl() with M_DEINTERLACE_TYPE. They can be applied to the entire image or only to the pixels that are considered part of a moving object. To apply the algorithm to the latter, select the adaptive version of the algorithm (for example, M_ADAPTIVE_DISCARD).

Often, the order in which the fields are grabbed will depend on the camera being used. When using an algorithm that produces two output frames (for example, M_BOB or M_ADAPTIVE_BOB), it is important to set the field that is grabbed first using MimControl() with M_FIRST_FIELD. This will ensure that the sequence of output frames occur in chronological order and that the video stream is fluid.

The adaptive algorithms apply a motion detection algorithm to dynamically determine which pixels are part of an object in motion and which pixels are part of the background, prior to performing the deinterlacing algorithm. Using MimControl(), you can set the number of neighboring frames used for motion detection (M_MOTION_DETECT_NUM_FRAMES), the location of the frame to be processed within this group of frames (M_MOTION_DETECT_REFERENCE_FRAME), and the threshold value used to differentiate between pixels of objects in motion and pixels of background objects (M_MOTION_DETECT_THRESHOLD). To visualize which pixels are affected, you can have **MimDeinterlace**() output an image of these pixels, using MimControl() with M_MOTION_DETECT_OUTPUT; pixels that are part of an object in motion are set to the maximum value (0xFF for an a bit image), while the other pixels are set to 0. In this case, the deinterlacing part of the adaptive algorithm is not executed.

Parameters

DeinterlaceContextImId

Specifies the identifier of the image processing context to be used for deinterlacing. The image processing context must have been previously allocated on the system using MimAlloc().

SrcImageArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1) Required array size:(SrcImageCount)

Specifies an array containing the identifiers of the buffers of the images to deinterlace. Only 8-bit and 16-bit buffers are supported. All source images must be of the same type, same format, and on the same system. Note that all source and destination images must have the same size.

These images must not have a region of interest (ROI) associated with them. Using images with an ROI will cause an error.

DstImageArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>] (and a maximum array rank for C# of 1)

Required array size:(SrcImageCount)

Specifies an array containing the identifiers of the buffers of the images in which to save the deinterlaced images. Only 8-bit and 16-bit buffers are supported. All destination images must be of the same type, same format, and on the same system. Note that all source and destination images must have the same size.

These images must not have a region of interest (ROI) associated with them. Using images with an ROI will cause an error.

SrcImageCount

Specifies the number of images in the source sequence.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcImageArrayPtr parameter.

DstImageCount

Specifies the number of deinterlaced images in the destination sequence.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the DstImageArrayPtr parameter.

ControlFlag

Reserved for future use. Should be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDetectEvent

Synopsis

Detect if at least one of the pixels in the source image satisfies the specified condition.

Syntax

```
AIL_INT MimDetectEvent(
    AIL_ID SrcImageBufId, //in
    AIL_INT64 Condition,  //in
    AIL_DOUBLE CondLow,   //in
    AIL_DOUBLE CondHigh   //in
)
```

Description

This function detects if at least one of the pixels in the source image satisfies the specified condition. To establish the location of the event, you can use MimLocateEvent().

You can limit this function's result to a region of the image buffer using a region of interest set using MbufSetRegion().

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer must be a 1-band image buffer.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Condition

Specifies the condition under which pixel values are considered an event.

For conditions that use two limits, set the Condition parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For conditions that use two limits																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IN_RANGE	Specifies that pixels with values between CondLow and CondHigh, inclusive, are considered events.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_RANGE	Specifies that pixels with values less than CondLow, or greater than CondHigh, are considered events.	c		e	f	g	a		i		j	k	l	m	n	b

For conditions that use one limit, set the Condition parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For conditions that use one limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL	Specifies that only pixels with values equal to CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Specifies that only pixels with values greater than CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Specifies that only pixels with values greater than or equal to the CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b

M_LESS	Specifies that only pixels with values less than CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Specifies that only pixels with values less than or equal to CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_EQUAL	Specifies that only pixels with values not equal to CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b

CondLow

Specifies the lower limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the lower limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value	Specifies the lower limit.	c		e	f	g	a		i		j	k	l	m	n	b

CondHigh

Specifies the upper limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the upper limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no upper limit is required. This setting should be selected for conditions that use one limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is **M_TRUE** if a pixel with the specified condition is found and **M_FALSE** otherwise.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDifferential

Synopsis

Perform a differential operation.

Syntax

```
void MimDifferential(
    AIL_ID Drv1ImageBufIdOrFilterContextImId, //in
    AIL_ID SrcOrDrv2ImageBufId,               //in
    AIL_ID SrcOrDrv3ImageBufId,               //in
    AIL_ID Drv4ImageBufId,                    //in
    AIL_ID Drv5ImageBufId,                    //in
    AIL_ID Dst1ImageBufId,                    //out
    AIL_ID Dst2ImageBufId,                    //out
    AIL_DOUBLE Param,                         //in
    AIL_INT64 Operation,                      //in
    AIL_INT64 ControlFlag                     //in
)
```

Description

This function performs a differential operation on an image, based on either automatically calculated or specified derivative images. This function can be used, for example, to calculate the gradient magnitude and orientation (angle) of edges in an image, or to sharpen an image.

You can have **MimDifferential**() automatically calculate the required derivatives (in X and Y) of the source image by passing a linear IIR filter image processing context (previously allocated using MimAlloc() with M_LINEAR_FILTER_IIR_CONTEXT). **MimDifferential**() calculates the derivatives using IIR filters. You can partially define how the derivatives are calculated by configuring the IIR filter image processing context using MimControl(). **MimDifferential**() ignores the M_FILTER_OPERATION setting.

You can have **MimDifferential**() use precalculated derivatives by passing derivative image buffers: most operations require one image buffer with the derivative in X, and another with the derivative in Y (for the first derivatives, this is synonymous with passing the vertical and horizontal edge detection images, respectively). Passing precalculated derivatives is useful if you need to calculate the derivatives using a specific filter operation (for example, using MimConvolve() with a predefined FIR filter or a custom kernel), or use the derivatives with multiple functions (in which case, it is more efficient to calculate the derivatives only once).

> Note that some operations require the first derivative images, while others require the second derivative images.

Parameters

Drv1ImageBufIdOrFilterContextImId

Specifies the identifier of an image buffer that stores a derivative, or a linear IIR filter context. The image buffer must not have a region of interest (ROI) associated with it. Specifying an image buffer with an ROI will cause an error.

See the Parameter associations section for possible values that can be specified.

SrcOrDrv2ImageBufId

Specifies the identifier of an image buffer that stores the source image, or a derivative. The image buffer must not have a region of interest (ROI) associated with it. Specifying an image buffer with an ROI will cause an error.

See the Parameter associations section for possible values that can be specified.

SrcOrDrv3ImageBufId

Specifies the identifier of an image buffer that stores the source image (for a sharpening operation), or a derivative. The image buffer must not have a region of interest (ROI) associated with it. Specifying an image buffer with an ROI will cause an error.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Drv4ImageBufId

Reserved for future expansion and must be set to **M_NULL**.

Drv5ImageBufId

Reserved for future expansion and must be set to **M_NULL**.

Dst1ImageBufId

Specifies the identifier of the first destination image buffer. The image buffer must not have a region of interest (ROI) associated with it. Specifying an image buffer with an ROI will cause an error.

Set this parameter to **M_NULL** if not used. In this case, you cannot set Dst2ImageBufId to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Dst2ImageBufId

Specifies the identifier of the second destination image buffer. The image buffer must not have a region of interest (ROI) associated with it. Specifying an image buffer with an ROI will cause an error.

Set this parameter to **M_NULL** if not used. In this case, you cannot set Dst1ImageBufId to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Param

Specifies an attribute of the operation to perform. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the differential operation to perform.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies a control setting for the operation to perform. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **Drv1ImageBufIdOrFilterContextImId**, **SrcOrDrv2ImageBufId**, **SrcOrDrv3ImageBufId**, **Param**, **Operation**, and **ControlFlag** parameters and possible values returned to the **Dst1ImageBufId** and **Dst2ImageBufId** parameters.

- For specifying the differential operation

Set unused source and destination parameters to **M_NULL**. Set other unused parameters to **M_DEFAULT**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the differential operation																		
Operation			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Drv1ImageBufIdOrFilterContextImId																	
		SrcOrDrv2ImageBufId																
		SrcOrDrv3ImageBufId																
		Dst1ImageBufId - Possible values returned																
		Dst2ImageBufId - Possible values returned																
		Param																
		ControlFlag																
M_GRADIENT			Performs a gradient operation.															

Drv1ImageBufIdOrFilterContextImId - Image buffer ID		Specifies the identifier of a float or signed image buffer, indicating the first derivative in X.	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv2ImageBufId	Specifies the identifier of a float or signed image buffer, indicating the first derivative in Y.	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Dst1ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the gradient magnitude; that is: sqrt(*Ix*Ix+Iy*Iy*). To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst2ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Dst2ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the gradient orientation. For an unsigned destination buffer, the angle is returned from 0° to 360° (counter-clockwise) and mapped to the entire range of the destination buffer. For example, for an 8-bit unsigned buffer, the angles of 0°, 90°, 180°, and 270° map to 0, 64, 128, and 192, respectively. For a signed destination buffer, mapping is done in both the positive and negative range of the buffer and represents angle values from -180° to 180°. For example, for an 8-bit signed buffer, the angles of -180°, -90°, 0°, and 90° map to -128, -64, 0 and 64, respectively. For a floating-point buffer, the results are not mapped and are returned in the range of 0° to 360°. > Note that gradient angle values are meaningless at positions where the gradient magnitude is 0. To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst1ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Drv1ImageBufIdOrFilterContextImId - Linear IIR filter context ID		Specifies the identifier of a linear IIR filter context (M_LINEAR_FILTER_IIR_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv2ImageBufId	Specifies the identifier of the source image buffer (of any type) from which to extract the derivatives and compute the gradient.	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Dst1ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the gradient magnitude; that is: sqrt(*Ix*Ix+Iy*Iy*). To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst2ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

		Dst2ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the gradient orientation. For an unsigned destination buffer, the angle is returned from 0° to 360° (counter-clockwise) and mapped to the entire range of the destination buffer. For example, for an 8-bit unsigned buffer, the angles of 0°, 90°, 180°, and 270° map to 0, 64, 128, and 192, respectively. For a signed destination buffer, mapping is done in both the positive and negative range of the buffer and represents angle values from -180° to 180°. For example, for an 8-bit signed buffer, the angles of -180°, -90°, 0°, and 90° map to -128, -64, 0 and 64, respectively. For a floating-point buffer, the results are not mapped and are returned in the range of 0° to 360°. > Note that gradient angle values are meaningless at positions where the gradient magnitude is 0. To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst1ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRADIENT_SQR			Performs a square gradient operation.	c		e	f	g	a		i		j	k	l	m	n	b
	Drv1ImageBufIdOrFilterContextImId - Image buffer ID		Specifies the identifier of a float or signed image buffer, indicating the first derivative in X.	c		e	f	g	a		i		j	k	l	m	n	b
		SrcOrDrv2ImageBufId	Specifies the identifier of a float or signed image buffer, indicating the first derivative in Y.	c		e	f	g	a		i		j	k	l	m	n	b
		SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
		Dst1ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the square gradient magnitude; that is: *Ix*Ix+Iy*Iy* . To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst2ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
		Dst2ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the square gradient orientation. For an unsigned destination buffer, the angle is returned from 0° to 360° (counter-clockwise) and mapped to the entire range of the destination buffer. For example, for an 8-bit unsigned buffer, the angles of 0°, 90°, 180°, and 270° map to 0, 64, 128, and 192, respectively. For a signed destination buffer, mapping is done in both the positive and negative range of the buffer and represents angle values from -180° to 180°. For example, for an 8-bit signed buffer, the angles of -180°, -90°, 0°, and 90° map to -128, -64, 0 and 64, respectively. For a floating-point buffer, the results are not mapped and are returned in the range of 0° to 360°. > Note that gradient angle values are meaningless at positions where the gradient magnitude is 0. To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst1ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Drv1ImageBufIdOrFilterContextImId - Linear IIR filter context ID		Specifies the identifier of a linear IIR filter context (M_LINEAR_FILTER_IIR_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv2ImageBufId	Specifies the identifier of an image buffer (of any type) from which to extract the derivatives.	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Dst1ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the square gradient magnitude; that is: *Ix*Ix+Iy*Iy* . To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst2ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Dst2ImageBufId	Specifies the identifier of a float or unsigned image buffer in which to write the results of the square gradient orientation. For an unsigned destination buffer, the angle is returned from 0° to 360° (counter-clockwise) and mapped to the entire range of the destination buffer. For example, for an 8-bit unsigned buffer, the angles of 0°, 90°, 180°, and 270° map to 0, 64, 128, and 192, respectively. For a signed destination buffer, mapping is done in both the positive and negative range of the buffer and represents angle values from -180° to 180°. For example, for an 8-bit signed buffer, the angles of -180°, -90°, 0°, and 90° map to -128, -64, 0 and 64, respectively. For a floating-point buffer, the results are not mapped and are returned in the range of 0° to 360°. > Note that gradient angle values are meaningless at positions where the gradient magnitude is 0. To not return this information, set this parameter to **M_NULL**. In this case, you cannot set Dst1ImageBufId to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAPLACIAN		Performs a Laplacian operation.	c		e	f	g	a		i		j	k	l	m	n	b
Drv1ImageBufIdOrFilterContextImId - Image buffer ID		Specifies the identifier of a float or signed image buffer, indicating the second derivative in X.	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv2ImageBufId	Specifies the identifier of a float or signed image buffer, indicating the second derivative in Y.	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Dst1ImageBufId	Specifies the identifier of a float or signed image buffer in which to write the results of the Laplacian operation; that is: *Ixx+Iyy* .	c		e	f	g	a		i		j	k	l	m	n	b
Drv1ImageBufIdOrFilterContextImId - Linear IIR filter context ID		Specifies the identifier of a linear IIR filter context (M_LINEAR_FILTER_IIR_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b
	SrcOrDrv2ImageBufId	Specifies the identifier of an image buffer (of any type) from which to extract the derivatives.	c		e	f	g	a		i		j	k	l	m	n	b

SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Dst1ImageBufId	Specifies the identifier of a float or signed image buffer in which to write the results of the Laplacian operation; that is: *Ixx+Iyy* .	c		e	f	g	a		i		j	k	l	m	n	b
M_SHARPEN	Performs a sharpening operation.	c		e	f	g	a		i		j	k	l	m	n	b
Drv1ImageBufIdOrFilterContextImId - Image buffer ID	Specifies the identifier of a float or signed image buffer, indicating the second derivative in X.	c		e	f	g	a		i		j	k	l	m	n	b
SrcOrDrv2ImageBufId	Specifies the identifier of a float or signed image buffer, indicating the second derivative in Y.	c		e	f	g	a		i		j	k	l	m	n	b
SrcOrDrv3ImageBufId	Specifies the identifier of an image buffer (of any type) that stores the image to sharpen.	c		e	f	g	a		i		j	k	l	m	n	b
Dst1ImageBufId	Specifies the identifier of an image buffer (of any type) in which to write the results of the sharpening operation; that is: *Is- alpha*(Ixx*Iyy*). If you use an unsigned image buffer, the sharpening operation performs a saturation.	c		e	f	g	a		i		j	k	l	m	n	b
Dst2ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the amount of sharpening.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the sharpening value.	c		e	f	g	a		i		j	k	l	m	n	b
Drv1ImageBufIdOrFilterContextImId - Linear IIR filter context ID	Specifies the identifier of a linear IIR filter context (M_LINEAR_FILTER_IIR_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b
SrcOrDrv2ImageBufId	Specifies the identifier of an image buffer (of any type) from which to extract the derivatives.	c		e	f	g	a		i		j	k	l	m	n	b
SrcOrDrv3ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Dst1ImageBufId	Specifies the identifier of an image buffer (of any type) in which to write the results of the sharpening operation; that is: *Is-alpha*(Ixx*Iyy*). If you use an unsigned image buffer, the sharpening operation performs a saturation.	c		e	f	g	a		i		j	k	l	m	n	b
Dst2ImageBufId	Set this parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param	Specifies the amount of sharpening.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that Aurora Imaging Library will establish an appropriate sharpening value.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the sharpening value.	c		e	f	g	a		i		j	k	l	m	n	b

		ControlFlag		Specifies whether to perform the sharpening operation, or to explicitly preprocess the linear IIR filter context.	c		e	f	g	a		i		j	k	l	m	n	b
			M_DEFAULT	Specifies to perform the sharpening operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_PREPROCESS	Specifies to preprocesses the linear IIR filter context explicitly. Note, if not called explicitly, Aurora Imaging Library performs this operation automatically upon the first call to **MimDifferential**().	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDilate

Synopsis

Perform a binary or grayscale dilation-type morphological operation.

Syntax

```
void MimDilate(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbIteration,  //in
    AIL_INT64 ProcMode    //in
)
```

Description

This function performs a binary or grayscale dilation on the given source image for the specified number of iterations.

In binary mode, this function uses a 3x3 full rectangular structuring element; in grayscale mode, a 3x3 empty one.

The overscan pixels are automatically set to the lowest possible buffer value, which will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

NbIteration

Specifies the number of times to iterate the operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of iterations to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that each non-zero (white) blob should be dilated until its background is about to disappear. This value is only available when ProcMode is set to M_BINARY_ULTIMATE or M_BINARY_ULTIMATE_ACCUMULATE. M_DEFAULT does not specify an explicit number of iterations to perform, instead it indicates to the function that the blobs should be dilated until black holes and the background are about to disappear. For example, if M_DEFAULT is specified and you have two rectangular blobs with two black holes (120x6 and 150x17), the dilation will continue until the black holes are reduced to 116x2 and 134x1, respectively (the smallest they can be before they disappear), and the background is reduced to the point it is about to disappear. > Setting NbIteration to M_DEFAULT with any processing mode other than M_BINARY_ULTIMATE or M_BINARY_ULTIMATE_ACCUMULATE will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Value > = 0	Specifies the number of iterations. When this parameter is set to 0 and ProcMode is set to M_BINARY, the source image is binarized and the result is copied into the destination image buffer. When this parameter is set to 0 and ProcMode is set to M_GRAYSCALE, the source image is copied into the destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ProcMode

Specifies the processing mode to use. This parameter can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Performs a binary dilation on the white (non-zero) blobs in the image. In this case, non-zero pixels are treated as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY_ULTIMATE	Performs a binary dilation on the white (non-zero) blobs, while preventing the black (zero) holes and background from completely disappearing. When in this processing mode, this operation performs a binary dilation until the number of iterations NbIteration is reached or until the black holes and background are about to disappear. If NbIteration is not set to **M_DEFAULT**, the specified number of iterations serves as an upper bound limit, and the procedure continues until the specified number of iterations is reached or the black holes and background are about to disappear, whichever comes first. If NbIteration is set to **M_DEFAULT**, the procedure continues until the black holes and background are about to disappear. > Note that if the dilation merges any blobs, the process continues on the merged blob. The holes and the background, however, are still evaluated individually.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY_ULTIMATE_ACCUMULATE	Performs a binary erosion on the white (non-zero) blobs in the negative of the image, while preventing these blobs from completely disappearing and keeping track of the number of iterations. > Note that the negative of the image is taken and the operation is switched to an erosion, so that each remaining pixel in a hole or the background can track the number of iterations needed to reach the current state. The resulting non-zero pixels will be set to the iteration number + 1. For example, consider an erosion with NbIteration set to 2. After the first iteration, the resulting non-zero pixels will be set to 1 + 1 (iteration number) = 2. After the second iteration,the resulting non-zero pixels will have all bits set to 1 + 2 (iteration number) = 3. This also means that what was considered the foreground is now considered the background, represented in black (zero), and what was considered holes and the background is now non-zero (iteration count). When in this processing mode, this operation performs a binary erosion on the negative of the image until the number of iterations NbIteration is reached or until the blobs are about to disappear. If NbIteration is not set to **M_DEFAULT**, the specified number of iterations serves as an upper bound limit, and the procedure continues until the specified number of iterations is reached or until the blobs are about to disappear, whichever comes first. If NbIteration is set to **M_DEFAULT**, the procedure continues until the blobs are about to disappear. > Note that if the erosion splits any blobs, the process continues on each sub-blob, evaluating them separately. For more information on binary ultimate accumulate dilation, see the Erosion and dilation section of Chapter 3: Fundamental image processing.	c		e	f	g	a		i		j	k	l	m	n	b

M_GRAYSCALE	Performs a grayscale dilation on the white (non-zero) blobs. In this case, each pixel of the source image is replaced with the maximum value in its neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDistance

Synopsis

Perform a distance transformation.

Syntax

```
void MimDistance(
    AIL_ID SrcImageBufId,       //in
    AIL_ID DstImageBufId,       //out
    AIL_INT64 DistanceTransform //in
)
```

Description

This function determines the shortest distance between each blob pixel and the blob's background, and assigns this distance to the pixel. It produces a type of contour mapping of a blob.

Parameters

SrcImageBufId

Specifies the identifier of the source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DistanceTransform

Specifies the way in which the minimum distance from blob pixel to background pixel is calculated. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to calculate the minimum distance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CHAMFER_3_4	Approximates the minimum distance using horizontal, vertical or diagonal pixel steps. Horizontal and vertical steps are counted as 3; diagonal steps are counted as 4. Then, the resulting distance is normalized by a factor of 3. This transform provides the best approximation to Euclidean distance. This transform requires that the destination buffer be deep enough to hold a number at least three times the maximum distance from a blob pixel to its edge. For example, an 8-bit buffer (255 max) can be used for a maximum distance of 85 pixels and a 16-bit buffer (65535 max) for a maximum distance of 21845 pixels. 3x3 Distance Matrix: $\begin{bmatrix} 4 & 3 & 4 \\ 3 & 0 & 3 \\ 4 & 3 & 4 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_CHESSBOARD	Approximates the minimum distance using horizontal, vertical, or diagonal pixel steps. All steps count as 1. 3x3 Distance Matrix: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b

M_CITY_BLOCK	Approximates the minimum distance using only horizontal or vertical pixel steps. Horizontal and vertical steps count as 1. 3x3 Distance Matrix: $\begin{bmatrix} \infty & 1 & \infty \\ 1 & 0 & 1 \\ \infty & 1 & \infty \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b
M_EUCLIDEAN_DIST	Calculates the true minimum Euclidean distance using a straight-line path. The Euclidean distance is calculated using the following formula, where: $Euclidean\ distance = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ • x_1 and y_1 represent the coordinates of the blob pixel. • x_2 and y_2 represent the coordinates of the nearest background pixel. Note, the source buffer must be a 1-band, 8-bit image buffer of type M_UNSIGNED and the destination buffer must be a 1-band, 32-bit image buffer of type M_FLOAT.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimDraw

Synopsis

Draw the settings of an image processing context or the results of an image processing operation.

Syntax

```
void MimDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID Src1AilId,              //in
    AIL_ID Src2AilId,              //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_DOUBLE Param1,             //in
    AIL_DOUBLE Param2,             //in
    AIL_INT64 ControlFlag          //in
)
```

Description

The function draws the settings of an image processing context or the results of an Aurora Imaging Library image processing operation, into a destination image buffer.

> You can also use **MimDraw**() to draw the internal copy of a source image that was passed to MimControl() or an image processing function and saved in an image processing context or result buffer. Note that the drawn image might differ from the original source image (for example, in bit depth). This is because, for greater optimization, an image processing context or result buffer might store its internal image data in a different format than the original source image buffer.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use. This parameter must be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

Src1AilId

Specifies the identifier of the primary source image buffer, result buffer, or image processing context from which to extract the results to draw. The buffer must have been previously allocated on the required system, using MbufAlloc...(), MimAllocResult(), or MimAlloc(), respectively.

If an image buffer is specified, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

See the Parameter associations section for possible values that can be specified.

Src2AilId

Specifies the identifier of the secondary source image buffer for the drawing operation. The buffer must have been previously allocated on the required system using MbufAlloc...().

If this parameter is not required for the draw operation, it must be set to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

See the Parameter associations section for possible values that can be specified.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid Aurora Imaging Library image buffer allocated using MbufAlloc...(). However, when the source buffers are image buffers, the destination image buffer should be greater than or equal in size and bit depth to the source image buffers. If the destination image buffer is smaller than the source image buffers, the excess will not be part of the destination image.

If drawing an internal image contained in an image processing context or result buffer, the same destination buffer rule applies. In this case, you can use MimInquire() to find the size, depth, and other attributes of the internal image to correctly set the destination buffer.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

An operation can draw in a 2D graphics list only if it is specified in the operation's description. If you specify a 2D graphics list, the 2D graphics list must have been previously allocated using MgraAllocList(). You cannot use a 2D graphics list when drawing with wavelet transformations.

Operation

Specifies the type of operation to perform.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the operation to perform.

If this parameter is not required for the draw operation, it must be set to 0.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the operation to perform.

If this parameter is not required for the draw operation, it must be set to 0.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies drawing constraints. You must set this parameter to M_DEFAULT unless you are drawing MimLocatePeak1d() or MimWaveletTransform() results.

To specify the default drawing behavior, set this parameter to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the default drawing behavior																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default behavior. Except when drawing MimLocatePeak1d() or MimWaveletTransform() results, Aurora Imaging Library ignores this parameter. When drawing MimLocatePeak1d() results, the default is M_FIXED_POINT + n, where *n* = 0. When drawing MimWaveletTransform() results, the default is M_AUTO_SCALE.	c		e	f	g	a		i		j	k	l	m	n	b

To specify the number of fractional bits in the source values when using MimLocatePeak1d() results, set this parameter to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of fractional bits when using MimLocatePeak1d() results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIXED_POINT + *n*	Specifies the number of fractional bits in the source values, when they are in a fixed-point format. Set *n* to an *integer* between 0 and 7, inclusive.	c		e	f	g	a		i		j	k	l	m	n	b

To specify how to manage the range of pixel values when using MimWaveletTransform() results, set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to manage the range of pixel values when using MimWaveletTransform() results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AUTO_SCALE	Remaps pixel values (Src1AiId) according to the range of pixel values allowed in the destination (DstImageBufOrListGraId).	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Clips pixel values (Src1AiId) that are outside the range of pixel values allowed in the destination (DstImageBufOrListGraId).	c		e	f	g	a		i		j	k	l	m	n	b

Parameter associations

The tables below list possible values for the **Src1AiId**, **Src2AiId**, **Operation**, **Param1**, and **Param2** parameters.

- For specifying to perform the draw operation with the specified image processing context
- For specifying a draw operation that uses image processing results
- For specifying to perform the draw operation from image buffer(s) containing a depth map and/or intensity map
- For specifying to perform the draw operation with the result buffer(s) used in MimLocatePeak1d()

In the following table, Src1AiId specifies an image processing context and Operation specifies the operation to perform with the specified image processing context. Src2AiId is not required and must be set to **M_NULL**. Set Param1 and Param2 to 0, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

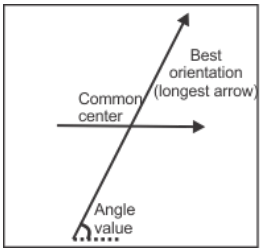
For specifying to perform the draw operation with the specified image processing context																
Src1AiId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Operation																
Src2AiId																
Param1																
Param2																
Dead pixel correction image processing context ID	Specifies a dead pixel correction image processing context used in MimDeadPixelCorrection() operations, allocated using MimAlloc() with M_DEAD_PIXEL_CONTEXT.															
M_DRAW_DEAD_PIXELS	Draws the dead pixels image from the dead pixel correction image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
Flat-field image processing context ID	Specifies an image processing context used in MimFlatField() operations, allocated using MimAlloc() with M_FLAT_FIELD_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_DARK_IMAGE	Draws the dark image from the flat-field image processing context. The dark image represents the thermal agitation of the camera's CCD.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_FLAT_IMAGE	Draws the flat image from the flat-field context. The flat image represents the variation in sensitivity of each element of the camera's CCD.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_OFFSET_IMAGE	Draws the offset image from the flat-field context. The offset image represents the electrical bias of the camera's CCD.	c		e	f	g	a		i		j	k	l	m	n	b
Match image processing context ID	Specifies an image processing context used in MimMatch() operations, allocated using MimAlloc() with M_MATCH_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_MASK	Draws the mask image from the match image processing context. All non-zero values in the image are considered mask pixels; these pixels will mask out corresponding pixels in the model image when MimMatch() performs the match.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_MODEL	Draws the model image from the match context. MimMatch() compares its source image against this model image.	c		e	f	g	a		i		j	k	l	m	n	b
Unwarp along path image processing context ID	Specifies an image processing context used in MimUnwarpAlongPath() operations, allocated using MimAlloc() with M_UNWARP_ALONG_PATH_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WORK_GRID_VERTICAL_LINES	Draws the vertical lines used to generate the source grid corresponding to destination pixels. You can retrieve the X- and Y-coordinates of the top and bottom of the vertical working grid lines using MimGet() with M_XY_WORK_TOP_GRID_VERTICAL_LINES and M_XY_WORK_BOTTOM_GRID_VERTICAL_LINES, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the index of the step to draw.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WORK_PATH	Draws the working path used to generate the source grid corresponding to destination pixels. You can retrieve the X- and Y-coordinates of the working path using MimGet() with M_XY_WORK_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WORK_QUADRILATERALS	Draws the quadrilaterals used to generate the source grid corresponding to destination pixels. You can retrieve the X- and Y-coordinates of the top and bottom working paths using MimGet() with M_XY_WORK_TOP_PATH and M_XY_WORK_BOTTOM_PATH, respectively.	c		e	f	g	a		i		j	k	l	m	n	b

In the following table, Src1AiId specifies a result buffer and Operation specifies the operation to perform with the specified result buffer. Src2AiId is not required and must be set to **M_NULL**. Set Param1 and Param2 to 0, unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a draw operation that uses image processing results																
Src1AiId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Operation																
Src2AiId																
Param1																
Param2																

Augmentation result buffer ID	Specifies an augmentation result buffer, allocated using MimAllocResult() with M_AUGMENTATION_RESULT, and used in MimAugment() operations. The DstImageBufOrListGrald parameter must specify an image buffer that can be processed (MbufAlloc...() with M_IMAGE + M_PROC). It must have the same number of bands and be the same type as the source used with MimAugment(). To specify an image buffer with an optimal size, so it can hold the results of all possible augmentations, call MimGetResult() with M_AUG_OPTIMAL_SIZE_X and M_AUG_OPTIMAL_SIZE_Y.														
M_DRAW_AUG_IMAGE	Draws the resulting image of the augmentation.	c		e	f	g	a		i		j	k	l	m	n	b
Find orientation result buffer ID	Specifies a find orientation image processing result buffer identifier, allocated using MimAllocResult() with M_FIND_ORIENTATION_LIST. The result buffer must contain the dominant orientations (angle values found using MimFindOrientation()) of the image and the orientation scores (between 0 and 100).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_IMAGE_ORIENTATION	Draws arrows to represent the found image orientations (viewing angles) calculated by MimFindOrientation(). The center of each arrow will be at the center of the destination image buffer. The arrow's length is proportional to its score and the available space in the destination buffer (DstImageBufOrListGrald). With multiple calls to **MimDraw**(), you can set each arrow to have a different color, dictated by the 2D graphics context specified (ContextGrald). Depending on your requirements, you would either change the foreground color associated with the 2D graphics context or allocate multiple 2D graphics context each with a different foreground color. This can be useful for debugging your application. This operation can draw in a 2D graphics list. 	c		e	f	g	a		i		j	k	l	m	n	b
Src2AiId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the index of the first orientation to draw from the result buffer. The first orientation in the result buffer is at index 0 and will have a score of 100%.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the number of orientations to draw from the result buffer. The number of orientations cannot exceed the number of entries allocated using MimAllocResult() with NbEntries.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics result buffer ID	Specifies a result buffer used to store MimStatCalculate() results, allocated using MimAllocResult() with M_STATISTICS_RESULT.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_GLCM_MATRIX	Draws the internally generated co-occurrence matrix. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Note that only co-occurrence matrix statistics calculated using the entire source image can be drawn. Using results from a co-occurrence matrix window will generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STAT_RESULT	Draws the specified statistical results of the image processing result buffer. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation.	c		e	f	g	a		i		j	k	l	m	n	b
Src2AiId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the type of statistical operation's result to draw.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CONTRAST	Draws the results of the co-occurrence contrast matrix statistic. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Only co-occurrence matrix statistics calculated using the entire source image can be drawn. If results from a co-occurrence matrix window, an error occurs.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CORRELATION	Draws the results of the co-occurrence correlation matrix statistic. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Only co-occurrence matrix statistics calculated using the entire source image can be drawn. If results from a co-occurrence matrix window, an error occurs.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_DISSIMILARITY	Draws the results of the co-occurrence dissimilarity matrix statistic. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Only co-occurrence matrix statistics calculated using the entire source image can be drawn. If results from a co-occurrence matrix window, an error occurs.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_ENERGY	Draws the results of the co-occurrence energy matrix statistic. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Only co-occurrence matrix statistics calculated using the entire source image can be drawn. If results from a co-occurrence matrix window, an error occurs.	c		e	f	g	a		i		j	k	l	m	n	b

			M_STAT_GLCM_ENTROPY	Draws the results of the co-occurrence entropy matrix statistic. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Only co-occurrence matrix statistics calculated using the entire source image can be drawn. If results from a co-occurrence matrix window, an error occurs.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_GLCM_HOMOGENEITY	Draws the results of the co-occurrence homogeneity matrix statistic. Note that the results are saturated to the maximum possible value for the specified destination buffer of the draw operation. The resulting co-occurrence matrix is resized to fit into the provided buffer, using closest neighbor interpolation. Only co-occurrence matrix statistics calculated using the entire source image can be drawn. If results from a co-occurrence matrix window, an error occurs.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_MAX	Draws the results of the maximum pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_MAX_ABS	Draws the results of the maximum absolute pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_MEAN	Draws the results of the mean value of the pixels operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_MIN	Draws the results of the minimum pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_MIN_ABS	Draws the results of the minimum absolute pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_STANDARD_DEVIATION	Draws the results of the standard deviation operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_SUM	Draws the results of the sum of the pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_SUM_ABS	Draws the results of the sum of the absolute pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b
			M_STAT_SUM_OF_SQUARES	Draws the results of the sum of the squared pixel operation.	c		e	f	g	a		i		j	k	l	m	n	b

Wavelet transformation result buffer ID	Specifies a result buffer used to store MimWaveletTransform() results, allocated using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT. The DstImageBufOrListGraId parameter must specify an image buffer that can be processed (MbufAlloc...() with M_IMAGE + M_PROC). It must have the same number of bands and be the same type as the source used with MimWaveletTransform(). For dyadic modes (MimControl() with M_TRANSFORMATION_MODE set to M_DYADIC), drawings are in the top-right (vertical coefficient), bottom-right (diagonal coefficient), and bottom-left (horizontal coefficient) corners of the display. This drawing pattern repeats for each level calculated (MimGetResult() with M_NUMBER_OF_LEVELS). Since dyadic transformations sample wavelet coefficients by a factor of 2 per level, drawings are resized at each level. Aurora Imaging Library also draws the approximation (the low frequency rendition) of the wavelet transformation at the last level, in the top-left corner of the display. For undecimated modes (M_UNDECIMATED), drawings are in one row, per level. Each row is split into three columns, representing the horizontal (left column), diagonal (middle column), and vertical (right column) wavelet coefficients for that level. Since undecimated transformations are not sampled, drawings are all the same size, regardless of level. Aurora Imaging Library also draws the approximation (the low frequency rendition) of the wavelet transformation at the last level, in the first column of the first row. The middle and right columns in this row are blank.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WAVELET +	Draws the resulting image of the wavelet transformation. During the wavelet transformation, calculations can require Aurora Imaging Library to internally add padding data to the image's border. M_DRAW_WAVELET does not draw this padding with the resulting image. To retrieve the image size required to perform this operation, use MimGetResult() with M_WAVELET_DRAW_SIZE_X and M_WAVELET_DRAW_SIZE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WAVELET_WITH_PADDING +	Draws the resulting image of the wavelet transformation, with padding. During the wavelet transformation, calculations can require Aurora Imaging Library to internally add padding data to the image's border. M_DRAW_WAVELET_WITH_PADDING draws the resulting image with this padding. To retrieve the image size required to perform this operation, use MimGetResult() with M_WAVELET_DRAW_SIZE_X_WITH_PADDING and M_WAVELET_DRAW_SIZE_Y_WITH_PADDING.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when Src1AiId=Wavelet transformation result buffer ID: M_DRAW_WAVELET; M_DRAW_WAVELET_WITH_PADDING].

You can add one of the following values to the above-mentioned values to specify whether the drawing operation uses the real or imaginary numbers in the wavelet result.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to draw using the real or imaginary numbers in the wavelet result																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_IMAGINARY_PART	Draws using the imaginary part of the values in the wavelet result. Only available for complex wavelet transformations (MimGetResult() with M_TRANSFORMATION_DOMAIN must return M_COMPLEX).	c		e	f	g	a		i		j	k	l	m	n	b
M_REAL_PART	Draws using only the real numbers in the result. Available for any type of wavelet transformation. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

In the following table, Src1AiId, and optionally Src2AiId, specify image buffer(s) and Operation specifies the operation to perform with the specified image buffer(s).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to perform the draw operation from image buffer(s) containing a depth map and/or intensity map																
Src1AiId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Operation																
Src2AiId																
Param1																
Param2																
Uncorrected depth map image buffer ID	Specifies an image buffer containing an uncorrected depth map. The image buffer must be properly formatted. This is the format that would result if you iterated calls to MimLocatePeak1d() and then drew the results using **MimDraw**() with M_DRAW_DEPTH_MAP_ROW. For more information, see the Generating an uncorrected depth map subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing.															
M_DRAW_PEAKS +	Draws the peaks at the position at which they were found in the original grayscale source image. It draws each peak using the peak's calculated intensity value or using the foreground color of the specified 2D graphics context. This operation cannot be rendered in world units (MgraControl() with M_INPUT_UNITS cannot be set to M_WORLD) when drawing in either an image buffer or a 2D graphics list. Additionally, this operation cannot be drawn with offset or zoom values (MgraControl() with M_DRAW_OFFSET_X or M_DRAW_OFFSET_Y set to values other than 0.0, and M_DRAW_ZOOM_X or M_DRAW_ZOOM_Y set to values other than 1.0). This operation can draw in a 2D graphics list if no intensity buffer is specified (Src2AiId set to M_NULL). You must specify a combination value from each of the following tables: • To specify the direction in which the peak detection was performed • To specify the style in which to draw	c		e	f	g	a		i		j	k	l	m	n	b
Src2AiId	Specifies whether the intensity results will be used when drawing the peaks.	c		e	f	g	a		i		j	k	l	m	n	b

			M_NULL	Specifies to draw the peaks in the foreground color of the 2D graphics context specified using ContextGraId.	c		e	f	g	a		i		j	k	l	m	n	b
			Uncorrected intensity map image buffer ID	Specifies an image buffer containing an uncorrected intensity map. When this buffer is specified, the peaks are drawn using their corresponding intensity value. The image buffer must be properly formatted. This is the format that would result if you iterated calls to MimLocatePeak1d() and then drew the results using **MimDraw**() with M_DRAW_INTENSITY_MAP_ROW. For more information, see the Generating an uncorrected depth map subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing. If you supply an intensity buffer, the intensity buffer defines the drawing color when drawing each position, regardless of the setting of ContextGraId. ContextGraId must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1		Specifies the row in the source image buffers from which to read the results.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2		Specifies the size of the drawn representation of valid peaks. When drawing the peaks as dots (M_DOTS), this parameter specifies the diameter of the dots that are drawn at valid positions; when connecting the peaks using lines (M_LINES), this parameter specifies the width of the line segments that connect valid positions.	c		e	f	g	a		i		j	k	l	m	n	b
			M_FROM_LINE_THICKNESS	Specifies to use the M_LINE_THICKNESS setting of the 2D graphics context as the size of the drawn representation of valid peaks.	c		e	f	g	a		i		j	k	l	m	n	b
			Value >= 1	Specifies the size of the drawn representation of valid peaks, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

In the following table, Src1AiId specifies the result buffer and Operation specifies the operation to perform with the specified result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to perform the draw operation with the result buffer(s) used in MimLocatePeak1d()																		
Src1AiId			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	Operation																	
		Src2AiId																
		Param1																
		Param2																
Locate peak 1D result buffer ID			Specifies a result buffer allocated using MimAllocResult() with M_LOCATE_PEAK_1D_RESULT and used in MimLocatePeak1d() operations.															

M_DRAW_DEPTH_MAP_ROW	Specifies to write the position values of the specified result buffer into a single row of the specified image buffer. Note that if the result buffer contains results from multiple frames, and you specify to draw all results (M_ALL), each frame's results will be drawn into a separate row, beginning at the row index specified with Param1.	c		e	f	g	a		i		j	k	l	m	n	b
Src2AiId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the row of the specified image buffer in which to write the position values. In the case of multiple peaks, the position of the first peak is drawn, as defined using MimControl() with M_SORT_CRITERION. When drawing results from multiple frames, this parameter specifies the first row in which to draw.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the index of the frame(s) for which to draw position values. Set this parameter to 0 if not drawing results from multiple frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to draw results from all frames whose results have been accumulated in the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_FRAMES	Specifies the index of the specific frame for which to draw results.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_INTENSITY_MAP_ROW	Specifies to write the intensity values of the specified result buffer into a single row of the specified image buffer. The default intensity value for missing peaks, M_INVALID, corresponds to the value -1 (or an unsigned buffer's maximum value); to specify a different value, use MimControl() with M_PEAK_INTENSITY_INVALID_VALUE. Note that if the result buffer contains results from multiple frames, and you specify to draw all results (M_ALL), each frame's results will be drawn into a separate row, beginning at the row index specified with Param1.	c		e	f	g	a		i		j	k	l	m	n	b
Src2AiId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the row of the specified image buffer in which to write the intensity values. In the case of multiple peaks, the intensity value of the first peak is drawn, as defined using MimControl() with M_SORT_CRITERION. When drawing results from multiple frames, this parameter specifies the first row in which to draw.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the index of the frame(s) for which to draw intensity values. Set this parameter to 0 if not drawing results from multiple frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to draw results from all frames whose results have been accumulated in the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* < M_NUMBER_OF_FRAMES	Specifies the index of the specific frame for which to draw results.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_PEAKS	Draws the peaks at the position at which they were found in the original source image. It draws each peak using the foreground color of the specified 2D graphics context. This operation cannot be rendered in world units (MgraControl() with M_INPUT_UNITS cannot be set to M_WORLD) when drawing in either an image buffer or a 2D graphics list. Additionally, this operation cannot be drawn with offset or zoom values (MgraControl() with M_DRAW_OFFSET_X or M_DRAW_OFFSET_Y set to values other than 0.0, and M_DRAW_ZOOM_X or M_DRAW_ZOOM_Y set to values other than 1.0).	c		e	f	g	a		i		j	k	l	m	n	b
Src2AilId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the rank index of the peak from which to read the results, or specifies the frame and rank index, in the case of a result buffer with multiple frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_SELECT_PEAK(AIL_INT Frame, AIL_INT Rank)	Specifies the frame and rank of the peak(s) for which to draw the requested result. This macro is required if a MimLocatePeak1d() operation accumulated results from multiple frames in the result buffer (MimControl() with M_NUMBER_OF_FRAMES set to a value greater than 1).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Frame Specifies the index of the frame for which the result was stored.															
	Value >= 0 — Specifies the index of the specific frame.															
	Rank Specifies the peak(s) for which to draw the requested result.															
	M_ALL — Specifies all peaks.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0 — Specifies the rank of the specific peak.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to draw all the peaks.	c		e	f	g	a		i		j	k	l	m	n	b

			Value >= 0	Specifies the index of the peak to draw.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2		Specifies the size of the drawn representation of valid peaks. When drawing the peaks as dots (M_DOTS), this parameter specifies the diameter of the dots that are drawn at valid positions; when connecting the peaks using lines (M_LINES), this parameter specifies the width of the line segments that connect valid positions.	c		e	f	g	a		i		j	k	l	m	n	b
			M_FROM_LINE_THICKNESS	Specifies to use the M_LINE_THICKNESS setting of the 2D graphics context as the size of the drawn representation of valid peaks.	c		e	f	g	a		i		j	k	l	m	n	b
			Value >= 1	Specifies the size of the drawn representation of valid peaks, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when Src1AiId=Uncorrected depth map image buffer ID: M_DRAW_PEAKS].

You must add one of the following values to the above-mentioned value to specify the style in which to draw.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the style in which to draw																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CROSS	Specifies to draw the valid result positions as crosses. The destination buffer must be a 2D graphics list. When M_CROSS is specified, only valid positions are drawn; if a peak was not found for a row/column, nothing will be drawn for that row/column.	c		e	f	g	a		i		j	k	l	m	n	b
M_DOTS	Specifies to draw the valid result positions as dots. When M_DOTS is specified, only valid positions are drawn; if a peak was not found for a row/column, nothing will be drawn for that row/column.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINES	Specifies to draw line segments between valid positions. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when Src1AiId=Uncorrected depth map image buffer ID: M_DRAW_PEAKS].

You must add one of the following values to the above-mentioned value to specify the direction in which the peak detection was performed.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the direction in which the peak detection was performed																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HORIZONTAL	Specifies that the peak detection was performed in the horizontal direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL	Specifies that the peak detection was performed in the vertical direction. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.

DLL	Requires ail11.dll; ailim11.dll.

MimEdgeDetect

Synopsis

Perform a specific edge detection operation and produce a gradient intensity and/or gradient angle image.

Syntax

```
void MimEdgeDetect(
    AIL_ID SrcImageBufId,          //in
    AIL_ID DstIntensityImageBufId, //out
    AIL_ID DstAngleImageBufId,     //out
    AIL_ID KernelId,               //in
    AIL_INT64 ControlFlag,         //in
    AIL_INT Threshold              //in
)
```

Description

This function performs an edge detection operation on the specified source image, using the specified filter. It produces a gradient intensity image and/or a gradient angle image in the specified image buffer(s). If one of the destination images is not required, specify **M_NULL** as its image buffer identifier.

For an unsigned destination buffer, the angle is returned from 0° to 360° (counter-clockwise) and mapped to the entire range of the destination buffer. For example, for an 8-bit unsigned buffer, the angles of 0°, 90°, 180°, and 270° map to 0, 64, 128, and 192, respectively. For a signed destination buffer, mapping is done in both the positive and negative range of the buffer and represents angle values from -180° to 180°. For example, for an 8-bit signed buffer, the angles of -180°, -90°, 0°, and 90° map to -128, -64, 0 and 64, respectively.

This function can also be performed on a floating-point buffer. However, the results are not mapped and are returned in the range of 0° to 360°. The degree of precision is equal to the precision of the floating-point buffer for regular computation or to +/- 0.4° for fast computation. The value for undefined results is the maximum positive value of the floating-point buffer.

You can perform the operation using a combination of fast and full gradient and angle computations.

Type of computation	Description of computation
Full gradient computation	Gradient = sqrt(GradientX*GradientX + GradientY*GradientY)
Fast gradient computation	Gradient = (abs(GradientX) + abs(GradientY))/2
Full angle computation	Angle = arctan(-GradientY/GradientX)
Fast angle computation	The fast angle approximation is based on the same equation as the full angle computation; however, the value is determined from a LUT mapping.

Parameters

SrcImageBufId

Specifies the identifier of the source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstIntensityImageBufId

Specifies the identifier of the destination buffer for the resulting gradient intensity image. This parameter must be given an image buffer identifier or **M_NULL**. Saturation is performed on this buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstAngleImageBufId

Specifies the identifier of the destination buffer for the resulting gradient angle image. This parameter must be given an image buffer identifier or **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

> Note that this parameter cannot be set to **M_NULL** if DstIntensityImageBufId is also set to **M_NULL**, otherwise an error will occur.

KernelId

Specifies the filter to use. This parameter must be set to the value below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the filter to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SOBEL +	Specifies to use the predefined 3x3 edge detection filters. The following kernel is convolved with the image to produce GradientX, which is an intermediate image used in the gradient or angle computation. For each pixel, GradientX contains directional derivative values calculated from horizontal (X-direction) intensity changes. $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ The following kernel is convolved with the image to produce GradientY, which, like GradientX, is an intermediate image used in the gradient or angle computation. For each pixel, GradientY contains directional derivative values calculated from vertical (Y-direction) intensity changes. $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the filter to use.

You can add one of the following values to the above-mentioned value to specify the overscan settings to use for the operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the overscan setting																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_DISABLE	Specifies to disable overscanning for this operation. You should disable overscanning to accelerate the operation for applications in which the overscan data is not important in the resulting buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Specifies to enable overscanning for this operation. The overscan uses pixels from the ancestor of the source buffer. If the source buffer is not a child buffer, or if the point falls outside the ancestor buffer, a mirror type overscan is used. A mirror overscan specifies that the overscan pixels will be a mirror copy of the source buffer's border pixels. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies how to perform the operation.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to perform the operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_FAST_EDGE_DETECT.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST_ANGLE + M_REGULAR_GRADIENT	Specifies a full gradient computation and a fast angle approximation.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST_EDGE_DETECT	Specifies a fast gradient computation and a fast angle approximation. The gradient is computed as an approximation, using the average of the absolute value of the two directional components (X and Y).	c		e	f	g	a		i		j	k	l	m	n	b
M_REGULAR_ANGLE + M_FAST_GRADIENT	Specifies a fast gradient computation and a full angle computation.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGULAR_EDGE_DETECT	Specifies a full gradient computation and a full angle computation. The gradient is computed as the square root of the sum of the square of each directional component (X and Y). This calculation is slower but more precise.	c		e	f	g	a		i		j	k	l	m	n	b

Threshold

Specifies the threshold value for calculating the gradient angle. For a gradient intensity value lower than the threshold value, the angle is not computed (not considered a significant edge) and the resulting angle for that pixel is undetermined. To perform the full operation, set this parameter to zero or **M_NULL**.

Note that if Aurora Imaging Library determines that calculating the gradient angle of all pixels is faster than excluding some pixels from the computation, the threshold value is simply ignored.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimErode

Synopsis

Perform a binary or grayscale erosion-type morphological operation.

Syntax

```
void MimErode(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbIteration,  //in
    AIL_INT64 ProcMode    //in
)
```

Description

This function performs a binary or grayscale erosion on the given source image for the specified number of iterations.

In binary mode, this function uses a 3x3 full rectangular structuring element; in grayscale mode, a 3x3 empty one.

The overscan pixels are automatically set to the highest possible buffer value, which will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

NbIteration

Specifies the number of times to iterate the operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of iterations to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that each blob should be eroded until it is about to disappear. This value is only available when ProcMode is set to M_BINARY_ULTIMATE or M_BINARY_ULTIMATE_ACCUMULATE. M_DEFAULT does not specify an explicit number of iterations to perform; instead, it indicates to the function that each blob should be eroded until it is about to disappear. For example, if M_DEFAULT is specified and you have two rectangular blobs (120x6 and 150x17), the erosion will continue until the blobs are reduced to 116x2 and 134x1, respectively (the smallest they can be before they disappear). Setting NbIteration to M_DEFAULT with any processing mode other than M_BINARY_ULTIMATE or M_BINARY_ULTIMATE_ACCUMULATE will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Value > = 0	Specifies the number of iterations. When this parameter is set to 0 and ProcMode is set to M_BINARY, the source image is binarized and the result is copied into the destination image buffer. When this parameter is set to 0 and ProcMode is set to M_GRAYSCALE, the source image is copied into the destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ProcMode

Specifies the processing mode to use. This parameter can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Performs a binary erosion on the white (non-zero) blobs in the image. In this case, non-zero pixels are treated as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY_ULTIMATE	Performs a binary erosion on the white (non-zero) blobs, while preventing them from completely disappearing. When in this processing mode, this operation performs a binary erosion until the number of iterations NbIteration is reached or until the blob is about to disappear. If NbIteration is not set to **M_DEFAULT**, the specified number of iterations serves as an upper bound limit, and the procedure continues until the specified number of iterations is reached or the blob is about to disappear, whichever comes first. If NbIteration is set to **M_DEFAULT**, the procedure continues until the non-zero blob is about to disappear. Note that if the erosion splits any blobs, the process continues on each sub-blob evaluating them separately.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY_ULTIMATE_ACCUMULATE	Performs a binary erosion on the white (non-zero) blobs, while preventing them from completely disappearing and keeping track of the number of iterations. When in this processing mode, this operation performs a binary erosion until the number of iterations NbIteration is reached or until the blob is about to disappear. If NbIteration is not set to **M_DEFAULT**, the specified number of iterations serves as an upper bound limit, and the procedure continues until the specified number of iterations is reached or the blob is about to disappear, whichever comes first. If NbIteration is set to **M_DEFAULT**, the procedure continues until the non-zero blob is about to disappear. > Note that if the erosion splits any blobs, the process continues on each sub-blob, evaluating them separately. The resulting non-zero pixels will be set to the iteration number + 1. For example, consider an erosion with NbIteration set to 2. After the first iteration, the resulting non-zero pixels will be set to 1 + 1 (iteration number) = 2. After the second iteration,the resulting non-zero pixels will be set to 1 + 2 (iteration number) = 3. For more information on erosion see the Erosion and dilation section of Chapter 3: Fundamental image processing. For more information on binary ultimate accumulate erosion, see the Erosion and dilation section of Chapter 3: Fundamental image processing.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Performs a grayscale erosion on the white (non-zero) blobs in the image. In this case, each pixel of the source image is replaced with the minimum value in its neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFilterAdaptive

Synopsis

Perform adaptive filtering.

Syntax

```
void MimFilterAdaptive(
    AIL_ID AdaptiveFilterContextImId, //in
    AIL_ID SrcImageBufId,             //in
    AIL_ID DstImageBufId,             //out
    AIL_DOUBLE Param1,                //in
    AIL_DOUBLE Param2,                //in
    AIL_DOUBLE Param3,                //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function performs adaptive filtering to remove noise in an image, while preserving edges as much as possible.

Depending on the application, **MimFilterAdaptive**() will require different value settings for the Param1, Param2, Param3 and ControlFlag parameters. It is recommended that you experiment with different settings to achieve the best level of noise reduction and edge preservation for your application.

Note that adaptive filters are slower than other similar operations, such as rank filters (MimRank()) and most of the filters available using MimConvolve(). If edge preservation is less important than speed for your application, try the predefined or custom filters available in MimConvolve().

Parameters

AdaptiveFilterContextImId

Specifies the predefined adaptive filter context to use.

See the Parameter associations section for possible values that can be specified.

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must be 1-band and not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

This image buffer must be 1-band and not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Param1

Specifies an attribute of the operation to perform.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the operation to perform.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the operation to perform.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies how to determine the new value for a pixel determined to be noise.

If not used, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **AdaptiveFilterContextImId**, **Param1**, **Param2**, **Param3**, and **ControlFlag** parameters.

- For specifying parameters of a bilateral, noise peak elimination, guided, or fast guided adaptive filter operation

The following AdaptiveFilterContextImId and associated Param1, Param2, Param3, and ControlFlag settings are used to determine the type of adaptive filtering to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying parameters of a bilateral, noise peak elimination, guided, or fast guided adaptive filter operation																
AdaptiveFilterContextImId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Param1																
Param2																
Param3																
ControlFlag																
M_BILATERAL	Specifies a bilateral adaptive filter operation. A bilateral filter smooths an image while preserving its features. It is effectively an edge-preserving filter. Bilateral adaptive filtering is based on spatial and intensity weights given to each pixel in a neighborhood, and results in greater or lesser degrees of smoothing and edge preservation, depending on the values set with Param1, Param2, and Param3. This function performs an operation similar to MimConvolve(), except the kernel used in calculations is variable and adapts to the content of the source image.															
Param1	Sets the amount of spatial smoothing to perform.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the amount of smoothing. A larger value applies more smoothing than a smaller value. Note that, beyond a certain value, all the spatial weights in the neighborhood become equal; further increasing Param1 would be useless, but not detrimental.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets how well pixel differences (edges) will be preserved by the smoothing operation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the amount of acceptable edge loss. To preserve edges, set Param2 to a low value. > Typically, this parameter is set to a decimal value much less than one.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the spatial window (neighborhood size) for the operation.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the spatial window (neighborhood size), in pixels. This parameter should typically be set to an odd value so the kernel is centered on a whole pixel. Even values are supported but not advised.	c		e	f	g	a		i		j	k	l	m	n	b

M_FAST_GUIDED	Specifies a fast guided adaptive filter operation. A fast guided filter smooths an image while preserving its edges. Fast guided filtering separates image information into edges and homogenous areas. Two sub-images are created, one by blurring the homogenous areas of the image and the other containing the location of the edges. A new image is then constructed by combining these two sub-images with the original image. The fast guided filter improves the computational speed of the guided filter by sub-sampling the original image and then up-sampling before constructing the final image. This is at the cost of a slightly degraded result. Note, only 8-bit and 16-bit unsigned image buffers are supported for guided filter operations.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the sub-sampling scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1.0	Specifies the scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets how well pixel differences (edges) will be preserved by the smoothing operation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the amount of acceptable edge loss. To preserve edges, set Param2 to a low value. > Typically, this parameter is set to a decimal value much less than one.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the spatial window (neighborhood size) for the operation.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the spatial window (neighborhood size), in pixels. This parameter should typically be set to an odd value so the kernel is centered on a whole pixel. Even values are supported but not advised.	c		e	f	g	a		i		j	k	l	m	n	b
M_GUIDED	Specifies a guided adaptive filter operation. A guided filter smooths an image while preserving its edges. Guided filtering separates image information into edges and homogenous areas. Two sub-images are created, one by blurring the homogenous areas of the image and the other containing the location of the edges. A new image is then constructed by combining these two sub-images with the original image. Note, only 8-bit and 16-bit unsigned image buffers are supported for guided filter operations.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	This parameter must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets how well pixel differences (edges) will be preserved by the smoothing operation.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 1.0	Specifies the amount of acceptable edge loss. To preserve edges, set Param2 to a low value. > Typically, this parameter is set to a decimal value much less than one.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the spatial window (neighborhood size) for the operation.	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0	Specifies the spatial window (neighborhood size), in pixels. This parameter should typically be set to an odd value so the kernel is centered on a whole pixel. Even values are supported but not advised.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOISE_PEAK_REMOVAL	Specifies a noise peak elimination adaptive filter operation. A noise peak elimination filter iteratively replaces the central pixel in a 3x3 neighborhood, if the pixel is a minimum or maximum within that neighborhood, and differs from the replacement value by more than a specified threshold. The replacement value is based on the neighboring pixels; how the replacement value is calculated can be set using the ControlFlag parameter. Optionally, you can specify a gap value. A pixel must be smaller/greater than the minimum/maximum of its neighbors by at least this amount to be considered the minimum/maximum of its neighborhood. The operation performed is similar to using MimRank() with M_3X3_RECT, except that the rank value is variable and adapts to the content of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Sets the number of iterations to perform.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT		c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Sets the gap value. A pixel must be smaller/greater than the minimum/maximum of its neighbors by at least this amount to be considered the minimum/maximum of its neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the gap value.	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Sets the replacement threshold. A pixel is only determined to be noise if its value differs from the replacement value by more than this threshold. The ControlFlag parameter determines how the replacement value is calculated.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the replacement threshold.	c		e	f	g	a		i		j	k	l	m	n	b
ControlFlag	Sets how to calculate the replacement value for each pixel. The replacement value is used to determine whether the pixel is noise. If the pixel is determined to be noise, it is replaced with this value.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_MEDIAN.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTREME	Specifies that the replacement value for each pixel is the minimum or maximum value of the neighboring pixels. The extreme closest to the pixel's current value is used.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies that the replacement value for each pixel is the average value of the neighboring pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIAN	Specifies that the replacement value for each pixel is the median value of the neighboring pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFilterBinomial

Synopsis

Perform binomial filtering.

Syntax

```
void MimFilterBinomial(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT KernelSizeX,  //in
    AIL_INT KernelSizeY,  //in
    AIL_INT Overscan,     //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function performs binomial filtering on the specified source image using a specified kernel size in X and Y.

The binomial filter approximates a Gaussian filter using only integer kernel values, resulting in faster computation than other filtering operations. This function also allows for different dimensions in X and Y, unlike other filtering operations.

The dimensions of the kernel (KernelSizeX and KernelSizeY) must be odd integers to ensure there is a central pixel.

This function is useful, for example, to filter out false edges in an image while preserving major foreground edges.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must be an 8-bit unsigned image buffer and not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

This image buffer must be an 8 or 16-bit integer image buffer and not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

KernelSizeX

Specifies the width of the kernel (X-size).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the kernel width																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 63	Specifies the width of the kernel. This must be an odd integer.	c		e	f	g	a		i		j	k	l	m	n	b

KernelSizeY

Specifies the height of the kernel (Y-size).

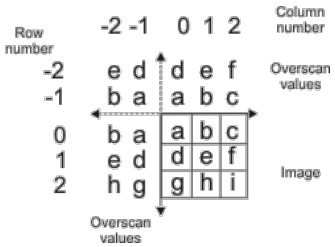
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the kernel height																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 63	Specifies the height of the kernel. This must be an odd integer.	c		e	f	g	a		i		j	k	l	m	n	b

Overscan

Specifies the type of overscan used to handle the source image's bordering pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of overscan																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_TRANSPARENT.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIRROR	Specifies that the border pixels of a source image are processed using overscan pixel values that mirror the source buffer pixel values. That is, the overscan pixel values will be a mirror copy of the source buffer's borders. For example: 	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSPARENT	Specifies that the border pixels of a source image are processed using transparent overscan pixel values. That is, the overscan pixel values will be those of the parent buffer. If they are not available, a mirror type overscan is used instead.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFilterMajority

Synopsis

Replace the value of each pixel with the value that occurs most often in its neighborhood.

Syntax

```
void MimFilterMajority(
    AIL_ID MajorityFilterContextImId, //in
    AIL_ID SrcImageBufId,             //in
    AIL_ID DstImageBufId,             //out
    AIL_ID StructElemBufId,           //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function performs a majority filter operation on the specified source image, using a custom or predefined structuring element. The structuring element defines a neighborhood around each pixel of the source image buffer, and the value that appears most often in the neighborhood replaces the value of the central pixel.

When using a custom structuring element, you can specify weights for the pixels in the neighborhood. The weights define the number of times to count the value of the corresponding pixel when establishing the frequency of the different pixel values. If the weights in the custom structuring element buffer are not uniform, then certain neighborhood pixels will have more influence during the majority filter operation. For example, if you set the central value to 2, and all others to 1, then the central pixel's value will be counted twice during the majority filter operation. When using a custom majority filter image processing context, instead of passing a structuring element, you associate a weight image with the context (MimControl() with M_WEIGHT_IMAGE). It is used similar to a structuring element, but allows you to save the weights with the context when calling MimSave() or MimStream().

By default, if two or more pixel values tie for the greatest frequency, the smallest value present among the tied values is used. If you are using a custom context, you can use MimControl() with M_TIE_BREAKER_MODE to change the default behavior. If you have specified weights for the pixels in your neighborhood, it is possible that two pixel values are tied despite not occurring with equal frequency in the neighborhood. In this case, you can chose to break the tie based on the non-weighted frequency of the values in the source image rather than their values, using MimControl() with M_USE_FREQUENCY_TIE_BREAKER.

If the source and destination are multi-band buffers, the same structuring element (neighborhood and weights) is applied to every band of the source buffer.

Once the majority filter image processing context is configured, preprocess the context by calling **MimFilterMajority**() with M_PREPROCESS. If the preprocess operation is not done explicitly (using M_PREPROCESS), it will be done when **MimFilterMajority**() is first called. When preprocessing explicitly, the source and destination image buffers can be set to **M_NULL**. If, however, the source or destination image buffer is provided, it will be used in the preprocess operation, to better optimize future calls.

Parameters

MajorityFilterContextImId

Specifies the identifier of the majority filter image processing context.

You can specify to use a predefined image processing context, or you can pass the identifier of a custom context. If you want to use a custom image processing context, it must have been previously allocated, using MimAlloc() with M_FILTER_MAJORITY_CONTEXT.

See the Parameter associations section for possible values that can be specified.

SrcImageBufId

Specifies the identifier of the source image buffer.

The source image buffer must not have a region of interest (ROI) associated with it; using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

The destination image buffer must not have a region of interest (ROI) associated with it; using an image buffer with an ROI will cause

an error.

StructElemBufId

Specifies the identifier of the structuring element that defines the neighborhood around each pixel, and, optionally, the weights of the pixels in the neighborhood.

Note that floating-point buffers are not supported.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies whether to preprocess a custom image processing context or to execute the filtering operation.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **MajorityFilterContextImId**, **StructElemBufId**, and **ControlFlag** parameters.

- For specifying the settings of the majority filter operation

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the settings of the majority filter operation																
MajorityFilterContextImId StructElemBufId ControlFlag	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to use a predefined majority filter image processing context that replaces the value of each pixel with the most common value in its neighborhood, as defined by the structuring element. You can optionally specify weights in the structuring element. When using a predefined image processing context, you must set ControlFlag to M_DEFAULT.															
StructElemBufId	Specifies which pixels are included in the neighborhood surrounding a central pixel. Only valid neighboring pixels are included in the neighborhood. For example, the pixel in the top-left corner of the source image will not have any neighboring pixels above it or to its left.	c		e	f	g	a		i		j	k	l	m	n	b
M_3X3_CROSS	Specifies a cross (+) structuring element whose neighborhood size is 3x3 pixels. In this case, there are 5 valid neighborhood pixels; the function ignores the pixels located at the four corners of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_3X3_RECT	Specifies a structuring element whose neighborhood size is 3x3 pixels. In this case, there are 9 valid neighborhood pixels; the function does not ignore any pixels in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_5X5_RECT	Specifies a structuring element whose neighborhood size is 5x5 pixels. In this case, there are 25 valid neighborhood pixels; the function does not ignore any pixels in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b

Custom structuring element identifier	Specifies to define the neighborhood using a custom structuring element. You must first allocate this structuring element, using MbufAlloc2d() with M_STRUCT_ELEMENT. You can use MbufPut2d() to populate your custom structuring element buffer with structuring element values. Note that the custom structuring element must not be a floating-point buffer. The structuring element defines the size and the weights of the neighborhood. If the weights in the custom structuring element buffer are not uniform, then certain neighborhood pixels will have more influence during the majority filter operation. For example, if you set the central value to 2, and all others to 1, then the central pixel's value will be counted twice during the majority filter operation.	c		e	f	g	a		i		j	k	l	m	n	b
ControlFlag	This parameter must be set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
Custom majority filter image processing context identifier	Specifies a custom majority filter image processing context, allocated using MimAlloc() with M_FILTER_MAJORITY_CONTEXT. When using a custom majority filter image processing context, you must set StructElemBufId to M_NULL. To specify the neighborhood (with weights, if applicable), you must pass the identifier of an image buffer to MimControl() with M_WEIGHT_IMAGE prior to calling this function.	c		e	f	g	a		i		j	k	l	m	n	b
StructElemBufId	This parameter must be set to M_NULL since the neighborhood is defined using an image.	c		e	f	g	a		i		j	k	l	m	n	b
ControlFlag	Specifies whether to preprocess the custom image processing context or to execute the filtering operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to execute the majority filter operation. If you have not yet preprocessed your custom image processing context, it will be preprocessed upon the first call to **MimFilterMajority**().	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Specifies to preprocess your custom image processing context. Preprocessing the context can speed up subsequent calls to **MimFilterMajority**().	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFindExtreme

Synopsis

Find an image buffer's extremes (minimum and/or maximum pixel values).

Syntax

```
void MimFindExtreme(
    AIL_ID SrcImageBufId,     //in
    AIL_ID ExtremeResultImId, //out
    AIL_INT64 ExtremeType     //in
)
```

Description

This function finds the maximum and/or minimum value of the specified source image and stores results in the specified extreme result buffer.

You can read the minimum and/or maximum from the result buffer, using MimGetResult1d() or MimGetResult(), specifying M_VALUE as the result type.

You can limit this function's results to a region of the source image buffer using a region of interest (ROI) set using MbufSetRegion().

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier. The buffer must be 1-band.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

ExtremeResultImId

Specifies the identifier of the buffer in which to store the extreme values. This parameter must be given the identifier of an image processing result buffer that was allocated with MimAllocResult() and that has an M_EXTREME_LIST type. If just the maximum or minimum is calculated, only one entry is needed. If both the minimum and maximum are calculated, the result buffer must have two entries. The minimum value is stored in the first entry and the maximum value is stored in the second.

ExtremeType

Specifies the type of extreme(s) to find.

To find the actual minimum and/or maximum pixel value(s) in the buffer, specify one or a combination of the values from the following table. For example, to find both the maximum and minimum values, set the ExtremeType parameter to M_MIN_VALUE + M_MAX_VALUE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to find the minimum value, maximum value, or both																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_VALUE	Finds the maximum value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_VALUE	Finds the minimum value.	c		e	f	g	a		i		j	k	l	m	n	b

To find the minimum absolute value, maximum absolute value, or both, specify one or a combination of the values from the following table. For example, to find both the maximum absolute and minimum absolute values, set the ExtremeType parameter to M_MIN_ABS_VALUE + M_MAX_ABS_VALUE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to find the minimum absolute value, maximum absolute value, or both

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAX_ABS_VALUE	Finds the maximum value, as an absolute value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_ABS_VALUE	Finds the minimum value, as an absolute value.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFindOrientation

Synopsis

Find the dominant orientations of an image buffer.

Syntax

```
void MimFindOrientation(
    AIL_ID OrientationContextImId, //in
    AIL_ID SrcImageBufId,          //in
    AIL_ID OrientationResultImId,  //out
    AIL_INT64 ControlFlag          //in
)
```

Description

This function finds the dominant orientations of the source image using the consistent spatial patterns in the image. Aurora Imaging Library writes the list of dominant orientations (best viewing angles) and their associated score in the specified result buffer. The number of orientations found depends on the number of entries allocated in the find orientation result buffer using MimAllocResult() with M_FIND_ORIENTATION_LIST.

You can read the calculated orientations (angle values) and their associated scores using MimGetResult1d() with M_ANGLE and M_SCORE, respectively.

This function requires the source image buffer to have dimensions that are a power of 2. If the image buffer's dimensions do not meet this criteria, you can use MimControl() with M_MODE and MimControl() with M_INTERPOLATION_MODE to specify how Aurora Imaging Library should clip or resize the image for calculations. This will not alter the original image.

Parameters

OrientationContextImId

Specifies the find orientation image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the find orientation image processing context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default M_FIND_ORIENTATION_CONTEXT context with all the controls in the context set to their default value.	c		e	f	g	a		i		j	k	l	m	n	b
Find orientation image processing context ID	Specifies a valid find orientation image processing context identifier, previously allocated using MimAlloc() with M_FIND_ORIENTATION_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageBufId

Specifies the identifier of the source image. The buffer used must be a 1-band image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

OrientationResultImId

Specifies the identifier of the find orientation result buffer. The buffer must have been allocated using MimAllocResult() with M_FIND_ORIENTATION_LIST.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFlatField

Synopsis

Applies a flat-field correction to remove artifacts from the specified source image.

Syntax

```
void MimFlatField(
    AIL_ID FlatFieldContextImId, //in
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageBufId,        //out
    AIL_INT64 ControlFlag        //in
)
```

Description

This function applies a per-pixel gain and offset correction to the source image and writes the results in the destination image. This process removes artifacts from grabbed images, such as electrical bias, thermal agitation, and sensitivity variations.

The following equation defines the relationship between the source and destination pixel values.

$$\text{DstImage} = \text{Gain} \times \left(\frac{(\text{SrcImg} - \text{ElectricalBias}) - (\text{ThermalAgitation} - \text{ElectricalBias})}{\text{CCDSensitivity} - \text{ElectricalBias}} \right)$$

Before using this function, you must configure the specified image processing context with the gain to apply, and the electrical bias, thermal agitation, and CCD sensitivity to remove, using MimControl(); otherwise, an error is reported. You can specify each of these on a per-pixel level using an image or on a global level using a constant.

Once the flat-field correction image processing context is configured, preprocess the context by calling **MimFlatField**() with M_PREPROCESS. If the preprocess operation is not done explicitly (using M_PREPROCESS), it will be done when **MimFlatField**() is first called. When preprocessing explicitly, the source and destination image buffers can be set to **M_NULL**. If, however, the source or destination image buffer is provided, it will be used in the preprocess operation, to better optimize future calls.

Parameters

FlatFieldContextImId

Specifies the identifier of a flat-field image processing context, previously allocated using MimAlloc() with M_FLAT_FIELD_CONTEXT.

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer must be either an 8- or 16-bit unsigned image buffer.

When preprocessing a flat-field image processing context, set this parameter to the identifier of a typical source image buffer. Otherwise, to exclude the buffer from preprocessing, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer. The buffer must be either an 8- or 16-bit unsigned image buffer.

When preprocessing a flat-field image processing context, set this parameter to the identifier of a typical destination image buffer. Otherwise, to exclude the destination image buffer from preprocessing, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ControlFlag

Specifies the operation to perform. This parameter must be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation to perform.																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Performs the flat-field correction operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_PREPROCESS	Preprocesses the specified image processing context. Note that, if not called explicitly, this operation will be performed automatically upon the first call to **MimFlatField**(). If the identifier of the source and/or destination buffer is not set to **M_NULL**, information about the source and/or destination image buffer will be used in the preprocessing operation.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFlip

Synopsis

Perform a horizontal or vertical image-flipping operation.

Syntax

```
void MimFlip(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT64 Operation,  //in
    AIL_INT64 OpFlag      //in
)
```

Description

This function flips the source image to the destination image according to the specified operation.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

> Note that after performing the operation, the destination image will not be calibrated, even if the source image was calibrated.

Operation

Specifies the operation to perform. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

for specifying the operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLIP_HORIZONTAL	Flips the image in a horizontal direction (left to right, along a vertical axis).	c		e	f	g	a		i		j	k	l	m	n	b
M_FLIP_VERTICAL	Flips the image in a vertical direction (top to bottom, along a horizontal axis).	c		e	f	g	a		i		j	k	l	m	n	b

OpFlag

Reserved for future expansion. Must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimFree

Synopsis

Free an image processing context or result buffer.

Syntax

```
void MimFree(
    AIL_ID ContextOrResultImId //in
)
```

Description

This function frees the specified image processing context or result buffer, previously allocated with MimAlloc() or MimAllocResult(), respectively. It should also be used if an image processing context was restored using MimRestore() or MimStream() (The control flag in MimStream() should be set to M_RESTORE in this case).

All image processing contexts and result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResultImId

Specifies the identifier of the image processing context or result buffer to free. These must have been successfully allocated (with MimAlloc() or MimAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimGet

Synopsis

Gets arrays of values from an image processing context.

Syntax

```
AIL_INT MimGet(
    AIL_ID ContextImId,   //in
    AIL_INT64 GetType,    //in
    AIL_INT ArraySize,    //in
    void *Param1Ptr,      //out
    void *Param2Ptr,      //out
    AIL_INT64 ControlFlag //in
)
```

Description

This function gets array(s) of values from an image processing context.

The number of arrays and the nature of the values depend on the image processing context used and on the specified type of information to get. Before you can inquire the content of the array(s), they must be set using MimPut(), unless otherwise specified.

To determine the size of the arrays required to store the requested values, call this function and set Param1Ptr and Param2Ptr to **M_NULL**. Alternatively, when dealing with a dead pixel correction image processing context, to determine the size of the arrays required to store the X- and Y- coordinates of the dead pixels, you can also use MimInquire() with M_XY_DEAD_PIXELS_ARRAY_SIZE. When dealing with a rearrangement image processing context, to determine the size of the arrays required to store the X- and Y- coordinates of the areas to move from the source to the destination and to determine the number of areas to be copied, you can also use MimInquire() with M_XY_DESTINATION_ARRAY_SIZE, M_XY_SIZE_ARRAY_SIZE, and M_XY_SOURCE_ARRAY_SIZE, respectively.

Parameters

ContextImId

Specifies the identifier of the image processing context. The image processing context must have been previously allocated with MimAlloc().

See the Parameter associations section for possible values that can be specified.

GetType

Specifies the type of information in the array(s).

See the Parameter associations section for possible values that can be specified.

ArraySize

Specifies the number of values in the array(s).

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

Param1Ptr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]

Specifies the address of the array in which to store the first set of values. Note that this parameter can be set to **M_NULL** when trying to establish the required size for the arrays.

See the Parameter associations section for possible values that can be returned.

Param2Ptr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- M_NULL

Specifies the address of the array in which to store the second set of values. This parameter can be set to **M_NULL** when trying to establish the required size for the arrays.

> The second array, when required, must have the same number of elements as the first array.

See the Parameter associations section for possible values that can be returned.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **ContextImId** and **GetType** parameters and possible values returned to the **Param1Ptr** and **Param2Ptr** parameters.

- For getting arrays of value(s) from an image processing context

The following ContextImId, GetType, Param1Ptr, and Param2Ptr settings can be specified to get values from an image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting arrays of value(s) from an image processing context																	
ContextImId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
GetType																
Param1Ptr - Possible values returned																
Param2Ptr - Possible values returned																
Augmentation context ID +	Specifies an augmentation context, allocated using MimAlloc() with M_AUGMENTATION_CONTEXT, and used in MimAugment() operations. You need not use MimPut() to inquire about an augmentation context.															
M_AUG_OPERATIONS_ENABLED +	Retrieves the list of the operations that were enabled for MimAugment(). Operations are enabled using MimControl() with M_AUG_..._OP.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the enabled operations. The returned list is sorted according to the priority that MimAugment() uses to perform the specified image processing operations.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Dead pixel correction image processing context ID +	Specifies a dead pixel correction image processing context, allocated using MimAlloc() with M_DEAD_PIXEL_CONTEXT and used in MimDeadPixelCorrection() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_DEAD_PIXELS +	Retrieves the X- and Y-coordinates of the dead pixels in the source image, relative to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the X-coordinates of the dead pixels, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the Y-coordinates of the dead pixels, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Rearrangement image processing context ID +	Specifies a rearrangement image processing context, allocated using MimAlloc() with M_REARRANGE_CONTEXT and used in MimRearrange() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_DESTINATION +	Retrieves the X- and Y-offsets of the areas in the destination image buffer into which to copy the source areas, relative to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the X-offsets of the destination areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the Y-offsets of the destination areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_SIZE +	Retrieves the width and height of the areas to be copied, relative to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the width of the areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the height of the areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_SOURCE +	Retrieves the X- and Y-offsets of the areas to be copied from the source image buffer into the destination image buffer, relative to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

		Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the X-offsets of the source areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* Returns an array with the Y-offsets of the source areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Unwarp along path image processing context ID +			Specifies an unwarp along path image processing context, allocated using MimAlloc() with M_UNWARP_ALONG_PATH_CONTEXT and used in MimUnwarpAlongPath() operations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_PATH +		Retrieves the X- and Y-coordinates of the path in the source image, relative to the pixel coordinate system. To get the required array size, use MimInquire() with M_PATH_ARRAY_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the X-coordinates of the path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the Y-coordinates of the path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_WORK_BOTTOM_GRID_VERTICAL_LINES +		Retrieves the X- and Y-coordinates of the bottom of the working vertical lines used to generate the grid in the source image, relative to the pixel coordinate system. These points lie along the bottom working path and are mapped to points along the bottom of the destination buffer. The bottom of the path is defined one-half-width below the working path relative to the direction of the path. To get the required array size, use MimInquire() with M_DESTINATION_SIZE_X. You can draw the vertical grid lines of the working path using MimDraw() with M_DRAW_WORK_GRID_VERTICAL_LINES.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the X-coordinates of the bottom of the working vertical lines, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the Y-coordinates of the bottom of the working vertical lines, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_XY_WORK_BOTTOM_PATH +	Retrieves the X- and Y-coordinates of the bottom internal working path in the source image, relative to the pixel coordinate system. The bottom path is defined one-half-width below the working path relative to the direction of the path. To get the required array size, use MimInquire() with M_WORK_PATH_ARRAY_SIZE. You can draw the quadrilaterals formed from the intersection of the top and bottom working paths using MimDraw() with M_DRAW_WORK_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the X-coordinates of the bottom working path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the Y-coordinates of the bottom working path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_WORK_PATH +	Retrieves the X- and Y-coordinates of the internal working path in the source image, relative to the pixel coordinate system. To get the required array size, use MimInquire() with M_WORK_PATH_ARRAY_SIZE. You can draw the working path using MimDraw() with M_DRAW_WORK_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the X-coordinates of the working path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the Y-coordinates of the working path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_WORK_TOP_GRID_VERTICAL_LINES +	Retrieves the X- and Y-coordinates of the top of the working vertical lines used to generate the grid in the source image, relative to the pixel coordinate system. These points lie along the top working path and are mapped to points along the top of the destination buffer. The top path is defined one-half-width above the working path relative to the direction of the path. To get the required array size, use MimInquire() with M_DESTINATION_SIZE_X. You can draw the vertical grid lines of the working path using MimDraw() with M_DRAW_WORK_GRID_VERTICAL_LINES.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the X-coordinates of the top of the working vertical lines, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

		Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the Y-coordinates of the top of the working vertical lines, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_WORK_TOP_PATH +		Retrieves the X- and Y-coordinates of the top internal working path in the source image, relative to the pixel coordinate system. The top path is defined one-half-width above the working path relative to the direction of the path. To get the required array size, use MimInquire() with M_WORK_PATH_ARRAY_SIZE. You can draw the quadrilaterals formed from the intersection of the top and bottom working paths using MimDraw() with M_DRAW_WORK_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the X-coordinates of the top working path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* Returns an array with the Y-coordinates of the top working path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For getting arrays of value(s) from an image processing context.

You can add one of the following values to the above-mentioned values to cast the specified array to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of Param1Ptr and Param2Ptr																	
ContextImId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
GetType																
Param1Ptr - Possible values returned																
Param2Ptr - Possible values returned																
M_TYPE_AIL_DOUBLE	Specifies that the array is an *AIL_DOUBLE*.															
Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Specifies that the array is an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT	Specifies that the array is an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Specifies that the array is an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Specifies that the array is an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
Param1Ptr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
Param2Ptr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the number of entries in the array(s). If Param1Ptr and Param2Ptr are both set to **M_NULL**, **MimGet**() returns the size of the required arrays.

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the values listed above are not available in Aurora Imaging Library Lite. See the value's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimGetResult1d

Synopsis

Get the specified number of result entries from an image processing result buffer.

Syntax

```
void MimGetResult1d(
    AIL_ID ResultImId,    //in
    AIL_INT OffEntry,     //in
    AIL_INT NbEntries,    //in
    AIL_INT64 ResultType, //in
    void *UserArrayPtr    //out
)
```

Description

This function retrieves the specified number of result entries from the specified result buffer and stores them in the specified one-dimensional destination user array.

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

> If your target image was associated with a camera calibration context, but you want to retrieve positional and dimensional results in pixel units, use MimControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MimGetResult1d**() will generate an error.

Parameters

ResultImId

Specifies the identifier of the image processing result buffer from which to get results.

OffEntry

Specifies the offset of the first result to read. The given value must be within the number of allocated result entries.

NbEntries

Specifies the number of result entries to get, starting from the entry specified by the OffEntry parameter.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

ResultType

Specifies the type of result(s) to retrieve.

See the Parameter associations section for possible values that can be specified.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the one-dimensional array in which to write the results read from the Aurora Imaging Library result buffer.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **ResultType** parameter and possible values returned to the **UserArrayPtr** parameter.

- For specifying the type of result to retrieve

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to retrieve																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																	
M_ANGLE +	Retrieves the best orientations (angle values) calculated by MimFindOrientation() and stored in an M_FIND_ORIENTATION_LIST result buffer. Aurora Imaging Library sorts the resulting angles according to their associated score (M_SCORE); angles with higher scores are listed first.																
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CUMULATIVE_VALUE +	Retrieves the cumulative histogram values from an M_HIST_LIST result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_EVENT +	Retrieves the number of events from an M_EVENT_LIST result buffer. Note that if the result buffer did not have the required number of entries to hold the total number of events that satisfied the condition, the number returned is limited to the number of entries in the result buffer. In this case, results from a multi-core processing application might differ from run to run as results are added globally from the multiple threads.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Retrieves the number of differences from an M_LOCATE_DIFFERENCE_RESULT result buffer. Note that if the result buffer did not have the required number of entries to hold the total number of differences that satisfied the condition, the number returned is limited to the number of entries in the result buffer. In this case, results from a multi-core processing application might differ from run to run as results are added globally from the multiple threads.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_PERCENTILE_VALUE +	Retrieves the inverse cumulative histogram from an M_HIST_LIST result buffer. Each value retrieved is the pixel value whereby *n* % of the pixels have a value less than or equal to that value, where *n* is the index of the user array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +	Retrieves the image X-coordinate of each event from an M_EVENT_LIST or M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_POSITION_Y +	Retrieves the image Y-coordinate of each event from an M_EVENT_LIST or M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE +	Retrieves the score attributed to each orientation (angle) value calculated by MimFindOrientation() and stored in an M_FIND_ORIENTATION_LIST result buffer. The scores are represented as percentages relative to the other orientations stored in the result buffer. Aurora Imaging Library lists higher scores first. The first orientation in the buffer will always be the most dominant orientation with a score of 100.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC1_VALUE +	Retrieves source 1 values for each differences from M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC2_VALUE +	Retrieves source 2 values for each differences from M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_VALID +	Retrieves the validity of projection data, from an M_PROJ_LIST result buffer. This is only useful when a region of interest is associated with the source image buffer of the projection operation. A lane of pixels will contain no valid pixels if it is excluded from the region of interest associated with the source image; this means that projection data cannot be calculated for the lane. However, the M_PROJ_LIST result buffer contains an entry for each lane of pixels found in the source image, regardless of the use of a region of interest. So when your source image buffer has a region of interest, you should use M_VALID to check the validity of the data for the lane in the result buffer. Note that for an M_SUM projection operation, if a lane of pixels does not contain valid projection data, a neutral pixel value of 0 is used for that lane. In this case, M_VALID will indicate valid projection data for all lanes of pixels in the image.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies invalid projection data for the lane due to no valid pixels residing in the lane of pixels in the source image corresponding to the M_PROJ_LIST entry.	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0	Specifies valid projection data for the lane.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE +	Retrieves values from an M_HIST_LIST, M_PROJ_LIST, M_EXTREME_LIST, M_EVENT_LIST, or M_COUNT_LIST result buffer. If an M_EXTREME_LIST type result buffer contains both the minimum and maximum value of an image, the minimum is written first, followed by the maximum.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CUMULATIVE_VALUE; M_VALUE.

You can add the following value to the above-mentioned values to get the requested results as a percentage.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results as a percentage																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_PERCENTAGE	Retrieves the histogram values or cumulative histogram values, as a percentage. The values are multiplied by 100 and then divided by the total number of elements in the histogram.															
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the type of result to retrieve.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the ResultType parameter to cast the required results to the requested data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- *[Aurora Imaging Library Lite]*

Note that some of the values listed above are not available in Aurora Imaging Library Lite. See the result's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimGetResult2d

Synopsis

Get data from a statistics result buffer, and place it in a user-supplied array.

Syntax

```
void MimGetResult2d(
    AIL_ID ResultImId,     //in
    AIL_INT OffsetX,       //in
    AIL_INT OffsetY,       //in
    AIL_INT SizeX,         //in
    AIL_INT SizeY,         //in
    AIL_INT64 ResultType,  //in
    AIL_INT64 ControlFlag, //in
    void *UserArrayPtr     //out
)
```

Description

This function retrieves, from an M_STATISTICS_RESULT result buffer, the results obtained for a specified region of the series of source buffer passed to MimStatCalculate() that uses an M_STATISTICS_CUMULATIVE_CONTEXT. One value is returned for every pixel in the specified region.

Parameters

ResultImId

Specifies the identifier of the statistics image processing result buffer from which to get results.

OffsetX

Specifies the X-offset of the region for which to retrieve results. The specified value must be between 0 and the SizeX parameter value.

OffsetY

Specifies the Y-offset of the region for which to retrieve results. The specified value must be between 0 and the SizeY parameter value.

SizeX

Specifies the width (X-size) of the region for which to retrieve results.

SizeY

Specifies the height (Y-size) of the region for which to retrieve results.

ResultType

Specifies the type of result(s) to retrieve.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]

Specifies the address of the user-specified array in which to write the results read from the multiple statistics result buffer.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **ResultType** parameter and possible values returned to the **UserArrayPtr** parameter.

- For specifying the type of result to retrieve

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to retrieve																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																	
M_STAT_MAX +	Retrieves the maximum value of each pixel, given the different source images.																
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS +	Retrieves the maximum absolute value of each pixel, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN +	Retrieves the mean value of each pixel, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN +	Retrieves the minimum value of each pixel, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS +	Retrieves the minimum absolute value of each pixel, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION +	Retrieves the standard deviation value of each pixel, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM +	Retrieves the sum of each pixel's different values, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS +	Retrieves the sum of the absolute value of each pixel's different value, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_SUM_OF_SQUARES +	Retrieves the sum of the square of each pixel's different value, given the different source images.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the type of result to retrieve.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the ResultType parameter to cast the required results to the requested data type																
ResultType UserArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimGetResult

Synopsis

Get the specified type of results from an image processing result buffer.

Syntax

```
void MimGetResult(
    AIL_ID ResultImId,    //in
    AIL_INT64 ResultType, //in
    void *UserArrayPtr    //out
)
```

Description

This function retrieves all the results of the specified type from the specified result buffer and stores them in the specified one-dimensional destination user array. Results are only available after calling MimCountDifference(), MimFindExtreme(), MimHistogram(), MimLocateEvent(), MimLocateDifference(), MimProjection(), MimStatCalculate(), or MimWaveletTransform().

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

> If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MimControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MimGetResult**() will generate an error.

If your results were calculated using MimStatCalculate(), the M_STATISTICS_RESULT buffer contains results from an operation performed on a single image (M_STATISTICS_CONTEXT context), or from cumulative operations performed on numerous images (M_STATISTICS_CUMULATIVE_CONTEXT context). Single image results are calculated from all pixels, a region of interest (ROI), or tiled regions of the single image; whereas, cumulative results are calculated for the same pixel location across multiple images. Use MimGetResult2d() to obtain most cumulative results. Note that M_STAT_NUMBER and M_TOTAL_NUMBER are available for cumulative results using **MimGetResult**(). Also use **MimGetResult**() to retrieve statistics results from a single image divided into tiles (specified using MimControl() with M_TILE_SIZE... and M_STEP_SIZE...). For a tiled image, one result per tile is returned for most of the supported statistics.

Parameters

ResultImId

Specifies the identifier of the image processing result buffer from which to retrieve results.

ResultType

Specifies the type of results to retrieve.

See the Parameter associations section for possible values that can be specified.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the one-dimensional array in which to write the results read from the Aurora Imaging Library result buffer.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **ResultType** parameter and possible values returned to the **UserArrayPtr** parameter.

- For a result buffer

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a result buffer																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserArrayPtr - Possible values returned																
M_AUG_ASPECT_RATIO +		Retrieves the aspect ratio applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_ASPECT_RATIO_OP.															
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_MODE +		Retrieves the mode of the aspect ratio applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_ASPECT_RATIO_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BOTH	Specifies that Aurora Imaging Library sets the aspect ratio mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVERT	Specifies to apply the aspect ratio as height/width. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NORMAL	Specifies to apply the aspect ratio as width/height. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_ANGLE +		Retrieves the motion blur angle applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_BLUR_MOTION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_SIZE +		Retrieves the size of the motion blur kernel applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_BLUR_MOTION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_CROP_BOTTOM_RIGHT_X +	Retrieves the X-coordinate of the bottom-right corner of the cropping applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_BOTTOM_RIGHT_Y +	Retrieves the Y-coordinate of the bottom-right corner of the cropping applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_TOP_LEFT_X +	Retrieves the X-coordinate of the top-left corner of the cropping applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_TOP_LEFT_Y +	Retrieves the Y-coordinate of the top-left corner of the cropping applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_ASYM_NB_ITERATIONS +	Retrieves the number of iterations of the asymmetrical dilation applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_DILATION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_ASYM_SUBAREA +	Retrieves the area selected for the asymmetrical dilation applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_DILATION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_LEFT_HALF	Specifies the left half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOWER_HALF	Specifies the lower half of the image.	c		e	f	g	a		i		j	k	l	m	n	b

	M_RIGHT_HALF	Specifies the right half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPPER_HALF	Specifies the upper half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_NB_ITERATIONS +		Retrieves the number of iterations of the dilation applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_DILATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_ASYM_NB_ITERATIONS +		Retrieves the number of iterations of the asymmetrical erosion applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_EROSION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_ASYM_SUBAREA +		Retrieves the area selected for the asymmetrical erosion applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_EROSION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_LEFT_HALF	Specifies the left half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOWER_HALF	Specifies the lower half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RIGHT_HALF	Specifies the right half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UPPER_HALF	Specifies the upper half of the image.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_NB_ITERATIONS +		Retrieves the number of iterations of the erosion applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_EROSION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_FLIP_DIRECTION +		Retrieves the direction of the flip applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_FLIP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_BOTH	Specifies to flip in either the horizontal or vertical direction.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FLIP_HORIZONTAL	Specifies to flip exclusively in the horizontal direction.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FLIP_VERTICAL	Specifies to flip exclusively in the vertical direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_VALUE_BAND_0 +		Retrieves the value of the gamma correction for band 0 applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_VALUE_BAND_1 +		Retrieves the value of the gamma correction for band 1 applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_VALUE_BAND_2 +		Retrieves the value of the gamma correction for band 2 applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_VALUE_BAND + *n* +		Retrieves the value of the gamma correction for band *n* applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_GAMMA_OP. Note that *n* is an integer between 0 and the value returned by M_AUG_GAMMA_VALUE_NB_BANDS - 1.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_VALUE_NB_BANDS +		Retrieves the number of bands for which gamma correction was applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_HSV_VALUE_GAIN +	Retrieves the value with which the value band (in the HSV color space) was multiplied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_HSV_VALUE_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET +	Retrieves the angular value added to the hue band (in the HSV color space) by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_HUE_OFFSET_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_VALUE +	Retrieves the value of the intensity addition (offset) applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_INTENSITY_ADD_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_VALUE +	Retrieves the value of the intensity multiplication (gain) applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_INTENSITY_MULTIPLY_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_ANGLE +	Retrieves the angle of the directional lighting (add-ramp) applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_LIGHTING_DIRECTIONAL_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_STDDEV +	Retrieves the standard deviation of the Gaussian additive noise applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_NOISE_MULTIPLICATIVE_STDDEV +	Retrieves the standard deviation of the multiplicative noise applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_NOISE_MULTIPLICATIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_SALT_PEPPER_DENSITY +	Retrieves the density of the salt and pepper noise applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_NOISE_SALT_PEPPER_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_OPERATION_ASSOCIATED_WITH_RESULT_TYPES +	Retrieves the list of the operations, and their associated result types, that MimAugment() applied. The returned list is sorted according to the priority that MimAugment() uses to perform the specified image processing operations.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_OPERATION_RESULT_TYPES +	Retrieves the list of the result types that are related to the operations that MimAugment() applied. The returned list is sorted according to the priority that MimAugment() uses to perform the specified image processing operations.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_OPERATION_RESULT_VALUES +	Retrieves the list of the values that were used for the operations that MimAugment() applied. The returned list is sorted according to the priority that MimAugment() uses to perform the specified image processing operations.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_OPERATIONS_APPLIED +	Retrieves a list of **M_TRUE** and **M_FALSE** values, indicating whether the corresponding enabled operations, returned by M_AUG_OPERATIONS_ENABLED, were actually applied by MimAugment(). For example, if M_AUG_OPERATIONS_APPLIED returns [**M_TRUE**, **M_TRUE**, **M_FALSE**] and M_AUG_OPERATIONS_ENABLED returns [M_AUG_ROTATION_OP, M_AUG_SCALE_OP, M_AUG_ASPECT_RATIO_OP], it means that MimAugment() applied the rotation and scale operations, but did not apply the aspect ratio operation. The order of the **M_TRUE** and **M_FALSE** values that M_AUG_OPERATIONS_APPLIED returns is the same as the order of the operations returned by M_AUG_OPERATIONS_ENABLED.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_OPERATIONS_ENABLED +	Retrieves the list of the operations that were enabled for MimAugment(). Operations are enabled using MimControl() with M_AUG_..._OP. The returned list is sorted according to the priority that MimAugment() uses to perform the specified image processing operations.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_OPTIMAL_SIZE_X +	Retrieves the suggested optimal width of an image buffer in which to produce all possible augmentation results. To establish this width, Aurora Imaging Library considers every image processing operation that is currently enabled for the augmentation (MimControl() with M_AUG_..._OP). The optimal image buffer size can be useful, for example, when drawing the resulting image of the augmentation (M_DRAW_AUG_IMAGE).	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_OPTIMAL_SIZE_Y +	Retrieves the suggested optimal height of an image buffer in which to produce all possible augmentation results. To establish this height, Aurora Imaging Library considers every image processing operation that is currently enabled for the augmentation (MimControl() with M_AUG_..._OP). The optimal image buffer size can be useful, for example, when drawing the resulting image of the augmentation (M_DRAW_AUG_IMAGE).	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_REDUCE_FACTOR +	Retrieves the reducing scale factor applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OFFSET_X +	Retrieves the X offset applied to by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OFFSET_Y +	Retrieves the Y offset applied to by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_ANGLE +	Retrieves the rotation angle applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_ROTATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN +	Retrieves the value with which the saturation band (in the HSV color space) was multiplied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SATURATION_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_FACTOR +	Retrieves the scale applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SCALE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SEED_USED +	Retrieves the value (seed) used to generate the randomized settings to perform the augmentation.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the seed.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_SHARPEN_DERICHE_VALUE +	Retrieves the value of the Deriche sharpening filter applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SHARPEN_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X +	Retrieves the horizontal direction of the shear applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SHEAR_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y +	Retrieves the vertical direction of the shear applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SHEAR_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_DERICHE_VALUE +	Retrieves the value of the Deriche filter smoothing applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SMOOTH_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_STDDEV +	Retrieves the standard deviation of the Gaussian smoothing applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_SMOOTH_GAUSSIAN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_X +	Retrieves the horizontal translation applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_TRANSLATION_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_Y +	Retrieves the vertical translation applied by MimAugment(), from an M_AUGMENTATION_RESULT result buffer. This result's related operation is specified using MimControl() with M_AUG_TRANSLATION_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b

	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_CUMULATIVE_VALUE +		Retrieves the cumulative histogram values from an M_HIST_LIST result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_REAL_SIZE +		Retrieves the number of values (entries) used by the histogram, from an M_HIST_LIST result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_VALUE_OFFSET +		Retrieves the intensity value in the source that corresponds to the first bin (entry 0) in the histogram, from an M_HIST_LIST result buffer. This result depends on the specified histogram mode, set using MimControl() with M_HIST_BIN_SIZE_MODE. This result returns an error if you set the mode to M_REGULAR.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_VALUE_RANGE +		Retrieves the number of intensity values in the source that were used to generate the histogram, from an M_HIST_LIST result buffer. This result depends on the specified histogram mode, set using MimControl() with M_HIST_BIN_SIZE_MODE. This result returns an error if you set the mode to M_REGULAR.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_EVENT +		Retrieves the number of events from an M_EVENT_LIST result buffer. Note that if the result buffer did not have the required number of entries to hold the total number of events that satisfied the condition, the number returned is limited to the number of entries in the result buffer. In this case, results from a multi-core processing application might differ from run to run as results are added globally from the multiple threads.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +		Retrieves the number of differences from an M_LOCATE_DIFFERENCE_RESULT result buffer. Note that if the result buffer did not have the required number of entries to hold the total number of differences that satisfied the condition, the number returned is limited to the number of entries in the result buffer. In this case, results from a multi-core processing application might differ from run to run as results are added globally from the multiple threads.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_OF_LEVELS +	Retrieves the total number of transformation levels calculated by the wavelet transformation, from an M_WAVELET_TRANSFORM_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIGINAL_IMAGE_SIZE_X +	Retrieves the unaltered width of the image used by the wavelet transformation, from an M_WAVELET_TRANSFORM_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIGINAL_IMAGE_SIZE_Y +	Retrieves the unaltered height of the image used by the wavelet transformation, from an M_WAVELET_TRANSFORM_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_PERCENTILE_VALUE +	Retrieves the inverse cumulative histogram from an M_HIST_LIST result buffer. Each value retrieved is the pixel value whereby *n* % of the pixels have a value less than or equal to that pixel value, where *n* is the index of the user array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +	Retrieves the image X-coordinate of each event from an M_EVENT_LIST or M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +	Retrieves the image Y-coordinate of each event from an M_EVENT_LIST or M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +	Retrieves the number of tiles along the X-dimension, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_SIZE_Y +	Retrieves the number of tiles along the Y-dimension, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC1_VALUE +	Retrieves source 1 values for each differences from M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC2_VALUE +	Retrieves source 2 values for each differences from M_LOCATE_DIFFERENCE_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANGULAR_DATA_COHERENCE +	Retrieves the coherence, from an M_STATISTICS_RESULT result buffer, of pixels representing angles and as such, treated as unit vectors. Coherence represents the directional trend of a group of vectors and is a measure of how parallel or coherent these vectors are with respect to each other.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANGULAR_DATA_MEAN +	Retrieves the angle of the vector sum, from an M_STATISTICS_RESULT result buffer, of pixels representing angles, and as such, treated as unit vectors. The angular component of a vector sum, calculated from the vector addition of unit vectors, is a representation of the average or mean angle of the unit vectors.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CONTRAST +	Retrieves the contrast of all the grayscale pixel values within a specified neighborhood in relation to the normalized co-occurrence probability of each pixel, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CORRELATION +	Retrieves the correlation within the normalized co-occurrence probability in a specified neighborhood, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_DISSIMILARITY +	Retrieves the dissimilarity between the normalized co-occurrence probability and the gray level co-occurrence matrix of a specified neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_ENERGY +	Retrieves the energy between two points in the gray level co-occurrence matrix of a given neighborhood, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_ENTROPY +	Retrieves the inverse of the energy between two points in the normalized co-occurrence probability of a specified neighborhood, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_HOMOGENEITY +	Retrieves the similarity between the normalized co-occurrence probability and the gray level co-occurrence matrix of a given neighborhood, from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX +	Retrieves the maximum pixel value from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS +	Retrieves the maximum absolute pixel value from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN +	Retrieves the mean pixel value from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_MIN +	Retrieves the minimum pixel value from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS +	Retrieves the minimum absolute pixel value from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_NUMBER +	Retrieves the number of pixels or the number of images, depending on the context type. For an M_STATISTICS_RESULT result buffer used with an M_STATISTICS_CONTEXT, this value represents the number of pixels that satisfied the condition specified when calling MimStatCalculate(). For an M_STATISTICS_RESULT result buffer used with an M_STATISTICS_CUMULATIVE_CONTEXT, this value represents the number of images used to calculate the current result. This does not include images that have previously been removed from the calculation.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ORIENTATION_DATA_COHERENCE +	Retrieves the coherence of the orientation data from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ORIENTATION_DATA_MEAN +	Retrieves the dominant orientation of the unit vectors from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION +	Retrieves the pixel standard deviation value from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM +	Retrieves the sum of the pixel values from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS +		Retrieves the sum of the absolute pixel values from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES +		Retrieves the sum of the squared pixel values from an M_STATISTICS_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TOTAL_NUMBER +		Retrieves the total number of images that were used to calculate the results in an M_STATISTICS_RESULT result buffer. This number is the sum of M_STAT_NUMBER and all the images that have previously been removed from the calculation using MimStatCalculate() with M_REMOVE. This is only available for an result buffer whose results were obtained using an M_STATISTICS_CUMULATIVE_CONTEXT context.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFORMATION_DOMAIN +		Retrieves whether the mathematical domain of the resulting wavelet transformation values consist of complex numbers or only real numbers, from an M_WAVELET_TRANSFORM_RESULT result buffer. Complex numbers have an imaginary part (imaginary numbers), in addition to a real part (real numbers).	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPLEX	Specifies that the mathematical domain of the values consist of real and imaginary numbers. Such transformations come from using a complex type of wavelet filter (MimControl() with M_WAVELET_TYPE), or a custom wavelet filter containing real and imaginary numbers (MimWaveletSetFilter()).	c		e	f	g	a		i		j	k	l	m	n	b
	M_REAL	Specifies that the mathematical domain of the values consist of only real numbers. Such transformations come from using a non-complex type of wavelet filter (MimControl() with M_WAVELET_TYPE), or a custom wavelet filter containing real numbers only (MimWaveletSetFilter()).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFORMATION_MODE +		Retrieves the wavelet transformation mode, from an M_WAVELET_TRANSFORM_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DYADIC	Specifies a dyadic wavelet transformation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNDECIMATED +	Specifies an undecimated wavelet transformation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VALID +		Retrieves the validity of projection data for each lane, from an M_PROJ_LIST result buffer. This is only useful when a region of interest is associated with the source image buffer of the projection operation. A lane of pixels will contain no valid pixels if it is excluded from the region of interest associated with the source image; this means that projection data cannot be calculated for the lane. However, the M_PROJ_LIST result buffer contains an entry for each lane of pixels found in the source image, regardless of the use of a region of interest. So when your source image buffer has a region of interest, you should use M_VALID to check the validity of the data for the lane in the result buffer. Note that for an M_SUM projection operation, if a lane of pixels does not contain valid projection data, a neutral pixel value of 0 is used for that lane. In this case, M_VALID will indicate valid projection data for all lanes of pixels in the image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	0	Specifies invalid projection data for the lane due to no valid pixels residing in the lane of pixels in the source image corresponding to the M_PROJ_LIST entry.	c		e	f	g	a		i		j	k	l	m	n	b
	Value != 0	Specifies valid projection data for the lane.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALUE +		Retrieves values from an M_HIST_LIST, M_PROJ_LIST, M_EXTREME_LIST, M_EVENT_LIST, or M_COUNT_LIST result buffer. If an M_EXTREME_LIST type result buffer contains both the minimum and maximum value of an image, the minimum is written first, followed by the maximum.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_WAVELET_DRAW_SIZE_X +	Retrieves the width required to draw, for every level calculated, all wavelet coefficient results (diagonal, horizontal, vertical, and the approximation), from an M_WAVELET_TRANSFORM_RESULT result buffer. You can perform the related drawing operation using MimDraw() with M_DRAW_WAVELET.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_DRAW_SIZE_X_WITH_PADDING +	Retrieves the width required to draw, for every level calculated, all wavelet coefficient results (diagonal, horizontal, vertical, and the approximation), from an M_WAVELET_TRANSFORM_RESULT result buffer. You can perform the related drawing operation using MimDraw() with M_DRAW_WAVELET_WITH_PADDING.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_DRAW_SIZE_Y +	Retrieves the height required to draw, for every level calculated, all wavelet coefficient results (diagonal, horizontal, vertical, and the approximation), from an M_WAVELET_TRANSFORM_RESULT result buffer. You can perform the related drawing operation using MimDraw() with M_DRAW_WAVELET.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_DRAW_SIZE_Y_WITH_PADDING +	Retrieves the height required to draw, for every level calculated, all wavelet coefficient results (diagonal, horizontal, vertical, and the approximation), from an M_WAVELET_TRANSFORM_RESULT result buffer. You can perform the related drawing operation using MimDraw() with M_DRAW_WAVELET_WITH_PADDING.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_SIZE +	Retrieves the size of the wavelet transformation, from an M_WAVELET_TRANSFORM_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_TYPE +	Retrieves the type of the wavelet transformation, from an M_WAVELET_TRANSFORM_RESULT result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CUSTOM	Specifies a custom wavelet processing.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DAUBECHIES_1	Specifies a Daubechies wavelet filter that uses 1 vanishing moment and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_2	Specifies a Daubechies wavelet filter that uses 2 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_3	Specifies a Daubechies wavelet filter that uses 3 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_3_COMPLEX	Specifies a Daubechies wavelet filter that uses 3 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_4	Specifies a Daubechies wavelet filter that uses 4 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_5	Specifies a Daubechies wavelet filter that uses 5 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_5_COMPLEX	Specifies a Daubechies wavelet filter that uses 5 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_6	Specifies a Daubechies wavelet filter that uses 6 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_7	Specifies a Daubechies wavelet filter that uses 7 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_7_COMPLEX	Specifies a Daubechies wavelet filter that uses 7 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_8	Specifies a Daubechies wavelet filter that uses 8 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HAAR	Specifies a Haar wavelet filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_1	Specifies a Symlet wavelet filter that uses 1 vanishing moment and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_2	Specifies a Symlet wavelet filter that uses 2 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_3	Specifies a Symlet wavelet filter that uses 3 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_4	Specifies a Symlet wavelet filter that uses 4 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SYMLET_5	Specifies a Symlet wavelet filter that uses 5 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_6	Specifies a Symlet wavelet filter that uses 6 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_7	Specifies a Symlet wavelet filter that uses 7 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_8	Specifies a Symlet wavelet filter that uses 8 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_AUG_ASPECT_RATIO; M_AUG_ASPECT_RATIO_MODE; M_AUG_BLUR_MOTION_ANGLE; M_AUG_BLUR_MOTION_SIZE; M_AUG_CROP_BOTTOM_RIGHT_X; M_AUG_CROP_BOTTOM_RIGHT_Y; M_AUG_CROP_TOP_LEFT_X; M_AUG_CROP_TOP_LEFT_Y; M_AUG_DILATION_ASYM_NB_ITERATIONS; M_AUG_DILATION_ASYM_SUBAREA; M_AUG_DILATION_NB_ITERATIONS; M_AUG_EROSION_ASYM_NB_ITERATIONS; M_AUG_EROSION_ASYM_SUBAREA; M_AUG_EROSION_NB_ITERATIONS; M_AUG_FLIP_DIRECTION; M_AUG_GAMMA_VALUE_BAND_0; M_AUG_GAMMA_VALUE_BAND_1; M_AUG_GAMMA_VALUE_BAND_2; M_AUG_GAMMA_VALUE_BAND + n; M_AUG_GAMMA_VALUE_NB_BANDS; M_AUG_HSV_VALUE_GAIN; M_AUG_HUE_OFFSET; M_AUG_INTENSITY_ADD_VALUE; M_AUG_INTENSITY_MULTIPLY_VALUE; M_AUG_LIGHTING_DIRECTIONAL_ANGLE; M_AUG_NOISE_GAUSSIAN_ADDITIVE_STDDEV; M_AUG_NOISE_MULTIPLICATIVE_STDDEV; M_AUG_NOISE_SALT_PEPPER_DENSITY; M_AUG_OPERATION_ASSOCIATED_WITH_RESULT_TYPES; M_AUG_OPERATION_RESULT_TYPES; M_AUG_OPERATION_RESULT_VALUES; M_AUG_OPERATIONS_APPLIED; M_AUG_OPERATIONS_ENABLED; M_AUG_OPTIMAL_SIZE_X; M_AUG_OPTIMAL_SIZE_Y; M_AUG_REDUCE_FACTOR; M_AUG_REDUCE_OFFSET_X; M_AUG_REDUCE_OFFSET_Y; M_AUG_ROTATION_ANGLE; M_AUG_SATURATION_GAIN; M_AUG_SCALE_FACTOR; M_AUG_SEED_USED; M_AUG_SHARPEN_DERICHE_VALUE; M_AUG_SHEAR_X; M_AUG_SHEAR_Y; M_AUG_SMOOTH_DERICHE_VALUE; M_AUG_SMOOTH_GAUSSIAN_STDDEV; M_AUG_TRANSLATION_X; M_AUG_TRANSLATION_Y.

You can add the following value to the above-mentioned values to specify whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested result type is available for retrieval.															
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CUMULATIVE_VALUE; M_VALUE.

You can add the following value to the above-mentioned values to get the requested results as a percentage. Alternatively, you can add a value from For determining the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results as a percentage																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																

M_PERCENTAGE	Retrieves the histogram values or cumulative histogram values, as a percentage. The values are multiplied by 100 and then divided by the total number of elements in the histogram.															
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ResultType=M_TRANSFORMATION_MODE: M_UNDECIMATED].

The following value might be combined with the above-mentioned value to determine if undecimated wavelet transformations are centered.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether undecimated wavelet transformations are centered																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CENTER	Specifies undecimated wavelet transformations that are centered. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_AUG_OPERATION_ASSOCIATED_WITH_RESULT_TYPES; M_AUG_OPERATION_RESULT_TYPES; M_AUG_OPERATION_RESULT_VALUES; M_AUG_OPERATIONS_APPLIED; M_AUG_OPERATIONS_ENABLED; M_CUMULATIVE_VALUE; M_PERCENTILE_VALUE; M_POSITION_X; M_POSITION_Y; M_STAT_ANGULAR_DATA_COHERENCE; M_STAT_ANGULAR_DATA_MEAN; M_STAT_GLCM_CONTRAST; M_STAT_GLCM_CORRELATION; M_STAT_GLCM_DISSIMILARITY; M_STAT_GLCM_ENERGY; M_STAT_GLCM_ENTROPY; M_STAT_GLCM_HOMOGENEITY; M_STAT_MAX; M_STAT_MAX_ABS; M_STAT_MEAN; M_STAT_MIN; M_STAT_MIN_ABS; M_STAT_NUMBER; M_STAT_ORIENTATION_DATA_COHERENCE; M_STAT_ORIENTATION_DATA_MEAN; M_STAT_STANDARD_DEVIATION; M_STAT_SUM; M_STAT_SUM_ABS; M_STAT_SUM_OF_SQUARES; M_VALID; M_VALUE.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values. Alternatively, you can add a value from For retrieving results as a percentage.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For a result buffer.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the values listed above are not available in Aurora Imaging Library Lite. See the result's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimGetResultSingle

Synopsis

Get a result from a wavelet transformation or 1D locate peak result buffer.

Syntax

```
void MimGetResultSingle(
    AIL_ID ResultImId,    //in
    AIL_INT64 Index1,     //in
    AIL_INT64 Index2,     //in
    AIL_INT64 ResultType, //in
    void *UserArrayPtr    //out
)
```

Description

This function retrieves a result of the specified type from a wavelet transformation or 1D locate peak result buffer. Aurora Imaging Library performs wavelet transformations with MimWaveletTransform() and 1D peak intensity detection with MimLocatePeak1d().

Parameters

ResultImId

Specifies the identifier of the wavelet transformation or 1D locate peak result buffer from which to get results.

Index1

Specifies the first index with which to indicate the result to retrieve.

For a wavelet transformation result, this parameter specifies a specific wavelet coefficient and level for which to get information. To retrieve the actual number of levels calculated, call MimGetResult() with M_NUMBER_OF_LEVELS. You can also use this parameter to retrieve information about the approximation (the low frequency rendition) of the wavelet transformation.

Set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the first index, for which to retrieve the wavelet transformation result

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIAGONAL_LEVEL(AIL_INT Level)	Retrieves results about the diagonal wavelet coefficient, at the specified transformation level.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Level Specifies the transformation level.															
	0 < *Value* <= M_NUMBER_OF_LEVELS Specifies the valid transformation level.	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL_LEVEL(AIL_INT Level)	Retrieves results about the horizontal wavelet coefficient, at the specified transformation level.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Level Specifies the transformation level.															

	0 < *Value* <= M_NUMBER_OF_LEVELS Specifies the valid transformation level.	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL_LEVEL(AIL_INT Level)	Retrieves results about the vertical wavelet coefficient, at the specified transformation level.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Level Specifies the transformation level.															
	0 < *Value* <= M_NUMBER_OF_LEVELS Specifies the valid transformation level.	c		e	f	g	a		i		j	k	l	m	n	b
M_APPROXIMATION	Retrieves results about the approximation (the low frequency rendition) of the wavelet transformation at the last calculated level.	c		e	f	g	a		i		j	k	l	m	n	b

For a 1D locate peak result, set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of the peak in the lane(s), for which to retrieve the 1D locate peak result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no information is required for the Index1 parameter. This setting is only valid if you are retrieving the number of scan lanes (M_NUMBER_OF_SCAN_LANES) or when retrieving the initialization state of the result buffer with M_STATUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_SELECT_PEAK(AIL_INT Frame, AIL_INT Rank)	Specifies the frame(s) and rank of the peak(s) for which to return the requested result (for the lane(s) specified with the Index2 parameter). This macro is required if a MimLocatePeak1d() operation accumulated results from multiple frames in the result buffer (MimControl() with M_NUMBER_OF_FRAMES set to a value greater than 1).	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Frame Specifies the index of the frame(s) for which the result was stored.															
	M_ALL Specifies all frames.															
	Value >= 0 Specifies the index of the specific frame.	c		e	f	g	a		i		j	k	l	m	n	b
	Rank Specifies the peak(s) for which to return the requested result.															
	M_ALL Specifies all peaks in the lane(s).	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0 Specifies the rank of the specific peak in the lane(s).	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Retrieves the requested result for all peaks in the scan lane(s) specified using the Index2 parameter. If peaks are absent from a lane, Aurora Imaging Library skips it unless M_INCLUDE_MISSING_DATA is specified. Ordering of the results is dependent on MimControl() with M_SORT_CRITERION. Note that this setting is equivalent to using the M_SELECT_PEAK() macro above, with the Frame and Rank parameters set to 0 and M_ALL, respectively.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER + M_INCLUDE_MISSING_DATA	Specifies the rank of the peak for which to retrieve the requested result, for the scan lane(s) specified using the Index2 parameter. Ordering of the results is dependent on MimControl() with M_SORT_CRITERION. If the peak is absent from the lane, Aurora Imaging Library skips it, unless M_INCLUDE_MISSING_DATA is specified.	c		e	f	g	a		i		j	k	l	m	n	b

Index2

Specifies the second index for which to retrieve results. For a wavelet transformation result, this information is not required; set this parameter to M_NULL.

For a 1D locate peak result, set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of the lane for which to retrieve the 1D locate peak result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no information is required for the Index2 parameter. For a 1D locate peak result, this setting is only valid if you are retrieving the number of scan lanes (M_NUMBER_OF_SCAN_LANES) or when retrieving the initialization state of the result buffer with M_STATUS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to retrieve the requested result for all scan lanes. If the specified peak(s) are absent from a lane, Aurora Imaging Library skips it unless M_INCLUDE_MISSING_DATA is specified. Ordering of the results is dependent on MimControl() with M_SORT_CRITERION.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_SCAN_LANES	Specifies the lane for which to retrieve the requested result. If the specified peak(s) are absent from a lane, Aurora Imaging Library skips it unless M_INCLUDE_MISSING_DATA is specified. Empty elements are added after the other elements, at the end of the returned array. Ordering of the results is dependent on MimControl() with M_SORT_CRITERION.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]

- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the one-dimensional array in which to write the results read from the Aurora Imaging Library result buffer.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **UserArrayPtr** parameter.

- For specifying the type of result to retrieve, when using a wavelet transformation result
- For specifying the type of result to retrieve, when using a 1D locate peak result

To retrieve a result from the specified wavelet coefficient and detail level (Index1), set ResultType to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to retrieve, when using a wavelet transformation result																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																	
M_WAVELET_COEFFICIENTS_IMAGE_ID +	Retrieves the identifier of the internal image buffer that holds the wavelet data specified by the Index1 parameter. For example, if you use M_WAVELET_COEFFICIENTS_IMAGE_ID with the Index1 parameter set to M_DIAGONAL_LEVEL(3), you will retrieve the identifier of the internal image buffer that holds the diagonal wavelet coefficient at level three. The type and depth (per band) of the data in each internal wavelet buffer is 32-bit floating-point. There is a separate internal image buffer for every wavelet coefficient result, at every level, and for each numerical domain. You can specify the domain by combining M_WAVELET_COEFFICIENTS_IMAGE_ID with M_REAL_PART (the default) or M_IMAGINARY_PART. To display an internal image buffer, you must inquire its system using MbufInquire() with M_OWNER_SYSTEM, and allocate a display on it using MdispAlloc(). You can then use this allocated display to visualize the content of M_WAVELET_COEFFICIENTS_IMAGE_ID.																
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the internal image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_LEVEL_PADDING_OFFSET_X +	Retrieves the X-offset of the padding used to calculate the result.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_LEVEL_PADDING_OFFSET_Y +	Retrieves the Y-offset of the padding used to calculate the result.	c		e	f	g	a		i		j	k	l	m	n	b

	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_LEVEL_SIZE_X +		Retrieves the width of the undistorted (non-padded) region that was used to calculate the result.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_LEVEL_SIZE_Y +		Retrieves the height of the undistorted (non-padded) region that was used to calculate the result.	c		e	f	g	a		i		j	k	l	m	n	b
	UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_WAVELET_COEFFICIENTS_IMAGE_ID.

You can add one of the following values to the above-mentioned value to specify whether to retrieve information about the real or imaginary numbers in the wavelet result.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to retrieve information about the real or imaginary numbers in the wavelet result																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IMAGINARY_PART	Retrieves information about the imaginary part of the values in the wavelet result. Only available for complex wavelet transformations (MimGetResult() with M_TRANSFORMATION_DOMAIN must return M_COMPLEX).	c		e	f	g	a		i		j	k	l	m	n	b
M_REAL_PART	Retrieves information about the real numbers in the result. Available for any type of wavelet transformation. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve a result from a 1D locate peak result buffer, ResultType can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of result to retrieve, when using a 1D locate peak result																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_NUMBER +	Retrieves the number of elements that would be returned, given the values specified for the Index1 and Index2 parameters.															
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_SCAN_LANES +	Retrieves the number of scan lanes. When using M_NUMBER_OF_SCAN_LANES, you must set the Index1 and Index2 parameters to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_PEAK_INTENSITY +	Retrieves the peak's intensity. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, empty elements are set to the value specified by M_PEAK_INTENSITY_INVALID_VALUE, and placed at the end of the returned array. The default intensity value for missing peaks is -1 (or an unsigned buffer's maximum value).	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_POSITION +	Retrieves the peak's position along the scan lane, in pixels. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, empty elements are set to M_INVALID and placed at the end of the returned array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_POSITION_X +	Retrieves the X-coordinate of the peak's position along the image's X-axis, in pixels. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, and M_SCAN_LANE_DIRECTION is set to M_HORIZONTAL, empty elements are set to M_INVALID and placed at the end of the returned array. If, however, M_SCAN_LANE_DIRECTION is set to M_VERTICAL, the lane indices of the empty elements are placed at the end of the returned array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_POSITION_Y +	Retrieves the Y-coordinate of the peak's position along the image's Y-axis, in pixels. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, and M_SCAN_LANE_DIRECTION is set to M_VERTICAL, empty elements are set to M_INVALID and placed at the end of the returned array. If, however, M_SCAN_LANE_DIRECTION is set to M_HORIZONTAL, the lane indices of the empty elements are placed at the end of the returned array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_WIDTH +	Retrieves the peak's detected width. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, empty elements are set to M_INVALID and placed at the end of the returned array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_RANK_INDEX +	Retrieves the peak's index along the scan lane. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, the rank indices of the empty elements are placed at the end of the returned array.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_LANE_INDEX +	Retrieves the lane's index of the data. Note that if the M_INCLUDE_MISSING_DATA combination value is specified, the lane indices of the empty elements are placed at the end of the returned array.	c		e	f	g	a		i		j	k	l	m	n	b

UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +	Retrieves the initialization state of a 1D locate peak result buffer. When using M_STATUS, you must set the Index1 and Index2 parameters to zero; otherwise, Aurora Imaging Library will generate an error.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies that the result buffer has been preprocessed and contains data from a previous operation. Once data is accumulated using MimLocatePeak1d(), M_STATUS will return M_COMPLETE.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNINITIALIZED	Specifies that the result buffer has not been preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_VALID	Specifies that the result buffer has been properly preprocessed, but has not been used in any operation. After preprocessing, but before accumulating any data, M_STATUS will return M_VALID. To preprocess a result buffer without accumulating data, use MimLocatePeak1d() with M_PREPROCESS.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_NUMBER; M_PEAK_INTENSITY; M_PEAK_POSITION; M_PEAK_POSITION_X; M_PEAK_POSITION_Y; M_PEAK_WIDTH; M_RANK_INDEX; M_SCAN_LANE_INDEX.

You can add one of the following values to the above-mentioned values to specify how to handle missing data, when using a 1D locate peak result.

If you do not specify any of these combination constants, Aurora Imaging Library does not include missing data when returning information about the specified result type. That is, empty scan lanes and ranks are skipped.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to handle missing data, when using a 1D locate peak result																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_INCLUDE_MISSING_DATA +	Specifies that missing data is included as part of the information returned by the result type.															
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an empty lane or rank.	c		e	f	g	a		i		j	k	l	m	n	b
M_ONLY_MISSING_DATA +	Specifies that only empty elements are returned by the result type. Alignment with other peaks is not checked. Empty lanes for rank *i* are always those where at most *i*-1 peaks were detected even when data inspection suggests that the last detected peak should be part of rank *i*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies an empty lane or rank.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PEAK_POSITION; M_PEAK_POSITION_X; M_PEAK_POSITION_Y.

You can add the following value to the above-mentioned values to specify returned data using fixed point with fractional bits, when using a 1D locate peak result.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For using fixed point with fractional bits, when using a 1D locate peak result																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIXED_POINT + *n*	Expresses results using fixed point with *n* fractional bits. Results must be an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For specifying the type of result to retrieve, when using a 1D locate peak result; and for the following values: M_INCLUDE_MISSING_DATA; M_ONLY_MISSING_DATA.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
UserArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the type of result to retrieve, when using a wavelet transformation result, For specifying the type of result to retrieve, when using a 1D locate peak result.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the ResultType parameter to cast the required results to the requested data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]* **|** *address of an AIL_FLOAT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimHistogram

Synopsis

Generate the intensity histogram of an image.

Syntax

```
void MimHistogram(
    AIL_ID SrcImageBufId, //in
    AIL_ID HistResultImId //out
)
```

Description

This function calculates the histogram (or pixel intensity distribution) of the specified source image and stores the results in the specified result buffer.

You can control the bin size of the calculated histogram, and the number of smoothing iterations to perform on it, by calling MimControl() with M_HIST_BIN_SIZE_MODE and M_HIST_SMOOTHING_ITERATIONS. By default, a histogram has 1 value per bin, and has not been smoothed.

To read the resulting histogram values from the result buffer, use MimGetResult1d() or MimGetResult(), and specify M_CUMULATIVE_VALUE, M_HIST_..., M_PERCENTILE_VALUE, or M_VALUE as the result type.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer. The image buffer must be 1-band.

Buffer data will be treated as unsigned, if MimControl() with M_HIST_BIN_SIZE_MODE is set to M_DEFAULT or M_REGULAR. That is, floating-point values will be cast to *AIL_UINT* values before generating the histogram. In this case, unexpected results can occur if a floating-point value exceeds the *AIL_UINT* range.

Signed buffers are remapped in the result such that the minimum value is 0, if you set M_HIST_BIN_SIZE_MODE to M_FIT_SRC_DATA, M_FIT_SRC_RANGE, or M_FREEDMAN.

8-, 16-, and 32-bit source image buffers are supported, unless you set M_HIST_BIN_SIZE_MODE to M_FREEDMAN. In this case, 8- and 16-bit source image buffers are supported.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

HistResultImId

Specifies the identifier of the destination for the histogram results. This parameter must be given the identifier of an image processing result buffer that was allocated with MimAllocResult() and has an M_HIST_LIST type.

If the result buffer has fewer entries than the full range of source values, the pixel values that are out of range will not be included in the histogram. If, however, the result buffer has more entries than the full range of source values, the entries that are out of range will have an undetermined value.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimHistogramEqualize

Synopsis

Perform a histogram equalization of an image.

Syntax

```
void MimHistogramEqualize(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT64 Operation,  //in
    AIL_DOUBLE Alpha,     //in
    AIL_DOUBLE Min,       //in
    AIL_DOUBLE Max        //in
)
```

Description

This function performs a histogram equalization of the specified source image. Results are written to a destination buffer, which can be either an image buffer or a LUT buffer.

This function first performs a histogram of the source image buffer. The histogram is then used to calculate a transformation LUT that can be used to enhance the source image (with MimLutMap()). If the destination buffer is a LUT, the transformation LUT is copied into the destination LUT. If the destination buffer is an image, the source buffer is remapped through the transformation LUT to produce the destination image.

Note that floating-point values will be cast to *AIL_UINT32* values before performing the histogram. Therefore, unexpected results can occur if a floating-point value is larger than the *AIL_UINT32* range.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

If you specify multiple image buffers with an ROI, this function performs a histogram using the source ROI, and writes to the destination only those pixels that intersect between the source ROI and the destination ROI. This is consistent with the behavior of this function when ROIs are not specified.

If **MimHistogramEqualize**() is not producing the results you expect, try performing an adaptive histogram equalization, using MimHistogramEqualizeAdaptive()). Though an adaptive histogram equalization can take longer to execute (more calculations to process), it can help minimize the over-amplification of noise and the potential for saturation. Unlike **MimHistogramEqualize**(), MimHistogramEqualizeAdaptive() is not available with Aurora Imaging Library Lite.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter will be treated as an unsigned image buffer identifier.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer or LUT buffer identifier.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Operation

Specifies the equalization operation to perform. This parameter can be set to one of the following values. Note that the cumulative probability distribution, $P_f(f)$, of the input image is approximated by its cumulative histogram:

$$P_f(f) = \sum_{m=0}^{j} H_F(m) \in [0, 1]$$

Refer to "Pratt, William K. *Digital Image Processing* . United States: John Wiley & Sons, 1978. 318.".

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of equalization to perform

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EXPONENTIAL	Specifies an equalization density function which generates an Exponential distribution. Output probability density model: $P_g(g) = \alpha e^{\{-\alpha(g-g_{min})\}} \quad g \geq g_{min}, \alpha > 0$ Transfer function: $g = \begin{cases} min(\ (g_{min} - \frac{1}{\alpha}\ln\{1 - P_f(f)\})\ ,\ g_{max}) & \text{if } P_f(f) < 1.0 \\ g_{max} & \text{if } P_f(f) = 1.0 \end{cases}$	c		e	f	g	a		i		j	k	l	m	n	b
M_HYPER_CUBE_ROOT	Specifies an equalization density function which generates a Hyperbolic Cube Root distribution. Output probability density model: $P_g(g) = \frac{1}{3}\left[\frac{g^{-2/3}}{\sqrt[3]{g_{max}} - \sqrt[3]{g_{min}}}\right]$ Transfer function: $g = \left[\left(\sqrt[3]{g_{max}} - \sqrt[3]{g_{min}}\right)\left(P_f(f)\right) + \sqrt[3]{g_{min}}\right]^3$	c		e	f	g	a		i		j	k	l	m	n	b
M_HYPER_LOG	Specifies an equalization density function which generates a Hyperbolic Logarithmic distribution. Output probability density model: $P_g(g) = \frac{1}{g\left[\ln(g_{max}) - \ln(g_{min})\right]}$ Transfer function: $g = g_{min}\left[\frac{g_{max}}{g_{min}}\right]^{P_f(f)}$	c		e	f	g	a		i		j	k	l	m	n	b
M_RAYLEIGH	Specifies an equalization density function which generates a Rayleigh distribution. Output probability density model: $P_g(g) = \frac{g - g_{min}}{\alpha^2}\left[e^{\left[-\frac{(g-g_{min})^2}{2\alpha^2}\right]}\right] \quad g \geq g_{min}, \alpha > 0$ Transfer function: $g = \begin{cases} min(\ (g_{min} + \left[2\alpha^2 \ln\left\{\frac{1}{1-P_f(f)}\right\}\right]^{\frac{1}{2}})\ ,\ g_{max}) & \text{if } P_f(f) < 1.0 \\ g_{max} & \text{if } P_f(f) = 1.0 \end{cases}$	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies an equalization density function which generates a Uniform distribution. Output probability density model: $P_g(g) = \frac{1}{g_{max} - g_{min}}$ $g_{min} \leq g \leq g_{max}$ Transfer function: $g = \left[g_{max} - g_{min}\right]P_f(f) + g_{min}$	c		e	f	g	a		i		j	k	l	m	n	b

Alpha

Specifies the value to use for alpha (α) in the equations for M_EXPONENTIAL and M_RAYLEIGH. For other Operation parameter values, set Alpha to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the alpha value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value > 0	Specifies the value to use for alpha (α). For an M_EXPONENTIAL operation, a greater alpha value yields a lower occurrence of the most frequent pixels of the histogram in the resulting image buffer. When operating on an 8-bit unsigned image, a typical value for alpha is between 0.001 and 0.1. For an M_RAYLEIGH operation, the greater the alpha is, the greater the occurrence of the most frequent pixels of the histogram in the resulting image buffer. When operating on an 8-bit unsigned image, a typical value for alpha is between 10 and 1000.	c		e	f	g	a		i		j	k	l	m	n	b

Min

Specifies the lowest pixel value to equalize.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the lowest pixel value to equalize																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value >= 0	Specifies the lowest pixel value.	c		e	f	g	a		i		j	k	l	m	n	b

Max

Specifies the highest pixel value to equalize.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the highest pixel value to equalize																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Min <= Value <= max buffer value	Specifies the highest pixel value.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimHistogramEqualizeAdaptive

Synopsis

Perform an adaptive histogram equalization on an image.

Syntax

```
void MimHistogramEqualizeAdaptive(
    AIL_ID AdaptiveEqualizeContextImId, //in
    AIL_ID SrcImageBufId,               //in
    AIL_ID DstImageBufId,               //out
    AIL_INT64 ControlFlag               //in
)
```

Description

This function enhances the contrast of the specified source image by applying a Contrast Limited Adaptive Histogram Equalization (CLAHE). The resulting image is placed in the destination image buffer.

This function first partitions the specified source image into a set of rectangular sections of equal size, referred to as tiles, and calculates histograms for each one. The histograms are then used to transform the image and produce an enhanced (equalized) version of it. By default, **MimHistogramEqualizeAdaptive**() uses 8 tiles along the image's X- and Y-direction. To modify the number of tiles, call MimControl() with M_NUMBER_OF_TILES_X and M_NUMBER_OF_TILES_Y. Though **MimHistogramEqualizeAdaptive**() can take longer to execute than a conventional histogram equalization (MimHistogramEqualize()), it can help minimize the over-amplification of noise and the potential for saturation, particularly in images with inconsistent lighting where some sections are significantly brighter than others.

By default, the maximum percentage of pixels that can be represented by a histogram bin is limited to 1% of all the pixels in its respective tile. Exceeding values are distributed evenly among that histogram's other bins. To modify this behavior, use the M_CLIP_LIMIT control.

You can also modify the number of histogram bins for each tile, as well as the equalization operation performed, using the M_HIST_SIZE and M_OPERATION controls. By default, **MimHistogramEqualizeAdaptive**() determines the number of histogram bins automatically according to the specified source image and performs a uniform-type of equalization.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

AdaptiveEqualizeContextImId

Specifies the identifier of the adaptive histogram equalization context. The context must have been allocated using MimAlloc() with M_HISTOGRAM_EQUALIZE_ADAPTIVE_CONTEXT.

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer must be a 1-band, 8- or 16-bit image buffer allocated using MbufAlloc...() with M_IMAGE + M_PROC. The type of the source image buffer must be the same as the destination image buffer. To inquire this information, use MbufInquire() with M_TYPE.

DstImageBufId

Specifies the identifier of the destination image buffer. The buffer must be a 1-band, 8- or 16-bit image buffer allocated using MbufAlloc...() with M_IMAGE + M_PROC. The type of the destination image buffer must be the same as the source image buffer. To inquire this information, use MbufInquire() with M_TYPE.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimInquire

Synopsis

Inquire about an image processing context or result buffer setting.

Syntax

```
AIL_INT MimInquire(
    AIL_ID ContextOrResultImId, //in
    AIL_INT64 InquireType,      //in
    void *UserVarPtr            //out
)
```

Description

This function inquires information about an image processing context or result buffer setting. The image processing context or result buffer must have been allocated with MimAlloc() or MimAllocResult(), respectively.

Parameters

ContextOrResultImId

Specifies the identifier of the image processing context or result buffer about which to inquire information.

See the Parameter associations section for possible values that can be specified.

InquireType

Specifies the type of information about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the variable in which to write the requested information. Since the **MimInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ContextOrResultImId** and **InquireType** parameters and possible values returned to the **UserVarPtr** parameter.

- For inquiring about any image processing result buffer
- For inquiring about any image processing context
- For inquiring about a specific type of image processing context
- For inquiring about a specific type of image processing result buffer

For any image processing result buffer, the ContextOrResultImId and InquireType parameters can be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about any image processing result buffer																	
ContextOrResultImId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType																
UserVarPtr - Possible values returned																

Image processing result buffer ID	Specifies an image processing result buffer, allocated using MimAllocResult().														
M_ALLOCATED_TYPE +	Inquires the image processing result buffer's data type, as was set using MimAllocResult() upon allocation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the image processing result buffer's data-type cannot be determined. Note that any result buffer that does not support allocation with a combination value from the For M_PROJ_LIST, M_EXTREME_LIST, or M_EVENT_LIST to change the type of data to store table will return this value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Specifies that the image processing result buffer was allocated with an *AIL_FLOAT* data-type.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Specifies that the image processing result buffer was allocated with an *AIL_INT* data-type.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_COUNT +	Inquires the number of modifications made to the result buffer since it was allocated.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of modifications.	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_SYSTEM +	Inquires the identifier of the system on which the result buffer is allocated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_SIZE +	Inquires the number of entries in the result buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of buffer entries. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_TYPE +	Inquires the attribute or nature of the result buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUGMENTATION_RESULT	Specifies that the buffer can hold MimAugment() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_COUNT_LIST	Specifies that the buffer can hold MimCountDifference() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_EVENT_LIST	Specifies that the buffer can hold MimLocateEvent() results.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTREME_LIST	Specifies that the buffer can hold MimFindExtreme() results.	c		e	f	g	a		i		j	k	l	m	n	b

		M_FIND_ORIENTATION_LIST	Specifies that the buffer can hold MimFindOrientation() results.	c		e	f	g	a		i		j	k	l	m	n	b
		M_HIST_LIST	Specifies that the buffer can hold MimHistogram() results.	c		e	f	g	a		i		j	k	l	m	n	b
		M_LOCATE_DIFFERENCE_RESULT	Specifies that the buffer can hold MimLocateDifference() results.	c		e	f	g	a		i		j	k	l	m	n	b
		M_LOCATE_PEAK_1D_RESULT	Specifies that the buffer can hold MimLocatePeak1d() results. INFO	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROJ_LIST	Specifies that the buffer can hold MimProjection() results.	c		e	f	g	a		i		j	k	l	m	n	b
		M_STATISTICS_RESULT	Specifies that the buffer can hold MimStatCalculate() results.	c		e	f	g	a		i		j	k	l	m	n	b
		M_WAVELET_TRANSFORM_RESULT	Specifies that the buffer can hold results from MimWaveletTransform() and MimWaveletDenoise(). INFO	c		e	f	g	a		i		j	k	l	m	n	b

For any image processing context, the ContextOrResultImId and InquireType parameters can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about any image processing context																	
ContextOrResultImId		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	InquireType																
		UserVarPtr - Possible values returned															
Image processing context ID		Specifies an image processing context, allocated using MimAlloc().															
	M_CONTEXT_TYPE +	Inquires the type of the image processing context. SET	c		e	f	g	a		i		j	k	l	m	n	b
		UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
		M_AUGMENTATION_CONTEXT	Specifies an image processing context that can be used with MimAugment().	c		e	f	g	a		i		j	k	l	m	n	b
		M_BINARIZE_ADAPTIVE_CONTEXT	Specifies an image processing context that can be used with MimBinarizeAdaptive() to perform an adaptive binarization.	c		e	f	g	a		i		j	k	l	m	n	b
		M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT	Specifies an image processing context that can be used with MimBinarizeAdaptive() to perform an adaptive binarization using seeds.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEAD_PIXEL_CONTEXT	Specifies an image processing context that can be used with MimDeadPixelCorrection().	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEINTERLACE_CONTEXT	Specifies an image processing context that can be used with MimDeinterlace().	c		e	f	g	a		i		j	k	l	m	n	b

		M_FILTER_MAJORITY_CONTEXT	Specifies an image processing context that can be used with MimFilterMajority().	c		e	f	g	a		i		j	k	l	m	n	b
		M_FIND_ORIENTATION_CONTEXT	Specifies an image processing context that can be used with MimFindOrientation().	c		e	f	g	a		i		j	k	l	m	n	b
		M_FLAT_FIELD_CONTEXT	Specifies an image processing context that can be used with MimFlatField().	c		e	f	g	a		i		j	k	l	m	n	b
		M_HISTOGRAM_EQUALIZE_ADAPTIVE_CONTEXT	Specifies an image processing context that can be used with MimHistogramEqualizeAdaptive().	c		e	f	g	a		i		j	k	l	m	n	b
		M_IM_REMAP_CONTEXT	Specifies an image processing context that can be used with MimRemap().	c		e	f	g	a		i		j	k	l	m	n	b
		M_LINEAR_FILTER_IIR_CONTEXT	Specifies an image processing context that can be used with MimConvolve() or MimDifferential().	c		e	f	g	a		i		j	k	l	m	n	b
		M_LOCATE_PEAK_1D_CONTEXT	Specifies an image processing context that can be used with MimLocatePeak1d().	c		e	f	g	a		i		j	k	l	m	n	b
		M_MATCH_CONTEXT	Specifies an image processing context that can be used with MimMatch().	c		e	f	g	a		i		j	k	l	m	n	b
		M_REARRANGE_CONTEXT	Specifies an image processing context that can be used with MimRearrange().	c		e	f	g	a		i		j	k	l	m	n	b
		M_STATISTICS_CONTEXT	Specifies an image processing context that can be used with MimStatCalculate().	c		e	f	g	a		i		j	k	l	m	n	b
		M_STATISTICS_CUMULATIVE_CONTEXT	Specifies an image processing context that can be used with MimStatCalculate() when calculating statistics from multiple images.	c		e	f	g	a		i		j	k	l	m	n	b
		M_UNWARP_ALONG_PATH_CONTEXT	Specifies an image processing context that can be used with MimUnwarpAlongPath().	c		e	f	g	a		i		j	k	l	m	n	b
		M_WAVELET_TRANSFORM_CONTEXT	Specifies an image processing context that can be used with MimWaveletTransform() and MimWaveletDenoise().	c		e	f	g	a		i		j	k	l	m	n	b
		M_WAVELET_TRANSFORM_CUSTOM_CONTEXT	Specifies a custom image processing context that can be used with MimWaveletTransform() and MimWaveletDenoise().	c		e	f	g	a		i		j	k	l	m	n	b

For an image processing context of a specific type, the ContextOrResultImId and InquireType parameters can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a specific type of image processing context																		
ContextOrResultImId		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	InquireType																	

	UserVarPtr - Possible values returned														
Adaptive binarize context ID		Specifies an adaptive binarize context, allocated using MimAlloc() with M_BINARIZE_ADAPTIVE_CONTEXT, and used in MimBinarizeAdaptive() operations.													
M_AVERAGE_MODE +		Inquires how Aurora Imaging Library establishes average pixel values. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_GAUSSIAN	Specifies a Gaussian type average.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNIFORM	Specifies a uniform type average.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +		Inquires whether the objects to binarize are lighter or darker than the background. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_BLACK	Specifies that the objects to binarize are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_WHITE	Specifies that the objects to binarize are lighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_MAX +		Inquires the maximum threshold value. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that there is no maximum threshold value restriction.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the maximum threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_MIN +		Inquires the minimum threshold value. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that there is no minimum threshold value restriction.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum threshold value.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_OFFSET +		Inquires the offset to add to each established threshold value. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b

M_GLOBAL_OFFSET_SECOND_PASS +	Inquires the offset to apply to the threshold values for the second pass of a hysteresis adaptive binarization. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL_DIMENSION +	Inquires the size of the neighborhood that the threshold mode uses to establish threshold values. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the size of the neighborhood, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST +	Inquires the minimum contrast between background and foreground pixels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b
M_NIBLACK_BIAS +	Inquires the bias for Niblack's binarization mode. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the bias. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NIBLACK_BIAS_SECOND_PASS +	Inquires the bias for the second pass of a Niblack adaptive binarization that uses a hysteresis process. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the bias.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +	Inquires how Aurora Imaging Library establishes the threshold values with which to binarize the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_BERNSEN	Specifies that threshold values are established using Bernsen's adaptive threshold algorithm. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL_MEAN	Specifies that threshold values are established using adaptive local mean calculations. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NIBLACK	Specifies that threshold values are established using Niblack's adaptive threshold algorithm. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_PSEUDOMEDIAN	Specifies that threshold values are established using adaptive pseudomedian calculations. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_TYPE +	Inquires the threshold type used for the adaptive binarization. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_HYSTERESIS	Specifies that the adaptive binarization uses a hysteresis process. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_IN_RANGE	Specifies to use the values inside the range defined by the two passes as the foreground.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_RANGE	Specifies to use the values outside the range defined by the two passes as the foreground.	c		e	f	g	a		i		j	k	l	m	n	b
M_SINGLE	Specifies to use a single pass of the specified threshold. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Adaptive binarize from seed context ID	Specifies an adaptive binarize context that uses seeds, allocated using MimAlloc() with M_BINARIZE_ADAPTIVE_FROM_SEED_CONTEXT, and used in MimBinarizeAdaptive() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_VALUE +	Inquires whether the objects to binarize are lighter or darker than the background. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the objects to binarize are darker than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the objects to binarize are lighter than the background.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL_OFFSET +	Inquires the offset to add to each established threshold value. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_ITERATIONS +	Inquires the number of times to perform the adaptive threshold process specified with M_THRESHOLD_MODE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TO_IDEMPOTENCE	Specifies that the threshold process iterates until idempotence is reached. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of iterations. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_SEED_ITERATIONS +	Inquires the number of iterations with which to internally establish seeds. SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of iterations. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +	Inquires how Aurora Imaging Library uses seeds to establish the threshold values with which to binarize the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_LEVEL	Specifies that threshold values are established using an adaptive leveling. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RECONSTRUCT	Specifies that threshold values are established using an adaptive geodesic reconstruction. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TOGGLE	Specifies that threshold values are established as one of two possibilities, defined by the seeds. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Augmentation context ID	Specifies an augmentation context, allocated using MimAlloc() with M_AUGMENTATION_CONTEXT, and used in MimAugment() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP +	Inquires whether to enable the aspect ratio operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_ASPECT_RATIO_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_ASPECT_RATIO_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP_MAX +	Inquires the maximum aspect ratio to apply for M_AUG_ASPECT_RATIO_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 16.0	Specifies the maximum aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP_MIN +	Inquires the minimum aspect ratio to apply for M_AUG_ASPECT_RATIO_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 16.0	Specifies the minimum aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ASPECT_RATIO_OP_MODE +	Inquires the aspect ratio mode to apply for M_AUG_ASPECT_RATIO_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTH	Specifies that Aurora Imaging Library sets the aspect ratio mode. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_INVERT	Specifies to apply the aspect ratio as height/width. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NORMAL	Specifies to apply the aspect ratio as width/height. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP +		Inquires whether to enable the operation that applies a motion blur kernel with random direction and kernel size. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_BLUR_MOTION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_BLUR_MOTION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_ANGLE_MAX +		Inquires the maximum value for the motion blur angle's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the maximum value for the motion blur angle's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_ANGLE_MIN +		Inquires the minimum value for the motion blur angle's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the minimum value for the motion blur angle's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_SIZE_MAX +		Inquires the maximum value for the kernel size's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 15	Specifies the maximum value for the kernel size's random distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_BLUR_MOTION_OP_SIZE_MIN +		Inquires the minimum value for the kernel size's random distribution [min, max], to apply for M_AUG_BLUR_MOTION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	5 <= *Value* <= 15	Specifies the min value for the kernel size's random distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP +		Inquires whether to enable the crop operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies to disable M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_CROP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP_FACTOR_X +		Inquires the crop width factor, to apply for M_AUG_CROP_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the width factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP_FACTOR_Y +		Inquires the crop height factor, to apply for M_AUG_CROP_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the height factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_CROP_OP_RESIZE +		Inquires the crop rese option, to apply for M_AUG_CROP_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies to not resize.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies to resize the crop dimensions according to the destination buffer size (MimAugment()).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_ASYM_OP +		Inquires whether to enable the asymmetrical dilation operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_DILATION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_DILATION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_ASYM_OP_NB_ITERATIONS_MAX +		Inquires the maximum number of iterations for dilation, to apply for M_AUG_DILATION_ASYM_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_OP +		Inquires whether the dilation operation was enabled. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_DILATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to enable M_AUG_DILATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_DILATION_OP_NB_ITERATIONS_MAX +	Inquires the maximum number of iterations, to apply for M_AUG_DILATION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_ASYM_OP +	Inquires whether to enable the asymmetrical erosion operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_EROSION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_EROSION_ASYM_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_ASYM_OP_NB_ITERATIONS_MAX +	Inquires the maximum number of iterations for dilation, to apply for M_AUG_EROSION_ASYM_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_OP +	Inquires whether to enable the erosion operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_EROSION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_EROSION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_EROSION_OP_NB_ITERATIONS_MAX +	Inquires the maximum number of iterations, to apply for M_AUG_EROSION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_FLIP_OP +	Inquires whether to enable the flip operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_FLIP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_FLIP_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_FLIP_OP_DIRECTION +	Inquires the maximum number of iterations, to apply for M_AUG_EROSION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the maximum number of iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP +		Inquires whether to enable the gamma correction operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_GAMMA_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP_DELTA +		Inquires the range for the random distribution of gamma values [gamma - delta, gamma + delta], to apply for M_AUG_GAMMA_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 10.0	Specifies the range for the random distribution of gamma values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP_MODE +		Inquires the gamma correction mode, to apply for M_AUG_GAMMA_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL_BANDS	Specifies to apply the same gamma correction to each band.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PER_BAND	Specifies to apply a different random gamma correction to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_GAMMA_OP_VALUE +		Inquires the gamma value, to apply for M_AUG_GAMMA_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 10.0	Specifies the gamma value.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HSV_VALUE_GAIN_OP +		Inquires whether to enable the operation that multiplies (gain) the color image's value band (HSV) by a random factor. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_HSV_VALUE_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_HSV_VALUE_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HSV_VALUE_GAIN_OP_MAX +		Inquires the maximum value for the random value factor's distribution [min, max], to use for M_AUG_HSV_VALUE_GAIN_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0.0	Specifies the maximum value for the random value factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HSV_VALUE_GAIN_OP_MIN +		Inquires the minimum value for the random value factor's distribution [min, max], to use for M_AUG_HSV_VALUE_GAIN_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the minimum value for the random value factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET_OP +		Inquires whether to enable the operation that adds a random hue angle to hue values. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_HUE_OFFSET_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_HUE_OFFSET_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET_OP_MAX +		Inquires the maximum value for the random hue angles' distribution [min, max], to use for M_AUG_HUE_OFFSET_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the maximum value for the random hue angles' distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_HUE_OFFSET_OP_MIN +		Inquires the minimum value for the random hue angles' distribution [min, max], to use for M_AUG_HUE_OFFSET_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the minimum value for the random hue angles' distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP +		Inquires whether to enable the intensity addition (offset) operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_INTENSITY_ADD_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_INTENSITY_ADD_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP_DELTA +		Inquires the range for the random distribution of intensity addition (offset) values [offset - delta, offset + delta], to apply for M_AUG_INTENSITY_ADD_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_AUTO_VALUE	Specifies a value that is automatically established according to the data type and data depth of the source image buffer with which to perform MimAugment(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of intensity addition (offset) values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP_MODE +	Inquires the intensity addition value (offset) mode, to apply for M_AUG_INTENSITY_ADD_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies to apply the intensity addition value (offset) to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the number of bands of the source image buffer with which to perform MimAugment(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies to apply the intensity addition value (offset) to the luminance band only. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_ADD_OP_VALUE +	Inquires the intensity addition value (offset), to apply for M_AUG_INTENSITY_ADD_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the intensity addition value (offset).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP +	Inquires whether to enable the intensity multiplication (gain) operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_INTENSITY_MULTIPLY_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_INTENSITY_MULTIPLY_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP_DELTA +	Inquires the range for the random distribution of the intensity multiplication (gain) values [gain - delta, gain + delta], to apply for M_AUG_INTENSITY_MULTIPLY_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of the intensity multiplication (gain) values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP_MODE +	Inquires the multiplication value (gain) mode, to apply for M_AUG_INTENSITY_MULTIPLY_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_ALL_BANDS	Specifies to apply the multiplication value (gain) to each band.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_VALUE	Specifies a value that is automatically established according to the number of bands of the source image buffer with which to perform MimAugment(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LUMINANCE	Specifies to apply the multiplication value (gain) to the luminance band only. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_INTENSITY_MULTIPLY_OP_VALUE +		Inquires the intensity multiplication value (gain), to apply for M_AUG_INTENSITY_MULTIPLY_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the intensity multiplication value (gain).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP +		Inquires whether to enable the directional lighting (add ramp) operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_LIGHTING_DIRECTIONAL_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_LIGHTING_DIRECTIONAL_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP_ANGLE_MAX +		Inquires an illumination to add to the image at a random angle (ramp), to apply for M_AUG_LIGHTING_DIRECTIONAL_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the maximum angle (ramp) with which to add the illumination, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP_INTENSITY_MAX +		Inquires the maximum light intensity, to use for M_AUG_LIGHTING_DIRECTIONAL_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum light intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_LIGHTING_DIRECTIONAL_OP_INTENSITY_MIN +		Inquires the minimum light intensity, to use for M_AUG_LIGHTING_DIRECTIONAL_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the minimum light intensity.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_LIGHTING_DIRECTIONAL_OP_MODE +	Inquires the directional lighting operation mode, to apply for M_AUG_LIGHTING_DIRECTIONAL_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies to apply the directional lighting to each band.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the number of bands of the source image buffer with which to perform MimAugment(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies to apply the directional lighting to the luminance band only. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP +	Inquires whether to enable the operation that applies a Gaussian additive noise with the option of a random noise standard deviation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP_STDDEV +	Inquires standard deviation for noise, to apply for M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the data type and data depth of the source image buffer with which to perform MimAugment(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the standard deviation for noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP_STDDEV_DELTA +	Inquires the range for the random distribution of noise standard deviation values [stddev - delta, stddev + delta], to apply for M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_VALUE	Specifies a value that is automatically established according to the data type and data depth of the source image buffer with which to perform MimAugment(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of noise standard deviation values.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_NOISE_MULTIPLICATIVE_OP +	Inquires whether to enable the operation that applies a multiplicative noise with option of a random noise standard deviation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_NOISE_MULTIPLICATIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_NOISE_MULTIPLICATIVE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_DISTRIBUTION +	Inquires the uniform distribution for noise generation, to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_GAUSSIAN	Specifies a Gaussian distribution for noise generation.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies a uniform distribution for noise generation.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_INTENSITY_MIN +	Inquires an optional minimum noise intensity value (white noise threshold), to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies an optional minimum noise intensity value (white noise threshold).	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_STDDEV +	Inquires the standard deviation for noise, to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the standard deviation for noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_MULTIPLICATIVE_OP_STDDEV_DELTA +	Inquires the range for the random distribution of noise standard deviation values [stddev - delta, stddev + delta], to apply for M_AUG_NOISE_MULTIPLICATIVE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the range for the random distribution of noise standard deviation values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_SALT_PEPPER_OP +	Inquires whether to enable salt and pepper noise with the option of random density. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies to disable M_AUG_NOISE_SALT_PEPPER_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_NOISE_SALT_PEPPER_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_SALT_PEPPER_OP_DENSITY +	Inquires the density value for salt and pepper noise, to use for M_AUG_NOISE_SALT_PEPPER_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the density value for salt and pepper noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_NOISE_SALT_PEPPER_OP_DENSITY_DELTA +	Inquires the range for the random distribution of noise density values [density - delta, density + delta], to use for M_AUG_NOISE_SALT_PEPPER_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the range for the random distribution of noise density values.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP +	Inquires whether to enable the reduce operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_REDUCE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP_BACKGROUND_COLOR +	Inquires the background color to use for M_AUG_REDUCE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLACK	Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE	Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN	Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE	Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN	Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN	Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_RED	Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_DONT_CARE_CLASS	Specifies the color (255, 255, 255).	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GRAY	Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_GREEN	Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_BLUE	Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GRAY	Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_GREEN	Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_LIGHT_WHITE	Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b
	Byte-encoded RGB value	Specifies an encoded RGB value. To verify if the value is a byte-encoded RGB value, use the **M_IS_RGB888** macro. To retrieve the R, G, and B bands, use the **M_RGB888_R**, **M_RGB888_G**, and **M_RGB888_B** macros.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies a grayscale value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP_FACTOR_MAX +		Inquires the maximum scale factor by which to reduce the image for M_AUG_REDUCE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 1.0	Specifies the maximum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_REDUCE_OP_FACTOR_MIN +		Inquires the minimum scale factor by which to reduce the image for M_AUG_REDUCE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 1.0	Specifies the minimum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b

M_AUG_RNG_INIT_VALUE +	Inquires the initialization value for the internal random number generator, when M_AUG_SEED_MODE is set to M_RNG_AUTO or M_RNG_INIT_VALUE. If you specified M_RNG_AUTO, you must preprocess the augmentation context before inquiring the initialization value. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < *AIL_INT32_MAX*	Specifies the explicit initialization value for the internal random number generator.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP +	Inquires whether to enable the rotation operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_ROTATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_ROTATION_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_DELTA +	Inquires the delta angle, to apply for M_AUG_ROTATION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the delta angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_MAX +	Inquires the maximum rotation angle, to apply for M_AUG_ROTATION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the maximum rotation angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_MIN +	Inquires the minimum rotation angle, to apply for M_AUG_ROTATION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the minimum rotation angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_REF +	Inquires the reference angle. All other angles for M_AUG_ROTATION_OP will be calculated relative to this angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the reference angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_ROTATION_OP_ANGLE_STEP +	Inquires the step size, to apply for M_AUG_ROTATION_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the reference angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN_OP +		Inquires whether to enable the multiplication (gain) of saturation band (HSV) operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SATURATION_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SATURATION_GAIN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN_OP_MAX +		Inquires the maximum value for the random saturation factor's distribution [min, max], to use for M_AUG_SATURATION_GAIN_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum value for the random saturation factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SATURATION_GAIN_OP_MIN +		Inquires the minimum value for the random saturation factor's distribution [min, max], to use for M_AUG_SATURATION_GAIN_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the minimum value for the random saturation factor's distribution.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_OP +		Inquires whether to enable the scale operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SCALE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SCALE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_OP_FACTOR_MAX +		Inquires the maximum scale factor, to apply for M_AUG_SCALE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 16.0	Specifies the maximum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SCALE_OP_FACTOR_MIN +		Inquires the minimum scale factor, to apply for M_AUG_SCALE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 16.0	Specifies the minimum scale factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SEED_MODE +		Inquires the mode with which to initialize seeds. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_RNG_AUTO	Specifies that the initialization value of the internal random number generator is chosen at random.	c		e	f	g	a		i		j	k	l	m	n	b
	M_RNG_INIT_VALUE	Specifies an explicit initialization value for the internal random number generator, using M_AUG_RNG_INIT_VALUE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED_SEED	Specifies a user-defined value for the seeds, using MimAugment() with the SeedValue parameter.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHARPEN_DERICHE_OP +		Inquires whether to enable the operation that applies a Deriche sharpening filter with a random filter smoothness value. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SHARPEN_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SHARPEN_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHARPEN_DERICHE_OP_FACTOR_MAX +		Inquires the maximum value for the sharpen filter smoothness value's random distribution [min, max], to apply for M_AUG_SHARPEN_DERICHE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the maximum value for the sharpen filter smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHARPEN_DERICHE_OP_FACTOR_MIN +		Inquires the minimum value for the sharpen filter smoothness value's random distribution [min, max], to apply for M_AUG_SHARPEN_DERICHE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the minimum value for the sharpen filter smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X_OP +		Inquires whether to enable the X-direction shear operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies to disable M_AUG_SHEAR_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SHEAR_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X_OP_MAX +		Inquires the maximum value for the X-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_X_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the maximum value for the X-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_X_OP_MIN +		Inquires the minimum value for the X-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_X_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the minimum value for the X-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y_OP +		Inquires whether to enable the Y-direction shear operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_SHEAR_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_SHEAR_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y_OP_MAX +		Inquires the maximum value for the Y-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_Y_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the maximum value for the Y-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SHEAR_Y_OP_MIN +		Inquires the minimum value for the Y-coefficient's random distribution [min, max], to use for M_AUG_SHEAR_Y_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 1.0	Specifies the minimum value for the Y-coefficient's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_DERICHE_OP +		Inquires whether to enable the operation that performs a Deriche filter smoothing with random filter smoothness value. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies to disable M_AUG_SMOOTH_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_SMOOTH_DERICHE_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_DERICHE_OP_FACTOR_MAX +	Inquires the maximum value for the Deriche smoothness value's random distribution [min, max], to use for M_AUG_SMOOTH_DERICHE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the maximum value for the Deriche smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_DERICHE_OP_FACTOR_MIN +	Inquires the minimum value for the Deriche smoothness value's random distribution [min, max], to use for M_AUG_SMOOTH_DERICHE_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the minimum value for the Deriche smoothness value's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_OP +	Inquires whether to enable the operation that performs a blurring of the image with a Gaussian blurring kernel calculated using a Gaussian function with a random standard deviation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable M_AUG_SMOOTH_GAUSSIAN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to enable M_AUG_SMOOTH_GAUSSIAN_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_OP_STDDEV_MAX +	Inquires the maximum value for the Gaussian standard deviation's random distribution [min, max], to apply for M_AUG_SMOOTH_GAUSSIAN_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the maximum value for the Gaussian standard deviation's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_SMOOTH_GAUSSIAN_OP_STDDEV_MIN +	Inquires the minimum value for the Gaussian standard deviation's random distribution [min, max], to apply for M_AUG_SMOOTH_GAUSSIAN_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the minimum value for the Gaussian standard deviation's random distribution [min, max].	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_X_OP +		Inquires whether to enable the X-direction translation operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_TRANSLATION_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_TRANSLATION_X_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_X_OP_MAX +		Inquires the maximum translation in the X-direction, to apply for M_AUG_TRANSLATION_X_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum translation in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_Y_OP +		Inquires whether to enable the Y-direction translation operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable M_AUG_TRANSLATION_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable M_AUG_TRANSLATION_Y_OP.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUG_TRANSLATION_Y_OP_MAX +		Inquires the maximum translation in the Y-direction, to apply for M_AUG_TRANSLATION_Y_OP. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum translation in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +		Inquires whether the augmentation context was preprocessed (that is, whether MimAugment() was called with M_PREPROCESS).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the augmentation context was not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the augmentation context was preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b

Cumulative statistics image processing context ID	Specifies a cumulative statistics image processing context, allocated using MimAlloc() with M_STATISTICS_CUMULATIVE_CONTEXT, and used in MimStatCalculate() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +	Inquires whether the cumulative statistics image processing context was preprocessed (that is, whether MimStatCalculate() was called).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the cumulative statistics image processing context was not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the cumulative statistics image processing context was preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_X +	Inquires the width of a typical source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the width has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the width of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_Y +	Inquires the height of a typical source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the height has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the height of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX +	Inquires whether to calculate the maximum pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS +	Inquires whether to calculate the maximum absolute pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN +	Inquires whether to calculate the mean value of the pixels as the statistical operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN +	Inquires whether to calculate the minimum pixel value as the statistical operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS +	Inquires whether to calculate the minimum absolute pixel value as the statistical operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_NUMBER +	Inquires whether to keep track of the number of source images. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION +	Inquires whether to calculate the standard deviation from the value of each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM +	Inquires whether to calculate the sum of the pixel values from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS +	Inquires whether to calculate the sum of the absolute pixel values from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES +	Inquires whether to calculate the sum of the squared pixel values from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
Dead pixel correction image processing context ID	Specifies a dead pixel correction image processing context, allocated using MimAlloc() with M_DEAD_PIXEL_CONTEXT, and used in MimDeadPixelCorrection() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEAD_PIXELS_IMAGE_ATTRIBUTE +	Inquires the attribute of the dead pixels image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that dead pixels have not been specified using MimControl() with M_DEAD_PIXELS or using MimPut() with M_XY_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies a bit-encoded value that indicates the image buffer's attributes, set at allocation.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEAD_PIXELS_IMAGE_HEIGHT +	Inquires the height of the dead pixels image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that dead pixels have not been specified using MimControl() with M_DEAD_PIXELS or using MimPut() with M_XY_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_DEAD_PIXELS_IMAGE_NB_BANDS +	Inquires the number of bands of the dead pixels image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that dead pixels have not been specified using MimControl() with M_DEAD_PIXELS or using MimPut() with M_XY_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEAD_PIXELS_IMAGE_TYPE +	Inquires the bit depth and data type of the dead pixels image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that dead pixels have not been specified using MimControl() with M_DEAD_PIXELS or using MimPut() with M_XY_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_FLOAT	Specifies the data depth and that the data type is floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_SIGNED	Specifies the data depth and that the data type is signed.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_UNSIGNED	Specifies the data depth and that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEAD_PIXELS_IMAGE_WIDTH +	Inquires the width of the dead pixels image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that dead pixels have not been specified using MimControl() with M_DEAD_PIXELS or using MimPut() with M_XY_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_MODE +	Inquires the interpolation mode used to establish the new value for dead pixels. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies to overwrite a dead pixel with an interpolation performed using a weighted average of all its neighboring pixels in the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_DEAD_PIXELS_ARRAY_SIZE +	Inquires the number of elements in the arrays containing the X- and Y-coordinates of the dead pixels.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of elements in the arrays.	c		e	f	g	a		i		j	k	l	m	n	b

Deinterlacing image processing context ID	Specifies a deinterlacing image processing context, allocated using MimAlloc() with M_DEINTERLACE_CONTEXT, and used in MimDeinterlace() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEINTERLACE_TYPE +	Inquires the deinterlacing algorithm. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_AVERAGE	Specifies that the average algorithm is applied to the pixels that are considered to be part of a moving object and leaves the background pixels unchanged.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_BOB	Specifies that the bob algorithm is applied to the pixels that are considered to be part of a moving object and leaves the background pixels unchanged.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_DISCARD	Specifies that the discard algorithm is applied to the pixels that are considered to be part of a moving object and leaves the background pixels unchanged.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE	Specifies the averaging algorithm. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BOB	Specifies the bob algorithm. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISCARD	Specifies the discard algorithm. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISCARD_FIELD +	Inquires the field that is discarded when M_DEINTERLACE_TYPE is set to M_DISCARD or M_ADAPTIVE_DISCARD. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_EVEN_FIELD	Specifies that the even field is discarded.	c		e	f	g	a		i		j	k	l	m	n	b
M_ODD_FIELD	Specifies that the odd field is discarded.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_FIELD +	Inquires the field that is processed first when M_DEINTERLACE_TYPE is set to M_BOB or M_ADAPTIVE_BOB. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_EVEN_FIELD	Specifies that the even field is processed first.	c		e	f	g	a		i		j	k	l	m	n	b
M_ODD_FIELD	Specifies that the odd field is processed first.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_NUM_FRAMES +	Inquires the number of frames to use to perform the motion detection. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 1	Specifies the number of frames.	c		e	f	g	a		i		j	k	l	m	n	b

M_MOTION_DETECT_OUTPUT +	Inquires whether the output images will be the deinterlaced images or images indicating which pixels are considered to be part of an object in motion (the internal motion detection mask). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the output images are the deinterlaced images.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the output images indicate the background pixels and the pixels that are considered to be part of the object in motion.	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_REFERENCE_FRAME +	Inquires the index of the frame to use for deinterlacing within the group of frames that are used for motion detection. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER_FRAME	Specifies that the center frame in the group is used as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_FRAME	Specifies that the first frame in the group is used as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST_FRAME	Specifies that the last frame in the group is used as the reference frame.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_MOTION_DETECT_NUM_FRAMES	Specifies the index of the frame relative to the first frame of the group. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MOTION_DETECT_THRESHOLD +	Inquires the threshold value used for motion detection. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the threshold.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_FIRST_IMAGE +	Inquires the index of the input image used to generate the first deinterlaced image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the image in the source image array.	c		e	f	g	a		i		j	k	l	m	n	b
Filter majority image processing context ID	Specifies a filter majority image processing context identifier, allocated using MimAlloc() with M_FILTER_MAJORITY_CONTEXT, and used in MimFilterMajority() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_IGNORED_VALUE +	Inquires the pixel value to ignore. If a pixel value in the neighborhood matches the ignored value, it will not be counted; the pixel is essentially excluded from the neighborhood. This value is only ignored if M_USE_IGNORED_VALUE is enabled. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the pixel value to ignore.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIE_BREAKER_MODE +		Inquires the method used to break a tie in the neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX	Specifies to replace the central pixel value with the greatest value present among the tied values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIAN	Specifies to replace the central pixel value with the median value present among the tied values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN	Specifies to replace the central pixel value with the smallest value present that appears among the tied values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_FREQUENCY_TIE_BREAKER +		Inquires whether the control type M_TIE_BREAKER_MODE should break a tie in the neighborhood using the values present or, if weights are specified, using the non-weighted frequency at which the tied values appear. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to choose the value with which to replace the central pixel by comparing the values present in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to choose the value with which to replace the central pixel by comparing the non-weighted frequency at which the values are found in the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_IGNORED_VALUE +		Inquires if Aurora Imaging Library should ignore the value in the source image set by M_IGNORED_VALUE when calling MimFilterMajority(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to ignore any value when calling MimFilterMajority().	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to ignore the pixels in the source image whose values match the ignored value when calling MimFilterMajority().	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_IMAGE_ATTRIBUTE +		Inquires the attributes of the weight image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the weight image buffer has not been set, using MimControl() with M_WEIGHT_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies a bit-encoded value that indicates the image buffer's attributes, set at allocation.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_IMAGE_HEIGHT +		Inquires the height of the weight image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the weight image buffer has not been set, using MimControl() with M_WEIGHT_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height of the weight image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_IMAGE_NB_BANDS +		Inquires the number of bands of the weight image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the weight image buffer has not been set, using MimControl() with M_WEIGHT_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_IMAGE_TYPE +		Inquires the bit depth and date type of the weight image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the weight image buffer has not been set, using MimControl() with M_WEIGHT_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the bit depth and data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_WEIGHT_IMAGE_WIDTH +		Inquires the width of the weight image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the weight image buffer has not been set, using MimControl() with M_WEIGHT_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width of the weight image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Find orientation image processing context ID		Specifies a find orientation image processing context identifier, allocated using MimAlloc() with M_FIND_ORIENTATION_CONTEXT, and used in MimFindOrientation() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BORDER_ATTENUATION +		Inquires whether MimFindOrientation() must process the image's borders, or if the operation can ignore noise occurring close to the ends of the image buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that MimFindOrientation() cannot ignore the image's borders. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that MimFindOrientation() can ignore the image's borders. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FREQUENCY_CUTOFF_RATIO_HIGH +		Inquires the upper limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency. Larger images will have higher maximum frequencies. Frequencies higher than this percentage will be ignored during computation. This can be useful for discarding noise in the image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the upper limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FREQUENCY_CUTOFF_RATIO_LOW +		Inquires the lower limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency. Larger images will have higher maximum frequencies. Frequencies lower than this percentage will be ignored during computation. This can be useful for discarding noise in the image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the lower limit of frequencies in which to look for dominant orientations, as a percentage of the maximum frequency. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_MODE +		Inquires the interpolation mode for the resizing used if the source image is of an inappropriate size. This will only be used if the source image has dimensions that are not a power of 2 (X-size and Y-size). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AVERAGE	Specifies averaging interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BICUBIC	Specifies bicubic interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BILINEAR	Specifies bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERPOLATE	Specifies interpolated resizing. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MAX	Specifies an interpolation based on the maximum pixel value in the source image area. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIN	Specifies an interpolation based on the minimum pixel value in the source image area. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE +		Inquires the resizing mode used if the source image is of an inappropriate size. Resizing will only occur if the source image has dimensions that are not a power of 2 (X-size and Y-size). The find orientation operation is then performed on the resized image, which is stored in a temporary image buffer. The original image is not altered. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CLIP_CENTER	Specifies to perform the find orientation operation using the largest centered portion of the image with dimensions that are a power of 2 (X-size and Y-size).	c		e	f	g	a		i		j	k	l	m	n	b
	M_RESIZE_DOWN	Specifies to perform the find orientation operation on a subsampled version of the image with the closest possible dimensions that are a power of 2 (X-size and Y-size).	c		e	f	g	a		i		j	k	l	m	n	b
	M_RESIZE_UP	Specifies to perform the find orientation operation on a zoomed version of the image with the closest possible dimensions that are a power of 2 (X-size and Y-size).	c		e	f	g	a		i		j	k	l	m	n	b
Flat-field image processing context ID		Specifies a flat-field image processing context, allocated using MimAlloc() with M_FLAT_FIELD_CONTEXT, and used in MimFlatField() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_CONST +		Inquires the dark constant value. This is used to remove thermal agitation recorded in the grabbed image (from the CCD) and to remove the darkest possible shade of black when removing uneven lighting from grabbed images. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID_CONST	Specifies that the constant has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 65535	Specifies the constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_IMAGE_HEIGHT +		Inquires the height of the dark image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the dark image buffer has not been set, using MimControl() with M_DARK_IMAGE. This value is also returned when M_DARK_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_IMAGE_NB_BANDS +		Inquires the number of bands of the dark image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_DARK_IMAGE. This value is also returned when M_DARK_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_IMAGE_TYPE +	Inquires a combination of two values: data type and data depth of the dark image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_DARK_IMAGE. This value is also returned when M_DARK_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_DARK_IMAGE_WIDTH +	Inquires the width of the dark image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_DARK_IMAGE. This value is also returned when M_DARK_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_EFFECTIVE_GAIN_CONST +	Inquires the gain factor used during the last call to MimFlatField() (when M_GAIN_CONST set to M_AUTOMATIC). If the gain factor was set manually, inquire the value using M_GAIN_CONST.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_CONST	Specifies that the gain factor has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the automatically calculated gain factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_CONST +	Inquires the flat constant value. This is used to remove the variations of CCD sensitivity recorded in the grabbed image (from the CCD) and to reduce the gray in the grabbed image when removing uneven lighting. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_CONST	Specifies that the constant has not been set.	c		e	f	g	a		i		j	k	l	m	n	b

	0 <= *Value* <= 65535	Specifies the constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_IMAGE_HEIGHT +		Inquires the height of the flat image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_FLAT_IMAGE. This value is also returned when M_FLAT_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_IMAGE_NB_BANDS +		Inquires the number of bands in the flat image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_FLAT_IMAGE. This value is also returned when M_FLAT_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_IMAGE_TYPE +		Inquires the identifier of the flat image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_FLAT_IMAGE. This value is also returned when M_FLAT_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
M_FLAT_IMAGE_WIDTH +		Inquires the width of the flat image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_FLAT_IMAGE. This value is also returned when M_FLAT_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAIN_CONST +		Inquires the gain factor used to normalize (or scale) the result of the flat field calculation back to the full dynamic range of the destination image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_AUTOMATIC	Specifies that the gain factor has been set to be automatically calculated. To inquire the automatically calculated value, use M_EFFECTIVE_GAIN_CONST.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID_CONST	Specifies that the gain factor has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the gain factor.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_CONST +		Inquires the offset constant value. This is used to remove the electrical bias recorded in the grabbed image (from the CCD) and to reduce the black in the flat image when removing uneven lighting. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID_CONST	Specifies that the constant has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= 65535	Specifies the constant.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_IMAGE_HEIGHT +		Inquires the height of the offset image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_OFFSET_IMAGE. This value is also returned when M_OFFSET_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_IMAGE_NB_BANDS +		Inquires the number of bands in the offset image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_OFFSET_IMAGE. This value is also returned when M_OFFSET_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_IMAGE_TYPE +		Inquires a combination of two values: data type and data depth of the offset image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_OFFSET_IMAGE. This value is also returned when M_OFFSET_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b

M_OFFSET_IMAGE_WIDTH +	Inquires the width of the offset image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the image buffer has not been set, using MimControl() with M_OFFSET_IMAGE. This value is also returned when M_OFFSET_CONST is set.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Histogram equalization adaptive context ID	Specifies a histogram equalization adaptive context, allocated using MimAlloc() with M_HISTOGRAM_EQUALIZE_ADAPTIVE_CONTEXT, and used in MimHistogramEqualizeAdaptive() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALPHA_VALUE +	Inquires the adjustment factor for M_EXPONENTIAL and M_RAYLEIGH operations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the adjustment factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIP_LIMIT +	Inquires the maximum percentage of values that a tile's histogram bin can represent. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 100.0	Specifies the maximum percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_SIZE +	Inquires the number of bins for each tile's histogram. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_SOURCE	Specifies that MimHistogramEqualizeAdaptive() automatically determines the number of bins for each tile's histogram, according to the number of intensities that are possible in the specified image buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 2	Specifies the number of bins for each tile's histogram. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TILES_X +	Inquires the number of tiles along the X-direction of the source image specified with MimHistogramEqualizeAdaptive(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 2	Specifies the number of tiles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TILES_Y +	Inquires the number of tiles along the Y-direction of the source image specified with MimHistogramEqualizeAdaptive(). SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 2	Specifies the number of tiles. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OPERATION +	Inquires the equalization operation that MimHistogramEqualizeAdaptive() uses. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXPONENTIAL	Specifies an equalization density function which generates an Exponential distribution. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HYPER_CUBE_ROOT	Specifies an equalization density function which generates a Hyperbolic Cube Root distribution. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HYPER_LOG	Specifies an equalization density function which generates a Hyperbolic Logarithmic distribution. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RAYLEIGH	Specifies an equalization density function which generates a Rayleigh distribution. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies an equalization density function which generates a Uniform distribution. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Linear IIR filter image processing context ID	Specifies a linear IIR filter image processing context, allocated using MimAlloc() with M_LINEAR_FILTER_IIR_CONTEXT, and used in MimConvolve() and MimDifferential() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_DEFAULT_SHARPEN_PARAM +	Inquires the default sharpening operation value established by Aurora Imaging Library (MimDifferential()).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the sharpening value.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_OPERATION +	Inquires the linear filter operation to perform (MimConvolve()). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_DERIVATIVE_X	Specifies to compute the first derivative of the image, with respect to X, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIRST_DERIVATIVE_Y	Specifies to compute the first derivative of the image, with respect to Y, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SECOND_DERIVATIVE_X	Specifies to compute the second derivative of the image, with respect to X, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SECOND_DERIVATIVE_XY	Specifies to compute the second derivative of the image, with respect to X and Y, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SECOND_DERIVATIVE_Y	Specifies to compute the second derivative of the image, with respect to Y, using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SMOOTH	Performs a smoothing operation on the image using the specified filter type.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_RESPONSE_TYPE +		Inquires the filter response type (MimConvolve()). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_SLOPE	Specifies a response that is normalized according to the input slope. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_STEP	Specifies a response that is normalized according to a step input. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS +		Inquires the degree of smoothness (strength of the denoising) to apply, according to the filter smoothness type (MimConvolve()). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the smoothness value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_SMOOTHNESS_TYPE +		Inquires the filter smoothness type (MimConvolve()). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NATIVE	Specifies to control the alpha (α), beta (β), or sigma (σ) value in the Deriche, Shen-Castan, or Vliet weighting function, depending on the filter type specified with M_FILTER_TYPE. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NORMALIZED	Specifies to use a normalized smoothness value that is between 0 and 100.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIZE	Specifies to use a smoothness value driven by a spatial size. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_TYPE +		Inquires the type of predefined IIR filter to use for the convolution operation (MimConvolve()). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DERICHE	Specifies a Deriche infinite support filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SHEN	Specifies a Shen-Castan infinite support exponential filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_VLIET	Specifies a Vliet infinite support filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Locate difference image processing result ID	Specifies a locate difference image processing result buffer, allocated using MimAllocResult() with M_LOCATE_DIFFERENCE_RESULT, and used in MimLocateDifference() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESULT_OUTPUT_UNITS +	Inquires whether results are returned in pixel or world units. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Locate peak 1D image processing context ID	Specifies a 1D locate peak image processing context, allocated using MimAlloc() with M_LOCATE_PEAK_1D_CONTEXT, and used in MimLocatePeak1d() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRAME_SIZE +	Inquires the Y-size of the frames. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FULL_SIZE	Specifies to use the buffer as a single frame.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the Y-size of the frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMUM_CONTRAST +	Inquires the minimum contrast. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 255	Specifies the minimum contrast.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_FRAMES +	Inquires the total number of frames for which peaks can be extracted. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of frames.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_PEAKS +	Inquires the maximum number of peaks to find along a given lane. If more than the specified number of peaks are found, the highest peaks are kept. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0	Specifies the number of peaks. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_INTENSITY_RANGE +	Inquires the number of pixels used to calculate the average peak intensity. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_WIDTH_DELTA +	Inquires the peak width delta. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_WIDTH_NOMINAL +	Inquires the peak width nominal. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCAN_LANE_DIRECTION +	Inquires the direction in which to detect peaks. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_HORIZONTAL	Detects peaks along the image's X-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_VERTICAL	Detects peaks along the image's Y-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Match image processing context ID	Specifies a match image processing context, allocated using MimAlloc() with M_MATCH_CONTEXT, and used in MimMatch() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE_ATTRIBUTE +	Inquires the attribute of the mask image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the mask image buffer has not been set, using MimControl() with M_MASK_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies a bit-encoded value that indicates the image buffer's attributes, set at allocation.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE_HEIGHT +	Inquires the height of the mask image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the mask image buffer has not been set, using MimControl() with M_MASK_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_MASK_IMAGE_NB_BANDS +	Inquires the number of bands of the mask image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the mask image buffer has not been set, using MimControl() with M_MASK_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE_TYPE +	Inquires the bit depth and data type of the mask image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the mask image buffer has not been set, using MimControl() with M_MASK_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_FLOAT	Specifies the data depth and that the data type is floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_SIGNED	Specifies the data depth and that the data type is signed.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_UNSIGNED	Specifies the data depth and that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_MASK_IMAGE_WIDTH +	Inquires the width of the mask image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the mask image buffer has not been set, using MimControl() with M_MASK_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_SCORE +	Inquires the specified maximum score used to remap normalized correlation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_DEPTH	Specifies to establish the maximum score based on the pixel depth of the destination buffer (for example, when dealing with a 16-bit signed buffer, the maximum value would be 32, 767). INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the maximum score. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE +	Inquires the type of algorithm used when performing the match. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_CORRELATE	Specifies a grayscale correlation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CORRELATE_NORMALIZED	Specifies a normalized grayscale correlation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEAN_OF_ABS_DIFFERENCES	Specifies the sum of absolute differences divided by the number of pixels used to compute it, resulting in a 8-bit range result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SUM_OF_ABS_DIFFERENCES	Specifies sum of the absolute differences. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_IMAGE_ATTRIBUTE +		Inquires the attributes of the internal buffer of the model image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the model image buffer has not been set, using MimControl() with M_MODEL_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies a bit-encoded value that indicates the image buffer's attributes, set at allocation.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_IMAGE_HEIGHT +		Inquires the height of the model image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the model image buffer has not been set, using MimControl() with M_MODEL_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_IMAGE_NB_BANDS +		Inquires the number of bands of the model image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the model image buffer has not been set, using MimControl() with M_MODEL_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_IMAGE_TYPE +		Inquires the bit depth and data type of the model image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the model image buffer has not been set, using MimControl() with M_MODEL_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 8	Specifies 8-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + 16	Specifies 16-bit unsigned data.	c		e	f	g	a		i		j	k	l	m	n	b

M_MODEL_IMAGE_WIDTH +	Inquires the width of the model image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the model image buffer has not been set, using MimControl() with M_MODEL_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_STEP +	Inquires whether every pixel or every other pixel in the model image is used when matching the source image to the model image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies to take every pixel (on both axes) to compute the match.	c		e	f	g	a		i		j	k	l	m	n	b
2	Specifies to take every other pixel (on both axes) to compute the match.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE_TYPE +	Inquires how the final correlation score is computed. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM	Specifies to use normalized grayscale correlation to compute the final match score.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM_CLIP	Specifies to clip the results from a normalized grayscale correlation when computing the final correlation score. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NORM_CLIP_SQR	Specifies to clip the results from the square of the normalized grayscale correlation when computing the final correlation score. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SQR_NORM	Specifies to use the square of the normalized grayscale correlation to compute the final correlation score.	c		e	f	g	a		i		j	k	l	m	n	b
Rearrangement image processing context ID	Specifies a rearrangement image processing context, allocated using MimAlloc() with M_REARRANGE_CONTEXT, and used in MimRearrange() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODE +	Inquires the processing mode. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the processing mode has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINES	Specifies that each area to be rearranged is a single horizontal line.	c		e	f	g	a		i		j	k	l	m	n	b

M_RECTS	Specifies that each area to be rearranged is a single rectangle.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_DESTINATION_ARRAY_SIZE +	Inquires the size of the arrays containing the X- and Y-offsets of the areas in the destination image buffer in which to copy the source areas. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the arrays.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_SIZE_ARRAY_SIZE +	Inquires the size of the arrays containing the width and height of the areas to copy. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the arrays.	c		e	f	g	a		i		j	k	l	m	n	b
M_XY_SOURCE_ARRAY_SIZE +	Inquires the size of the arrays containing the X- and Y-offsets of the areas to copy from the source image buffer into the destination image buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the arrays.	c		e	f	g	a		i		j	k	l	m	n	b
Remap image processing context ID	Specifies a remap image processing context, allocated using MimAlloc() with M_IM_REMAP_CONTEXT, and used in MimRemap() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_DST_END_VALUE +	Inquires the end pixel value used to compute the destination range. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_DST_RANGE +	Inquires whether to use the destination image buffer or the remap image processing context to determine the destination range. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_BUFFER	Specifies to use the minimum and maximum pixel values of the destination image buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_CONTEXT	Specifies to use the context's destination start and end values specified with M_DST_START_VALUE and M_DST_END_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DST_START_VALUE +	Inquires the start pixel value used to compute the destination range. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the start pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMAP_FUNCTION +	Inquires the function used to map the source image buffer to the destination image buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIFORM	Specifies to perform a uniform linear mapping.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC_END_VALUE +	Inquires the end pixel value used to compute the source range. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the end pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC_RANGE +	Inquires whether to use the source image buffer or the remap image processing context to determine the source range. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_BUFFER	Specifies to use the minimum and maximum pixel values of the source image buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_CONTEXT	Specifies to use the context's source start and end values specified with M_SRC_START_VALUE and M_SRC_END_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SRC_START_VALUE +	Inquires the start pixel value used to compute the source range. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the start pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics image processing context ID	Specifies a statistics image processing context, allocated using MimAlloc() with M_STATISTICS_CONTEXT, and used by MimStatCalculate() as the operation to perform.	c		e	f	g	a		i		j	k	l	m	n	b
M_COND_HIGH +	Inquires the upper limit of the selected condition (inquired using M_CONDITION) that establishes which pixels to include in the statistical operation(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no upper limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit.	c		e	f	g	a		i		j	k	l	m	n	b

M_COND_LOW +	Inquires the lower limit of the selected condition (inquired using M_CONDITION) that establishes which pixels to include in the statistical operation(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that there is no lower limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the lower limit.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONDITION +	Inquires the condition for the statistical computation that establishes which pixels to include in the statistical operation(s). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no condition is set. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EQUAL	Specifies that only pixels with values equal to M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Specifies that only pixels with values greater than M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL	Specifies that only pixels with values greater than or equal to M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_IN_RANGE	Specifies that pixels with values between M_COND_LOW and M_COND_HIGH, inclusive, will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Specifies that only pixels with values less than M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL	Specifies that only pixels with values less than or equal to M_COND_LOW will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_EQUAL	Specifies that pixels with values equal to M_COND_LOW will be not used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_RANGE	Specifies that pixels with values less than M_COND_LOW, or greater than M_COND_HIGH, will be used for statistical calculations.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLCM_PAIR_OFFSET_X +	Inquires the displacement in X between pixel pairs used to generate the co-occurrence matrix. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the displacement in X.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLCM_PAIR_OFFSET_Y +		Inquires the displacement in Y between pixel pairs used to generate the co-occurrence matrix. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the displacement in Y.	c		e	f	g	a		i		j	k	l	m	n	b
M_GLCM_QUANTIFICATION +		Inquires the depth of the gray level of the co-occurrence matrix. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 10	Specifies the depth. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +		Inquires whether the statistics image processing context was preprocessed (that is, whether MimStatCalculate() was called).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the statistics image processing context was not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the statistics image processing context was preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_X +		Inquires the width of a typical source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the width has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the width of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SOURCE_SIZE_Y +		Inquires the height of a typical source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the height has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the height of a typical source image, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANGULAR_DATA_COHERENCE +		Inquires whether to calculate the coherence of the angular data. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ANGULAR_DATA_MEAN +		Inquires whether to calculate the dominant direction of the unit vectors. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CONTRAST +		Inquires whether to calculate the contrast of all the grayscale pixel values within a specified neighborhood in relation to the normalized co-occurrence probability of each pixel. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_CORRELATION +		Inquires whether to calculate the correlation within the normalized co-occurrence probability in a specified neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_DISSIMILARITY +		Inquires whether to calculate the dissimilarity between the normalized co-occurrence probability and the gray level co-occurrence matrix of a specified neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_ENERGY +		Inquires whether to calculate the energy between two points in the gray level co-occurrence matrix of a given neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_GLCM_ENTROPY +	Inquires whether to calculate the inverse of the energy between two points in the normalized co-occurrence probability of a specified neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_GLCM_HOMOGENEITY +	Inquires whether to calculate the similarity between the normalized co-occurrence probability and the gray level co-occurrence matrix of a given neighborhood. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX +	Inquires whether to calculate the maximum pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MAX_ABS +	Inquires whether to calculate the maximum absolute pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MEAN +	Inquires whether to calculate the mean value of the pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_MIN +	Inquires whether to calculate the minimum pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_MIN_ABS +	Inquires whether to calculate the minimum absolute pixel value from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_NUMBER +	Inquires whether to keep track of the number of pixels, from the source image, that satisfied the condition specified when calling MimStatCalculate(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to keep track.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ORIENTATION_DATA_COHERENCE +	Inquires whether to calculate the coherence of the orientation data. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_ORIENTATION_DATA_MEAN +	Inquires whether to calculate the dominant orientation of the unit vectors. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_STANDARD_DEVIATION +	Inquires whether to calculate the standard deviation from the value of each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM +	Inquires whether to calculate the sum of the pixel values from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_ABS +	Inquires whether to calculate the sum of the absolute pixel values from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_SUM_OF_SQUARES +	Inquires whether to calculate the sum of the squared pixel values from each pixel location across the source image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to perform this statistical operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_X +	Inquires the distance between the neighborhood along the X-axis in which to evaluate the specified statistic. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Width	Specifies the width of the neighborhood, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_STEP_SIZE_Y +	Inquires the distance between the neighborhood along the Y-axis in which to evaluate the specified statistic. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Height	Specifies the height of the neighborhood, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TILE_SIZE_X +	Inquires the X-size (width) of the neighborhood in which to evaluate the specified statistic. SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_CONST	Specifies that the constant (M_TILE_SIZE_X) has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
Width	Specifies the width of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_TILE_SIZE_Y +	Inquires the Y-size (height) of the neighborhood in which to evaluate the specified statistic. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID_CONST	Specifies that the constant (M_TILE_SIZE_Y) has not been set.	c		e	f	g	a		i		j	k	l	m	n	b
Height	Specifies the height of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
Unwarp along path image processing context ID	Specifies an unwarp along image processing context, allocated using MimAlloc() with M_UNWARP_ALONG_PATH_CONTEXT, and used by MimUnwarpAlongPath() as the operation to perform.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESTINATION_SIZE_X +	Inquires the width of the destination buffer if one was specified when calling MimUnwarpAlongPath() with M_PREPROCESS. If a destination buffer was not specified, this is the length (X-size) of the path, rounded up to the nearest integer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the width of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DESTINATION_SIZE_Y +	Inquires the height of the destination buffer if one was specified when calling MimUnwarpAlongPath() with M_PREPROCESS. If a destination buffer was not specified, this is the width (Y-size) of the path, rounded up to the nearest integer, set using MimControl() with M_SINGLE_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the height of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATION_MODE +	Inquires the interpolation mode used when unwarping along the path. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC	Specifies bicubic interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR	Specifies bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_MODE +	Inquires the type of path. This path must be set using MimPut(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_POLYLINE	Specifies that the path is defined by a polyline. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN +	Inquires how to handle pixels that map outside of the source image buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_CLEAR	Specifies to set the destination pixel to 0 if the corresponding point falls outside of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Specifies to leave the destination pixels as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Specifies to use pixels from the source buffer's ancestor buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PATH_ARRAY_SIZE +	Inquires the required array size to store the values retrieved using MimGet() with M_XY_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the array.	c		e	f	g	a		i		j	k	l	m	n	b
M_SINGLE_WIDTH +	Inquires the single width for all points along the path. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the width of the path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNWARP_SAMPLING_MODE +	Inquires which algorithm to use for the unwarp along path operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_ALONG_PATH	Specifies to spread the deformation over the entire path. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ADAPTIVE_AT_JUNCTION	Specifies to deform only at the junction of each path segment. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WORK_PATH_ARRAY_SIZE +	Inquires the required array size to store the values retrieved using MimGet() with M_XY_WORK_PATH, M_XY_WORK_TOP_PATH, and M_XY_WORK_BOTTOM_PATH.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the array.	c		e	f	g	a		i		j	k	l	m	n	b

Wavelet image processing context ID	Specifies a wavelet image processing context, allocated using MimAlloc() with M_WAVELET_TRANSFORM_CONTEXT or M_WAVELET_TRANSFORM_CUSTOM_CONTEXT, and used in MimWaveletTransform() and MimWaveletDenoise() operations. Unless otherwise specified, inquire types apply to both M_WAVELET_TRANSFORM_CONTEXT and M_WAVELET_TRANSFORM_CUSTOM_CONTEXT context types.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_FORWARD_HIGH_PASS_ID +	Inquires the identifier of the internal buffer containing the values used for the high-pass filter of a forward transformation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no information to inquire about.	c		e	f	g	a		i		j	k	l	m	n	b
Kernel Buffer ID	Specifies the identifier of the kernel buffer containing the filter values.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_FORWARD_LOW_PASS_ID +	Inquires the identifier of the internal buffer containing the values used for the low-pass filter of a forward transformation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no information to inquire about.	c		e	f	g	a		i		j	k	l	m	n	b
Kernel Buffer ID	Specifies the identifier of the kernel buffer containing the filter values.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_REVERSE_HIGH_PASS_ID +	Inquires the identifier of the internal buffer containing the values used for the high-pass filter of a reverse transformation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no information to inquire about.	c		e	f	g	a		i		j	k	l	m	n	b
Kernel Buffer ID	Specifies the identifier of the kernel buffer containing the filter values.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILTER_REVERSE_LOW_PASS_ID +	Inquires the identifier of the internal buffer containing the values used for the low-pass filter of a reverse transformation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no information to inquire about.	c		e	f	g	a		i		j	k	l	m	n	b
Kernel Buffer ID	Specifies the identifier of the kernel buffer containing the filter values.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRANSFORMATION_DOMAIN +	Inquires whether the mathematical domain of the filter specified to perform the wavelet transformation consists of complex numbers or only real numbers. Complex numbers have an imaginary part (imaginary numbers), in addition to a real part (real numbers).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLEX	Specifies that the mathematical domain of the filter consists of real and imaginary numbers. Such transformations use a complex type of wavelet filter (MimControl() with M_WAVELET_TYPE), or a custom wavelet filter containing real and imaginary numbers (MimWaveletSetFilter()).	c		e	f	g	a		i		j	k	l	m	n	b
M_REAL	Specifies that the mathematical domain of the filter consists of only real numbers. Such transformations use a non-complex type of wavelet filter (MimControl() with M_WAVELET_TYPE), or a custom wavelet filter containing real numbers only (MimWaveletSetFilter()).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSFORMATION_MODE +	Inquires the wavelet transformation mode. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DYADIC	Specifies a dyadic wavelet transformation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNDECIMATED +	Specifies an undecimated wavelet transformation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_CONTEXT_TYPE +	Inquires the type of wavelet transformation context allocated.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CUSTOM	Specifies that the wavelet context was allocated using M_WAVELET_TRANSFORM_CUSTOM_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDEFINED	Specifies that the wavelet context was allocated using M_WAVELET_TRANSFORM_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_WAVELET_SIZE +	Inquires the size of the wavelet defined in the context. If the context type is M_WAVELET_TRANSFORM_CONTEXT, the size depends on the specified M_WAVELET_TYPE control. If the context type is M_WAVELET_TRANSFORM_CUSTOM_CONTEXT, the size depends on the filters specified with MimWaveletSetFilter().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the size of the wavelet.	c		e	f	g	a		i		j	k	l	m	n	b

M_WAVELET_TYPE +		Inquires the type of wavelet filter used by the wavelet transformation context. This setting is only available for an M_WAVELET_TRANSFORM_CONTEXT context type. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CUSTOM	Specifies a custom wavelet filter. This indicates that you are using an M_WAVELET_TRANSFORM_CUSTOM_CONTEXT context type. Custom wavelet filters are set with MimWaveletSetFilter().	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_1	Specifies a Daubechies wavelet filter that uses 1 vanishing moment and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_2	Specifies a Daubechies wavelet filter that uses 2 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_3	Specifies a Daubechies wavelet filter that uses 3 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_3_COMPLEX	Specifies a Daubechies wavelet filter that uses 3 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_4	Specifies a Daubechies wavelet filter that uses 4 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_5	Specifies a Daubechies wavelet filter that uses 5 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_5_COMPLEX	Specifies a Daubechies wavelet filter that uses 5 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_6	Specifies a Daubechies wavelet filter that uses 6 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_7	Specifies a Daubechies wavelet filter that uses 7 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_7_COMPLEX	Specifies a Daubechies wavelet filter that uses 7 vanishing moments and complex coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DAUBECHIES_8	Specifies a Daubechies wavelet filter that uses 8 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HAAR	Specifies a Haar wavelet filter. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_1	Specifies a Symlet wavelet filter that uses 1 vanishing moment and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SYMLET_2	Specifies a Symlet wavelet filter that uses 2 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b

		M_SYMLET_3	Specifies a Symlet wavelet filter that uses 3 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SYMLET_4	Specifies a Symlet wavelet filter that uses 4 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SYMLET_5	Specifies a Symlet wavelet filter that uses 5 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SYMLET_6	Specifies a Symlet wavelet filter that uses 6 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SYMLET_7	Specifies a Symlet wavelet filter that uses 7 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b
		M_SYMLET_8	Specifies a Symlet wavelet filter that uses 8 vanishing moments and real coefficients.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ContextOrResultImId=Augmentation context ID: M_AUG_ASPECT_RATIO_OP; M_AUG_BLUR_MOTION_OP; M_AUG_CROP_OP; M_AUG_DILATION_ASYM_OP; M_AUG_DILATION_OP; M_AUG_EROSION_ASYM_OP; M_AUG_EROSION_OP; M_AUG_FLIP_OP; M_AUG_GAMMA_OP; M_AUG_HSV_VALUE_GAIN_OP; M_AUG_HUE_OFFSET_OP; M_AUG_INTENSITY_ADD_OP; M_AUG_INTENSITY_MULTIPLY_OP; M_AUG_LIGHTING_DIRECTIONAL_OP; M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP; M_AUG_NOISE_MULTIPLICATIVE_OP; M_AUG_NOISE_SALT_PEPPER_OP; M_AUG_REDUCE_OP; M_AUG_ROTATION_OP; M_AUG_SATURATION_GAIN_OP; M_AUG_SCALE_OP; M_AUG_SHARPEN_DERICHE_OP; M_AUG_SHEAR_X_OP; M_AUG_SHEAR_Y_OP; M_AUG_SMOOTH_DERICHE_OP; M_AUG_SMOOTH_GAUSSIAN_OP; M_AUG_TRANSLATION_X_OP; M_AUG_TRANSLATION_Y_OP].

You can add the following value to the above-mentioned values to determine the priority (order) with which MimAugment() performs the corresponding operation. Alternatively, you can add a value from For inquiring the probability that MimAugment() performs the corresponding operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the priority (order) with which MimAugment() performs the corresponding operation																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_PRIORITY		Inquires the priority with which MimAugment() performs the corresponding operation.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the priority (order). The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ContextOrResultImId=Augmentation context ID: M_AUG_ASPECT_RATIO_OP; M_AUG_BLUR_MOTION_OP; M_AUG_CROP_OP; M_AUG_DILATION_ASYM_OP; M_AUG_DILATION_OP; M_AUG_EROSION_ASYM_OP; M_AUG_EROSION_OP; M_AUG_FLIP_OP; M_AUG_GAMMA_OP; M_AUG_HSV_VALUE_GAIN_OP; M_AUG_HUE_OFFSET_OP; M_AUG_INTENSITY_ADD_OP; M_AUG_INTENSITY_MULTIPLY_OP; M_AUG_LIGHTING_DIRECTIONAL_OP; M_AUG_NOISE_GAUSSIAN_ADDITIVE_OP; M_AUG_NOISE_MULTIPLICATIVE_OP; M_AUG_NOISE_SALT_PEPPER_OP; M_AUG_REDUCE_OP; M_AUG_ROTATION_OP; M_AUG_SATURATION_GAIN_OP; M_AUG_SCALE_OP; M_AUG_SHARPEN_DERICHE_OP; M_AUG_SHEAR_X_OP; M_AUG_SHEAR_Y_OP; M_AUG_SMOOTH_DERICHE_OP; M_AUG_SMOOTH_GAUSSIAN_OP; M_AUG_TRANSLATION_X_OP; M_AUG_TRANSLATION_Y_OP].

You can add the following value to the above-mentioned values to determine the probability that MimAugment() performs the corresponding operation. Alternatively, you can add a value from For inquiring the priority (order) with which MimAugment() performs the corresponding operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the probability that MimAugment() performs the corresponding operation																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_PROBABILITY	Inquires the probability that MimAugment() performs the corresponding operation.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the probability, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when ContextOrResultImId=Wavelet image processing context ID, and InquireType=M_TRANSFORMATION_MODE, : M_UNDECIMATED].

The following value might be combined with the above-mentioned value to specify undecimated wavelet transformations that are centered.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether undecimated wavelet transformations are centered																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CENTER	Specifies undecimated wavelet transformations that are centered. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when ContextOrResultImId=Wavelet image processing context ID: M_FILTER_FORWARD_HIGH_PASS_ID; M_FILTER_FORWARD_LOW_PASS_ID; M_FILTER_REVERSE_HIGH_PASS_ID; M_FILTER_REVERSE_LOW_PASS_ID].

You can add one of the following values to the above-mentioned values to specify which internal buffer to inquire about (real or imaginary numbers).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which internal buffer to inquire about (real or imaginary numbers)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IMAGINARY_PART	Inquires about the internal buffer containing the imaginary part of the values used by the wavelet filter.	c		e	f	g	a		i		j	k	l	m	n	b
M_REAL_PART	Inquires about the internal buffer containing the real part of the values used by the wavelet filter. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

For a specific type of image processing result buffer, the ContextOrResultImId and InquireType parameters can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a specific type of image processing result buffer																
ContextOrResultImId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType																
UserVarPtr - Possible values returned																

Event list image processing result ID	Specifies an event list image processing result buffer, allocated using MimAllocResult() with M_EVENT_LIST, and used in MimLocateEvent() operations.														
M_RESULT_OUTPUT_UNITS +	Inquires whether results are returned in pixel or world units. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Intensity histogram image processing result ID	Specifies an intensity histogram image processing result buffer, allocated using MimAllocResult() with M_HIST_LIST, and used in MimHistogram() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_BIN_SIZE_MODE +	Inquires the number of values each histogram bin can hold. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_SRC_DATA	Specifies that MimHistogram() determines the resulting bin size according to the source image's minimum and maximum intensity values, and the total number of bins. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_SRC_RANGE	Specifies that MimHistogram() determines the resulting bin size according to the full range of possible values in the source buffer, and the total number of bins. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FREEDMAN	Specifies that MimHistogram() determines the resulting bin size according to the Freedman–Diaconis rule. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_REGULAR	Specifies that each histogram bin can hold 1 value.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIST_SMOOTHING_ITERATIONS +	Inquires the number of smoothing iterations to perform on the histogram after it has been generated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of smoothing iterations. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Locate peak 1D image processing result ID	Specifies a 1D locate peak image processing result buffer, allocated using MimAllocResult() with M_LOCATE_PEAK_1D_CONTEXT, and used in MimLocatePeak1d() operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PEAK_INTENSITY_INVALID_VALUE +	Inquires the intensity value to use for missing peaks. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_INVALID	Specifies the value -1. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies any integer value, for applications with specific needs. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_CRITERION +		Inquires the characteristic with which to sort peaks. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_PEAK_INTENSITY +	Orders peaks according to their intensity.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PEAK_POSITION +	Orders peaks according to their position.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when ContextOrResultImId=Locate peak 1D image processing result ID, and InquireType=M_SORT_CRITERION, : M_PEAK_INTENSITY; M_PEAK_POSITION].

One of the following values might be combined with the above-mentioned values to specify the order to sort peaks.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the order to sort peaks																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SORT_DOWN	Sorts peaks in descending order.	c		e	f	g	a		i		j	k	l	m	n	b
M_SORT_UP	Sorts peaks in ascending order.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For inquiring about any image processing result buffer
- For inquiring about any image processing context
- For inquiring about a specific type of image processing context
- For inquiring about a specific type of image processing result buffer

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to a required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*. Note that M_TYPE_AIL_ID should only be used with M_OWNER_SYSTEM.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, if it is an *AIL_INT*. If the requested information is an *AIL_DOUBLE* or if it is an *AIL_INT64* on a 32-bit system, this function will return **M_NULL**.

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the values listed above are not available in Aurora Imaging Library Lite. See the value's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimLabel

Synopsis

Label objects in an image buffer.

Syntax

```
void MimLabel(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT64 ProcMode    //in
)
```

Description

This function labels all objects (or blobs) in the specified source image, starting from the top-left corner, with unique consecutive values beginning with the value 1. Each pixel in the same blob is given the same numerical value.

If you want to distinguish between touching blobs, you should separate the blobs. For example, you can use an erosion operation before performing the labeling operation.

Parameters

SrcImageBufId

Specifies identifier of the data source of the operation. This parameter must be given an image buffer identifier. If the source buffer is not binary, all non-zero pixels are considered as part of an object or blob.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer identifier. The destination buffer should be large enough to hold the maximum number of objects (blobs). For example, an 8-bit buffer can be used for a maximum of 254 blobs and a 16-bit buffer can be used for a maximum of 65534 blobs). If the destination buffer depth is too small, several blobs might be given the same value.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ProcMode

Specifies the type of connectivity (lattice) to use for processing. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of connectivity to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_4_CONNECTED +	Specifies that each pixel has 4 neighbors. If two blobs touch on the vertical and horizontal axis, they are considered one blob.	c		e	f	g	a		i		j	k	l	m	n	b
M_8_CONNECTED +	Specifies that each pixel has 8 neighbors. If two blobs touch on the vertical, horizontal, or diagonal, they are considered one blob.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the ProcMode parameter.

You can add one of the following values to the above-mentioned values to specify the processing mode.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the ProcMode parameter																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_BINARY	Specifies the binary processing mode. If the source image is a binarized image (containing 0 or the maximum value of the buffer, achieved for example with MimBinarize() or MimBinarizeAdaptive()), you can add M_BINARY to the connectivity mode to accelerate processing, particularly if the labeling is done by the Host.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Specifies the grayscale processing mode. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimLocateDifference

Synopsis

Locate the differences between 2 buffers, with some tolerance.

Syntax

```
AIL_INT MimLocateDifference(
    AIL_ID LocateDifferenceContextImId, //in
    AIL_ID Src1ImageBufId,              //in
    AIL_ID Src2ImageBufId,              //in
    AIL_ID LocateDifferenceResultImId,  //out
    AIL_DOUBLE AbsTolerance,            //in
    AIL_DOUBLE RelTolerance,            //in
    AIL_INT64 ControlFlag               //in
)
```

Description

This function locates the differences between 2 buffers, with some tolerance, and put the positions and values of the 2 sources in the result object. Absolute and relative tolerances will be taken in account to determine if the pixels are different. You can limit this function's results to a region of the source image buffers using a region of interest (ROI) set using MbufSetRegion().

Parameters

LocateDifferenceContextImId

Specifies the predefined locate difference context to use.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the locate difference context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_LOCATE_DIFFERENCE_CONTEXT_TOLERANCE_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_DIFFERENCE_CONTEXT_TOLERANCE_MAX	Specifies a predefined locate difference context that takes the maximum of the absolute tolerance and the relative tolerance applied on the maximum absolute value.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCATE_DIFFERENCE_CONTEXT_TOLERANCE_SUM	Specifies a predefined locate difference context that takes the sum of the absolute tolerance and the relative tolerance applied on the maximum absolute value.	c		e	f	g	a		i		j	k	l	m	n	b

Src1ImageBufId

Specifies the identifier of the first source image buffer.

This image buffer must be 1-band. This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Src2ImageBufId

Specifies the identifier of the second source image buffer.

This image buffer must be 1-band. This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

LocateDifferenceResultImId

Specifies the identifier of the image processing locate difference result buffer in which to write the results. The object must have previously allocated using MimAllocResult() with M_LOCATE_DIFFERENCE_RESULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the image processing locate difference result buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to not use an image processing locate difference result buffer. In this case, the function will return the number of differences.	c		e	f	g	a		i		j	k	l	m	n	b
LocateDifferenceResultImId	Specifies the identifier of the image processing locate difference result buffer in which to write the results.	c		e	f	g	a		i		j	k	l	m	n	b

AbsTolerance

Specifies an absolute tolerance.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the absolute tolerance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0.0	Specifies the absolute tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

RelTolerance

Specifies a relative tolerance. Relative tolerance is applied on the highest absolute value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the relative tolerance																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the relative tolerance.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Return value

If LocateDifferenceResultImId is **M_NULL**, the returned value is the number of differences. If LocateDifferenceResultImId is not **M_NULL**, the returned value is **M_INVALID**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimLocateEvent

Synopsis

Find pixel coordinates or values that satisfy a specified condition.

Syntax

```
AIL_INT MimLocateEvent(
    AIL_ID SrcImageBufId,   //in
    AIL_ID EventResultImId, //out
    AIL_INT64 Condition,    //in
    AIL_DOUBLE CondLow,     //in
    AIL_DOUBLE CondHigh     //in
)
```

Description

This function finds the coordinates and value of the pixels that satisfy the specified condition in the specified source image. Results are stored in the specified event result buffer.

You can retrieve the values, X- or Y-coordinates, and the number of pixels that satisfy the condition, using MimGetResult() or MimGetResult1d(). If the result buffer does not have the required number of entries to hold the total number of events that satisfy the condition, the number returned will be limited to the number of entries in the result buffer.

You can limit this function's results to a region of the source image buffer using a region of interest (ROI) set using MbufSetRegion().

Parameters

SrcImageBufId

Specifies the identifier of the source image for the operation. This parameter must be given the identifier of a 1-band buffer.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

EventResultImId

Specifies the identifier of the buffer in which to store the event results. This parameter must be given the identifier of an image processing result buffer, allocated using MimAllocResult() with an M_EVENT_LIST type. The buffer must have enough entries to hold the expected number of events.

If the number of pixels found is less than the number of event result buffer entries, the remaining result buffer entries are set to **M_INVALID**. If the number of events is greater than the number of allocated result buffer entries, the extra entries are discarded; when using multi-processing, the events kept might not be the first events in the source image. To determine the number of events so that you can allocate an appropriate result buffer, call **MimLocateEvent**() with EventResultImId set to **M_NULL**. The returned value is the number of pixels that satisfy the condition. Use this value to allocate a result buffer of the correct size. You then call this function a second time, specifying a result buffer with the correct size.

Condition

Specifies the condition under which pixel values are considered an event. This parameter must be set to one of the values below.

For conditions that use two limits, set the Condition parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For conditions that use two limits																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_IN_RANGE +	Specifies that pixels with values between CondLow and CondHigh, inclusive, are considered events.	c		e	f	g	a		i		j	k	l	m	n	b
M_OUT_RANGE	Specifies that pixels with values less than CondLow, or greater than CondHigh, are considered events.	c		e	f	g	a		i		j	k	l	m	n	b

For conditions that use one limit, set the Condition parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For conditions that use one limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_EQUAL	Specifies that only pixels with values equal to CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER	Specifies that only pixels with values greater than CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_GREATER_OR_EQUAL +	Specifies that only pixels with values greater than or equal to the CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS	Specifies that only pixels with values less than CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_LESS_OR_EQUAL +	Specifies that only pixels with values less than or equal to CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_EQUAL	Specifies that only pixels with values not equal to CondLow will be considered an event.	c		e	f	g	a		i		j	k	l	m	n	b

For conditions that use no limit, set the Condition parameter to the value below. Typically, this is only useful when specifying a local maxima or local minima condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For conditions that use no limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL +	Specifies that all pixels satisfying the local maxima or local minima condition will be considered an event. If you do not specify a local maxima or local minima condition, M_ALL results in the coordinates and value of all the source pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IN_RANGE; M_GREATER_OR_EQUAL; M_LESS_OR_EQUAL; M_ALL.

You can add one of the following values to the above-mentioned values to specify a local maxima or local minima as part of the condition under which pixel values are considered an event.

The local maxima or local minima condition only applies to pixels that meet the initial condition (for example, M_IN_RANGE).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a local maxima or local minima as part of the condition																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

<table>
<tr>
<td>M_LOCAL_MAX_NOT_STRICT</td>
<td>
Specifies that within a 3x3 neighborhood of a pixel, the pixel must be equal to or greater than all of its neighbors for it to be considered an event.
<table>
<tr><td>=<</td><td>=<</td><td>=<</td></tr>
<tr><td>=<</td><td>X</td><td>=<</td></tr>
<tr><td>=<</td><td>=<</td><td>=<</td></tr>
</table>
The following is an example that shows a pixel, at the center of its 3x3 neighborhood, that satisfies the M_LOCAL_MAX_NOT_STRICT condition.
<table>
<tr><td>5</td><td>5</td><td>5</td></tr>
<tr><td>5</td><td>5</td><td>5</td></tr>
<tr><td>5</td><td>5</td><td>5</td></tr>
</table>
</td>
<td>c</td><td></td><td>e</td><td>f</td><td>g</td><td>a</td><td></td><td>i</td><td></td><td>j</td><td>k</td><td>l</td><td>m</td><td>n</td><td>b</td>
</tr>
<tr>
<td>M_LOCAL_MAX_STRICT_MEDIUM</td>
<td>
Specifies that within a 3x3 neighborhood of a pixel, the pixel must be equal to or greater than all of its neighbors, and it must also be greater than the pixels on its left and the pixel below, for it to be considered an event.
<table>
<tr><td><</td><td>=<</td><td>=<</td></tr>
<tr><td><</td><td>X</td><td>=<</td></tr>
<tr><td><</td><td><</td><td>=<</td></tr>
</table>
The following is an example that shows a pixel, at the center of its 3x3 neighborhood, that satisfies the M_LOCAL_MAX_STRICT_MEDIUM condition.
<table>
<tr><td>4</td><td>5</td><td>5</td></tr>
<tr><td>4</td><td>5</td><td>5</td></tr>
<tr><td>4</td><td>4</td><td>5</td></tr>
</table>
</td>
<td>c</td><td></td><td>e</td><td>f</td><td>g</td><td>a</td><td></td><td>i</td><td></td><td>j</td><td>k</td><td>l</td><td>m</td><td>n</td><td>b</td>
</tr>
</table>

M_LOCAL_MIN_NOT_STRICT	Specifies that within a 3x3 neighborhood of a pixel, the pixel must be equal to or less than all of its neighbors for it to be considered an event. => => => => X => => => => The following is an example that shows a pixel, at the center of its 3x3 neighborhood, that satisfies the M_LOCAL_MIN_NOT_STRICT condition. 4 4 4 4 4 4 4 4 4	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL_MIN_STRICT_MEDIUM	Specifies that within a 3x3 neighborhood of a pixel, the pixel must be equal to or less than all of its neighbors, and it must also be less than the pixels on its left and the pixel below, for it to be considered an event. > => => > X => > > => The following is an example that shows a pixel, at the center of its 3x3 neighborhood, that satisfies the M_LOCAL_MIN_STRICT_MEDIUM condition. 5 4 4 5 4 4 5 5 4	c		e	f	g	a		i		j	k	l	m	n	b

CondLow

Specifies the lower limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the lower limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no lower limit is required. This setting should be selected for conditions that use M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the lower limit.	c		e	f	g	a		i		j	k	l	m	n	b

CondHigh

Specifies the upper limit of the selected condition.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the upper limit																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no upper limit is required. This setting should be selected for conditions that use one limit or M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the upper limit.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

If EventResultImId is **M_NULL**, the returned value is the number of pixels that satisfy the specified condition in the specified source image. If EventResultImId is not **M_NULL**, the returned value is **M_INVALID**. In that case, use either MimGetResult() or MimGetResult1d(), with M_NB_EVENT as the result type, to retrieve the number of pixels that satisfy the specified condition.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimLocatePeak1d

Synopsis

Performs peak intensity detection in every lane (row or column) of the source image.

Syntax

```
void MimLocatePeak1d(
    AIL_ID ContextId,         //in
    AIL_ID SrcImageBufId,     //in
    AIL_ID ResultId,          //out
    AIL_INT PeakWidthNominal, //in
    AIL_INT PeakWidthDelta,   //in
    AIL_DOUBLE MinContrast,   //in
    AIL_INT64 ControlFlag,    //in
    AIL_DOUBLE ControlValue   //in
)
```

Description

This function detects the position and the intensity of a peak in every lane (row or column) of an image. It is well suited for applications that generate 3D data based on laser line or sheet of light profiling.

The function iterates through each lane. For each lane, the function determines the pixels that are in the peak neighborhood, as defined by PeakWidthNominal, PeakWidthDelta, and MinContrast. For each such region, it finds the position of the highest intensity with sub-pixel accuracy, and calculates the average intensity around that position. The position can be retrieved in a fixed-point representation, with 0 to 7 fractional bits.

For optimal performance, preprocess the result buffer by calling **MimLocatePeak1d**() with M_PREPROCESS. You can choose to preprocess with or without an image. When preprocessing with an image, specify a typical image in the series of images that you will process with **MimLocatePeak1d**(). The preprocessed image's size and bit depth are stored. When preprocessing without an image (SrcImageBufId set to **M_NULL**), you must specify the number of scan lanes in a typical image in the series with ControlFlag. When preprocessing without an image, it is assumed that all images are 8-bit unsigned.

If the preprocess operation is not done explicitly (using M_PREPROCESS), **MimLocatePeak1d**() will preprocess the result buffer internally when it is first called. If a new image in the series has a different size or bit depth than the image used to preprocess, **MimLocatePeak1d**() will preprocess the result buffer again, given the new image's characteristics.

In a noisy image, or any image that might have multiple peaks in a single lane, such as when the object or surface is highly reflective, **MimLocatePeak1d**() can record the position and intensity of multiple peaks in a lane. You can create a custom application that sifts through the multiple peaks in each lane to determine the best peak in the lane for your needs. To record multiple peaks in a lane, use MimControl() with M_NUMBER_OF_PEAKS set to the maximum number of peaks to record. For more information, see the Multiple peaks in a single lane subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing.

You can retrieve the calculated results from the result buffer using MimGetResultSingle().

MimLocatePeak1d() can accumulate results from multiple frames in one result buffer. To do so, you must set MimControl() with M_NUMBER_OF_FRAMES, which is the maximum number of frames for which results can be accumulated in the result buffer. When calling **MimLocatePeak1d**(), set M_FRAME_INDEX to the appropriate index in the result buffer which corresponds to the frame for which to store results.

MimLocatePeak1d() supports multi-frame image buffers. When passing a multi-frame image buffer, you must set MimControl() with M_FRAME_SIZE to the Y-size of each individual frame. For information on multi-frame image buffers, see the Specifying the dimensions of a multi-frame image buffer subsection of the Specifying the dimensions of a data buffer section of Chapter 23: Data buffers. Note that the total number of frames (M_NUMBER_OF_FRAMES) can be different from the frame capacity of the source image buffer. For example, if the source image buffer has a 5-frame capacity and you have 10 total frames to process, 2 calls to **MimLocatePeak1d**() can be done before calling MimGetResultSingle(). Typically, you would set M_FRAME_INDEX to frame 0 on the first call to **MimLocatePeak1d**(); then, set it to frame 5 on the second call. Finally, it is important to set M_FRAME_INDEX such that the result buffer can hold all results produced from the call. That is, there must be enough indices available in the result buffer to accommodate results from all frames in the source image buffer. For more information on peak extraction from a multi-frame image buffer, see the Extraction of peaks from a multi-frame buffer subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing.

For results obtained from laser line images, you can draw an uncorrected depth map or intensity map using MimDraw() with

M_DRAW_DEPTH_MAP_ROW or M_DRAW_INTENSITY_MAP_ROW, respectively. For more information, see the Generating an uncorrected depth map subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing.

To verify that the peak was found at the right location, you can draw the calculated peak intensities at the calculated positions into an image buffer using MimDraw() with M_DRAW_PEAKS.

Parameters

ContextId

Specifies the identifier of the image processing context.

The image processing context must have been previously allocated using MimAlloc() with M_LOCATE_PEAK_1D_CONTEXT.

SrcImageBufId

Specifies the identifier of the source image buffer.

Note that this function only supports 8- and 16-bit unsigned source image buffers.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

This image buffer must not have an associated camera calibration context. Using an image buffer with an associated camera calibration context will cause an error.

When preprocessing (M_PREPROCESS), you can specify an image in the series, or you can set this parameter to **M_NULL**.

ResultId

Specifies the identifier of the image processing result buffer in which to store results. The result buffer must have been allocated using MimAllocResult() with M_LOCATE_PEAK_1D_RESULT.

PeakWidthNominal

Specifies the nominal (expected average) width of the peak neighborhood. In laser line images, this is the average width of the laser line.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the nominal width of the peak																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to use the value of the control type M_PEAK_WIDTH_NOMINAL, set using MimControl(). The default value of M_PEAK_WIDTH_NOMINAL is 20.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to ignore this parameter. This setting should be used exclusively when ControlFlag is set to M_PREPROCESS; this is the only possible setting in this case.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the average width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

PeakWidthDelta

Specifies the number of pixels that can be added to or subtracted from the nominal width, when determining the range of allowable widths of the peak neighborhood.

PeakWidthNominal plus PeakWidthDelta defines the maximum width of the peak neighborhood. PeakWidthNominal minus PeakWidthDelta defines the minimum width of the peak neighborhood.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the variance of the width of the peak																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies to use the value of the control type M_PEAK_WIDTH_DELTA, set using MimControl(). The default value of M_PEAK_WIDTH_DELTA is 20.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to ignore this parameter. This setting should be used exclusively when ControlFlag is set to M_PREPROCESS; this is the only possible setting in this case.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum amount of variance from the nominal width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

MinContrast

Specifies the minimum contrast (difference) between the intensity of the local background and the minimum acceptable intensity of pixels in the peak neighborhood.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the contrast that defines the peak																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to use the value of the control type M_MINIMUM_CONTRAST, set using MimControl(). The default value of M_MINIMUM_CONTRAST is 100.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to ignore this parameter. This setting should be used exclusively when ControlFlag is set to M_PREPROCESS; this is the only possible setting in this case.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= 65534	Specifies the minimum difference in intensity. > Note that the default value is 100, regardless of the bit depth of the image. To keep the default functionality in a 16-bit image, set this value to 25,600.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies whether to explicitly preprocess the image processing result buffer; or specifies a frame index for which to extract peaks.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the ControlFlag setting.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlFlag** and **ControlValue** parameters.

- For specifying preprocessing or for specifying the frame index into which peaks will be extracted

The following ControlFlag and corresponding ControlValue parameter settings are used to control preprocessing and for specifying the frame index.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying preprocessing or for specifying the frame index into which peaks will be extracted																
ControlFlag	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																

M_DEFAULT	Specifies to perform the peak intensity detection on the specified image. When the specified image processing result buffer has not been preprocessed or has been preprocessed with an image that has different characteristics than SrcImageBufId, (for instance, size or bit depth), **MimLocatePeak1d**() will automatically preprocess the image processing result buffer before performing the peak intensity detection. If a multi-frame image buffer is used (MimControl() with M_FRAME_SIZE not set to M_FULL_SIZE), peak extraction begins at the first frame (frame 0).															
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_FRAME_INDEX	Specifies the frame index in the M_LOCATE_PEAK_1D_RESULT result buffer into which extractions will be accumulated. If processing a multi-frame image buffer, this sets the first frame index in the result buffer into which extractions will be accumulated. When accumulating results from multiple calls to **MimLocatePeak1d**(), set M_FRAME_INDEX to the appropriate index in the result buffer. If using a multi-frame image buffer, specify a frame index that is subsequent to the frames processed in the preceding call, so as not to overwrite previous results.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_FRAMES	Specifies the frame index into which peaks will be accumulated.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Specifies to preprocess the specified image processing result buffer. When M_PREPROCESS is specified, set PeakWidthNominal, PeakWidthDelta, and MinContrast to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior, when preprocessing with a source image.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the number of scan lanes in subsequent source images, when preprocessing without an image (ControlFlag set to M_PREPROCESS and SrcImageBufId set to **M_NULL**).	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimLutMap

Synopsis

Perform a point-to-point LUT mapping operation.

Syntax

```
void MimLutMap(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_ID LutBufId       //in
)
```

Description

This function maps each pixel in the specified source image to values determined by the specified lookup table (LUT).

If the source image is signed, negative numbers are treated as unsigned values and address the LUT accordingly. For example, -1 is represented as 0xff for 8-bit image data and 0xffff for 16-bit image data and will therefore address LUT entry 0xff or 0xffff, respectively.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer identifier. If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

LutBufId

Specifies the LUT through which to map source input values. This parameter must be given a valid LUT buffer identifier.

If the LUT has fewer entries than the full range of source values, unexpected results will occur. If the LUT buffer depth is greater than that of the destination buffer, results will be truncated to fit the destination buffer. LUT entries must have been previously initialized (for example, using MgenLutRamp() or MbufPut1d()).

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimMatch

Synopsis

Generates a series of correlation values between the source image and a model image.

Syntax

```
void MimMatch(
    AIL_ID MatchContextImId, //in
    AIL_ID SrcImageBufId,    //in
    AIL_ID DstImageBufId,    //out
    AIL_INT64 ControlFlag    //in
)
```

Description

This function generates a series of correlation values (match scores) resulting from the model image being compared to every pixel in your image. The **MimMatch**() operation results in an image of correlation data. The closer the resulting pixel value is to the specified maximum score, the closer the match between the source image buffer and the model image.

This function uses a match image processing context to match a pixel in the source image with a given model image. Specify the model image using MimControl() with M_MODEL_IMAGE. You can also specify the algorithm to use to perform the correlation (the mode), using MimControl() with M_MODE. In addition, you can limit the region in the model image with which to perform the correlation, using a mask image; specify the mask image using MimControl() with M_MASK_IMAGE. In the match mask image, all non-zero pixels are considered mask pixels that are ignored during the match.

When dealing with large images, you can reduce the number of comparisons performed from 1 every pixel to 1 every other pixel in your model image, using MimControl() with M_MODEL_STEP.

Before you call this function to perform a match operation, preprocess the match image processing context by calling **MimMatch**() with M_PREPROCESS. If the preprocess operation is not done explicitly (using M_PREPROCESS), it will be done when **MimMatch**() is first called. When preprocessed explictly, the source and destination image buffers can be set to **M_NULL**. If, however, the source or destination image buffer is provided, it will be used in the preprocess operation, to better optimize future calls.

Parameters

MatchContextImId

Specifies the identifier of the match image processing context. The context must have been allocated using MimAlloc() with M_MATCH_CONTEXT. In addition, the match image processing context must already be configured with the identifier of the model image.

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer must be an 8-bit unsigned image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When preprocessing the match image processing context, set this parameter to a typical source image buffer. To exclude this buffer from preprocessing, set this parameter to **M_NULL**.

DstImageBufId

Specifies the identifier of the destination image buffer. The buffer must be either a 16-bit signed or a 32-bit floating-point image buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When preprocessing the match image processing context, set this parameter to a typical destination image buffer. To exclude this buffer from preprocessing, set this parameter to **M_NULL**.

ControlFlag

Specifies the operation to perform. This parameter must be set to one of the following values.

For specifying the operation to perform.																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Performs the match operation according to the settings of the match image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Preprocess the specified image processing context. Note that, if not called explicitly, this operation will be performed automatically with the first call to **MimMatch**(). If the identifier of the source and/or destination buffer is not set to **M_NULL**, information about the source and/or destination image buffer will be used in the preprocessing operation.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimMorphic

Synopsis

Perform a morphological transformation using a user-defined or predefined structuring element.

Syntax

```
void MimMorphic(
    AIL_ID SrcContainerOrImageBufId, //in
    AIL_ID DstContainerOrImageBufId, //out
    AIL_ID StructElemBufId,          //in
    AIL_INT64 Operation,             //in
    AIL_INT NbIterationOrArea,       //in
    AIL_INT64 ProcMode               //in
)
```

Description

This function performs one of several morphological transformations on the specified source image or depth map container, using a user-defined or predefined structuring element.

If the source and destination are multi-band buffers, the same structuring element is applied to every band of the source buffer.

If the source is a depth map container, invalid pixels are copied unchanged into the destination buffer. Also, for the supported operations, the M_GRAYSCALE processing mode is used.

Parameters

SrcContainerOrImageBufId

Specifies the identifier of the source image buffer or depth map container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source image buffer or depth map container identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Source depth map container identifier	Specifies the identifier of the source depth map container. The depth map container must have been previously allocated using MbufAllocContainer() with M_PROC, and must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE). The container must not have a region component. Using a container with a region component will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Source image buffer identifier	Specifies the identifier of the source image buffer. The image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstContainerOrImageBufId

Specifies the identifier of the destination buffer in which to put the result of the operation. If SrcContainerOrImageBufId is an image buffer, this parameter must be given an image buffer identifier. If SrcContainerOrImageBufId is a depth map container, this parameter must be given the identifier of a depth map image buffer or depth map container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination image buffer, depth map image buffer, or depth map container identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Destination depth map container identifier	Specifies the identifier of the destination depth map container. The depth map container must have been previously allocated using MbufAllocContainer() with M_PROC, must store data in a 3D-processable depth map format (that is, if you call MbufInquireContainer() with M_3D_PROCESSABLE_DEPTH_MAP, the function returns M_TRUE), and must not be a child container.	c		e	f	g	a		i		j	k	l	m	n	b
Destination depth map image buffer identifier	Specifies the identifier of the destination depth map image buffer. The depth map image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned image buffer with the same dimensions (SizeX and SizeY) as the source depth map container's range component. It must be 3D-corrected (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE), and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
Destination image buffer identifier	Specifies the identifier of the destination image buffer. The image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

StructElemBufId

Specifies the structuring element to use. For all operations except area opening and area closing operations, you must specify the identifier of the structuring element buffer which defines the required structuring element.

For M_AREA_OPEN or M_AREA_CLOSE operations, you must specify either an M_3X3_CROSS or an M_3X3_RECT structuring element. Note that these predefined structuring elements are not available for any other type of operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the structuring element																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_3X3_CROSS	Specifies a cross (+) structuring element whose neighborhood size is 3x3 pixels. In this case, there are 5 valid neighborhood pixels; the function ignores the pixels located at the four corners of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_3X3_RECT	Specifies a structuring element whose neighborhood size is 3x3 pixels. In this case, there are 9 valid neighborhood pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Structuring element buffer identifier	Specifies a custom structuring element to use, allocated using MbufAlloc...() with an M_STRUCT_ELEMENT attribute. This structuring element should be loaded with structuring-element values, using MbufPut(). These values can be set to **M_DONT_CARE** to specify that the corresponding source image pixels should not be taken into account during the operation. You can set the overscan type and the center of the structuring element, using MbufControl(). For window leveling operations, the structuring element buffer must contain only 0 or **M_DONT_CARE** values. The center structuring element value must not be an **M_DONT_CARE** value. When the source is a depth map container, the structuring element buffer must contain only 0 or **M_DONT_CARE** values. The center structuring element value must be 0 (to avoid introducing new invalid pixels). For M_BOTTOM_HAT, M_CLOSE, M_OPEN, and M_TOP_HAT operations, the structuring element value at position *Size* - *Center* - 1 must also be 0, since it will be the center of the second operation.	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the operation to perform. Supported operations are given in the table below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation to perform

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AREA_CLOSE	Specifies an area close operation. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_AREA_OPEN	Specifies an area open operation. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOTTOM_HAT +	Specifies a bottom-hat operation. This operation is the equivalent to a close operation (M_CLOSE), followed by a subtraction of the source from the results.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSE +	Specifies a close operation. This operation is equivalent to a dilation followed by an erosion.	c		e	f	g	a		i		j	k	l	m	n	b
M_DILATE +	Specifies a dilation operation. When performing a binary dilation, if any of the structuring element values match the corresponding neighborhood values, the center pixel is set to the maximum value of the buffer (for example, 0xff for an 8-bit buffer); otherwise, it remains unchanged. In effect, binary dilation adds layers to the objects. When performing a grayscale dilation, this operation adds each structuring element value to the corresponding pixel value in the neighborhood, and then replaces the center pixel of the neighborhood with the maximum value from the resulting neighborhood values. The pixels that correspond to **M_DONT_CARE** in the structuring element are ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_ERODE +	Specifies an erosion operation. When performing a binary erosion, if the structuring element does not match the corresponding neighborhood values exactly, the center pixel is set to zero; otherwise, it remains unchanged. In effect, binary erosion peels off layers of objects. When performing a grayscale erosion, this operation subtracts each structuring element value from the corresponding pixel value in the neighborhood, and then replaces the center pixel of the neighborhood with the minimum value from the resulting neighborhood values. The pixels that correspond to **M_DONT_CARE** in the structuring element are ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_HIT_OR_MISS	Specifies a hit or miss transformation. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_LEVEL	Specifies a window leveling operation on a pixel neighborhood basis. This operation finds the minimum and maximum pixel values in a neighborhood, and then creates a linear mapping such that these map to the minimum and maximum possible values of the image buffer. It then replaces the center pixel of the neighborhood with its corresponding value in the mapping. If the destination image buffer is a floating-point buffer, each pixel will be set to a value between 0 and 1. The pixels that correspond to **M_DONT_CARE** in the structuring element are ignored. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_MATCH	Specifies a matching operation. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OPEN +	Specifies an open operation. This operation is equivalent to an erosion, followed by a dilation.	c		e	f	g	a		i		j	k	l	m	n	b
M_THICK	Specifies a thickening operation. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_THIN	Specifies a thinning operation. > Note that this operation is only available when the source is an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_TOP_HAT +	Specifies a top-hat operation. This operation is the equivalent to an open operation (M_OPEN) followed by a subtraction of the results from the source.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_BOTTOM_HAT; M_CLOSE; M_DILATE; M_ERODE; M_OPEN; M_TOP_HAT.

You can add the following value to the above-mentioned values to specify to combine multiple iterations of an operation into a larger equivalent structuring element and to perform the combined operation once.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to combine multiple iterations of an operation																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ITER_COMBINED	Specifies to combine multiple iterations of an operation into a larger equivalent structuring element and to perform the combined operation once. This can improve operation performance for multiple iterations if the equivalent structuring element is not too large. > Note that the resulting borders might be different than when not combining the iterations into a single operation.	c		e	f	g	a		i		j	k	l	m	n	b

NbIterationOrArea

Specifies a value that is dependent on the morphological operation chosen.

For an M_AREA_OPEN or M_AREA_CLOSE operation, this parameter specifies the minimum area, in pixels. For an M_HIT_OR_MISS, M_MATCH, or M_LEVEL operation, this parameter must be set to 1. For the other morphological operations, this parameter specifies the number of times to iterate the operation.

If the number of iterations is set to 0, and ProcMode is set to M_BINARY, the source is binarized and the result is copied into the destination buffer. If the number of iterations is set to 0, and ProcMode is set to M_GRAYSCALE, the source is copied into the destination buffer.

To control how the overscan operation is handled when calling **MimMorphic**() with a custom structuring element and specifying multiple iterations, use MbufControl() with M_ITER_OVERSCAN.

ProcMode

Specifies the processing mode to use. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Treats non-zero pixels as ones (1) during processing.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Uses the source's gray values for processing. The resulting pixels will be grayscale. > You must use this processing mode to perform an M_TOP_HAT, M_BOTTOM_HAT, or M_LEVEL operation, or when the source is a depth map container.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimMorphicShape

Synopsis

Perform a morphological transformation using a predefined shape.

Syntax

```
void MimMorphicShape(
    AIL_ID ContextId,     //in
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_DOUBLE Dimension, //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function performs morphological transformations on the specified source image, using a circular shape.

Parameters

ContextId

Specifies a predefined context used to perform the morphological transformation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the predefined context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CIRCULAR_CONTEXT_BINARY_CLOSE	Specifies a binary close operation using a circle. This operation is equivalent to a circular dilation followed by a circular erosion.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCULAR_CONTEXT_BINARY_DILATE	Specifies a binary dilate operation using a circle.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCULAR_CONTEXT_BINARY_ERODE	Specifies a binary erode operation using a circle.	c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCULAR_CONTEXT_BINARY_OPEN	Specifies a binary open operation using a circle. This operation is equivalent to a circular erosion followed by a circular dilation.	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageBufId

Specifies the identifier of the source image buffer.

This image buffer must be 1-band, 8-bit unsigned and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination buffer in which to put the result of the operation.

This image buffer must be 1-band, 8-bit unsigned and must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Dimension

Specifies the diameter of the circle element, in pixels.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the diameter

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value > = 1	Specifies the diameter of the circle element used for the morphological transformation. The value must be an odd integer.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimOpen

Synopsis

Perform a binary or grayscale opening-type morphological operation.

Syntax

```
void MimOpen(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbIteration,  //in
    AIL_INT64 ProcMode    //in
)
```

Description

This function performs a binary or grayscale opening operation on the given source image for the specified number of iterations. An opening is an erosion followed by a dilation.

In binary mode, this function uses a 3x3 full rectangular structuring element; in grayscale mode, a 3x3 empty one.

The overscan pixels are automatically set to the highest possible buffer value for the erosion and the lowest possible buffer value for the dilation; this will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

NbIteration

Specifies the number of times to iterate the operation.

When this parameter is set to 0 and ProcMode is set to M_BINARY, the source image is binarized and the result is copied into the destination image buffer.

When this parameter is set to 0 and ProcMode is set to M_GRAYSCALE, the source image is copied into the destination image buffer.

ProcMode

Specifies the processing mode to use. This parameter can be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Treats non-zero pixels as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Uses the source image's gray values for processing. The resulting buffer also contains gray values.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimPolarTransform

Synopsis

Perform a polar-to-rectangular or rectangular-to-polar transformation on an image or generate LUTs used to perform the transformation.

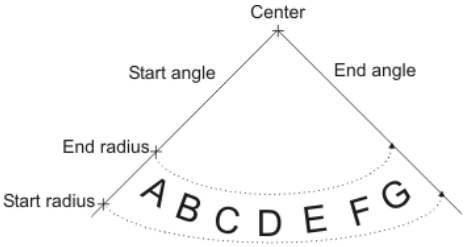
Syntax

```
void MimPolarTransform(
    AIL_ID SrcImageOrDstXLutBufId, //in
    AIL_ID DstImageOrYLutBufId,    //out
    AIL_DOUBLE CenterPosX,         //in
    AIL_DOUBLE CenterPosY,         //in
    AIL_DOUBLE StartRadius,        //in
    AIL_DOUBLE EndRadius,          //in
    AIL_DOUBLE StartAngle,         //in
    AIL_DOUBLE EndAngle,           //in
    AIL_INT64 OperationMode,       //in
    AIL_INT64 InterpolationMode,   //in
    AIL_DOUBLE *DstSizeXPtr,       //out
    AIL_DOUBLE *DstSizeYPtr        //out
)
```

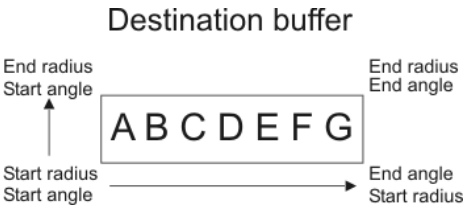
Description

This function performs a rectangular-to-polar or polar-to-rectangular transformation. Alternatively, this function can generate the X- and Y-coordinate LUTs required to perform the transformation using MimWarp().

For a rectangular-to-polar transformation, the zone of interest to convert is based on a circle centered at CenterPosX and CenterPosY, with the part to convert having the specified start and end radius (StartRadius and EndRadius) and specified start and end angle (StartAngle and EndAngle), measured with respect to the X-axis of the image.



The result will be mapped to the destination buffer as follows:



The increment in angle is determined by the length, in pixels, of the outside arc, calculated as follows:

(endangle - startangle) / arclength.

A polar-to-rectangular transform performs the reverse of the transform described above. It takes a polar buffer and maps it to a rectangular buffer. Use the CenterPosX, CenterPosY, StartAngle, EndAngle, StartRadius, EndRadius parameters to specify where in the destination buffer the contents of the polar buffer will be mapped.

To determine the required size of the destination image or LUT buffer, call the function with DstImageOrYLutBufId set to **M_NULL**; in this case, DstSizeXPtr and DstSizeYPtr will return the required X- and Y- size of the buffer, respectively. Note that DstSizeXPtr and DstSizeYPtr will return *AIL_DOUBLE* values; to allocate an appropriate buffer using these values, you should use a ceiling function, such as `ceil()`, to obtain the X- and Y- sizes as *integer (AIL_INT)* values.

Parameters

SrcImageOrDstXLutBufId

Specifies the identifier of the source image buffer or the destination X-coordinate LUT buffer.

See the Parameter associations section for possible values that can be specified.

DstImageOrYLutBufId

Specifies the identifier of the destination image buffer or the Y-coordinate LUT buffer.

See the Parameter associations section for possible values that can be returned.

CenterPosX

Specifies the X-coordinate of the circle's center in the rectangular image.

CenterPosY

Specifies the Y-coordinate of the circle's center in the rectangular image.

StartRadius

Specifies the start radius of the zone of interest in the circle, in pixels.

EndRadius

Specifies the end radius of the zone of interest in the circle, in pixels.

StartAngle

Specifies the start angle of the zone of interest. The scan starts at this angle.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the starting angle																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
-360.0 <= *Value* <= 720.0	Specifies the start angle. If the start angle (StartAngle) is less than the end angle (EndAngle), the direction of the scan will be counter clockwise. If the start angle is greater than the end angle, the direction of the scan will be clockwise. There is no maximum span limit between StartAngle and EndAngle.	c		e	f	g	a		i		j	k	l	m	n	b

EndAngle

Specifies the end angle of the zone of interest. The scan ends at this angle.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the ending angle																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
-360.0 <= *Value* <= 720.0	Specifies the end angle. If the end angle (EndAngle) is greater than the start angle (StartAngle), the direction of the scan will be counter clockwise. If the end angle is less than the start angle, the direction of the scan will be clockwise. There is no maximum span limit between StartAngle and EndAngle.	c		e	f	g	a		i		j	k	l	m	n	b

OperationMode

Specifies the transform to perform.

See the Parameter associations section for possible values that can be specified.

InterpolationMode

Specifies the interpolation mode and overscan used when transforming the source image. When generating the LUTs to perform the transformation, this parameter is ignored and should be set to M_DEFAULT.

This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the interpolation mode used in the conversion																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default interpolation mode and overscan. When transforming the source image, the default is the same as M_NEAREST_NEIGHBOR + M_OVERSCAN_ENABLE. When only generating the LUTs to perform the transformation, this is the only possible value.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC +	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR +	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR +	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the InterpolationMode parameter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to determine the value of a destination pixel when its associated point falls outside the source buffer																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_CLEAR	Sets the destination pixel to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Leaves the destination pixel as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Uses pixels from the source buffer's ancestor buffer. If the source buffer is not a child buffer or if the point falls outside the ancestor buffer, leave the destination pixel as is. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Points falling outside the source image are undefined.										j	k	l	m		
M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

DstSizeXPtr

Specifies the address of the variable in which the required width of the destination buffer is written. If you don't need this information, set this parameter to **M_NULL**.

DstSizeYPtr

Specifies the address of the variable in which the required height of the destination buffer is written. If you don't need this information, set this parameter to **M_NULL**.

Parameter associations

The table below lists possible values for the **SrcImageOrDstXLutBufId** and **OperationMode** parameters and possible values returned to the **DstImageOrYLutBufId** parameter.

- For selecting the Operation Mode

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For selecting the Operation Mode																	
OperationMode SrcImageOrDstXLutBufId DstImageOrYLutBufId - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_POLAR_TO_RECTANGULAR	Performs a polar-to-rectangular transformation.															
SrcImageOrDstXLutBufId	Specifies the identifier of the source image buffer containing the polar image. This buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageOrYLutBufId	Specifies the identifier of the destination buffer in which to store the transformed rectangular image. Use the CenterPosX, CenterPosY, StartAngle, EndAngle, StartRadius, EndRadius parameters to specify the zone in which to write the rectangular version of the polar image. This buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. > Note that after performing the operation, the destination image will not be calibrated, even if the source image was calibrated.	c		e	f	g	a		i		j	k	l	m	n	b
M_POLAR_TO_RECTANGULAR_LUT +	Generates X- and Y-coordinate LUTs for use with MimWarp() to perform the specified polar-to-rectangular transformation.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageOrDstXLutBufId	Specifies the identifier of the destination buffer in which to store the X-coordinate LUT. This buffer must be a signed 16- or 32-bit buffer with an M_LUT attribute.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageOrYLutBufId	Specifies the identifier of the destination buffer in which to store the Y-coordinate LUT. This buffer must be a signed 16- or 32-bit buffer with an M_LUT attribute.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGULAR_TO_POLAR	Performs a rectangular-to-polar transformation.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageOrDstXLutBufId	Specifies the identifier of the source image buffer containing the rectangular image. Use the CenterPosX, CenterPosY, StartAngle, EndAngle, StartRadius, EndRadius parameters to define the image's zone of interest to transform. This buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageOrYLutBufId	Specifies the identifier of the destination image buffer in which to place the transformed zone of interest. This image is a polar image. This buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_RECTANGULAR_TO_POLAR_LUT +	Generates X- and Y-coordinate LUTs for use with MimWarp() to perform the specified rectangular-to-polar transformation.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageOrDstXLutBufId	Specifies the identifier of the destination buffer in which to store the X-coordinate LUT. This buffer must be a signed 16- or 32-bit buffer with an M_LUT attribute.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageOrYLutBufId	Specifies the identifier of the destination buffer in which to store the Y-coordinate LUT. This buffer must be a signed 16- or 32-bit buffer with an M_LUT attribute.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_POLAR_TO_RECTANGULAR_LUT; M_RECTANGULAR_TO_POLAR_LUT.

You can add the following value to the above-mentioned values to specify the number of fractional bits in the coordinates of the source point.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For generating LUTs																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIXED_POINT + *n*	Specifies the number of fractional bits for the source point (x_s , y_s). To do so, add the define M_FIXED_POINT + n to M_POLAR_TO_RECTANGULAR_LUT or M_RECTANGULAR_TO_POLAR_LUT where *n* >= 0. If nothing is added to M_POLAR_TO_RECTANGULAR_LUT or M_RECTANGULAR_TO_POLAR_LUT, it is assumed that there are no fractional bits in the coordinates of the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimProjection

Synopsis

Project a 2D image into 1D.

Syntax

```
void MimProjection(
    AIL_ID SrcImageBufId,           //in
    AIL_ID DstImageBufOrResultImId, //out
    AIL_DOUBLE ProjectionAxisAngle, //in
    AIL_INT64 Operation,            //in
    AIL_DOUBLE OperationValue       //in
)
```

Description

This function projects a two-dimensional image into one dimension, based on the specified projection operation and projection axis angle, writing results to the specified projection image processing result buffer or a destination image buffer. The results are generated based on the specified operation and calculated along each lane of pixels in the image, perpendicular to the specified projection axis angle. The 90° projection of the image (lanes = rows) using a summation operation is known as the row profile; the 0° projection (lanes = columns) as the column profile.

Writing results to a projection image processing result buffer allows you to manipulate results using an array once you have retrieved the results using MimGetResult() or MimGetResult1d().

You can limit this function's results to a region of the source image buffer using a region of interest (ROI) set using MbufSetRegion().

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer. The image buffer must be a 1-band buffer.

This image buffer can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

DstImageBufOrResultImId

Specifies the destination buffer in which to write the projection results.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the projection result buffer identifier or a destination image buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Destination image buffer ID	Specifies the identifier of an image buffer allocated with MbufAlloc...(). The destination image buffer must have the same data type (unsigned, signed, or float) as the source buffer. For an M_SUM operation, this image buffer must be of 32-bit depth. For an M_MAX, M_MEAN, M_MEDIAN, M_MIN, M_RANK, or M_RANK_PERCENTILE operation, this image buffer can have a depth greater than or equal to that of the source image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Destination result buffer ID	Specifies the identifier of an image processing result buffer allocated using MimAllocResult() with M_PROJ_LIST. The buffer should have as many locations as there are lanes to project in the image perpendicular to the specified projection axis angle. You can read the projection values from the result buffer using MimGetResult1d() or MimGetResult() with M_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b

ProjectionAxisAngle

Specifies the angle of the projection axis. Lanes are perpendicular to this angle. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the angle of projection in degrees																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_0_DEGREE	Specifies a projection onto the axis at 0° (lanes = columns).	c		e	f	g	a		i		j	k	l	m	n	b
M_90_DEGREE	Specifies a projection onto the axis at 90° (lanes = rows).	c		e	f	g	a		i		j	k	l	m	n	b

Operation

Specifies the operation to perform.

The following operations are supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_SUM.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX	Specifies to find the maximum pixel value along each lane.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEAN	Specifies to find the mean pixel value along each lane.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEDIAN	Specifies to find the median pixel value along each lane.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Specifies to find the minimum pixel value along each lane.	c		e	f	g	a		i		j	k	l	m	n	b
M_RANK	Specifies to find the pixel value with the specified rank (OperationValue) along each lane. The pixels in a lane are ranked in ascending order based on pixel value. The rank cannot be larger than the lane size of the image. This can be inquired using MbufInquire() with M_SIZE_X or M_SIZE_Y, depending on the projection axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_RANK_PERCENTILE	Specifies to find the pixel value with the specified rank (OperationValue) along each lane; the rank is specified as a percentage of the lane length. The pixels in a lane are ranked in ascending order based on pixel value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM	Specifies to sum all pixel values along each lane. Note that, if the sum of pixels in any lane is larger than the depth of the result buffer or destination image buffer, the result will be undefined.	c		e	f	g	a		i		j	k	l	m	n	b

OperationValue

Specifies the rank for M_RANK, and the percentile for M_RANK_PERCENTILE. For other operations, this parameter must be set to **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimPut

Synopsis

Puts arrays of values into an image processing context.

Syntax

```
void MimPut(
    AIL_ID ContextImId,    //out
    AIL_INT64 PutType,     //in
    AIL_INT ArraySize,     //in
    const void *Param1Ptr, //in
    const void *Param2Ptr, //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function puts arrays of values into a specified image processing context.

The number of arrays and the nature of the values depend on the image processing context used and on the specified type of information to set. After setting the array(s), their values can be inquired using MimGet().

Parameters

ContextImId

Specifies the identifier of the image processing context. The image processing context must have been previously allocated with MimAlloc().

See the Parameter associations section for possible values that can be returned.

PutType

Specifies the type of information in the array(s).

See the Parameter associations section for possible values that can be specified.

ArraySize INQ

Specifies the number of values in the array(s).

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the Param1Ptr or Param2Ptr parameter.

Param1Ptr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a constant std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a constant std::vector<AIL_INT64>]

Specifies the address of the first array of values.

See the Parameter associations section for possible values that can be specified.

Param2Ptr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>]
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a constant std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>]

- array of type AIL_INT64 [optionally, in C++: a reference to a constant std::vector<AIL_INT64>]

Specifies the address of the second array of values.

> Note that the second array must have the same number of elements as the first array.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The table below lists possible values for the **PutType**, **Param1Ptr**, and **Param2Ptr** parameters and possible values returned to the **ContextImId** parameter.

- For setting array(s) of values into an image processing context

The following ContextImId, PutType, Param1Ptr, and Param2Ptr parameter settings can be specified to put values into an image processing context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For setting array(s) of values into an image processing context																			
ContextImId - Possible values returned			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	PutType																		
		Param1Ptr																	
		Param2Ptr																	
Dead pixel correction image processing context ID +			Specifies a dead pixel correction image processing context, allocated using MimAlloc() with M_DEAD_PIXEL_CONTEXT, and used in MimDeadPixelCorrection() operations.																
	M_XY_DEAD_PIXELS +		Specifies the X- and Y-coordinates of the dead pixels in the source image, relative to the pixel coordinate system. Note that you should only use this setting if you have not specified an image that identifies the dead pixels, using MimControl() with M_DEAD_PIXELS.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the X-coordinates of the dead pixels, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the Y-coordinates of the dead pixels, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Rearrangement image processing context ID +			Specifies a rearrangement image processing context, allocated using MimAlloc() with M_REARRANGE_CONTEXT, and used in MimRearrange() operations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_DESTINATION +		Specifies the X- and Y-offsets of the areas in the destination image buffer into which to copy the source areas, relative to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b

		Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the X-offsets of the destination areas, in pixels. When in line mode (MimControl() with M_MODE set to M_LINES), all values in this array must be set to 0, because MimRearrange() can only copy entire rows in M_LINES mode. For example, when dealing with 3 areas, the X-offset array must contain the following: [0, 0, 0].	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the Y-offsets of the destination areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_SIZE +		Specifies the width and height of the areas to copy, relative to the pixel coordinate system. Note that this settings is only available when in rectangle mode (MimControl() with M_MODE set to M_RECTS).	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the width of the areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the height of the areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	M_XY_SOURCE +		Specifies the X- and Y-offsets of the areas to copy from the source image buffer into the destination image buffer, relative to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the X-offsets of the source areas, in pixels. When in line mode (MimControl() with M_MODE set to M_LINES), all values in this array must be set to 0, because MimRearrange() can only copy entire rows in M_LINES mode. For example, when dealing with 3 areas, the X-offset array must contain the following: [0, 0, 0].	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]* Specifies an array with the Y-offsets of the source areas, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
Unwarp along path image processing context ID +			Specifies an unwarp along path image processing context, allocated using MimAlloc() with M_UNWARP_ALONG_PATH_CONTEXT, and used in MimUnwarpAlongPath() operations.	c		e	f	g	a		i		j	k	l	m	n	b

	M_XY_PATH +		Specifies the X- and Y-coordinates of the path in the source image, relative to the pixel coordinate system. To set the width of the path, use MimControl() with M_SINGLE_WIDTH.	c		e	f	g	a		i		j	k	l	m	n	b
		Param1Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]* Specifies an array with the X-coordinates of the path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
		Param2Ptr	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]* Specifies an array with the Y-coordinates of the path, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For setting array(s) of values into an image processing context.

One of the following values might be combined with the above-mentioned values to cast the specified array to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of Param1Ptr and Param2Ptr																		
ContextImId - Possible values returned			Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	PutType																	
		Param1Ptr																
		Param2Ptr																
M_TYPE_AIL_DOUBLE			Specifies that the array is an *AIL_DOUBLE*.															
	Param1Ptr		Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr		Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT			Specifies that the array is an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr		Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr		Data type info: Data type: *array of type AIL_FLOAT [optionally, in C++: a reference to a constant std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT			Specifies that the array is an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr		Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr		Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16			Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr		Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a constant std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b

	Param2Ptr	Data type info: Data type: *array of type AIL_INT16 [optionally, in C++: a reference to a constant std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32		Specifies that the array is an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a constant std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Specifies that the array is an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1Ptr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a constant std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Param2Ptr	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a constant std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- *[Aurora Imaging Library Lite]*
  Note that some of the values listed above are not available in Aurora Imaging Library Lite. See the value's corresponding operation function for Aurora Imaging Library Lite availability.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimRank

Synopsis

Perform a rank filter on the pixels in an image.

Syntax

```
void MimRank(
    AIL_ID SrcImageBufId,   //in
    AIL_ID DstImageBufId,   //out
    AIL_ID StructElemBufId, //in
    AIL_INT Rank,           //in
    AIL_INT64 ProcMode      //in
)
```

Description

This function performs a rank filter operation on the specified source buffer. It replaces each pixel with the pixel in its neighborhood whose value is the specified *rankth* value relative to others. It uses the specified structuring element as a mask. This mask determines the neighborhood size and which pixels in the neighborhood to ignore.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

StructElemBufId

Specifies the identifier of the structuring element buffer to use as a mask. You can either define a custom structuring element as a mask or use a predefined mask buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the structuring element buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_3X3_CROSS +	Applies a cross (+) mask and sets the neighborhood size to 3x3 pixels. In this case, there are 5 valid neighborhood pixels; the cross mask makes the function ignore the pixels located at the four corners of the neighborhood.	c		e	f	g	a		i		j	k	l	m	n	b
M_3X3_RECT +	Applies no mask and sets the neighborhood size to 3x3 pixels. In this case, there are 9 valid neighborhood pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_5X5_RECT +	Applies no mask and sets the neighborhood size to 5x5 pixels. In this case, there are 25 valid neighborhood pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Structuring element buffer identifier	Applies a mask specified by the custom structuring element, having been previously allocated with MbufAlloc...() with an M_STRUCT_ELEMENT attribute. This structuring element is to be loaded with structuring-element values, using MbufPut(). The structuring element buffer dimensions are used as the neighborhood size. Values in the structuring element set to **M_DONT_CARE** represent neighborhood pixels not to be considered in the ranking operation. All other values in the structuring element must be set to 1, representing neighborhood pixels to be considered in the ranking operation.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_3X3_CROSS; M_3X3_RECT; M_5X5_RECT.

You can add one of the following values to the above-mentioned values to specify the overscan settings to use for the operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the overscan setting																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_DISABLE	Specifies to disable overscanning for this operation. You should disable overscanning to accelerate the operation for applications in which the overscan data is not important in the resulting buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Specifies to enable overscanning for this operation. The overscan uses pixels from the ancestor of the source buffer. If the source buffer is not a child buffer, or if the point falls outside the ancestor buffer, a mirror type overscan is used. A mirror overscan specifies that the overscan pixels will be a mirror copy of the source buffer's border pixels. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

Rank

Specifies which of the pixel values to select after valid neighborhood values are sorted in increasing order; valid neighborhood pixel values are those that are not masked-out.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which pixel to select																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEDIAN	Specifies that **MimRank**() performs a median filter (that is, each pixel is replaced with the median neighborhood value).	c		e	f	g	a		i		j	k	l	m	n	b
0 < *Value* <= *number of valid neighborhood pixels*	Specifies the rank of the pixel value to select. If the number of valid pixels is less than the given rank, the number of valid pixels is used as the rank.	c		e	f	g	a		i		j	k	l	m	n	b

ProcMode

Specifies the processing mode to use. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Treats non-zero pixels as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Uses the source image's gray values for processing. The resulting buffer will also contain gray values. Performance between this and M_GRAYSCALE_BIG_STRUCT_ELEMENT should be analyzed on your own applications.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE_BIG_STRUCT_ELEMENT	Uses the source image's gray values for processing. The resulting buffer will also contain gray values. This processing mode is specially designed for large structuring elements. The source image must be an 8-bit, unsigned buffer. Performance between this and M_GRAYSCALE should be analyzed on your own applications.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimRearrange

Synopsis

Copies specified areas of a source buffer to the specified areas of a destination buffer.

Syntax

```
void MimRearrange(
    AIL_ID RearrangeContextImId, //in
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageBufId,        //out
    AIL_INT64 ControlFlag        //in
)
```

Description

This function copies specified areas of a source buffer to specified areas of a destination buffer. The rest of the source image buffer is ignored and not copied.

Before using this function, you must configure the specified image processing context with which operation to perform, using MimControl() with M_MODE. This specifies whether to copy rectangular areas or rows of the source buffer. In addition, you must specify the offsets and sizes of the source and destination areas, using MimPut() with M_XY_SOURCE, M_XY_DESTINATION, and M_XY_SIZE. Note that each array in the rearrangement image processing context must have the same number of elements. The source and destination image buffers need not be of the same size, but must be equal to or larger than the largest supplied source and destination coordinates.

Once the rearrangement image processing context is configured, preprocess the context by calling **MimRearrange**() with M_PREPROCESS. If the preprocess operation is not done explicitly (using M_PREPROCESS), it will be done when **MimRearrange**() is first called. When preprocessing explicitly, the source and destination image buffers can be set to **M_NULL**. If, however, the source or destination image buffer is provided, it will be used in the preprocess operation, to better optimize future calls.

Parameters

RearrangeContextImId

Specifies the identifier of the rearrangement image processing context that indicates how to perform the operation and the locations to copy. This context should be previously allocated using MimAlloc() with M_REARRANGE_CONTEXT.

SrcImageBufId

Specifies the identifier of the source image buffer.

If preprocessing the rearrangement image processing context (using M_PREPROCESS), set this parameter to the identifier of a typical source image buffer. Otherwise, to exclude this buffer from preprocessing, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

If preprocessing the rearrangement image processing context (using M_PREPROCESS), set this parameter to the identifier of a typical destination image buffer. Otherwise, to exclude this buffer from preprocessing, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ControlFlag

Specifies the operation to perform. This parameter must be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Performs the rearrangement operation.	c		e	f	g	a		i		j	k	l	m	n	b

M_PREPROCESS	Preprocesses the specified image processing context. Note that, if not called explicitly, this operation will be performed automatically upon the first call to **MimRearrange**(). If the identifier of the source and/or destination image buffer is not set to **M_NULL**, information about the source and/or destination image buffer will be used in the preprocessing operation.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimRemap

Synopsis

Remap source pixels to destination pixels according to the specified mode.

Syntax

```
void MimRemap(
    AIL_ID RemapContextImId, //in
    AIL_ID SrcImageBufId,    //in
    AIL_ID DstImageBufId,    //out
    AIL_INT64 ControlFlag    //in
)
```

Description

This function performs window leveling by linearly remapping source pixels to destination pixels according to the specified mode. By default, Aurora Imaging Library performs remapping calculations using the possible minimum and maximum values of the full value range of the specified destination image buffer. You can also use a custom remap image processing context and specify different values.

When using a custom remap image processing context, you can choose the range of source pixel values that are remapped, using MimControl() with M_SRC_START_VALUE and M_SRC_END_VALUE, as well as the range of destination pixel values to which they are remapped using M_DST_START_VALUE and M_DST_END_VALUE.

By default, Aurora Imaging Library determines the remapping for the destination image buffer. However, when not using a remap image processing context, you can specify to remap the values such that the value zero in the source buffer maps to the value zero in the destination buffer with M_CENTERED. Note that saturation can occur when M_CENTERED is specified.

Parameters

RemapContextImId

Specifies the identifier of a remap image processing context. If a custom remap image processing context is not used, this parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the remap context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies a linear remapping from source pixel to corresponding destination pixel. Note, Aurora Imaging Library performs remapping calculations using the possible minimum and maximum values of the specified destination image buffer. By default, this is the full value range according to the buffer type. To control these values, use MbufControl() with M_MIN and M_MAX. For the source image buffer, the minimum and maximum values used for calculations depend on the specified remapping mode (ControlFlag).	c		e	f	g	a		i		j	k	l	m	n	b
Remap image processing context ID	Specifies the identifier of a remap image processing context, previously allocated using MimAlloc() with M_IM_REMAP_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageBufId

Specifies the identifier of the source image buffer.

If the image buffer has a region of interest (ROI) associated with it, the ROI must be in M_RASTER or M_VECTOR_AND_RASTER format. Using an image buffer with an M_VECTOR ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination image buffer.

If the image buffer has a region of interest (ROI) associated with it, the ROI must be in M_RASTER or M_VECTOR_AND_RASTER format. Using an image buffer with an M_VECTOR ROI will cause an error.

ControlFlag

Specifies the remapping mode. When using a custom remap image processing context, this parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the remapping method to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. When using the default remap image processing context, the default remapping mode is M_FIT_SRC_RANGE. When using a custom remap image processing context, this is the only possible setting. Use MimControl() with M_SRC_RANGE to specify how the source range is established.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_SRC_DATA +	Specifies to remap according to the source data. The true minimum and maximum pixel values will be calculated from the source image buffer using MimFindExtreme().	c		e	f	g	a		i		j	k	l	m	n	b
M_FIT_SRC_RANGE +	Specifies to remap according to the minimum and maximum pixel values specified for the source image buffer (MbufControl() with M_MIN and M_MAX). By default, this is the full value range according to the buffer type. If the source buffer has data below the specified M_MIN or above the specified M_MAX, this data will be mapped to unpredictable values. If the goal is to clip this data to the M_MIN and M_MAX values of the destination buffer, use MimClip() with M_OUT_RANGE on the source buffer first to set the data outside the range to the specified source M_MIN and M_MAX values.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_FIT_SRC_DATA; M_FIT_SRC_RANGE.

You can add the following value to the above-mentioned values to specify to keep the range centered to zero.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the remapping centered to zero																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CENTERED	Specifies to center the range to zero.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimResize

Synopsis

Resize an image by the specified X- and Y-scale factors or generate the matrix to perform the transformation.

Syntax

```
void MimResize(
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageOrArrayBufId, //out
    AIL_DOUBLE ScaleFactorX,     //in
    AIL_DOUBLE ScaleFactorY,     //in
    AIL_INT64 InterpolationMode  //in
)
```

Description

This function resizes the source image by the specified factors and stores results in the destination buffer starting from the top-left corner. Alternatively, this function can generate the 3x3 or 3x2 matrix required to perform the transformation using MimWarp().

Parameters

SrcImageBufId

Specifies the source image to resize if performing the transformation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source image buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter; this parameter must be set to M_NULL when generating the matrix that you can use with MimWarp() to perform the resizing.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the source image buffer to resize. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageOrArrayBufId

Specifies the destination image buffer or array buffer, depending on whether performing the transformation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Array buffer identifier	Specifies the identifier of the destination M_ARRAY buffer when generating the matrix. When generating a warp matrix, this parameter must be given a single-band, 32-bit floating-point buffer that has an M_ARRAY attribute and that has dimensions 3x2, for a first-order polynomial warping, or 3x3, for a perspective polynomial warping. If you pass this parameter a buffer with dimensions 3x2, MimWarp() will assume the third row to be (0, 0, 1) when performing the warping.	c		e	f	g	a		i		j	k	l	m	n	b

Image buffer identifier	Specifies the identifier of the destination image buffer when resizing the source image. This buffer must not have a region of interest (ROI) associated with it. Using a buffer with an ROI will cause an error. > Note that after performing the operation, the destination image will not be calibrated, even if the source image was calibrated.	c		e	f	g	a		i		j	k	l	m	n	b

ScaleFactorX

Specifies the horizontal scaling factor. This parameter can be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the scaling factor (width)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILL_DESTINATION	Specifies to use a scaling factor that resizes the source image to fill the entire width of the destination buffer. When set to M_FILL_DESTINATION, the source image is resized to fill the entire width of the destination buffer. You can only use this value when resizing an image. In this case, you must pass SrcImageBufId a source image buffer identifier, and not M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the horizontal scaling factor. A value greater than 1.0 enlarges the source image, while a factor less than 1.0 reduces it. You must specify a specific value for the horizontal scaling factor when only generating the matrix to perform the transformation.	c		e	f	g	a		i		j	k	l	m	n	b

ScaleFactorY

Specifies the vertical scaling factor. This parameter can be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the scaling factor (height)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILL_DESTINATION	Specifies to use a scaling factor that resizes the source image to fill the entire height of the destination buffer. When set to M_FILL_DESTINATION, the source image is resized to fill the entire height of the destination buffer. You can only use this value when resizing an image. In this case, you must pass SrcImageBufId a source image buffer identifier, and not M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the vertical scaling factor. A value greater than 1.0 enlarges the source image, while a factor less than 1.0 reduces it. You must specify a specific value for the vertical scaling factor when only generating the matrix to perform the transformation.	c		e	f	g	a		i		j	k	l	m	n	b

InterpolationMode

Specifies the interpolation mode when resizing the source image. When only generating the matrix, this parameter is ignored and should be set to M_DEFAULT.

This parameter can be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the mode of interpolation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies the default value. When resizing the source image, the default is the same as M_NEAREST_NEIGHBOR + M_OVERSCAN_ENABLE. When only generating the coefficients matrix, this is the only possible value.	c		e	f	g	a		i		j	k	l	m	n	b
M_AVERAGE +	Specifies averaging interpolation. The scaling factors for the width and height of the source image must be less than or equal to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC +	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR +	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERPOLATE +	Specifies interpolated resizing. For resizing up, this is equivalent to bilinear; for resizing down, this is equivalent to averaging. This gives the best speed/result compromise for interpolated resizing.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX +	Specifies an interpolation based on the maximum pixel value in the specified source image. The scaling factors for the width and height of the source image must be less than or equal to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN +	Specifies an interpolation based on the minimum pixel value in the specified source image. The scaling factors for the width and height of the source image must be less than or equal to 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR +	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the InterpolationMode parameter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For overscan																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_CLEAR	Sets the destination pixel to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Leaves the destination pixel as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Uses pixels from the source buffer's ancestor buffer. If the source buffer is not a child buffer or if the point falls outside the ancestor buffer, it leaves the destination pixel as is. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Points falling outside the source image are undefined.										j	k	l	m		

M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimRestore

Synopsis

Restore an image processing context from disk.

Syntax

```
AIL_ID MimRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextImIdPtr       //out
)
```

Description

This function restores an image processing context that was previously saved to a file using MimSave() or MimStream(). This function restores all of the image processing context's settings that were in effect when the image processing context was saved.

> Note that this function allocates the image processing context for you. To load an image processing context into an existing image processing context, allocate the context using MimAlloc() and then use MimStream() with M_LOAD.

When the restored image processing context is no longer required, release it using MimFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored image processing context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the image processing context. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, image processing context files have an MIC extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the image processing context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextImIdPtr

Specifies the address of the variable in which to write the image processing context identifier or specifies the data type that the function should use to return the restored image processing context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated image processing context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated image processing context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_IM_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the image processing context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored image processing context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the restored image processing context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_IM_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimRotate

Synopsis

Rotate an image by a specified angle of rotation or generate the matrix to perform the transformation.

Syntax

```
void MimRotate(
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageOrArrayBufId, //out
    AIL_DOUBLE Angle,            //in
    AIL_DOUBLE SrcCenX,          //in
    AIL_DOUBLE SrcCenY,          //in
    AIL_DOUBLE DstCenX,          //in
    AIL_DOUBLE DstCenY,          //in
    AIL_INT64 InterpolationMode  //in
)
```

Description

This function rotates an image by the specified angle of rotation. Alternatively, this function can generate the 3x3 or 3x2 warp matrix required to perform the transformation using MimWarp(). If you specify a source image, it will be rotated using the specified interpolation mode. The center of rotation in the source image is determined by the specified X- and Y-source rotation-center coordinates. The rotated image will then be clipped to fit the destination buffer. It will be placed in the destination buffer with its center positioned at the specified X- and Y-destination center coordinates.

Parameters

SrcImageBufId

Specifies the source image to rotate if performing the transformation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source image buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter; this parameter must be set to M_NULL when generating the matrix that you can use with MimWarp() to perform the rotation.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the source image buffer to rotate. When rotating an image, this parameter must be given an image buffer identifier. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageOrArrayBufId

Specifies the destination image buffer or array buffer, depending on whether performing the transformation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

Array buffer identifier	Specifies the identifier of the destination M_ARRAY buffer when generating the matrix. When generating a warp matrix, this parameter must be given a single-band, 32-bit floating-point buffer that has an M_ARRAY attribute and that has dimensions 3x2, for a first-order polynomial warping, or 3x3, for a perspective polynomial warping. If you pass this parameter a buffer with dimensions 3x2, MimWarp() will assume the third row to be (0, 0, 1) when performing the warping.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the destination image buffer when rotating the source image. When rotating an image, this parameter must be given an image buffer identifier. This buffer must not have a region of interest (ROI) associated with it. Using a buffer with an ROI will cause an error. > Note that after performing the operation, the destination image will not be calibrated, even if the source image was calibrated.	c		e	f	g	a		i		j	k	l	m	n	b

Angle

Specifies the angle of rotation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the rotation angle																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
-360.0 <= *Value* <= 360.0	Specifies the angle of rotation, in degrees. When a positive angle is specified, the function rotates the image in a counter-clockwise direction, regardless of camera calibration settings.	c		e	f	g	a		i		j	k	l	m	n	b

SrcCenX

Specifies the X-coordinate to use as the center of rotation in the source image. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-coordinate (source)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Causes the image to rotate about its true center. When specifying M_DEFAULT as a center of rotation, it is computed as (*source buffer size* - 1) / 2. You can only use this value when rotating an image. In this case, you must pass SrcImageBufId a source image buffer identifier, and not M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate in the pixel coordinate system. You must specify a specific value for the X coordinate of the source buffer center when only generating the warp matrix to perform the transformation.	c		e	f	g	a		i		j	k	l	m	n	b

SrcCenY

Specifies the Y-coordinate to use as the center of rotation in the source image. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-coordinate (source)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Causes the image to rotate about its true center. When specifying M_DEFAULT as a center of rotation, it is computed as (*source buffer size* - 1) / 2. You can only use this value when rotating an image. In this case, you must pass SrcImageBufId a source image buffer identifier, and not M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate in the pixel coordinate system. You must specify a specific value for the Y coordinate of the source buffer center when only generating the warp matrix to perform the transformation.	c		e	f	g	a		i		j	k	l	m	n	b

DstCenX

Specifies the X-coordinate in the destination buffer to which the specified center of the rotated source image will be mapped. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the X-coordinate (destination)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Causes the true center of the destination buffer to be used. When specifying M_DEFAULT as a center of rotation, it is computed as (*destination buffer size* - 1) / 2. You can only use this value when rotating an image. In this case, you must pass SrcImageBufId a source image buffer identifier, and not M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate in the pixel coordinate system. You must specify a specific value for the X coordinate of the destination buffer center when only generating the warp matrix to perform the transformation.	c		e	f	g	a		i		j	k	l	m	n	b

DstCenY

Specifies the Y-coordinate in the destination buffer to which the specified center of the rotated source image will be mapped. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Y-coordinate (destination)																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Causes the true center of the destination buffer to be used. When specifying M_DEFAULT as a center of rotation, it is computed as (*destination buffer size* - 1) / 2. You can only use this value when rotating an image. In this case, you must pass SrcImageBufId a source image buffer, and not M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate in the pixel coordinate system. You must specify a specific value for the Y coordinate of the destination buffer center when only generating the warp matrix to perform the transformation.	c		e	f	g	a		i		j	k	l	m	n	b

InterpolationMode

Specifies the interpolation mode when rotating the source image. When only generating the matrix, this parameter is ignored and should be set to M_DEFAULT.

This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the interpolation mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies the default value. When rotating the source image, the default is the same as M_NEAREST_NEIGHBOR + M_OVERSCAN_ENABLE. When only generating the coefficients matrix, this is the only possible value.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC +	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR +	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR +	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the InterpolationMode parameter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For overscan																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_CLEAR	Sets the destination pixel to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Leaves the destination pixel as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Uses pixels from the source buffer's ancestor buffer. If the source buffer is not a child buffer or if the point falls outside the ancestor buffer, it leaves the destination pixel as is. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Points falling outside the source image are undefined.										j	k	l	m		
M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.

DLL	Requires ail11.dll; ailim11.dll.

MimSave

Synopsis

Save an image processing context to a file.

Syntax

```
void MimSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextImId,          //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves an image processing context to disk.

To load the saved image processing context, use either MimRestore() or MimStream().

Parameters

FileName

Specifies the name and path of the file in which to save the image processing context; it is recommended that you use the MIC file extension for easier use with other Zebra Aurora software products. The function internally handles the opening and closing of this file. If this file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, image processing context files have an MIC extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextImId

Specifies the identifier of the image processing context to save. The image processing context must have been previously allocated using MimAlloc().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimShift

Synopsis

Perform a point-to-point bit shift.

Syntax

```
void MimShift(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbBitsToShift //in
)
```

Description

This function performs left or right bit-shifting on each pixel in the specified image. The shift operation is signed or unsigned depending on the source image buffer's data type.

Note that floating-point values will be cast to *AIL_UINT32* before performing the shift operation (except when shifting by 0). Therefore, unexpected results can occur if a floating-point value is larger than the *AIL_UINT32* range.

Note that, if you shift by 0, only a copy operation will be performed.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DstImageBufId

Specifies the identifier of the destination of the results. This parameter must be given an image buffer identifier.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

NbBitsToShift

Specifies the number of bits to shift. If the given value is negative, each pixel in the specified image is right bit-shifted by the specified number of bits; otherwise, it is left bit-shifted.

Remark

- Optimized in-place processing is supported (source equals destination), but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimStatCalculate

Synopsis

Calculates and stores a variety of statistics from the specified source image(s).

Syntax

```
void MimStatCalculate(
    AIL_ID StatContextImId, //in
    AIL_ID SrcImageBufId,   //in
    AIL_ID StatResultImId,  //out
    AIL_INT64 ControlFlag   //in
)
```

Description

This function calculates a variety of statistics for the pixels in the source image(s). Results are stored in the specified statistics result buffer.

The specified image processing context establishes which statistics to calculate. This function can take statistics for the specified image (M_STATISTICS_CONTEXT context) or it can accumulate statistics for each pixel across multiple images (M_STATISTICS_CUMULATIVE_CONTEXT context). For an M_STATISTICS_CONTEXT context, you can specify a condition that a pixel must satisfy to be included in the calculation using M_CONDITION. Co-occurrence statistics (M_STAT_GLCM...) ignore any specified condition. If you need to calculate one type of single-image statistic and don't need to specify a condition, you can use a predefined context; otherwise, use MimAlloc() to set up a custom context that you can control using MimControl().

You can read the statistics from the result buffer using MimGetResult() or MimGetResult2d(), depending on the statistical operation performed. You can also draw some statistical results, using MimDraw().

Once the statistics image processing context is configured, preprocess the context and result buffer by calling **MimStatCalculate**() with M_PREPROCESS. If the preprocess operation is not done explicitly (using M_PREPROCESS), it will be done when **MimStatCalculate**() is first called. When explicitly preprocessing a cumulative statistics image processing context and result buffer, the source image buffer can be set to **M_NULL** if you set the size that the source buffer will be, using MimControl() with M_SOURCE_SIZE_X and M_SOURCE_SIZE_Y. Note that providing the source image buffer better optimizes future calls.

When working with an M_STATISTICS_CUMULATIVE_CONTEXT context, you can remove the calculated statistics associated with a specified source image from the result buffer by calling **MimStatCalculate**() with M_REMOVE. To reset the results generated from finding the extremes of each pixel's value (such as, M_STAT_MIN), use M_RESET_EXTREMES.

To reset all the results in the result buffer, free it using MimFree(), and then allocate a new result buffer using MimAllocResult() with M_STATISTICS_RESULT. These actions apply to either context (M_STATISTICS_CONTEXT or M_STATISTICS_CUMULATIVE_CONTEXT).

Note that when working with an M_STATISTICS_CUMULATIVE_CONTEXT context, your source images should have the same dimensions, so that statistics are calculated for the same pixel across the multiple images.

For an M_STATISTICS_CONTEXT, you can limit some of this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). Not all statistical operations support ROIs; for a list of those that do, see MimControl(). Note that the M_STATISTICS_CUMULATIVE_CONTEXT context does not support ROIs.

Parameters

StatContextImId

Specifies a statistics context, which determines the statistics to calculate. You can select the identifier of a previously allocated statistics context, or you can select a predefined statistics context to evaluate one type of single-image statistical calculation. The predefined statistics contexts do not need to be allocated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the statistics context, or specific statistic to calculate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_STAT_CONTEXT_MAX	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_MAX, which is set to M_ENABLE. Use this predefined context to calculate the maximum pixel value of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MAX_ABS	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_MAX_ABS, which is set to M_ENABLE. Use this predefined context to calculate the maximum absolute pixel value of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MEAN	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_MEAN, which is set to M_ENABLE. Use this predefined context to calculate the mean pixel value of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MIN	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_MIN, which is set to M_ENABLE. Use this predefined context to calculate the minimum pixel value of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_MIN_ABS	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_MIN_ABS, which is set to M_ENABLE. Use this predefined context to calculate the minimum absolute pixel value of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_NUMBER	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_NUMBER, which is set to M_ENABLE. Use this predefined context to calculate the number of valid pixels in the source image. This is mainly useful in the presence of a region.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_STANDARD_DEVIATION	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_STANDARD_DEVIATION, which is set to M_ENABLE. Use this predefined context to calculate the standard deviation of the pixel values of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_SUM	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_SUM, which is set to M_ENABLE. Use this predefined context to calculate the sum of the pixel values of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STAT_CONTEXT_SUM_ABS	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_SUM_ABS, which is set to M_ENABLE. Use this predefined context to calculate the sum of the absolute pixel values of the source image.	c		e	f	g	a		i		j	k	l	m	n	b

M_STAT_CONTEXT_SUM_OF_SQUARES	Specifies a predefined single-image statistics image processing context with all control types set to their default except for M_STAT_SUM_OF_SQUARES, which is set to M_ENABLE. Use this predefined context to calculate the sum of the squared pixel values of the source image.	c		e	f	g	a		i		j	k	l	m	n	b
Statistics context identifier	Specifies a statistics context identifier, previously allocated using MimAlloc() with M_STATISTICS_CONTEXT or M_STATISTICS_CUMULATIVE_CONTEXT. You can specify which statistics calculations are evaluated using MimControl().	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer used must be a 1-band image buffer.

When preprocessing a cumulative statistics image processing context and result buffer, set this parameter to the identifier of a typical source image buffer. To exclude this buffer from preprocessing, set this parameter to **M_NULL** and set the height and width of the area in the source buffer for which you need statistics, using MimControl() with M_SOURCE_SIZE_X and M_SOURCE_SIZE_Y; otherwise, an error will occur.

To process multiple images when performing a cumulative statistics operation, call **MimStatCalculate**() multiple times, each time with a different source image. Note that the source image must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI for a cumulative statistics operation will cause an error.

An image buffer specified for a single-image statistics operation can have an ROI associated with it. The ROI must be in raster format; otherwise, you will get an error. Note that not all statistics operations support ROIs; for a list of those that do, see MimControl().

StatResultImId

Specifies the identifier of the result buffer. The buffer must have been allocated using MimAllocResult() with M_STATISTICS_RESULT.

ControlFlag

Specifies the operation to perform on the source image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To specify the operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Calculates the statistics specified using MimControl(), or in a predefined statistics context. This is the only valid option for a predefined statistics context.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Preprocesses the specified image processing context and image processing result buffer. Note that, if not called explicitly, this operation will be performed automatically upon the first call to **MimStatCalculate**(). If the identifier of the source image buffer is not set to **M_NULL**, information about the source image buffer will be used in the preprocessing operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMOVE	Removes the results related to the specified source image. This removes the results of the calculations, performed on the specified source image, from the accumulated results in the result buffer. To reset the minimum and maximum statistical results, use M_RESET_EXTREMES. The minimum and maximum statistical results cannot be removed. > Note that this value only works when dealing with a cumulative statistics context (M_STATISTICS_CUMULATIVE_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b

M_RESET_EXTREMES	Resets the results generated from M_STAT_MIN, M_STAT_MIN_ABS, M_STAT_MAX, and M_STAT_MAX_ABS operations. Note that, when resetting the minimum and maximum results, the SrcImageBufId parameter is ignored. > Note that this value only works when dealing with a cumulative statistics context (M_STATISTICS_CUMULATIVE_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimStream

Synopsis

Load, restore, or save an image processing context from/to a file or a memory stream, or save an augmentation report to a file.

Syntax

```
void MimStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextImIdPtr,        //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a image processing context from/to a file or memory stream, or save an augmentation report to a file.

To inquire the number of bytes necessary to save an image processing context to memory stream, you should first call this function (**MimStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. This function is equivalent to a file saved using MimSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. When saving an image processing context, all information about the previously allocated image processing context is saved. The content saved to memory stream is equivalent to the content saved to file.

When loading or restoring an image processing context, all of the image processing context's settings that were in effect when the image processing context was saved will be restored. Note that, you can also restore an image processing context using MimRestore().

Using **MimStream**(), you can choose to save a backwards-compatible version of the image processing context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore image processing contexts saved using Aurora Imaging Library version 9.0 or above. Settings that do not exist in the lower version will be filled with default values when the image processing context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the image processing context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the image processing context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the image processing context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the image processing context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextImIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the image processing context. In the case of an M_SAVE_REPORT operation, this parameter specifies the address of the variable from which to read the identifier of the augmentation result buffer.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the image processing context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of an image processing context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextImIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextImIdPtr - Possible values passed or returned																

M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save an image processing context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.														
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the image processing context must be compatible when saved. The size of an image processing context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextImIdPtr	Specifies the address of the variable containing the identifier of the image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Image processing contexts typically have an MIC file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextImIdPtr	Specifies the address of the variable containing the identifier of the image processing context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores an image processing context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Image processing contexts typically have an MIC file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the image processing context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b

ContextImIdPtr	Specifies the address of the variable in which to write the identifier of the image processing context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves an image processing context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Zebra Aurora software products, when saving a image processing context to a file, use the MIC file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the image processing context must be compatible when saved. Note that the image processing context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b

ContextImIdPtr	Specifies the address of the variable containing the identifier of the image processing context to save.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_REPORT	Saves a report containing results from calling MimAugment() with an augmentation result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\MyReportFile.txt*"), when the StreamType parameter is set to M_FILE. You should use a TXT file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyReportFile.txt*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextImIdPtr	Specifies the address of the variable containing the identifier of the augmentation result buffer to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimThick

Synopsis

Perform a binary or grayscale thickening operation on an image.

Syntax

```
void MimThick(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbIteration,  //in
    AIL_INT64 ProcMode    //in
)
```

Description

This function performs a binary or grayscale thickening on the specified source image for the specified number of iterations.

The overscan pixels are automatically set to the lowest possible buffer value, which will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

NbIteration

Specifies the number of times to iterate the operation. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of times to iterate the operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TO_IDEMPOTENCE	Specifies the operation will iterate until idempotence is reached.	c		e	f	g	a		i		j	k	l	m	n	b
Value > = 0	Specifies the number of iterations. If this parameter is set to 0, and ProcMode is set to M_BINARY, the source image is binarized and the result is copied into the destination image buffer. If this parameter is set to 0, and ProcMode is set to M_GRAYSCALE, the source image is copied into the destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ProcMode

Specifies the processing mode to use. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BINARY	Treats non-zero pixels as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b

M_GRAYSCALE	Uses the source image's gray values for processing. The resulting pixels will be grayscale.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimThin

Synopsis

Perform a binary or grayscale thinning operation on an image.

Syntax

```
void MimThin(
    AIL_ID SrcImageBufId, //in
    AIL_ID DstImageBufId, //out
    AIL_INT NbIteration,  //in
    AIL_INT64 ProcMode    //in
)
```

Description

This function performs a binary or grayscale thinning on the specified source image for the specified number of iterations.

The overscan pixels are automatically set to the highest possible buffer value, which will produce the most accurate possible results for the image border pixels.

Parameters

SrcImageBufId

Specifies the identifier of the data source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

NbIteration

Specifies the number of times to iterate the operation. This parameter must be set to one of the values below.

> Note that when ProcMode is set to M_BINARY3, NbIteration must be set to 0 or M_TO_SKELETON.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of times to iterate the operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TO_SKELETON	Specifies that every object will be reduced to its skeleton.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of iterations. If this parameter is set to 0, and ProcMode is set to M_BINARY, the source image is binarized and the result is copied into the destination image buffer. If this parameter is set to 0, and ProcMode is set to M_GRAYSCALE, the source image is copied into the destination image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

ProcMode

Specifies the processing mode to use. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the processing mode to use																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_BINARY	Treats non-zero pixels as ones (1) during processing. The resulting non-zero pixels will have all bits set to one.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY2	Treats non-zero pixels as ones (1) during processing and produces a result that better preserves the original topology of the objects in the source image. Similar to M_BINARY, the resulting non-zero pixels will also have all bits set to one. However, this mode produces a result that is more consistent with the one produced using the M_GRAYSCALE processing mode. Note that this processing mode is slower than using the M_BINARY or M_BINARY3 processing modes.	c		e	f	g	a		i		j	k	l	m	n	b
M_BINARY3	Treats non-zero pixels as ones (1) during processing and produces a faster result than other processing modes. Similar to M_BINARY, the resulting non-zero pixels will also have all bits set to one. However, this mode produces a result that is more consistent with the one produced using the M_GRAYSCALE processing mode. Note that this processing mode does not preserve the original topology of objects in the source image as well as M_BINARY2. Overscan pixels are treated as zeroes instead of ones.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAYSCALE	Uses the source image's gray values for processing. The resulting pixels will be grayscale.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimTransform

Synopsis

Perform a Fast Fourier transform (FFT), a Discrete Cosine transform (DCT), or a polar-cartesian coordinates transformation.

Syntax

```
void MimTransform(
    AIL_ID SrcImageRBufId,   //in
    AIL_ID SrcImageIBufId,   //in
    AIL_ID DstImageRBufId,   //out
    AIL_ID DstImageIBufId,   //out
    AIL_INT64 TransformType, //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function performs a forward or reverse FFT or DCT transformation on an image. It also perfoms a cartesian-to-polar or polar-to-cartesian point-to-point coordinates transformation.

Parameters

SrcImageRBufId

Specifies the identifier of the source buffer for the real component of the image.

See the Parameter associations section for possible values that can be specified.

SrcImageIBufId

Specifies the identifier of the source buffer for the imaginary component of the image.

This parameter can be set to **M_NULL**. See the parameter association table for more details.

See the Parameter associations section for possible values that can be specified.

DstImageRBufId

Specifies the identifier of the destination buffer for the real component of the image.

This parameter can be set to **M_NULL**. See the parameter association table for more details.

See the Parameter associations section for possible values that can be returned.

DstImageIBufId

Specifies the identifier of the destination buffer for the imaginary component of the image.

This parameter can be set to **M_NULL**. See the parameter association table for more details.

See the Parameter associations section for possible values that can be returned.

TransformType

Specifies the type of transform to perform on the image.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies if the transform is a forward transform or a reverse transform.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **SrcImageRBufId**, **SrcImageIBufId**, **TransformType**, and **ControlFlag** parameters and possible values returned to the **DstImageRBufId** and **DstImageIBufId** parameters.

- For selecting the transformation type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For selecting the transformation type.																
TransformType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlFlag																
SrcImageRBufId																
SrcImageIBufId																
DstImageRBufId - Possible values returned																
DstImageIBufId - Possible values returned																
M_DCT8X8	Performs a discrete cosine transform on each 8x8 pixel block in the image.															
M_DEFAULT	Same as M_FORWARD.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORWARD +	Performs a forward transform on the image.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageRBufId	Specifies the identifier of the source buffer for the real component of the image in the spatial domain. The width and height of the source buffer must be a multiple of 8. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
8-bit signed/unsigned image buffer ID	Performs calculations in fixed-point format and returns to the destinations in 23.9 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
16-bit signed/unsigned image buffer ID	Performs calculations in fixed-point format and returns to the destinations in 25.7 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
32-bit signed image buffer ID	Conserves the fixed point format between the destination buffer and the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Floating-point image buffer ID	Specifies that processing will be performed in floating-point arithmetic. The destination buffer should also be a floating-point buffer.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageIBufId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageRBufId	Specifies the identifier of the destination buffer for the real component of the image in the frequency domain. If SrcImageRBufId is an integer buffer, this must be a 32-bit signed integer buffer. If SrcImageRBufId is a floating-point buffer, this must also be a floating-point buffer. The width and height of the destination buffer should be a multiple of 8. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageIBufId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE +	Performs a reverse transform on the image.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageRBufId	Specifies the identifier of the source buffer for the real component of the image in the frequency domain. This must be a 32-bit integer buffer or a floating-point buffer. The width and height of the source buffer must be a multiple of 8. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageIBufId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageRBufId	Specifies the identifier of the destination buffer for the real component of the image in the spatial domain. The width and height of the destination buffer should be a multiple of 8. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
8-bit signed/unsigned image buffer ID	The format of the source buffer is assumed to be in 23.9 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
16-bit signed/unsigned image buffer ID	The format of the source buffer is assumed to be in 25.7 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
32-bit signed image buffer ID	The fixed point format of the destination buffer is the same as that of the source.	c		e	f	g	a		i		j	k	l	m	n	b
Floating-point image buffer ID	If SrcImageRBufId is a floating-point buffer, this must also be a floating-point buffer and processing will be performed in floating-point arithmetic.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageIBufId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_FFT	Performs a Fast Fourier transform.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FORWARD.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORWARD +	Performs a forward transform on the image. For floating-point destination buffers, the processing will be performed in floating-point arithmetic.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageRBufId	Specifies the identifier of the source buffer for the real component of the image in the spatial domain. The width and height of the source buffer must be a power of 2. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

			8-bit signed/unsigned image buffer ID	Calculations are performed in fixed-point format and returned to the destinations in 23.9 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
			16-bit signed/unsigned image buffer ID	Calculations are performed in fixed-point format and returned to the destinations in 25.7 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
			32-bit signed image buffer ID	The fixed point format of the destination buffer will be the same as that of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
			Floating-point image buffer ID	The destination buffer should also be a floating-point buffer and processing will be performed in floating-point arithmetic.	c		e	f	g	a		i		j	k	l	m	n	b
		SrcImageIBufId		Specifies the identifier of the source buffer for the imaginary component of the image in the spatial domain. This buffer must be of the same type and pixel depth, and have the same dimensions and limitations as the buffer specified for SrcImageRBufId.	c		e	f	g	a		i		j	k	l	m	n	b
			M_NULL	Specifies that a faster version of the forward transform will be performed.	c		e	f	g	a		i		j	k	l	m	n	b
			Image buffer ID	Specifies the identifier of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
		DstImageRBufId		Specifies the identifier of the destination buffer for the real component of the image in the frequency domain. If SrcImageRBufId is an integer buffer, this can be a 32-bit signed integer buffer or a floating-point buffer. If SrcImageRBufId is a floating-point buffer, this must be a floating-point buffer also. The width and height of the destination buffer must be a power of 2. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
		DstImageIBufId		Specifies the identifier of the destination buffer for the imaginary component of the image in the frequency domain. This buffer must be of the same type and pixel depth, and have the same dimensions and limitations as the buffer specified for DstImageRBufId.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REVERSE +			Performs a reverse transform on the image.	c		e	f	g	a		i		j	k	l	m	n	b
		SrcImageRBufId		Specifies the identifier of the source buffer for the real component of the image in the frequency domain. This must be a 32-bit integer buffer or a floating-point buffer. The source buffer must be a 32-bit signed integer or a floating-point buffer. The width and height of the source buffer must be a power of 2. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageIBufId	Specifies the identifier of the source buffer for the imaginary component of the image in the frequency domain. This buffer must be of the same type and pixel depth, and have the same dimensions and limitations as the buffer specified for SrcImageRBufId.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageRBufId	Specifies the identifier of the destination buffer for the real component of the image in the spatial domain. The width and height of the destination buffer must be a power of 2. This image must not have a region of interest (ROI) associated with it. Using an image with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
8-bit signed/unsigned image buffer ID	The format of the source buffer is assumed to be in 23.9 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
16-bit signed/unsigned image buffer ID	The format of the source buffer is assumed to be in 25.7 fixed point format.	c		e	f	g	a		i		j	k	l	m	n	b
32-bit signed image buffer ID	The fixed point format of the destination buffer is the same as that of the source.	c		e	f	g	a		i		j	k	l	m	n	b
Floating-point image buffer ID	If SrcImageRBufId is a floating-point buffer, this must also be a floating-point buffer and processing will be performed in floating-point arithmetic.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageIBufId	Specifies the identifier of the destination buffer for the imaginary component of the image in the spatial domain. This buffer must be of the same type and pixel depth, and have the same dimensions and limitations as the buffer specified for DstImageRBufId.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that a faster version of the reverse transform will be performed.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer ID	Specifies the identifier of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_POLAR	Performs a cartesian-to-polar or a polar-to-cartesian transform on the coordinates of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FORWARD.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORWARD +	Converts the real and imaginary components of cartesian coordinates to the magnitude and phase components of polar coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageRBufId	Specifies the identifier of the source buffer for the real component of the cartesian coordinates. It must be a 32-bit floating-point buffer.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageIBufId	Specifies the identifier of the source buffer for the imaginary component of the cartesian coordinates. It must be provided.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageRBufId	Specifies the identifier of the destination buffer for the magnitude component of the polar coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that a faster version of the forward transform will be performed. > Only one of the destination buffers can be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer ID	Specifies the identifier of the destination buffer. It must be a 32-bit floating-point buffer.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageIBufId	Specifies the identifier of the destination buffer for the phase component of the polar coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that a faster version of the forward transform will be performed. > Only one of the destination buffers can be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer ID	Specifies the identifier of the destination buffer. The phase results are comprised in the range of 0-360.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE +	Converts the magnitude and phase components of polar coordinates to the real and imaginary components of cartesian coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageRBufId	Specifies the identifier of the source buffer for the magnitude component of the polar coordinates. It must be a 32-bit floating-point buffer.	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageIBufId	Specifies the identifier of the source buffer for the phase component of the polar coordaintes. It must be provided.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageRBufId	Specifies the identifier of the destination buffer for the real component of the cartesian coordinates. It must be a 32-bit floating-point buffers.	c		e	f	g	a		i		j	k	l	m	n	b
DstImageIBufId	Specifies the identifier of the destination buffer for the imaginary component of the cartesian coordinates. The phase results are comprised in the range of 0-360.	c		e	f	g	a		i		j	k	l	m	n	b

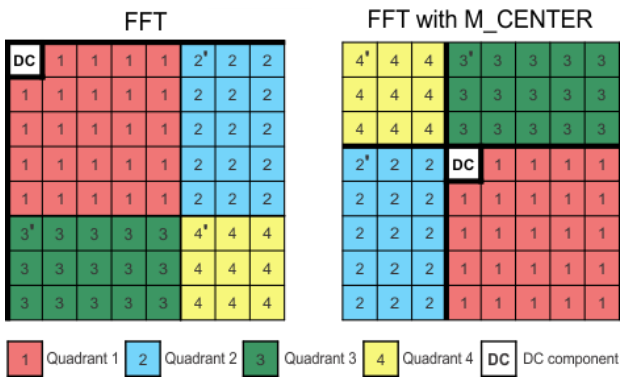
Combination values for [when TransformType=M_DCT8X8: M_FORWARD; M_REVERSE;], [when TransformType=M_FFT: M_FORWARD; M_REVERSE].

You can add one or more of the following values to the above-mentioned values to set the transform characteristics.

Note that only M_NORMALIZE is supported with both M_FFT and M_DCT8X8. All other values are supported only with M_FFT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the ControlFlag parameter																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_1D_COLUMNS	Performs a 1D transform on all columns of the image. If M_1D_COLUMNS has been specified with the forward transform, then it must be specified for the reverse transform as well.	c		e	f	g	a		i		j	k	l	m	n	b
M_1D_ROWS	Performs a 1D transform on all rows of the image. If M_1D_ROWS has been specified with the forward transform, then it must be specified for the reverse transform as well.	c		e	f	g	a		i		j	k	l	m	n	b
M_CENTER	Centers the real and imaginary components of the image in the frequency domain by repositioning their respective buffers' quadrants. The DC component is positioned at (SizeX/2-1, SizeY/2-1), where SizeX and SizeY are the width and height of the source buffer, respectively.  If the size of the destination buffers is different from that of the source buffer, data that falls outside of the destination buffers is clipped; exceeding areas of the destination buffers are unaffected. See the Managing data buffers section of Chapter 23: Data buffers for more information. If M_CENTER has been specified for the forward transform, then it must be specified for the reverse transform as well.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAGNITUDE +	Specifies whether to compute or use the magnitude. When used with M_FORWARD, computes the magnitude of the forward FFT (the square root of $R^2 + I^2$). The magnitude is returned in the DstImageRBufId destination buffer, thereby overwriting the real component of the destination image. When used with M_REVERSE, computes the real component of the transform from the magnitude and phase ($R = M \cos(P)$). The result will be used to perform the reverse FFT. For a reverse FFT, SrcImageRBufId and SrcImageIBufId must contain the magnitude and phase of a forward transform, respectively. M_MAGNITUDE must be used with M_PHASE for a reverse FFT. For a forward FFT, DstImageIBufId can be set to **M_NULL** if M_MAGNITUDE is used without M_PHASE. M_MAGNITUDE can only be used in transforms processed with floating-point buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE	Normalizes results (divide the final result by 8 for DCT and by (m x n) for FFT where m x n is the size of the image). M_NORMALIZE must be used with fixed-point arithmetic (all integer buffers) to avoid overflows. If M_NORMALIZE has been specified with the forward transform, then it must be specified for the reverse transform as well.	c		e	f	g	a		i		j	k	l	m	n	b

M_PHASE	Specifies whether to compute or use the phase. When used with M_FORWARD, computes the phase of the forward transform (P = atan(I/R)). The phase is returned in the DstImageIBufId buffer, thereby overwriting the imaginary component of the destination image. The phase is returned in the range of 0° to 360°. When used with M_REVERSE, computes the imaginary component of the transform from the magnitude and phase (I = M sin(P)). The result will be used to perform the reverse FFT. For a reverse FFT, SrcImageRBufId and SrcImageIBufId must contain the magnitude and phase of a forward transform, respectively. M_PHASE must be used with M_MAGNITUDE for a reverse FFT. For a forward FFT, DstImageRBufId can be set to **M_NULL** if M_PHASE is used without M_MAGNITUDE. M_PHASE can only be used in transforms processed with floating-point buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Clips (saturates) the results of a reverse FFT according to the destination buffer's data type. M_SATURATION can only be used with a reverse FFT (M_FFT + M_REVERSE) operation which uses floating-point source buffers and integer destination buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_MAGNITUDE.

You can add the following value to the above-mentioned value to specify that the magnitude of the forward FFT is scaled logarithmically.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_MAGNITUDE																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LOG_SCALE	Scales logarithmically the magnitude of the forward FFT to be in the range of 0-255. This value is used to scale the spectrum into a displayable range. M_LOG_SCALE can only be used in forward transforms processed with floating-point buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when TransformType=M_POLAR: M_FORWARD; M_REVERSE].

You can add one or more of the following values to the above-mentioned values to specify the transform characteristics.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the transform characteristics																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FAST_PHASE	Performs faster phase computation. For a forward polar transformation, it uses a faster computation of the phase results but slightly less accurate. For a reverse polar transformation, it has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZE_PHASE	Rescales the phase results. For a forward polar transformation, it rescales the phase results to be in the range of 0.0-1.0. For a reverse polar transformation, SrcImageRBufId must contain the phase values in the range of 0.0-1.0. This value cannot be used in combination with M_NORMALIZE_PHASE_255.	c		e	f	g	a		i		j	k	l	m	n	b

M_NORMALIZE_PHASE_255	Rescales the phase results. For a forward polar conversion, it rescales the phase results to be in the range of 0-255. For a reverse polar transformation, SrcImageIBufId must contain the phase values in the range of 0-255. This value cannot be used in combination with M_NORMALIZE_PHASE.	c		e	f	g	a		i		j	k	l	m	n	b
M_SQUARE_MAGNITUDE	Specifies that the magnitude values are returned or used squared. For a forward polar transformation, it gives the square of the magnitude results (that is, $R^2 + I^2$). For a reverse transformation, SrcImageRBufId must contain the magnitude values squared.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimTranslate

Synopsis

Translate an image by a specified X- and/or Y-displacement, or generate the matrix to perform the transformation.

Syntax

```
void MimTranslate(
    AIL_ID SrcImageBufId,        //in
    AIL_ID DstImageOrArrayBufId, //out
    AIL_DOUBLE DisplacementX,    //in
    AIL_DOUBLE DisplacementY,    //in
    AIL_INT64 InterpolationMode  //in
)
```

Description

This function translates the source image by the specified X- and Y-displacement, writing results to the destination buffer. Alternatively, this function can generate the 3x3 or 3x2 matrix required to perform the transformation using MimWarp().

This function can be used to align images to subpixel accuracy before, for example, subtracting them.

Parameters

SrcImageBufId

Specifies the source image to translate if performing the transformation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source image buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies to ignore this parameter; this parameter must be set to M_NULL when generating the matrix that you can use with MimWarp() to perform the translation.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of the source image buffer to translate. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b

DstImageOrArrayBufId

Specifies the destination image buffer or array buffer, depending on whether performing the transformation. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the destination buffer identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Array buffer identifier	Specifies the identifier of the destination M_ARRAY buffer when generating the matrix. When generating a warp matrix, this parameter must be given a single-band, 32-bit floating-point buffer that has an M_ARRAY attribute and that has dimensions 3x2, for a first-order polynomial warping, or 3x3, for a perspective polynomial warping. If you pass this parameter a buffer with dimensions 3x2, MimWarp() will assume the third row to be (0, 0, 1) when performing the warping.	c		e	f	g	a		i		j	k	l	m	n	b

Image buffer identifier	Specifies the identifier of the destination image buffer when translating the source image. This buffer must not have a region of interest (ROI) associated with it. Using a buffer with an ROI will cause an error. > Note that after performing the operation, the destination image will not be calibrated, even if the source image was calibrated.	c		e	f	g	a		i		j	k	l	m	n	b

DisplacementX

Specifies the amount, in pixel units, by which to displace the source image in the X-direction. This parameter can be set to any positive or negative value.

DisplacementY

Specifies the amount, in pixel units, by which to displace the source image in the Y-direction. This parameter can be set to any positive or negative value.

InterpolationMode

Specifies the interpolation mode when translating the source image. When only generating the matrix, this parameter is ignored and should be set to M_DEFAULT.

This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the interpolation mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. When translating the source image, the default is the same as M_BILINEAR + M_OVERSCAN_ENABLE. When only generating the coefficient matrix, this is the only possible value.	c		e	f	g	a		i		j	k	l	m	n	b
M_BICUBIC +	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR +	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR +	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the InterpolationMode parameter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For overscan																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_CLEAR	Sets the destination pixel to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Leaves the destination pixel as is.	c		e	f	g	a		i		j	k	l	m	n	b

M_OVERSCAN_ENABLE	Uses pixels from the source buffer's ancestor buffer. If the source buffer is not a child buffer or if the point falls outside the ancestor buffer, it leaves the destination pixel as is. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Points falling outside the source image are undefined.										j	k	l	m		
M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimUnwarpAlongPath

Synopsis

Unwarp an image along a path with a specified width.

Syntax

```
void MimUnwarpAlongPath(
    AIL_ID UnwarpAlongPathContextImId, //in
    AIL_ID SrcImageBufId,              //in
    AIL_ID DstImageBufId,              //out
    AIL_INT64 ControlFlag              //in
)
```

Description

This function unwarps an image along a path with a specified width.

Before calling this function, you must add a path to the context using MimPut() with M_XY_PATH and two arrays (one for the X-coordinates and one for the Y-coordinates). In addition, you must specify the width of the path using with using MimControl() with M_SINGLE_WIDTH.

This function generates quadrilateral regions from the intersections of the top and bottom lines, defined one half-width above and below the center of the path. When using the M_ADAPTIVE_ALONG_PATH sampling mode, the image is unwarped along the entire quadrilateral. If the M_ADAPTIVE_AT_JUNCTION sampling mode is used, this function will generate rectangles from the shortest side (top or bottom) of the quadrilateral, then generate triangles to fill in the rest of the original quadrilateral. In this case, the image is only distorted in the triangular region at the junction of each path segment.

Parameters

UnwarpAlongPathContextImId

Specifies the identifier of the unwarp along path image processing context. This context must have been previously allocated using MimAlloc() with M_UNWARP_ALONG_PATH_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the unwarp along path image processing context identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Unwarp along path image processing context ID	Specifies the identifier of the unwarp along path image processing context.	c		e	f	g	a		i		j	k	l	m	n	b

SrcImageBufId

Specifies the identifier of the source image buffer. The buffer must have been previously allocated with M_PROC. If this parameter is not used, it must be set to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination buffer in which to put the result of the operation. The buffer must have been previously allocated with M_PROC. If this parameter is not used, it must be set to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

ControlFlag

Specifies which unwarp operation to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which unwarp operation to perform																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Performs the unwarp along path operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESS	Preprocesses the unwarp along path image processing context specified by UnwarpAlongPathContextImId. Note that, if not called explicitly, this operation will be performed automatically upon the first call to **MimUnwarpAlongPath**(). If DstImageBufId is not specified, the optimal size of the destination buffer is determined according to the path length (X-size) and the path width (Y-size), rounded up to the nearest integer. You can use MimInquire() with M_DESTINATION_SIZE_X and M_DESTINATION_SIZE_Y to inquire the optimal destination image buffer size to allocate.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimWarp

Synopsis

Perform a warping.

Syntax

```
void MimWarp(
    AIL_ID SrcImageBufId,       //in
    AIL_ID DstImageBufId,       //out
    AIL_ID WarpParam1BufId,     //in
    AIL_ID WarpParam2BufId,     //in
    AIL_INT64 OperationMode,    //in
    AIL_INT64 InterpolationMode //in
)
```

Description

This function warps an image through either a 3x3 matrix-defined warping or LUTs. You can perform first-order polynomial warpings or perspective polynomial warpings using either a 3x3 coefficients matrix or lookup tables (LUTs). You can perform custom warpings using LUTs.

A warping associates each pixel position of the destination buffer, (x_d, y_d), with a specific point in the source buffer, (x_s, y_s), and then determines the pixel value of (x_d, y_d) from its associated point and from a specified interpolation mode.

When performing a first-order polynomial warping, (x_d, y_d) gets associated with (x_s, y_s) through the following equations:

$$x_s = a_0 x_d + a_1 y_d + a_2$$

$$y_s = b_0 x_d + b_1 y_d + b_2$$

When performing a perspective polynomial warping, (x_d, y_d) gets associated with (x_s, y_s) through the following equations:

$$x_s = (a_0 x_d + a_1 y_d + a_2)/(c_0 x_d + c_1 y_d + c_2)$$

$$y_s = (b_0 x_d + b_1 y_d + b_2)/(c_0 x_d + c_1 y_d + c_2)$$

When performing a 3x3 matrix-defined warping, WarpParam1BufId specifies the required coefficients (a_0...a_2, b_0...b_2) or (a_0...a_2, b_0...b_2, c_0...c_2) and WarpParam2BufId must be set to **M_NULL**. The coefficients can be automatically generated using MgenWarpParameter() or can be user-supplied.

When performing a warping using LUTs, x_s is determined from (x_d, y_d) through one LUT and y_s is determined from (x_d, y_d) through another LUT. In this case, WarpParam1BufId specifies the LUT for x_s and WarpParam2BufId specifies the LUT for y_s. The LUTs can be user-supplied or can be automatically generated from a 3x3 coefficient matrix (for a first-order or polynomial warping) using MgenWarpParameter().

Parameters

SrcImageBufId

Specifies the buffer on which to perform the warping. This buffer can be of any type.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the buffer in which to place the results of the warping. This buffer can be of any type.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

> Note that after performing the operation, the destination image will not be calibrated, even if the source image was calibrated.

WarpParam1BufId

Specifies the buffer containing the matrix coefficients or the LUT buffer from which x_s is determined.

When WarpParam1BufId specifies the (a_0...a_2, b_0...b_2) or (a_0...a_2, b_0...b_2, c_0...c_2) coefficients, the coefficients must be in a single-band, 32-bit floating-point buffer that has an M_ARRAY attribute and that has dimensions 3x2 or 3x3. The first row specifies the a_n coefficients, the second row specifies the b_n coefficients, and the third row specifies the c_n coefficients. If the buffer is 3x2, the third row is assumed to be (0, 0, 1).

When WarpParam1BufId specifies the LUT buffer from which x_s is determined, the buffer must be a signed 16- or 32-bit integer buffer or a 32-bit floating-point buffer, have the same X and Y size as the destination image buffer, and have an M_LUT attribute.

WarpParam2BufId

Specifies the LUT buffer from which y_s is determined. This buffer must be a signed 16- or 32-bit integer buffer or a 32-bit floating-point buffer, have the same X and Y size as the destination image buffer, and have an M_LUT attribute.

If you are not using LUTs to perform the warping, set WarpParam2BufId to **M_NULL**.

OperationMode

Specifies the mode of operation. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the mode of operation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_WARP_LUT +	Performs the warping through LUTs.	c		e	f	g	a		i		j	k	l	m	n	b
M_WARP_POLYNOMIAL	Performs the warping through a 3x3 coefficient matrix. This includes a first-order polynomial warping or a perspective polynomial warping. Note that with a perspective polynomial warping matrix, **MimWarp**() might return an error if the calculation of $(c_0x_d + c_1y_d + c_2)$ is 0 in the destination buffer, regardless of whether or not the matrix was generated using MgenWarpParameter().	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_WARP_LUT.

You can add the following value to the above-mentioned value to specify the number of fractional bits in the coordinates of the source point.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_WARP_LUT																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FIXED_POINT + *n*	Specifies the number of fractional bits for the source point (x_s , y_s). To do so, add the define M_FIXED_POINT + n to M_WARP_LUT where n >= 0. If nothing is added to M_WARP_LUT, it is assumed that there are no fractional bits in the coordinates of the source point (and therefore interpolation is not required).	c		e	f	g	a		i		j	k	l	m	n	b

InterpolationMode

Specifies the interpolation mode. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the interpolation mode																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_NEAREST_NEIGHBOR + M_OVERSCAN_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

M_BICUBIC +	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_BILINEAR +	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEAREST_NEIGHBOR +	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the InterpolationMode parameter.

You can add one of the following values to the above-mentioned values to specify how to determine the value of a destination pixel when its associated point falls outside the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For overscan																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OVERSCAN_CLEAR	Sets the destination pixel to 0, if the associated point falls outside the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_DISABLE	Leaves the destination pixel as is.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_ENABLE	Uses pixels from the source buffer's ancestor buffer. If the source buffer is not a child buffer or if the point falls outside the ancestor buffer, leave the destination pixel as is. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Points falling outside the source image are undefined.										j	k	l	m		
M_OVERSCAN_FAST	Specifies that Aurora Imaging Library will automatically select the overscan that optimizes speed, according to the specified operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_OVERSCAN_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimWarpList

Synopsis

Applies a warping matrix to a list of coordinates.

Syntax

```
void MimWarpList(
    AIL_ID WarpParamBufId,              //in
    AIL_INT64 TransformType,            //in
    AIL_INT NumPoints,                  //in
    const AIL_DOUBLE *SrcCoordXArrayPtr, //in
    const AIL_DOUBLE *SrcCoordYArrayPtr, //in
    AIL_DOUBLE *DstCoordXArrayPtr,      //out
    AIL_DOUBLE *DstCoordYArrayPtr,      //out
    AIL_INT64 ControlFlag               //in
)
```

Description

This function transforms a list of coordinates (points) using the specified warping matrix and stores the resulting coordinates in the destination arrays (M_REVERSE). Alternatively, this function can transform the coordinates using the inverse of the specified matrix (M_FORWARD).

When this function transforms the coordinates using the warping matrix as is (M_REVERSE), it is referred to as a reverse warping transformation because this is how MimWarp() associates each pixel position of its destination buffer, (x_d, y_d), with a specific point in its source buffer, (x_s, y_s). Therefore, you can use **MimWarpList**() to establish which points in a source image MimWarp() associates with destination pixels when using the same matrix. When this function transforms the coordinates using the inverse of the specified warping matrix (M_FORWARD), it is referred to as a forward warping transformation; you can use a forward warping transformation, for example, to see if a specific pixel in the source image of MimWarp() is actually mapped to a destination buffer pixel.

You can only specify the coefficients for a 3x3 or 3x2 matrix; this allows you to perform a first-order polynomial warping or a perspective polynomial warping.

Perform a first-order polynomial warping by passing a 3x3 matrix, where the third row has the coefficients [0,0,1], or by passing a 3x2 matrix, the last row is assumed to be [0,0,1]. Perform a perspective polynomial warping by passing a 3x3 matrix, where the third row does not have the coefficients [0,0,1]. The matrix is applied as follows:

$x_d = (a_0 x_s + a_1 y_s + a_2)/(c_0 x_s + c_1 y_s + c_2)$

$y_d = (b_0 x_s + b_1 y_s + b_2)/(c_0 x_s + c_1 y_s + c_2)$

If a 3x2 matrix, or a 3x3 matrix whose third row has the coefficients of [0,0,1] is specified (for a first-order polynomial warping operation), the denominator resolves to 1, effectively leaving only the numerator. The coefficients can be automatically generated using MgenWarpParameter(), MimResize(), MimRotate(), or MimTranslate(), or can be user-established.

Note that **MimWarpList**() will internally invert the coefficients matrix (a_0...a_2, b_0...b_2, c_0...c_2) for a forward transfomation (M_FORWARD).

Parameters

WarpParamBufId

Specifies the buffer containing the warp matrix coefficients.

The coefficients must be in a single-band, 32-bit floating-point buffer that has an M_ARRAY attribute and that has dimensions 3x2 or 3x3. The first row specifies the a_n coefficients, the second row specifies the b_n coefficients, and the third row specifies the c_n coefficients. If the buffer is 3x2, the third row is assumed to be (0, 0, 1).

TransformType

Specifies the transformation type to perform the warping.

For specifying the transformation type.																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_REVERSE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FORWARD	Specifies to perform a forward transformation. A forward transformation uses the inverse of the specified warping matrix to transform the source X- and Y-coordinates.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE	Specifies to perform a reverse transformation. A reverse transformation uses the specified warping matrix as is to transform the source X- and Y-coordinates.	c		e	f	g	a		i		j	k	l	m	n	b

NumPoints

Specifies the number of points in the arrays.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the SrcCoordXArrayPtr or SrcCoordYArrayPtr parameter.

SrcCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array that contains the source X-coordinates.

SrcCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array that contains the source Y-coordinates.

DstCoordXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to store the destination X-coordinates.

DstCoordYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPoints)

Specifies the address of the array in which to store the destination Y-coordinates.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.

DLL	Requires ail11.dll; ailim11.dll.

MimWatershed

Synopsis

Perform a watershed transformation.

Syntax

```
void MimWatershed(
    AIL_ID SrcImageBufId,    //in
    AIL_ID MarkerImageBufId, //in
    AIL_ID DstImageBufId,    //out
    AIL_INT MinVariation,    //in
    AIL_INT64 ControlFlag    //in
)
```

Description

This function performs a watershed transformation on the specified source buffer. The catchment basins of the source buffer can be determined from extrema (minima or maxima) in the source buffer and/or from a specified marker image. In the case when no marker image is specified, a valid catchment basin is determined from an extrema when the minimum variation in gray levels of an extremum's catchment basin is equal to or greater than the value specified by the MinVariation parameter. In the case when a marker image is specified, each marker of a marker image produces catchment basin(s) by forcing an extremum in the corresponding area(s) of the source buffer.

There are two types of marker images: non-labeled and labeled. In a non-labeled marker image, each group of touching pixels with the value zero in the marker image (known as a marker) starts a catchment basin in the corresponding area of the source image. Specifically, each marker in the marker image forces a extremum in the corresponding area of the source image. Pixels in the marker image are considered touching if they are vertically, horizontally, or diagonally adjacent, that is, if they are "8-connected". Note that each catchment basin in the destination image is given a unique grayscale value, starting at 1.

In a labeled marker image, a marker is a set of pixels that do not have to necessarily touch and that have all the same label value (pixel intensity). Each marker starts a catchment basin in the corresponding area of the source image and each catchment basin of the destination image is assigned the label value of the marker that generated it. Note that, in a labeled marker image, a marker can touch other markers, allowing you to specify adjacent catchment basins. Valid marker label values are 1 to $2^n - 2$ (where *n* refers to the number of bits per pixel in the buffer). Pixels with the label value of zero are interpreted as the region of the image that is not a marker. Finally, pixels with the label value of $2^n - 1$ are considered to be part of a "don't care" mask and are not processed, accelerating the watershed transformation.

You can specify that the destination buffer contain one of the following:

- Watershed lines. The watershed lines are given the value 0 and all other pixels are given the maximum value in the destination buffer.
- Labeled catchment basins, without watershed lines. Each catchment basin is assigned a grayscale value greater or equal to 1.
- Labeled catchment basins and watershed lines. Each catchment basin is assigned a grayscale value greater or equal to 1. Watershed lines are given the value 0.

For more information, see the Watershed transformations section of Chapter 4: Advanced image processing.

Parameters

SrcImageBufId

Specifies the buffer on which to perform the transform. This buffer can be 8-bit or 16-bit, signed or unsigned.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

MarkerImageBufId

Specifies the buffer to use as a marker image. This buffer can be 8-bit or 16-bit, signed or unsigned. If you are not using a marker image, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The default marker image type is non-labeled. If you want to use a labeled marker image, you must add M_LABELED_MARKER to the ControlFlag parameter.

DstImageBufId

Specifies the buffer in which to place the results of the transformation. This buffer can be 8-bit or 16-bit, signed or unsigned. The destination buffer can hold 2^n - 5 catchment basins, where *n* refers to the number of bits per pixel in the buffer. Therefore, an 8-bit destination buffer can hold 251 catchment basins and a 16-bit destination buffer can hold 65531 catchment basins.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

MinVariation

Specifies the minimum variation of gray-level of a catchment basin. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum variation of gray-level																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Produces a catchment basin from each extremum in the source buffer. Setting MinVariation to 1 will have the same effect.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF	Specifies that catchment basins will not be determined from extrema in the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Sets the minimum variation.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies how to perform the transformation. It can be set to any combination of the following seven sets of values below. It can also be set to M_DEFAULT, in which case the default value from each set is used (without M_SKIP_LAST_LEVEL, M_LABELED_MARKER and M_FILL_SOURCE). If no value from a set is specified, its default value is used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how to perform the transformation																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_WATERSHED + M_MINIMA_FILL + M_REGULAR + M_4_CONNECTED.	c		e	f	g	a		i		j	k	l	m	n	b
M_BASIN +	Labels catchment basins.	c		e	f	g	a		i		j	k	l	m	n	b
M_WATERSHED +	Calculates watershed lines.	c		e	f	g	a		i		j	k	l	m	n	b
M_WATERSHED + M_BASIN +	Labels catchment basin and calculates watershed lines.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the ControlFlag parameter.

You can add one of the following values to the above-mentioned values to specify whether to use the source buffer's minima or maxima when determining catchment basins from extrema in the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to use the source buffer's minima or maxima																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MAXIMA_FILL	Uses the source buffer's maxima.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINIMA_FILL	Uses the source buffer's minima. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the ControlFlag parameter.

You can add one of the following values to the above-mentioned values to specify how the watershed lines are drawn.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how the watershed lines are drawn																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_REGULAR	Traces the watershed lines exactly. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRAIGHT_WATERSHED	Forces the watershed lines to be straight. Note that selecting M_STRAIGHT_WATERSHED automatically selects the M_SKIP_LAST_LEVEL setting.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the ControlFlag parameter.

You can add one of the following values to the above-mentioned values to specify whether 4 or 8-connected watershed lines will be used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying 4 or 8-connected watershed lines																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_4_CONNECTED	Uses 4-connected watershed lines. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_8_CONNECTED	Uses 8-connected watershed lines.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the ControlFlag parameter.

You can add the following value to the above-mentioned values to specify that an extremum's zone of influence cannot extend past the minimum and maximum gray level.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For preventing an extremum's zone of influence from extending past the minimum and maximum gray level																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SKIP_LAST_LEVEL	Prevents an extremum's zone of influence from extending beyond L_{max} - 1 (for a minimum) or L_{min} - 1 (for a maximum), where L_{max} is the maximum gray level in the image and L_{min} is the minimum gray level.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the ControlFlag parameter.

You can add the following value to the above-mentioned values to specify that the marker image is a labeled marker image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For denoting that the marker image is labeled																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_LABELED_MARKER	Specifies that the marker image is a labeled marker image.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the ControlFlag parameter.

You can add the following value to the above-mentioned values to specify that any unwanted extrema from the source image is removed.

It removes the extrema by filling the catchment basins of unwanted extrema until they become plateaus.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For removing unwanted extrema																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILL_SOURCE	Fill all local basins of a valid catchment basin in the source image.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimWaveletDenoise

Synopsis

Perform a wavelet denoising.

Syntax

```
void MimWaveletDenoise(
    AIL_ID WaveletContextImId,             //in
    AIL_ID SrcImageBufOrWaveletResultImId, //in
    AIL_ID DstImageBufOrWaveletResultImId, //out
    AIL_INT Level,                         //in
    AIL_ID DenoisingType,                  //in
    AIL_INT64 ControlFlag                  //in
)
```

Description

This function performs a denoising on the specified source buffer using a wavelet shrinkage process and places the results in the specified destination buffer. The source and destination can either both be an image or a wavelet result.

This function supports in-place processing; you can specify the identifier of the same buffer for the source and the destination. To specify wavelet controls that can affect this function's results, use MimControl().

To specify wavelet controls that can affect this function's results, use MimControl(). If the destination of this function is a result, you can perform drawing operations with them, using MimDraw().

Parameters

WaveletContextImId

Specifies the identifier of a wavelet image processing context, when performing a wavelet denoising of an image. The context must have been allocated using MimAlloc() with either M_WAVELET_TRANSFORM_CONTEXT or M_WAVELET_TRANSFORM_CUSTOM_CONTEXT. When performing a wavelet denoising of a result, this information is not required; set this parameter to **M_NULL**.

If you specify a wavelet context, it indicates the filter and mode that this function uses to perform the wavelet denoising. For an M_WAVELET_TRANSFORM_CONTEXT context type, you can set a predefined type of wavelet filter, using MimControl() with M_WAVELET_TYPE. The default is a Haar wavelet filter type. For an M_WAVELET_TRANSFORM_CUSTOM_CONTEXT context type, you must specify custom wavelet filters, using MimWaveletSetFilter(). For either type of context, the default mode is dyadic. To change this (for example, to specify undecimated), use the M_TRANSFORMATION_MODE control.

To inquire about the information in a wavelet image processing context, call MimInquire(). For example, you can retrieve the identifier of the internal buffer containing the wavelet filter values (M_FILTER_..._ID).

SrcImageBufOrWaveletResultImId

Specifies the identifier of the source with which to perform the wavelet denoising. The identifier must be for an image buffer or a wavelet image processing result buffer. The image buffer can be of any type. The wavelet result buffer must have been allocated on the required system using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT. It must hold MimWaveletTransform() results. To retrieve information about wavelet transformation results, use MimGetResult() and MimGetResultSingle().

If the source is an image buffer, the destination must be an image buffer of the same type. If the source is a wavelet result buffer, the destination must be a wavelet result buffer.

If you specify an image buffer, there must not be a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufOrWaveletResultImId

Specifies the identifier of the destination with which to perform the wavelet denoising. The identifier must be for an image buffer or a wavelet image processing result buffer. The image buffer can be of any type. The wavelet result buffer must have been allocated on the required system using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT.

If the destination is an image buffer, the source must be an image buffer of the same type. If the destination is a wavelet result buffer, the source must be a wavelet result buffer.

If you specify an image buffer, there must not be a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Level

Specifies the transformation level at which to perform the denoising.

Set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the transformation level																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the level information is not required. Only valid when the source is a result. Denoising is performed at the transformation level indicated in the result.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the level. Only valid when the source is an image. This function decomposes the image according to the specified number of levels, performs the denoising, and recomposes the image using the same number of levels. Aurora Imaging Library establishes the maximum level to be the point at which denoising at subsequent levels does not provide relevant data. The maximum level depends on the source image and the wavelet image processing context. If the specified level exceeds the maximum level, the maximum level is used.	c		e	f	g	a		i		j	k	l	m	n	b

DenoisingType

Specifies the type of wavelet process with which to perform the denoising.

Set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the wavelet shrinkage process																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BAYES_SHRINK	Specifies predefined BayesShrink denoising techniques. Typically effective for minimizing Gaussian noise.	c		e	f	g	a		i		j	k	l	m	n	b
M_NEIGH_SHRINK	Specifies predefined NeighShrink denoising techniques. Typically effective when noise is difficult to characterize. Denoising mimics the general behavior of wavelet coefficients by taking neighborhood statistics and dismissing outliers. M_NEIGH_SHRINK is often faster than the other predefined denoising techniques, though results can be less precise.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURE_SHRINK	Specifies predefined SureShrink denoising techniques. Typically effective for minimizing Mean Square Errors (MSE).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimWaveletSetFilter

Synopsis

Set wavelet filters for a custom wavelet context.

Syntax

```
void MimWaveletSetFilter(
    AIL_ID WaveletContextImId,           //out
    AIL_ID LowForwardRealFilterId,       //in
    AIL_ID HighForwardRealFilterId,      //in
    AIL_ID LowReverseRealFilterId,       //in
    AIL_ID HighReverseRealFilterId,      //in
    AIL_ID LowForwardImaginaryFilterId,  //in
    AIL_ID HighForwardImaginaryFilterId, //in
    AIL_ID LowReverseImaginaryFilterId,  //in
    AIL_ID HighReverseImaginaryFilterId, //in
    AIL_INT64 ControlFlag                //in
)
```

Description

This function allows you to set wavelet filters for a custom wavelet context.

Filter parameters must specify the identifier of a kernel buffer allocated using MbufAlloc...() with M_KERNEL. Each kernel buffer type must be 32-bit float (32 + M_FLOAT). The height (SizeY) of each kernel buffer must be 1. The width (SizeX) of each kernel buffer must be identical. Kernels contain the filter values used by the wavelet transformation, when you call MimWaveletTransform() with a custom wavelet context.

Each filter parameter can be set to **M_NULL** if you do not require its information. At least one filter parameter must specify the identifier of a kernel.

Parameters

WaveletContextImId

Specifies the identifier of a custom wavelet image processing context allocated using MimAlloc() with M_WAVELET_TRANSFORM_CUSTOM_CONTEXT.

LowForwardRealFilterId

Specifies the identifier of the kernel buffer containing the real part of the complex numbers used for the low-pass filter of a forward wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

HighForwardRealFilterId

Specifies the identifier of the kernel buffer containing the real part of the complex numbers used for the high-pass filter of a forward wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

LowReverseRealFilterId

Specifies the identifier of the kernel buffer containing the real part of the complex numbers used for the low-pass filter of a reverse wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

HighReverseRealFilterId

Specifies the identifier of the kernel buffer containing the real part of the complex numbers used for the high-pass filter of a reverse wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

LowForwardImaginaryFilterId

Specifies the identifier of the kernel buffer containing the imaginary part of the complex numbers used for the low-pass filter of a forward wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels.

If you do not require the filter information related to this parameter, set it to **M_NULL**.

HighForwardImaginaryFilterId

Specifies the identifier of the kernel buffer containing the imaginary part of the complex numbers used for the high-pass filter of a forward wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

LowReverseImaginaryFilterId

Specifies the identifier of the kernel buffer containing the imaginary part of the complex numbers used for the low-pass filter of a reverse wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

HighReverseImaginaryFilterId

Specifies the identifier of the kernel buffer containing the imaginary part of the complex numbers used for the high-pass filter of a reverse wavelet transformation. The kernel must be 32-bit float, have a height of 1, and a width that is identical to the other kernels. If you do not require the filter information related to this parameter, set it to **M_NULL**.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimWaveletTransform

Synopsis

Perform a wavelet transformation.

Syntax

```
void MimWaveletTransform(
    AIL_ID WaveletContextImId,             //in
    AIL_ID SrcImageBufOrWaveletResultImId, //in
    AIL_ID DstImageBufOrWaveletResultImId, //out
    AIL_INT64 TransformType,               //in
    AIL_INT Level,                         //in
    AIL_INT64 ControlFlag                  //in
)
```

Description

This function performs a forward or reverse wavelet transformation using the specified source buffer and places the results in the specified destination buffer. The source and destination can be an image or wavelet result.

This function supports in-place processing; you can specify the identifier of the same buffer for the source and the destination.

Wavelets refer to mathematical algorithms that process data in both the space and frequency domain, offering a compromise between each. By using wavelets, **MimWaveletTransform**() results can therefore represent space and frequency. This makes **MimWaveletTransform**() especially effective at filtering images. It is typically useful for signal coding, signal denoising, data compression, and pattern recognition.

To specify wavelet controls that can affect **MimWaveletTransform**() results, use MimControl(). To perform drawing operations with this function's results, use MimDraw().

To retrieve information about a specific wavelet result, use MimGetResultSingle(). A specific wavelet result refers to a transformed rendition of the source, based on either a wavelet coefficient (diagonal, horizontal, or vertical) at a specific level, or on an approximation of the wavelet transformation. For general types of wavelet results, such as the actual number of transformation levels used, call MimGetResult().

Parameters

WaveletContextImId

Specifies the identifier of a wavelet image processing context. The context must have been allocated using MimAlloc() with M_WAVELET_TRANSFORM_CONTEXT or M_WAVELET_TRANSFORM_CUSTOM_CONTEXT.

The context indicates the filter and mode that this function uses to perform its transformation. For an M_WAVELET_TRANSFORM_CONTEXT context type, you can set a predefined type of wavelet filter, using MimControl() with M_WAVELET_TYPE. The default is a Haar wavelet filter type. For an M_WAVELET_TRANSFORM_CUSTOM_CONTEXT context type, you must specify custom wavelet filters, using MimWaveletSetFilter(). For either type of context, the default transformation mode is dyadic. To change this (for example, to specify undecimated), use the M_TRANSFORMATION_MODE control.

To inquire about the information in a wavelet image processing context, call MimInquire(). For example, you can retrieve the identifier of the internal buffer containing the wavelet filter values (M_FILTER_..._ID).

SrcImageBufOrWaveletResultImId

Specifies the identifier of the source with which to perform the wavelet transformation. The identifier must be for an image buffer or a wavelet image processing result buffer. The image buffer can be of any type. The wavelet result buffer must have been allocated on the required system using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT. It must hold **MimWaveletTransform**() results.

If the source is an image buffer, the destination must be a wavelet result buffer. If the source is wavelet result buffer, the destination can be an image or a wavelet result buffer.

If you specify an image buffer, there must not be a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufOrWaveletResultImId

Specifies the identifier of the destination in which to write the result of the wavelet transformation. The identifier must be for an image buffer or a wavelet image processing result buffer. The image buffer can be of any type. The wavelet result buffer must have been allocated on the required system using MimAllocResult() with M_WAVELET_TRANSFORM_RESULT.

If the destination is an image buffer, the source must be a wavelet result buffer. If the destination is a wavelet result buffer, the source can be an image or a wavelet result buffer.

If you specify an image buffer, there must not be a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

TransformType

Specifies the type of wavelet transformation to perform.

Set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the wavelet transformation type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FORWARD	Specifies a forward wavelet transformation. This decomposes the source according to the number of levels established. The source can be an image or result. The destination must be a result.	c		e	f	g	a		i		j	k	l	m	n	b
M_REVERSE	Specifies a reverse wavelet transformation. This recomposes the source according to the number of levels established. The source must be a result. The destination can be an image or a result.	c		e	f	g	a		i		j	k	l	m	n	b

Level

Specifies the number of transformation levels with which to calculate results. Results are written in the specified destination buffer. The level applies to either an M_FORWARD or M_REVERSE transformation.

Set this parameter to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the transformation level																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies a level of 0. Only supported when using M_REVERSE and the destination is an image. In any other case, M_DEFAULT (or 0) causes an error.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the transformation level. Values depend on the transformation, the source, and the destination. With M_FORWARD, the source can be an image or result; the destination must be a result. In all cases, the level must be greater than 0 and less than or equal to the maximum level. If the source is a result, the level must also be greater than the level in the source. If the specified level exceeds the maximum level, Aurora Imaging Library uses the maximum level instead. Aurora Imaging Library establishes the maximum level to be the point at which transformations at subsequent levels do not provide relevant data. This depends on the source and the wavelet image processing context. With M_REVERSE, the source must be a result; the destination can be a result or an image. If the destination is a result, the level must be greater than 0 and less than the level in the source. If the specified level exceeds the level in the source (maximum level), Aurora Imaging Library uses the level in the source instead. If the destination is an image, the level must be 0 (or M_DEFAULT). Setting the level to 0 (or M_DEFAULT) when the destination is a result causes an error. To determine the level in the source result, use it with MimGetResult() and M_NUMBER_OF_LEVELS. To determine the level in the destination result, use it with M_NUMBER_OF_LEVELS.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.

MimZoneOfInfluence

Synopsis

Perform a zone of influence detection.

Syntax

```
void MimZoneOfInfluence(
    AIL_ID SrcImageBufId,   //in
    AIL_ID DstImageBufId,   //out
    AIL_INT64 OperationFlag //in
)
```

Description

This function separates an image into zones, according to how much of a blob's surrounding background is within the blob's territorial boundaries, or "zone of influence". The image is considered to be binary, with background pixels equal to 0 (black), and all non-zero pixels treated as blobs. It gives every pixel in a blob's zone of influence the same value. Each zone of influence is numbered consecutively, beginning with 1. A blob's zone of influence consists of all pixels closer to that blob than to any other blob. There are as many zones as blobs.

Parameters

SrcImageBufId

Specifies the identifier of the source of the operation. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

DstImageBufId

Specifies the identifier of the destination of the resulting image. This parameter must be given an image buffer identifier.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The destination buffer should be deep enough to hold the maximum number of zones (blobs). The maximum label value is $2^n - 5$, where *n* is the depth of the destination buffer in bits. For example, an 8-bit buffer can be used for a maximum of 251 blobs and a 16-bit buffer can be used for a maximum of 65531 blobs. Note that if the destination buffer depth is too small, several zones might be given the same value.

OperationFlag

Specifies the type of 3x3 distance matrix used for the operation. The following table shows each value's respective 3x3 distance matrix for calculating the distance to a neighboring pixel, which is then used for the zone of influence computation.

This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the type of 3x3 distance matrix																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CHESSBOARD.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAMFER_3_4	Uses a distance algorithm that makes a better approximation to actual Euclidean distance, and is therefore more accurate than M_CHESSBOARD. This is the 3x3 distance matrix: $\begin{bmatrix} 4 & 3 & 4 \\ 3 & 0 & 3 \\ 4 & 3 & 4 \end{bmatrix}$	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_CHESSBOARD	Specifies a chessboard matrix. This operation is faster than M_CHAMFER_3_4. This is the 3x3 distance matrix: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$															

Remark

- In-place processing is supported, but the source and destination image buffers cannot partially overlap (a situation that can only occur when using child buffers).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailim11.lib.
DLL	Requires ail11.dll; ailim11.dll.